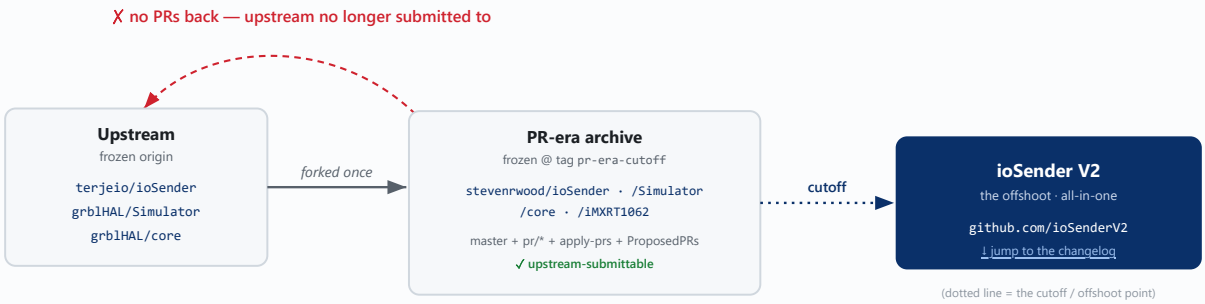
ioSender V2 — Overview

An enhanced **all-in-one** grblHAL / Grbl g-code sender. ioSender V2 is the ongoing offshoot of `stevenrwood/ioSender` (itself a one-time fork of `terjeio/ioSender`), severed from upstream at the **PR-era cutoff**. There are no composable feature branches and nothing is submitted upstream any more — the product is the `master` branch, consumed whole. This page maps how V2 came to be and how its repos wire together at runtime; the full changelog follows below.



One fork from each upstream → the PR-era archive (everything up to the cutoff, still submittable) → a dotted hand-off across the cutoff to **ioSender V2**, where all development now happens. Upstream is a frozen origin with the return path severed.

V2 repos & how they wire together at runtime

The app and simulator moved to the `ioSenderV2` org (original names kept, full git history preserved). The grblHAL **firmware** was *not* migrated — its deep submodule web makes it all-or-nothing, so it stays in the frozen fork, referenced by the `pr-era-cutoff` tag.

`ioSenderV2/ioSender`	The WPF sender app (this document lives here). Talks to the controller over USB serial or Telnet/WebSocket.
`ioSenderV2/Simulator`	**Option-matched simulator.** On connect the app reads the controller's build options → a 12-hex signature, then pulls a matching sim from this repo's `sim-<sig>` releases (a shared build cache), dispatching the `build-matched-sim` CI to produce one if absent. Every download is public; a token is only needed to *trigger* a build.
`stevenrwood/iMXRT1062` firmware · tag-only	The grblHAL firmware build (Teensy 4.1 driver superproject + `core` submodule at `srw/combined`, incl. the WCS-rotation transform fix) the app connects to. Stays in the fork — build branch `srw/local-build-config`, frozen reference at tag `pr-era-cutoff`.

THE FULL CHANGELOG — EVERY FEATURE & FIX IN THE V2 BUILD

Features & Fixes

Fork `stevenrwood/ioSender` · base branch `master` · **414 improvements in the `integration` build.** The fork ships as one all-in-one build — every change below is already in the `integration` branch and consumed wholesale. · all LF-normalized (`.gitattributes * text=auto eol=lf`).

Features at a glance — grouped by area. **NEW** new capability, **CHG** changes existing behaviour, **FIX** bug fix. Numbers link to the detail below.

Connectivity & reliability

Connect to USB / Ethernet / simulator, with a Connect/Reconnect menu, remembered host and opt-in prefer-network	NEW	15
Validate controller — streams a capability-tailored test in check mode (no motion), reports pass/fail + 3D preview	NEW	16
Restore main-window *position* across launches (was size only)	CHG	17
Survive connection loss / SD-card swap — guarded writes + auto-reconnect instead of crashing	FIX	4
Telnet / websocket modal-dialog UI deadlock	FIX	24
`-forgetNetwork` startup flag — ignore the saved connection target for one run and open the connect dialog (scan the network / pick any transport) instead of auto-connecting the remembered host; non-destructive (the saved value is untouched, and re-learned from the controller's `$I` on the next serial connect)	NEW	66
Demo-video capture aids (`-demomarker`) — an opt-in timestamped marker log (`SESSION_START` / `RUN_START` / `RUN_END` / `TIMELAPSE_ON/OFF`) plus a run-bar *Timelapse* toggle, so a video compositor can sync the app screen-recording to the machine-camera footage and time-compress the long cuts; hidden and near-free when off	NEW	67
OBS auto-record bridge & tool-change cut markers — with `-demomarker`, ioSender drives OBS Studio over obs-websocket (auto Start/Stop recording on program load / end, so a shoot needs no manual Record button); `RUN_START/RUN_END` now bracket the actual cutting, with a tool change emitting `RUN_END…RUN_START` around the M6. Plus `build.ps1` forwards trailing flags to the launched app	NEW	68
Per-Monitor V2 DPI awareness — the app reads the *live* per-monitor DPI (like Edge / VS Code) instead of a system-DPI value cached at logon, so it renders at the correct scale regardless of resolution / scale changes and is no longer thrown off by a stale DPI cache; adds `-debuglog=dpi` diagnostics	NEW	69
The connect handshake no longer fails a healthy controller, and refuses to carry on blind if it does.	FIX	235
The UI pump no longer spins unbounded, which twice reached the 32-bit ceiling and killed the app.	FIX	245
A controller wait whose worker threw would spin for ever — responsive, and completely ignoring you. Forty of them now end.	FIX	253
A controller that stopped answering could freeze the sender for the rest of the session; a settings write that was never acknowledged reported success anyway. Both fixed.	FIX	258
The migration probe now requires a status report back, and spends one attempt per app run	FIX	356

A soft-limit alarm now says which axis ran out of travel and by how much, with the sum worked out and the line that asked for it - and says nothing rather than guessing where it cannot resolve the frame honestly.	NEW	402

Jogging & main page

Configurable main page — Edit Main Page editor, pinnable flyouts, assignable jog panels, per-offset Go/Set buttons	NEW	9
Native Xbox / game-controller support — analog-stick jog, remappable buttons, live Controller tab	NEW	8
3D view: Ctrl+click to jog (retracts Z first) + per-axis $23 orientation fix	NEW	20
Keyboard jog keeps working when focus drifts off the Job view	CHG	22
High-DPI / 125–150% text-scaling layout	CHG	7
Assignable *Jog distance +/−* controller actions — step the jog distance preset (separate from jog speed)	NEW	35
Run-bar jog preset spinners — read-only *Dist* / *Feed* up/down selectors (wrap-around) on the Check line; startup splash stays on top so the full init sequence is visible	NEW	45
Drag a tab header left/right to reorder tabs — main bar and every sub-tab strip; the order persists across restarts (and agrees with the Edit Main Page editor)	NEW	60
Jog commands now wait for the previous $J's ok before sending another - rapid clicks were able to send an overlapping $J and wedge the controller (no response to anything, not just jog, until a power cycle).	FIX	121
Analog stick jog now runs on a dedicated background thread instead of the UI dispatcher - eliminates stutter during a continuous hold.	FIX	128
The run bar's "Cycle Start" button is now just "Run"; a small dropdown next to it replaces the old right-click menu for choosing Dry Run / Check Run	CHG	161
The status message no longer enlarges then shrinks 10s after every update (which shifted the whole window layout) - the font is now permanently large and a new message instead flashes the bar background light green (or light red for an error/alarm) for 5s.	CHG	163
The always-visible pinned-position flyouts (G28/G30/G59.3/etc.) no longer overlap the top tab labels; their vertical offset is now measured from the tab strip's real rendered height instead of a hand-tuned guess.	FIX	170
The Feed rate panel could only show a couple of digits of a 4-5 digit feed; the run bar trades its static unit label for a tooltip.	FIX	203
Mashing a D-pad direction or the run bar jog pad could leave the controller answering nothing at all; the guard against that now covers every input and can actually see its acknowledgement.	FIX	205
Thirteen top-level tabs cut to four — Setup, Job, Work Order, Offsets. The rest open as their own windows from File and Tools, and where each one lives is a layout slot you can change.	CHG	210
Hide the jog pad or just its five go-to buttons; the right-hand panels then fill one column instead of always spreading across two, and stop sitting under the flyouts.	CHG	213
A purpose-built run strip replaces the old bar; the bottom status bar is gone	CHG	220
A message history window, opened from the status line, that also pops itself when a message arrives	NEW	221
Feed Hold and Stop now outrank controller traffic instead of queueing behind it	FIX	225
A jog cancelled before the controller parsed it left jogging dead until restart	FIX	228

Commands sent during an in-flight jog no longer come back as a string of error:9.	FIX	241
A new install gets the busy nine-tab bar again; the compressed arrangement ships as a configuration overlay you apply instead.	CHG	248
The four corner buttons and the centre bullseye looked dead when they declined to move, and their safe-Z retract targeted the limit switch itself.	FIX	250
Settings > Top-level tabs let you move the Job view into a menu, which silently stopped the controller handshake from running at all.	FIX	252
A `NullReferenceException` before the window was usable — the field recording what had focus is only ever assigned on the way *out* of the view, and the block that reads it runs both ways.	FIX	298
Greyed-out Generate carries the tab's own explanation instead of generic text	FIX	331
Menu-hosted views that need Generate/Run or the jog pad open as a session tab; Calibration becomes a top-level view	CHG	332
Tabs opened from a menu can be closed again; ESC closes the current tab when enabled	NEW	333
The two buttons collapse when pressing them could do nothing, instead of sitting greyed out	CHG	355
Pause a running job at the next block boundary, park at G30 with the spindle off, look at the work, then carry on	NEW	372

Machine setup & settings

Machine Setup Wizard — guided first-run machine description (catalog, home-corner picker, per-axis table)	NEW	21
grbl:Settings — change-from-startup highlight, granular revert, per-Save restore points, bitfield tooltips	NEW	18
Stepper calibration (scratch) tab — long-span V-bit steps/mm	NEW	5
Safe-Z on Go To — lift / traverse / descend instead of a diagonal rapid	CHG	23
Editable G28/G30 predefined offsets (Set rapids to target, then stores)	CHG	26
Named G28 fixtures — a saved library of named machine positions in the G28 flyout (Set stores where you are, Go rapids back with Safe-Z); for recurring jigs & vices	NEW	70
Machine Setup restructure — Settings split (Grbl/App), new Tools tab, guided step-tabs + startup gate	NEW	28
Settings redesign — flat tab strip, free-text $-setting search (with match nav), grbl-settings autosave, dialog editors folded in as tabs	CHG	42
Per-user config folder + standalone `.xml` folded into `App.config` sections (`ConfigPanel<T>` wizards, curated defaults); console output search + font size, shared reset footer, bindable ResetAndUnlock	CHG	43
Lathe mode toggle — App › Main checkbox enables/disables lathe mode, adding or removing the Lathe wizards tab (restart to apply)	NEW	47
Restart-needed cue — the Restart button pulses red while a restart-only change is pending and offers to relaunch when you leave Settings	CHG	48
Connect: Network default + **Scan network** — finds grblHAL controllers on the LAN (confirms each by a `?`/`$I` handshake, not just an open port) and lists them in an editable host picker	NEW	59
grbl:Settings tree now populates when opened after connecting — an early activation during the connect handshake loaded the settings group-less (before `$I`	FIX	61

reported enum support), so the grblHAL settings tree never built; it now fetches the group metadata once enums are known and rebuilds the tree on entry		
… and the early activation itself is now suppressed at source — the Settings view ignores its inner tab-strip's startup selection churn until the view is actually shown, so `Activate` can no longer fire mid-handshake (belt-and-suspenders over #61)	FIX	65
Capability-gated tabs own their own "unavailable" reason — each gated view declares why it's missing, so the removal and the *Edit Main Page* listing can't drift (fixes the SD Card "needs SD" vs. "needs a filesystem" mismatch)	CHG	80
Restart prompt no longer stranded behind the relaunch splash when applying a restart-only change	FIX	81
App.config and Grbl settings snapshots now share one day-retained Backups folder instead of a single .bak and a count-retained settings-backups folder.	CHG	91
New Fixture definitions library (named, typed workholding positions) in Machine Setup, consumed by Start Job's fixture picker - retires the old single-slot G28 named-position combo.	NEW	94
GCodeProgramComments extracted as a shared parser for a loaded program's (TOOL)/(STOCK) comments, reused by touch-plate edge-radius compensation and CarveView's 3D carve simulation; touch-plate diameter fix.	FIX	96
Restore-from-file now previews only the rows that would actually change, colors them live as it applies, and offers automatic rollback on a mid-run failure; the G28 flyout gained a read-only fixture-position picker; every generated macro is now saved to disk; fixed a real crash switching to the Start Job tab.	NEW	104
3D probe/touch plate dialogs now both measure overall length (body/collet top to tip)	CHG	107
Lathe Wizards availability now follows the controller's own Mode-of-operation setting, not a separate ioSender toggle	CHG	108
Machine tab shows connected grblHAL version/build and can check GitHub for a newer SRW-fork build, then flash it	NEW	118
Corner Fence schematic is now a real 3-zone red/green/cream diagram sized off the active 3D probe; Position row gets a checkmark + directly-editable X/Y/Z fields instead of a raw coordinate string.	CHG	143
Clicking Machine Setup while the machine was moving (e.g. right after a G30) could freeze that tab for the rest of the session.	FIX	144
Settings > Restart now actually relaunches ioSender instead of just closing it; the unused AppLaunch.exe relaunch supervisor is gone	FIX	148
Title bar silently checks GitHub for a newer release and appends "(update available)"; Settings > Simulator's up-to-date status now updates live as options are toggled, not just on tab-show.	NEW	169
Offsets tab rebuilt as an inline-editable grid (Clr/Get MPos columns, orange/blue unsaved-vs-saved coloring, restore-to-startup) with corrected G28/G30/G92 commit behavior	NEW	171
Top-level tabs reordered around the daily workflow; Settings App/Jogging/G Code tabs get resizable columns; Simulator tab redesigned for readability.	CHG	179
Machine Setup's firmware section trimmed to two info lines + one Check for Updates button; results move into a message box with real per-release changelog text and custom Flash Firmware/Cancel buttons.	CHG	181
ioSender itself reads no environment variables anymore - OBS demo-recording config moves into Settings > App > Camera, the AI review key into a registry-	CHG	182

backed SecretStore with a Set/Update button, and the simulator's GitHub token is a compiled-in scoped PAT.		
Settings > App > Camera > Demo recording (OBS) gets a Validate button that connects to OBS now and confirms each configured source name actually exists.	NEW	183
Check for Updates now queries GitHub's real /releases/latest for firmware instead of a hand-rolled rolling tag, so a successfully-flashed build is correctly recognized as up to date.	FIX	187
The two standalone stepper-calibration wizards are gone; calibration is now a step inside Machine Setup alongside squareness.	CHG	197
Three fixes to the fixture editor - Test position no longer drives back to a stale saved point, the 3D Probe / Touch Plate choice survives closing, and dismissing the dialog no longer minimizes the main window.	FIX	206
Both tabs now use a searchable category tree instead of tab strips - every config panel is its own page, and search matches the words on the page, not just page names.	CHG	208
A search hit found in a tooltip now says which control that tooltip belongs to, so the answer is "hover Send comments", not "it is on this page somewhere".	CHG	209
How the app looks and behaves moved off the Application page into its own General page; the dead Theme dropdown is gone.	CHG	211
SD Card, Probing, Height Map and Lathe Tools were listed in the Tools menu but greyed out and unclickable; the two that could be opened then crashed on the way in.	FIX	215
Park positions checked against the envelope, work origins stored in the right frame, Validate corrected	FIX	226
Probe prompts name the probe and tip, and the corner probe has one code path	FIX	231
Reference points, homed state and the work envelope are mirrored from the real machine onto the simulator.	CHG	242
Layer a trimmed App.config onto an existing profile — adopt someone else's screen layout without touching your machine, macros or fixtures. Reversible.	NEW	247
With no configured target the Connect dialog lands on Serial, and a serial link is no longer handed over to the network behind your back.	CHG	249
A `$n=` typed at the MDI never reached ioSender's own copy, so Machine Setup — and the jog envelope — kept working from superseded numbers.	FIX	251
Pick a page from the filtered settings list and the matching controls on it are marked and scrolled to — dashed when the hit is in a tooltip.	NEW	257
A connect could finish with none of the controller's settings read — and say nothing. Travel limits, the soft-limit checks and the jog clamp all went quiet, and a probe was told to seek to Z −9998.	FIX	277
A fixture origin taught too close to a limit was accepted, then refused mid-job — after the tool had parked and the operator had been asked to fit the probe.	NEW	278
Two features asked the machine how far it could travel and called the answer the spoilboard. The board's extent is now its own machine fact, entered in Machine Setup.	NEW	289
"My machine setup is trashed", three times. The controller was correct every time; the page was inventing zeros for travel, steps/mm and max rate — and Apply was one press from sending `$130=0` with soft limits on.	FIX	292
A board left in check mode by an earlier session looped for ever — capabilities unread, reconnect, repeat — and a clone controller that resets when the port opens	FIX	293

failed every time with nothing wrong.		
Two things are snapshotted at the same moments — the controller's settings and ioSender's own configuration — and only one of them had a restore. Now each row is a point in time you can restore either half of, or both.	NEW	296
The V-bit scratch method returns, for machines with no 3D probe; both methods sit side by side	NEW	330
The mid-run "jog to a corner and Zero All" prompt is gone; the program inherits the active WCS	CHG	336
Typing into the Measured column reformatted mid-keystroke; real tooltips on the three cutting fields	FIX	337
Edited inputs did not discard a generated program; the scratch prolog parks at G30 and ends its own run	FIX	338
Two scratch depths instead of one, for a reference surface that is not flat	CHG	342
The rotation field was parsed only in the constructor, so a safety gate read a connect-time value	FIX	353
A second squareness tab where the square is the artifact and the probe is the reader, at 0.002 deg rather than 0.01	NEW	368
Flip the square, measure again, and the two halves split into the square's own error and the machine's	NEW	369
The acknowledgement wait raced the status poller; and the dialog asserted a cause it had never established	FIX	370
Non-focusable buttons raise no focus-lost, so a length field's pending edit never reached the value the handler read	FIX	371
The Overview page contradicted the navigation tree three inches to its left, and the manual	FIX	373
A fixture's Set button asks the controller where it is, instead of trusting the last status report.	FIX	380
The catalogue pick is a label now; it only fills fields in when there are no controller settings to keep.	CHG	397
Apply was live the moment Machine Setup opened, and one click recorded a generic machine nobody had picked.	FIX	398
Test position now lifts to the top of Z, travels in X and Y, and descends last - it used to cut the corner straight there from wherever the spindle happened to be.	FIX	399
Test position writes the corner it probed straight into the fixture, replacing the reference you jogged to by eye; the correction it used to keep alongside is deleted, and older fixtures are converted as they load.	CHG	400
Settings > User Interface > Sound attaches a sound to a limit switch, the probe, or the machine going into alarm - all off by default.	NEW	406
The TLO baseline is a machine Z again, so the derived seek depth works and no longer depends on G59.3	FIX	408
The Probing tab's tool-length seek now derives the same way the macro does, and stores the target surface from the same touch	FIX	409
Firmware 2.0.0 derives its version from a git tag, so [VER:] lost its build date; ten date-gated feature tests read 0 and two of them switched a bug workaround ON.	FIX	412

Commissioning & workflow tools

Surface Spoilboard — machine-referenced facing (Z-only touch-off, Reuse Z0, finish pass, leveling check)	NEW	29

Load Stock — corner-probe stock size/flatness/skew/thickness, results popup, Copy size → Fusion add-in	NEW	30
Squareness (Auto Square) — drill an L, measure, write the ganged-axis squaring offset + re-home	NEW	31
Probe definition diagrams — per-type schematic in the probe editor labelling the measurements to enter	NEW	34
Probe input follows the selected probe type — tool setter → `Q1`, else the main probe → `Q0` (guards a stale selection)	FIX	36
Start Job (was Load Stock) — first tab, connection-gated tabs, auto 3D-probe at the front-left origin, writes `start_job.macro`; new Verify skew pass (re-touch each corner in the rotated frame); flyout persistence & offset-Set fixes	NEW	44
Tools tab guidance — each Tools tab (tool table, spoilboard surfacing, scratch calibration, Trinamic, PID) now shows an on-screen "About" explainer of what it does and how to use it	NEW	52
Start Job tab no longer strands + program view stays a right column — a probe/hold that didn't cleanly clear could leave a gated tab disabled with no way back; the generated program overlay no longer grows to cover the whole tab	FIX	57
Probing & Tools sub-tabs share one look — instructions on the ambient grey, results in a white panel, consistent across the four Probing tabs and Stepper calibration	CHG	62
Check mode (`$C`) moved off the status bar to a checkable **Cycle Start** right-click item — a dry-run / validate toggle now sits with the button that starts a run	CHG	63
Smoother, faster corner probing — post-trigger back-offs are gentle feeds (no rapid-accel table shudder) and the Z probes assume ≤ 1" stock for a much shorter seek	CHG	71
Start Job estimated stock size — setup fields relabelled *Est.* plus a new *Est. thickness (Z)* field that warns when stock exceeds 1"	NEW	72
Consolidate the Start Job corner probe to a single macro (pcorner) and add optional sacrificial spacer/backer support so a thin sheet is probed on the metal, not the backer.	NEW	85
With Measure on and the estimated size too large, corners that fall over bare spoilboard now abort at once instead of driving a doomed side probe into air.	FIX	86
Machinist Vise is now a working fixture kind: a dedicated corner-probe macro drives Set/Test position, Start Job generates a real program for it, and its schematic tracks the fixture's own Jaw width/Max opening.	NEW	98
Root-caused the vise Start Job error:71 storm (a dangling-paren comment corrupting the grblHAL parser) and a real MDI-queue pacing bug it exposed; added a jaw keep-out toolpath check, spacer-aware Z math, and Cancel-issues-Feed-Hold during a corner probe.	FIX	102
Touch Plate as an alternative probe for Start Job/vise Set position; partial vise Measure; fixed a real TLO-reference near-miss.	NEW	103
Root-caused the Machinist Vise corner 4/2 face-probe margin flip-flop (a physical tip-clearance problem, not size-estimate slop) and fixed it with a live tool-diameter lookup; dropped the readout-only corners 1/3; first clean end-to-end Machinist Vise Start Job run on real hardware.	FIX	105
Start Job tab left-aligned, vise labels say exact not estimated, vise corner-probe motion after TLO ref no longer over-travels	CHG	109

Generates an O-word-loop .nc file for reproducing the controller-wedge-on-Reset bug over Serial/Telnet	NEW	110
Measure auto-disables for Touch Plate + non-conductive stock (can't detect edges electrically); measured size/flatness/skew readout now follows the selected mm/in units instead of always showing mm.	FIX	113
New Settings > Simulator tab picks axes/probe/rotation and builds a grblHAL simulator (via the existing CI workflow) into %AppData%\Simulator; the Connect dialog's Simulator tab is now gated on that exe existing, and auto-launches it on Ok (and on startup auto-reconnect) if not already running.	NEW	114
Settings > Simulator now defaults to the connected controller's actual options (8 compile-time toggles incl. Toolsetter/Safety door/E-stop/Lathe UVW/Ganged+Auto-square Y), copies its live settings + ATC macros into the build, and Machine Setup gains a one-button "Build simulator matching this machine" step with the same status readout. Several IP/hang/status bugs found and fixed along the way.	NEW	115
Vise corners 1/3 (previously Z-only, jaw-blocked) now also probe the left-edge X face and feed a real measured WCS origin + left-edge skew; fixed a size_y averaging bug that silently halved the measured stock height, and a schematic bug that never showed vise corner Z labels.	NEW	117
Fixture edit dialog's OK button is now disabled for the duration of a vise corner probe - it could previously close (Enter or an impatient click) before the async probe finished, copying the stale pre-probe position back and losing the fresh result. The Fixtures-tab list's own "Set position" shortcut (silently wrong for a vise - no vise-empty check, no corner-probe macro call) is removed.	FIX	124
Vise schematic Z labels now read relative to the true jaw origin; Start Job no longer silently overwrites entered stock size from a loaded program (offers a Copy-from-STOCK button instead); Corner Fence corners travel at a known-safe height instead of always retracting to machine top; new opt-in exact-size mode probes corners tightly instead of from a padded estimate; fixed a real pcorner.macro sign bug (GT 0 vs NE 0) found along the way that affected the vise path too.	CHG	125
Cycle Start menu gains Dry run: offsets Z clear of stock and forces spindle/coolant off for the whole run regardless of program content - built after an incomplete in-progress implementation left the spindle live during a real test and tore a probe cable out.	NEW	134
Start Job's DISCOVER pass correctly descended to the fixture's saved reference Z, then searched ~70mm off instead of the expected ~12mm and alarmed (Probe fail) right before the real surface - a leftover G92 offset, never cancelled by the macros' own G54 clear.	FIX	136
After setting the TLO reference the machine crossed to corner 2 at Safe Z instead of Z0 (undoing an explicit retract); the schematic's per-corner Zstock label and the modeless install-probe/PROMPT dialogs ignored the in/mm toggle and UiScale zoom respectively.	FIX	137
A real tool change mid-dry-run left dry run's Z-offset G92 active while tc.macro ran its own work-coordinate toolsetter move, pushing the target out of travel (Alarm:2) and leaving a stale hang-watchdog timer that force-reset the controller.	FIX	138
Touch-plate mode's DISCOVER corner skipped the safety rise to machine top entirely (it was bundled inside the 3D-probe-only spoilboard search block); corner 2's approach after setting the TLO reference now has the same structure as every other corner instead of a bespoke extra waypoint.	FIX	139

"Stock size is exact" was wired into Generate but never fed back into the field labels/hint text (stayed on "Est. ..."); touch-plate corners now show their own measured top Z on the schematic instead of a thickness value that mode can never actually compute.	FIX	140
G92 Zk does not set an absolute offset - it makes wherever the machine currently is read as work-Z=k - so a fixed "G92Z-17" only gave the intended clearance if the machine happened to already be sitting at the stock surface, which it never was; a machine parked ~67mm above the stock plus the intended 17mm gave an 84mm gap instead of 17mm.	FIX	141
Every Generate button now saves its program for post-mortem; fixed a real hardware ALARM:2 on Start Job's Corner 2 probe (renamed misleading "Corner travel margin" field to "Safe Z delta", warn below 10mm)	FIX	149
Corner Fence's origin-corner XY and the spoilboard Z are captured ONCE by Test position and reused every job, instead of re-locating them on every Start Job run	FIX	151
NumericField.Value is always mm now for mm/in fields; a new inheritable IsImperial flag flips a whole panel's display units at once, and typed unit suffixes (1", .5in, 12.5mm) are accepted	NEW	152
Corner Fence/Start Job's X/Y face probes now search a fixed 5mm below the measured top instead of the material midpoint (top - thickness/2)	CHG	153
Edge length labels doubled 12pt->24pt; per-corner angle moved inside the box while thickness/Z stays outside, both at 22pt instead of one combined 11pt label	CHG	154
Calibrate steps/mm by probing a reference block of known true size against a Corner Fence, instead of V-bit scratch marks measured by hand; Start Job now warns when a measured exact size implies bad calibration	NEW	155
Start Job / Stepper Calibration (Probe) no longer randomly abort right after the G30 park move with "controller did not return to idle"	FIX	158
Stepper Calibration (Probe) now parks at G30 when done (was left sitting at Z0) and gets its own Safe Z delta field; any Generate'd program hides its preview automatically once it finishes cleanly	CHG	159
Start Job's fixture dropdown gets a built-in "G28 (loose probe)" entry recreating the original pre-Fixture-library workflow, for jobs that don't have (or don't want) a saved Corner Fence fixture	NEW	160
Start Job / Stepper Calibration (Probe & Scratch) / Auto Square / Surface Spoilboard no longer have their own Generate button – the shared Run bar reads "Generate" (disabled until ready) then flips to "Run", with Dry Run/Check Run hidden while on those tabs.	CHG	162
Stepper Calibration (Probe) gains a Calibrate: XY steppers / Z stepper (1-2-3 block) toggle - probes the spoilboard once as a baseline, then the block's three sides in turn, fitting a new Z steps/mm by least-squares across all three measurements. Can reuse the last saved starting position to skip the manual jog on repeat runs.	NEW	164
Machine Setup's two optional wizard steps now color green instead of staying blank, and share the same macro-status query the SD/ATC gate uses.	FIX	167
The shared Run bar's green "press me" highlight came back for the "Generate" label, not just "Run"; Stepper Calibration's Z-gauge reuse-saved-position path now waits for the descend move to genuinely finish before probing.	FIX	173
All 5 Tools tabs' labeled inputs now colon-align on a shared column instead of each field's box hugging its own (possibly short) label; each tab's generated .macro file is now named for its own top-level selection (axis/fixture) so switching modes and generating no longer silently overwrites the other mode's saved copy.	CHG	174

Run > Simulate switches the live connection to the bundled simulator for one job, then reconnects to the real controller automatically when it ends.	NEW	176
Fusion 360 add-in (renamed ioSenderV2, one panel/two commands) exports CAM feeds/speeds to JSON; a new Feeds & Speeds tab analyzes it (material table + machine limits + optional AI review) and writes an apply file back; Help > Support gets a one-click add-in installer.	NEW	178
Esc/Ctrl+C during Ask AI to review now aborts the in-flight ~20-30s request immediately, with a Cancelling... line logged the instant the key is pressed.	FIX	184
Probing's "Max search distance" (G38.2 travel limit) is now editable right on Tool length offset, Edge finder (external/internal) and Center finder, instead of only via Machine Setup > Probe definitions.	NEW	188
Odd Jobs — describe a small job as toolpaths (line/circle/oval/square/rect) with operations under them (pocket, contour, drill, bore, chamfer, finishing) and it compiles to one program with a sane tool order	NEW	192
Work Order is now a top-level tab (New/Load menu items added); Setup's Touch Plate probing follows the selected Material's conductivity; tc.macro can use a fixed touch-plate block instead of a toolsetter.	CHG	193
New 4-flute chamfer bit option and a per-operation Climb/Conventional cut direction (correct internal/external orbit math) on Work Order operations, plus a Settings:App spindle-direction-capability setting for hardware that can't run both directions; shared day-of-week rotation for the debug/console/crash logs and App.config/Grbl backups; fixed a Spindle-panel crash.	NEW	195
Surfacing is a Work Order operation now, including Entire Spoilboard — not a separate wizard with its own idea of where the board is.	NEW	198
The fixed built-in tool enum is retired for a single editable tool list you can add your own tools to.	CHG	199
A run of real-hardware bugs: rapids from a false origin, traverses at cutting height, a dry run that did not protect, and alarms that did not abort.	FIX	201
Probing conductive stock with a real touch plate silently dropped the plate thickness and lip compensation — 12 mm of Z error.	FIX	202
Stepper calibration, Squareness and Surface spoilboard leave the Tools tab for where they already live; the tab itself disappears when the controller supports none of what remains.	CHG	204
Macros and g-code programs share a single streaming engine, and a Work Order is a single press	NEW	219
Engrave text on a Work Order with a single-stroke font and a V-bit	NEW	222
Any installed font, carved to depth-varying passes from its true outlines	NEW	223
Tick Text on a line, circle, oval, square or rectangle and text is fitted inside it	NEW	224
Generate loads the program and takes you to the Job tab, so Run is pressed against what is on screen.	CHG	236
V-carving a script font compiles in seconds instead of minutes.	CHG	239
Tick Corner reliefs on a square or rectangular toolpath and its pocket gets a dogbone in each inside corner, so a square-cornered part drops in instead of fouling on the radius a round cutter leaves.	NEW	259
A helical bore's feed is compensated where the cutting edge really does outrun the tool centre - a wall-following pass - and left exactly as entered on a full-immersion helix, where compensating it starves the cut and rubs the cutter to destruction.	FIX	261

Pick an .svg per toolpath and V-carve it — same engine, passes and preview as carved text.	NEW	262
Stopping a dry run left its G92 on the controller — through resets — shifting work Z on every job after.	FIX	264
Probe-fail crash, Generate refusing a validated fence, and a work order comment that failed the run.	FIX	265
A probe that failed to release said "error 9" three times and named nothing; now it says so in a sentence.	FIX	266
Changing an SVG's width or file reused the previous carve — the cache key had never learned about artwork.	FIX	267
Cap how deep a carve may go: hairline lettering keeps its full depth while wide areas flatten off and clear.	NEW	268
Duplicating an SVG toolpath lost its artwork — the copy routine listed its fields by hand and never learned the new ones.	FIX	269
Indirect toolpaths gain an Absolute/Relative offset mode — and lose an anchor dropdown that never did anything.	NEW	270
Collect toolpaths under a named group in the tree and enable or disable the set with one tick.	NEW	271
Copy a group of toolpaths as one, with per-member ticks — add a toolpath to the original and every copy gains it.	NEW	272
Carve levels follow the operation's depth of cut — a deep emblem need not be cut in six passes when one will do.	CHG	274
The greyed-out Generate button names what it is refusing, and metric tapping drills are no longer rejected as non-standard.	FIX	275
Generate reused the previous build's program after a code change — the cache now notices that the build itself changed.	FIX	276
The Height Map tab publishes the program it is about to send, follows the run through Steps → Program → Surface Map, logs every probed height, and can map a whole table without an origin being set first.	NEW	283
ALARM:2 on point 1, then on point 2, then a point that searched 50 mm and touched nothing — four faults in the probe geometry, each hidden by the one before it.	FIX	284
The 3D surface never rendered — bindings that could not work, and a camera aimed at nothing — and once it did, the colour ramp depended on the square of how flat the board was.	FIX	285
Retry carries on from the point that failed instead of discarding fifteen good readings; Stop works during the run's longest unattended moves; and a cancelled run no longer leaves the app "busy" indefinitely.	CHG	286
A probe succeeded, the controller acked, and the retract never reached the wire — the run then resumed on the next unrelated `ok` it saw, which was the operator's own jog, and stepped to the wrong line.	FIX	287
Every probing operation stopped to demand the operator trigger the probe by hand, then made them press Start a second time to get past it.	CHG	288
One slow plunge at the board's high point, a rule in the pocket, then OK to surface or Cancel with nothing cut but the pocket.	NEW	290
Fixture *Test position* died with `error:2` part-way into a probe. grblHAL does not hand back 0 for a named parameter nobody set — it fails the read.	FIX	295

The chart recommended 24000 mm/min on a gantry that maxes at 16510. grblHAL clamps rather than complains — so the chip load quoted was 0.400 mm/tooth and the tool actually saw 0.275.	FIX	297
Every PDF and Illustrator export wraps the page in a clip rectangle — imported as artwork, and being the largest thing in the file it captured the bounding box too. And a `transform=` was counted as unsupported while the path was imported anyway, untransformed.	FIX	300
Right-click the Work Order stock diagram to write a dimensioned PDF: the stock measured, every toolpath ballooned, and a feature table giving each one's position, geometry, operations and tooling.	NEW	303
Every toolpath draws in its own colour and is named by a lettered balloon — on screen and on the saved drawing — and the feature table repeats the same balloon, so finding a feature is matching a mark rather than reading one.	CHG	304
A contour's envelope now shows the band it actually removes instead of filling the whole area, and text/artwork envelopes draw the real glyphs rather than a 40×25 mm rectangle read from fields neither geometry ever sets.	FIX	305
Stock size lives in the .workorder file and the diagram is drawn against it, so loading a work order composed for a different blank no longer shows a layout that looks wrong for no visible reason. Setup is offered the size, never given it silently.	NEW	306
The PREREQ guard that warns when a stored G28/G30/G59.3 is outside the soft limits now actually runs - it had never once evaluated	FIX	317
A racked laser gantry is squared in software: enter the two diagonals and the side of a burned square, and every SVG job is sheared to match	NEW	320
A **Negative** tick on an SVG toolpath carves the space around the artwork instead of the artwork — the logo stands proud inside a recessed panel a chosen border wider than the ink.	NEW	322
A negative-carved logo traced 0.1–0.4 mm slivers for the first 45% of its program and floored its 8 mm panel in the last 5% — the spine detail passes now cut after the stepped rings and the flat clearing.	FIX	323
A **Panel** choice on the negative: *Artwork outline* uses the logo's own enclosing contour as the boundary, so a circular badge carves only inside its circle and nothing outside it is touched.	NEW	324
A new **Clear floor** operation on V-carved toolpaths pockets the carve's floor flat with an end mill, at exactly the depth the V-bit floored it — the raised-logo panel comes out smooth instead of ridged.	NEW	325
The end mill's face rode the surface the V-bit had already finished and rubbed rather than cut, scorching the floor of the first maple badge — a new **Below carve floor** field (0.1 mm) keeps it taking a chip, and a new Clear floor starts at one pass.	FIX	326
A new **Mark** operation on every shape geometry: a V-bit's tip rides the outline itself, as deep as the chosen groove width makes it on that bit — a 1 mm marking circle around a badge, a border on a rectangle.	NEW	327
A **Dashed** tick on the Mark operation, with Dash and Gap lengths: a closed shape gets a whole number of periods so the pattern meets itself, a line gets a dash at each end, and every dash is its own plunge.	NEW	328
Mark (#327) stepped by a hidden depth of cut and ran a one-pass groove as five	FIX	329
A corner probe with no trusted safe height rapided to an approach depth derived from machine travel	FIX	334
A touch-plate corner sequence prompted before corners 3 and 4 but not before 2	FIX	335

The finishing-tool bucket picked a ball end mill for an operation defined by leaving a flat bottom	FIX	347
A run-time substitution that emits a locating dimple at every hole centre, keeping the work order intact	NEW	348
A pattern's positions are printed on the drawing sheet instead of left to be reconstructed by hand	CHG	349
A warning named a bit with no picker to change it; another refused metric drill sizes the series simply skipped	FIX	350
Work orders generate under a rotated WCS again; the one toolpath that mixes frames clears the rotation at its source	CHG	354
Setup stored a raw origin and a negated angle; both are wrong for a rotation applied about machine zero	FIX	357
A firmware frame-conversion pair is guarded independently; every rotation write now repairs the parser position	FIX	358
The same firmware fault as #358 through a second door: no R word needed, only a WCS that HAS a rotation	FIX	359
The per-corner approach was a rapid to a work Z that a tool change invalidates; unticking Touch corners sights the frame instead	CHG	362
The four probed corners, the measured size and the spoilboard Z survive a restart, with the measurement's date on the readout	NEW	363
Three refinements from running it: a readable sight gap, the tool off the corner it is hiding, and the back-corner pairs compared without a retract	CHG	365
WPF resolves properties, not fields, so every row rendered empty and the selection never matched	FIX	366
A scribed rectangle inset 10 mm from the measured frame turns an angular error into a taper read with calipers	NEW	367
A measurement describes one piece of stock - saying the stock is a different size says the measurement is of something else	CHG	374
Probe a map during setup, then tick Apply height map on the work order — with a refusal when the map no longer matches the setup.	NEW	381
A tool change no longer looks like a moved work origin, and the origin is stamped with a tool offset that is actually current.	FIX	382
Setup hands off to Tools > Height map, which takes its area From setup and remembers how it was set up.	CHG	383
Absolute traverses, a safe lift that names Z, and a hold before EVERY point including the first.	FIX	384
An open path's chamfer traces BOTH top edges of the groove, offset by the roughing bit's radius.	FIX	389
A false trigger no longer passes as a measurement — the fast and slow probes have to land in the same place.	NEW	392
A clearance margin bigger than the gap to machine top aimed a rapid above the envelope; now clamped.	FIX	393
Pick a touch plate and the height map holds at every point; pick a 3D probe and it does not.	CHG	396
Machine Setup step 5's Probe it now descends by the smaller of the probe's own search distance and the travel actually left below you - and the dialog says which bound applies instead of silently clamping.	FIX	401

The TLO probe's downward seek is computed from the stored reference baseline instead of a fixed 90 mm, so changing the toolsetter no longer breaks it.	FIX	403
Every Setup program now states its own units, plane and work offset up front, and refuses to start on an alarm or an active G92 instead of inheriting it.	NEW	404
When a probe cycle aborts with the tool resting on the plate, Z lifts to machine zero automatically and reports what the probe pin says afterwards.	NEW	405

Files, SD card & ATC

Load Folder — assemble per-toolpath `.nc` files into one outlined program (+ Fusion add-in)	NEW	6
SD + LittleFS browser & file manager — multi-filesystem, View/Edit/Create/Upload via Notepad	NEW	14
SD Card first-line cleanup — the filesystem status no longer overlaps the checkbox, and the free-space banner is hidden when the controller reports no sizes (was a bare “mounted”)	FIX	51
ATC tool-change macro provisioning — Install ATC macros + seeded Start Job	NEW	25
On an ATC controller the setup gate could dump you at step 6 even though the macros were installed and turned green a moment later	FIX	88
The Machine Setup ATC-macro check now catches a 0-byte or truncated tc/pcorner/pvisecorner.macro on the controller - it used to read as Installed since only the file NAME was checked, not its size, so M6 T8 silently ran the firmware's default tool change instead of the macro.	FIX	123
SD Card tab queries its own mount status on activate instead of bailing out to "No card" before the controller has actually reported one.	FIX	168
Machine Setup's macro-provisioning gate no longer false-trips when a realtime status report interleaves with the checksum read.	FIX	175
Dead time between Setup steps removed, corner retreats cut from 20mm to 5mm, and the ATC macro update no longer wedges the app until it is restarted.	FIX	207
Silence is no longer read as a dead link during homing or a firmware upload	FIX	227
A listing that times out is no longer reported as a controller with no files on it.	FIX	243
A command answered with an error stops the wait immediately instead of running to timeout.	FIX	244
Short query helpers nested inside a file transfer and cleared its poll suspension	FIX	346
A per-file hash manifest replaces the single whole-set checksum; a one-line edit now uploads one macro	CHG	352

Program view & G-code

Reusable program view — file / folder / each wizard gets its own view; one Cycle Start runs the active one	CHG	32
Background file / folder loading — big programs parse on a worker thread; the UI stays responsive, rows fill as read	CHG	32
Continuous block numbering — explicit `N` words shown in the Block column and hidden from the G-code column	CHG	32
Explain G-code on hover — plain-language per line/selection (decodes parameters, O-word CALLs, expressions); click the balloon to copy	NEW	33
One in-place streaming path — every wizard/macro runs in its own view; a run never overwrites the loaded job	CHG	37

Live 3D carve view — watch the stock get machined away (program-sized stock, real cutter profile flat/ball/V-bit, Play / Cycle Start playback) instead of just toolpath lines	NEW	41
3-line run view — on Cycle Start a generated program collapses to previous / current / next line at the bottom of the tab; click the title bar to expand	NEW	76
Save program... now works for a wizard's generated preview, and the floating Program popup no longer shows a redundant second copy of the loaded job already docked in the Job tab.	FIX	93
The Fusion ioSenderBatchPost add-in always combines every operation into one .nc; Load File now recognizes its section markers and shows the same collapsible outline Load Folder used to, so Load Folder is retired.	CHG	97
(TOOL) comments carry a tool's overall length (L=), read from Fusion and exposed per-program	NEW	106
The Program view overlay (Load File/Folder, wizard output) was a fixed 560-720px panel that ran over half the client area on a narrower window - now always exactly the right third.	FIX	112
Double-click any Job-view center tab's header (Program, 3D View, Console) to pop it into its own borderless window, resizable and Ctrl+Alt+=/- zoomable; double-click that window's title bar to dock it back. Replaces the old Console-only popout special case.	NEW	116
ioSenderBatchPost now remaps Fusion's Air/Mist coolant setting to Flood before posting - grbl.cps has no code for Air/Mist and raised "Unsupported 'air' coolant", failing every operation using it.	FIX	122
Feed rate/Spindle/Outline could stretch wide enough (Outline's wrapped warning text especially) to squeeze the Program/3D View/Console workspace.	FIX	145
Hovering a program-list line now reliably explains THAT line (was matching an unrelated line's tokens); Machine/Program limits read as one compact interval per axis instead of two full-width fields.	FIX	177
The cut-trail line in the 3D carve view was invisible in every state; stock top-face color lightened for contrast.	FIX	180
Work Order's Run hands its program to the Job tab as the real loaded job (pops back after); Load File no longer freezes on a large file.	CHG	194
Setup no longer crashes on a first-ever/cold session; a Work Order run's live ok/* status now shows in the real docked Job tab list instead of a floating panel.	FIX	196
A 220,000-line program loads in 2.6 s instead of 32 s, and the status line now says what the load is doing.	FIX	216
On the Job screen with nothing loaded, the program list's Data column was a sliver with its own heading clipped; it now fills the list.	FIX	217
A 220k-line load went from 32s to under 3s, and the console no longer freezes the app	FIX	229
Compile progress and timing, plus an estimated run time for any loaded program	NEW	230
A block ioSender could not parse is now sent verbatim instead of being silently discarded.	FIX	234
The board is drawn at its declared size, and says so plainly when nothing declares one.	FIX	237
Show the program listing and the 3D view side by side on the Job tab.	NEW	238
SVG artwork becomes laser g-code — outlines, or shaded — for a plain Grbl diode machine.	NEW	273

A 32 mm job was refused for spanning 858 mm. Machine-coordinate park moves were being measured as if they were work coordinates.	FIX	279
A 600 mm ruler was run on a 400 mm machine. It drove 200 mm past the end of X and sat grinding against the stop — the extents were on screen the whole time and nothing ever compared them to the travel limits.	NEW	291
Opening a program containing `G92` while disconnected crashed ioSender through the 3D viewer, and so did every SVG laser job — the laser is plain Grbl, so its coordinate systems are never asked for.	FIX	294
The logo could only ever start at the origin, no matter what the SVG said — and on a laser homed to its back-left corner the ordinary placement ran the job off the back edge into the stop.	NEW	301
A fill at S600/F800 removed nearly 7 mm of cedar while every number on the Burn tab looked reasonable — a fill is a raster, so halving the interval doubles the energy per square millimetre and nothing said so.	CHG	302
Generated laser jobs declare power and feed at the top as #<s_line> etc., so a job is retuned by editing four numbers.	NEW	309
Two more pitches on the Copies tab ramp outline and fill power per copy - five exposures of one logo in a single job.	NEW	310
A wrong-signed Pitch Y emitted positive Y on a table that only has negative - and the summary understated the reach.	FIX	311
One header comment opened '(' and never closed it, and the loader refused the whole saved .nc after the first block.	FIX	312
The per-copy power ramp wrote an assignment before copy 1 that set the constant to the value the header had already given it.	FIX	313
The dialog's X/Y offsets are declared once as #<x_org> / #<y_org> instead of being added to every coordinate, so a job can be re-placed by editing two lines.	CHG	314
The lift before a G30 park no longer names X/Y at the live position - a parse-time read and a soft-limit trap	FIX	316
SVG-to-laser now lives in CNC.Svg, a WPF-free assembly with a net8 portability probe, so the EngravingBox appliance can build the same files on a Raspberry Pi	NEW	318
Generate makes the program the loaded job on the Job tab; the run's end gives the previous one back, and Esc discards it	CHG	343
A tool-length offset was reported as absent exactly when it was present, and one of two start-from paths restored no modal state	FIX	375
The program list groups into collapsible toolpaths on any file with M6, and either one of them or everything from one onward can be run on its own	NEW	376
One collection notification for a 43,000-block rebuild instead of 43,000, with a wait cursor and an elapsed time.	FIX	386
Two token types were missing from the g-code writer's switch and were rewritten to their command name.	FIX	387
"T1 - 1/4" 2 flute end mill" rather than the name of the first of its five operations.	CHG	388
The g-code emulator yields ONE frame — work coordinates — so the 3D view and the stock finally agree.	FIX	390

Macros, console & keys

Macro manager dialog (grid, F-key assign, confirm-on-run flag) + run-only macro flyout	NEW	10
Macro pre-processor directives — PREREQ / MBOX / WAITIDLE / PROMPT / `@path`	NEW	11

In-app Key Mappings editor + console state persistence	NEW	3
Console: inline terminal command prompt + output right-click menu (Clear / Save)	NEW	19
Bind any main + second-level tab to a key — right-click Bind-to-Key + header badges	NEW	79
Start Job no longer writes a start_job.macro file or seeds a catalog entry - it already ran entirely in-memory like the other Generate/Cycle Start wizards.	CHG	92
Two probe-tip-losing collision bugs fixed in pcorner.macro (found probing a Corner Fence fixture on hardware); MacroProcessor now truncates an oversized auto-generated comment instead of having the controller silently reject the whole line.	FIX	99
Console tab now mirrors to a durable, timestamped log file per run - survives a UI freeze or a firmware debug flood that used to leave no record behind.	NEW	111
Console shows a controller restart-after-hang notice (with a report-it dialog) and a firmware-change notice, right on reconnect.	NEW	126
OBS Control panel + F7-F12 hotkeys start/stop the 2 Tapo RTSP cameras and the app screen-capture source individually, filing each take into a named subfolder.	NEW	142
Works through the UI feedback Phil Barrett sent after his first sessions with V2.	CHG	190
One Top Level Tabs group holds every tab-strip and menu destination; a key reaches its target wherever it currently lives, menu commands can be bound too, and the menu shows the key.	CHG	212
Typed commands and streamed programs now share one dispatch path and one pacer.	CHG	240
Two new bindable actions in the Program group — MDI opens the console ready to type, Status shows the message history.	NEW	254
F12 now opens the MDI console rather than toggling it, dismissed with Esc like every other window here; the MDI input line is also sized separately from the log.	CHG	255
Find-in-console highlights every match, with the current one picked out — previously it marked one, and that one frequently did not render at all.	FIX	256
Keyboard jogging worked on the Job tab and nowhere else; shortcuts worked on the top-level tabs and nowhere else. Both now work in every window, dialogs included, and stop only when you are typing.	FIX	260
status_<stamp>.log + latest_status.log, with each line tagged app or firmware.	NEW	263
The one log an operator can read now says what a connect established, what a run did, and what a settings change altered — old value and new.	NEW	280
From the second connect onward the log repeated the last connection's line — a serial connect could report a network address — and the new connect detail never appeared.	FIX	281
Settings > Keyboard listed fifteen File and Help commands interleaved with the tabs. They now sit in their own group directly above.	CHG	282
The most explicit statement the app ever makes was the one thing that could not be recovered afterwards — and a dialog still *open* is exactly the state worth diagnosing.	CHG	299
Typing an MDI command left the console at the end of the log instead of snapping it to the top once the scrollback was full	FIX	319
Opening a program with (PROMPT param, default, label) rows asks for the values once, on every controller; [brackets] over constants fold to numbers so a parametric file runs on stock Grbl	NEW	321

The tool-length reference sequence becomes a controller macro instead of a second copy emitted inline	NEW	339
Two hardware failures from one root: the macro ends standing on the puck, and leaves G59.3 active	FIX	340
One of the macro's two callers never supplied the input its branch reads, and named parameters do not survive a reset	FIX	341
The identifying comment, prerequisite gate, modal line, park and end word come from one place	CHG	344
Directive lines skipped the paren sanitiser, so an inner ')' ran the rest of the sentence together	FIX	345
Setup's rotation line assigned a leading zero and left the rest of the expression in the block	FIX	351
Reading back through the log was impossible with a connected machine; the view is now held on the same TEXT	FIX	360
A few-pound camera remote answers prompts, feed-holds a running job and resumes a held one.	NEW	385
Bind one remote as a pendant; each button's job is yours to set per machine state, including in alarm.	NEW	391
In the Keyboard settings panel, a bound shortcut now scrolls to its row rather than doing its job.	NEW	394
The Height map tab's duplicate remote tick is gone, and the page no longer claims your volume will not move.	CHG	395
Tool-length offsets are logged as numbers rather than as the expression that produced them	FIX	410
A “driver: <name> version <n>” line at connect, which is what distinguishes one simulator build from the next	NEW	411
Every status log opens with one line naming the version, assembly, commit, configuration and bitness it came from.	NEW	414

Polish, onboarding & docs

Numeric fields select their value on focus — tab or click into any number field and just type the replacement (no clearing first); single-digit edits still work	CHG	58
High-DPI polish — fixed pixel sizes converted to Min/Auto so text doesn’t clip at 125–150% scaling	FIX	46
Lathe wizards rebuilt to the app layout idiom — the 5 upstream absolute-Margin wizards re-hosted in scaling StackPanel/Grid so they no longer clip/overlap at 125–150%	CHG	49
Tab strips fill their width — a StretchTabControl spreads each tab header proportionally across the row so tabs read as distinct (main bar + Settings / Tools / Lathe / Probing / Machine Setup)	CHG	50
Online user manual + in-app context help — `F1` (and the Help menu) open a task-oriented manual with novice / intermediate / machinist reading tracks, A–Z index, search, diagrams and screenshots; hosted at `iosenderv2.github.io/ioSender`	NEW	56
Onboarding tooltips — plain-language controller-state field + per-setting help	NEW	12
UI localization — all UI strings registered for 7 locales (cross-cutting)	NEW	9,10,12,14–16,18,19,21,23

Center owner-less modal dialogs (MPGPending, ProbeVerify)	FIX	27
Show Motor Fault / Warning signals in the Signals display	FIX	2
Build documentation — Mega-XL upgrade notes + repeatable tool-change guide	NEW	13
Assorted released-ioSender bug fixes (RP.Math build, LF endings, CSV null-ref, boolean settings, null-comms)	FIX	1
Headless build — `build.ps1` + VS Code tasks build/launch without opening the VS GUI (MSBuild found via vswhere); unhandled exceptions now dump to `%AppData%\ioSender\ioSender.crash.log` and exit `0xFA11` instead of blocking on a dialog	NEW	53
Opt-in diagnostic trace log — launch with `-debuglog` (or `IOSENDER_DEBUGLOG`) to write a timestamped flow trace to `%AppData%\ioSender\ioSender.debug.log`; off by default, so `DebugLog.Write(…)` call sites can live permanently in the code without a rebuild toggle	NEW	64
Code renamed LoadStock → Start Job to match the UI throughout (files, classes, `ViewType`, layout keys, config section)	CHG	54
Localization catch-up — later V2 features (Start Job, probing-redesign dialogs, main-page editor, Tools guidance, …) were `x:Uid`'d but had no LocBaml rows; every missing string backfilled into all 7 locale CSVs	NEW	55
Playbooks — the repeatable dev procedures (changelog entry, PDF/manual publish, builds, localization, firmware, demo video) collected into `docs/playbooks/`, one per file with ready-to-run steps	NEW	73
Startup dialogs no longer hidden behind the splash — the progress splash docks to the top so the connect dialog, message boxes and setup prompts stay visible	FIX	74
Status-bar messages shown at double height for 10 s when written, then shrink back — so an important message in the easily-missed status line gets noticed	NEW	75
Selected tab label goes bold and slightly larger — the active tab reads at a glance, not just by its white background (every StretchTabControl bar)	CHG	77
Run-bar jog pad restored — it had been squeezed to nothing by the Program button; run bar rebalanced into four columns with the button row filling the slack up to the jog pad	FIX	78
The deterministic dev playbooks are now one-command scripts — push to both remotes, regenerate the Overview PDF, run `gh`, and generate a changelog entry — so they are invoked, not re-derived each time	NEW	82
One live instance, a clean supervised restart, and opt-in crash reports with a minidump.	NEW	83
File menu dissolved onto the program view + right-click menu; toolbar row removed; Camera menu made opt-in.	CHG	84
Optional localhost server that drives the WPF UI by x:Uid so a change can be verified end to end without a human clicking	NEW	87
The optional test server grew from a proof slice into a real automation harness, and moved into its own app-agnostic assembly	NEW	89
The app-agnostic UI test server moved out to its own public repo and NuGet package; ioSender now consumes it as a package.	CHG	90

UiScale (the DPI/zoom setting) now applies to every modal dialog window, not just MainWindow.	FIX	95
WpfUiTestServer (v1.2.0) gains POST /handoff (shows a popup, drops the banner, and stops the server) so an agent can hand a UI it drove back to the operator with instructions; /idle now waits for an actual composed frame, not just a drained dispatcher queue.	NEW	100
Locale rows added/removed for every new or retired x:Uid this round (Fixture definitions, Fusion outline parity, Machinist Vise, Start Job's unit toggle and Jaw width/Max opening) across all 7 locales.	CHG	101
One-line PowerShell installer (`irm ... /install.ps1 | iex`) + rolling-release CI so a new user never touches Visual Studio or a build	NEW	119
Check for Updates now offers "Update now?" instead of just opening the releases page; new Help > Support > Roll back to previous version undoes the last update	NEW	120
Message boxes are now the app's own scaled window instead of the native OS dialog, so they read on a high-DPI display at whatever zoom Ctrl+Alt+Plus/Minus is set to.	NEW	127
Tooltip text now scales with UiScale zoom instead of staying fixed-size, unreadably small at high zoom.	FIX	129
-simulator connects to the bundled simulator on startup, launching it first if needed - joins the existing -port/-baud flags for serial/network.	NEW	130
-testserver launches no longer collide with (or steal focus from) another running instance; -simulator <port> picks which port the simulator listens/connects on, so parallel agents don't fight over one.	FIX	131
A fresh -testserver launch no longer grabs OS foreground focus from whatever the operator is doing, and WpfUiTestServer's /screenshot stays fully functional (previously trading one for the other).	FIX	132
ToolTips added to SpindleControl, Goto/Outline action buttons, the Macro Manager/Editor dialogs, and PortDialog's network connection settings.	NEW	133
ConsoleLog, DebugLog and the crash logger each reimplemented the same file-path resolution and 8MB rotation - now share one CNC.Core.LogFile; the crash log previously had no size cap at all, and console_*.log gains a live "latest_console.log" hard link.	CHG	135
AppMessageBox's text bumped 13pt -> 19.5pt (+50%), fixing every AppDialogs.Show call app-wide at once.	FIX	146
-message=text pops up an informational dialog once the main window is revealed - handy for telling someone what to test when launching a build for them.	NEW	147
Drag to resize the setup/drawing split on Start Job, Height Map, and 7 Probing/Tools sub-tabs, plus the Job tab's left sidebar / centre workspace / right sidebar	NEW	150
Every push to master now publishes a real GitHub Release (2.1, 2.2, ...) with notes listing which #N changelog entries shipped in it, instead of overwriting one unversioned rolling build; the TOC gains a Version column showing which release each entry arrived in	NEW	156
Release notes' install one-liner is now wrapped as powershell "irm ... | iex" so it runs from cmd.exe as well as PowerShell; the main window's title bar showed a stale hardcoded "2.0.1" instead of the actual published release version	FIX	157

Check for Updates on a local dev build (no embedded version to compare against "latest") now lists every published release with an installable ioSender.zip and lets you pick one to install over the dev build's own bin folder, for quick comparison against a real published build.	NEW	165
New color-by-function design system (AppleStyles.xaml) applied app-wide - one primary button per screen, pill-style nav tabs, colored panel-title dots.	NEW	166
App-wide oval checkbox style, NumericField read-only styling + one-time alternate-unit peek, tab-header-drag no longer leaves sibling flyouts/content repainting, two stale status-message fixes	CHG	172
Closing the window while a job is running/paused now explains why instead of silently refusing; a WPF crash typing fast in the Grbl settings search is fixed; hover tooltips for a line like T1M6 now read in the same order the words appear.	FIX	185
Every real screenshot in the User Manual gets a novice-friendly bullet breakdown of its visible UI regions; new tools for a multi-shot reshoot round-trip.	CHG	186
A saved keymap entry missing its Method no longer crashes startup; a probing-triggered status message can no longer crash the app from a background thread; the console shows the controller's $I response and typed $ commands again, while internal $CWD housekeeping traffic stays quiet.	FIX	189
Dragging a tab header no longer crashes the app or strands it with an invisible page and a stuck cursor.	FIX	191
A missing section now inherits the shipped template's curated value, and one unreadable section can no longer take every other section down with it.	CHG	200
A tab that multiplied on every launch, a setup prompt that reappeared for ever, Esc stolen from every window, and a scroll wheel that only worked over the scrollbar.	FIX	214
The machine engine is separated from the desktop UI and compiles as a plain .NET 8 library	CHG	218
A crash log that survives the crash, and three crashes on startup and connect	FIX	232
Fifteen small corrections across windows, offsets, the program list and Work Orders	FIX	233
The pop-up status window stays for a chosen time, takes the screen only for errors, and returns focus on close.	CHG	246
Close and reopen ioSender from the menu, settings saved first - some session state clears no other way.	NEW	307
An unowned message box left the app beeping at every click and refusing to close - Task Manager was the only way out.	FIX	308
Load SVG Laser Job is off by default again and enabled per launch with -enableSVGLaserJob, so nothing has to be remembered and reverted before a push.	CHG	315
The skew readout showed mojibake where the degree and delta signs should have been	FIX	361
The default category filter subtracted twenty channels instead of adding one	FIX	364

Every topic the 2026-09-17 audit listed as MISSING is now written, including the laser path, height maps and calibration.	CHG	377
A menu item and a bindable key for the manual, opening at the topic for the screen you are on.	NEW	378
Modal prompts now own the ACTIVE window, so one raised over a Topmost dialog cannot hide behind it.	FIX	379
A throw while connecting skipped the rest of startup - no key handling, no shortcuts - and the half-live process then blocked every later launch.	FIX	407
Every assembly moves to 3.0.0.0, and the release cutter can finally emit a major version instead of only minor+1.	CHG	413

#	Change	Diff
Version 2.1		
1	Bug Fixes for released ioSender	Files: 5 0 0 Lines: +28 -10
2	Surface Motor Fault / Warning signals	Files: 2 0 0 Lines: +5 -32
3	Console persistence + Key Mappings editor	Files: 12 0 +4 Lines: +1224 -15
4	Connection-loss handling + auto-reconnect	Files: 10 0 0 Lines: +522 -63
5	Stepper calibration (scratch) tab	Files: 13 0 +3 Lines: +773 -2
6	Load Folder + Fusion batch-post add-in	Files: 28 0 +8 Lines: +1634 -8
7	High-DPI / large-text layout	Files: 27 0 0 Lines: +126 -109
8	Xbox controller + Key Mappings editor polish	Files: 14 0 +4 Lines: +2255 -687
9	Configurable main page + flyouts + jog panels (XL)	Files: 31 0 +20 Lines: +2835 -119
10	Macro manager + run-only flyout	Files: 16 0 +2 Lines: +598 -26
11	Macro Processor Enhancements	Files: 8 0 +1 Lines: +723 -21
12	Onboarding tooltips	Files: 14 0 0 Lines: +283 -28
13	Build documentation (Mega-XL notes + tool-change guide)	Files: 1 0 +8 Lines: +2662 0
14	SD + LittleFS file browser & file manager	Files: 12 0 +3 Lines: +945 -135
15	USB / Ethernet / simulator connection support + Connect menu	Files: 26 0 +3 Lines: +1151 -68
16	Validate controller (check-mode g-code conformance test)	Files: 19 0 +1 Lines: +1208 -38

17	Restore main window position across launches	Files: 2 0 0 Lines: +60 -7
18	grbl:Settings — live edits, change highlight, revert & snapshots	Files: 12 0 +2 Lines: +659 -43
19	Console — inline terminal-style command prompt	Files: 13 0 0 Lines: +220 -21
20	3D view — Ctrl+click to jog + per-axis orientation	Files: 5 0 0 Lines: +206 -60
21	grbl:Settings — Machine Setup Wizard	Files: 12 0 +3 Lines: +1740 -1
22	Keyboard jog — keep active when Job-view focus drifts	Files: 2 0 0 Lines: +23 -1
23	Go To — optional safe-Z (lift, traverse, descend)	Files: 10 0 0 Lines: +54 -8
24	comms — fix telnet/websocket UI deadlock	Files: 2 0 0 Lines: +29 -10
25	ATC tool-change macro provisioning (Install ATC + Start Job)	Files: 7 0 +5 Lines: +804 -10
26	Offsets — editable G28/G30 (Set rapids & stores)	Files: 1 0 0 Lines: +21 -2
27	Dialogs — center owner-less modal dialogs	Files: 4 0 0 Lines: +4 -2
28	Machine Setup restructure + Tools tab + setup gate	Files: 4 0 +2 Lines: +237 -21
29	Surface Spoilboard — machine-referenced facing tool	Files: 2 0 +2 Lines: +673 0
30	Load Stock — corner-probe stock measurement	Files: 7 0 +7 Lines: +1125 0
31	Squareness (Auto Square) — ganged-gantry offset tool	Files: 2 0 +2 Lines: +686 0
32	Program view as a reusable object + background loading	Files: 20 0 +4 Lines: +830 -184
33	Explain G-code on hover (novice tooltips)	Files: 4 0 +1 Lines: +805 -39
34	Probe definition diagrams	Files: 2 0 0 Lines: +142 -22
35	Jog distance +/− controller actions	Files: 4 0 0 Lines: +17 -2
36	Probe input follows the selected probe type	Files: 3 0 0 Lines: +16 0
37	One in-place streaming path (stayPut removed)	Files: 3 0 0 Lines: +62 -110
38	Match the bundled simulator to the connected controller	Files: 2 0 0 Lines: +355 0

39	Re-establish modal state on a mid-program start	Files: 1 0 0 Lines: +22 0
40	Load Stock “apply work rotation” defaults off	Files: 1 0 0 Lines: +6 -1
41	Live 3D toolpath + material-removal carve view	Files: 4 0 +2 Lines: +1107 -62
42	Settings redesign — flat tab strip + searchable Grbl settings	Files: 23 2 0 Lines: +935 -542
43	Per-user config folder + standalone config folded into App.config	Files: 27 0 +3 Lines: +1269 -369
44	Start Job workflow + Verify skew, flyout & offset fixes	Files: 17 0 0 Lines: +369 -182
45	Run-bar jog preset spinners + startup splash + MDI Send align	Files: 4 0 +2 Lines: +132 -9
46	High-DPI audit — text no longer clips at 125–150% scaling	Files: 63 0 0 Lines: +196 -197
47	Lathe mode enable/disable checkbox	Files: 3 0 0 Lines: +56 -0
48	Restart button pulse + restart-on-leave prompt	Files: 2 0 0 Lines: +58 -1
49	Lathe wizards rebuilt to the app layout idiom (high-DPI)	Files: 5 0 0 Lines: +461 -387
50	Tab strips fill their width (StretchTabControl)	Files: 7 0 +1 Lines: +137 -14
51	SD Card first-line cleanup (overlap + free-space banner)	Files: 2 0 0 Lines: +9 -4
52	Per-tool guidance text on the Tools tabs	Files: 7 0 0 Lines: +46 -14
53	Headless build script + crash logging	Files: 2 0 +2 Lines: +255 -8
54	Rename LoadStock → Start Job (code matches the UI)	Files: 30 0 0 Lines: +177 -177
55	Localization sweep — V2 strings into all 7 locales	Files: 29 0 0 Lines: +1581 -1
56	Online user manual + in-app context help (F1)	Files: 2 0 +1 Lines: +126 -2
57	Fixes: Start Job tab strand + program-view overlay width	Files: 3 0 0 Lines: +26 -1
58	Numeric fields select their value on focus	Files: 1 0 0 Lines: +35 -0
59	Connect: Network default + LAN controller discovery	Files: 9 0 +1 Lines: +527 -9
60	Drag to reorder tabs (persisted across restarts)	Files: 6 0 +1 Lines: +306 -8

61	grbl:Settings tree populates after connecting	Files: 2 0 +0 Lines: +37 -4
62	Probing/Tools sub-tab layout standardized	Files: 6 0 +0 Lines: +38 -30
63	Check ($C) moved to Cycle Start context menu	Files: 11 0 +0 Lines: +48 -29
64	Opt-in diagnostic trace log (-debuglog)	Files: 3 0 +1 Lines: +179 -5
65	grbl:Settings premature activation prevented at source	Files: 1 0 +0 Lines: +19 -0
66	Startup flag -forgetNetwork	Files: 1 0 +0 Lines: +13 -0
67	Demo-video capture markers (-demomarker)	Files: 5 0 +1 Lines: +219 -1
68	OBS auto-record bridge + tool-change cut markers	Files: 5 0 +1 Lines: +211 -3
69	Per-Monitor V2 DPI awareness	Files: 3 0 +1 Lines: +79 -0
70	Named G28 fixtures	Files: 12 0 +1 Lines: +336 -3
71	Smoother, faster corner probing	Files: 2 0 +0 Lines: +30 -22
72	Start Job: estimated stock size & Z check	Files: 10 0 +0 Lines: +47 -16
73	Playbooks: one-stop procedure library	Files: 0 0 +14 Lines: +456 -0
74	Startup dialogs no longer hidden behind the splash	Files: 3 0 +0 Lines: +18 -17
75	Status-bar messages shown at double height	Files: 1 0 +0 Lines: +37 -0
76	Program view: 3-line run view	Files: 4 0 +0 Lines: +170 -2
77	Selected tab: bold, larger label	Files: 1 0 +0 Lines: +62 -1
78	Run-bar jog pad restored & row rebalanced	Files: 3 0 +0 Lines: +42 -39
79	Bind any tab to a key — right-click Bind-to-Key + header badges	Files: 13 0 +1 Lines: +716 -17
80	Capability-gated tabs own their own “unavailable” reason	Files: 12 0 +1 Lines: +129 -93
81	No more restart prompt hiding behind the relaunch splash	Files: 1 0 +0 Lines: +18 -1
82	Dev playbooks distilled to one-command scripts	Files: 5 0 +4 Lines: +384 -11

83	Relaunch supervisor, single-instance & crash reporting	Files: 5 0 +2 Lines: +850 -58
84	Main-menu overhaul	Files: 35 0 +0 Lines: +458 -65
85	Start Job: one probe macro + spacer support	Files: 19 2 +0 Lines: +80 -232
86	Start Job: fail fast over bare board	Files: 1 0 +0 Lines: +12 -4
87	UI test server (-testserver)	Files: 2 0 +1 Lines: +505 -1
88	Fix false Machine Setup step 6 (ATC macros)	Files: 2 0 +0 Lines: +17 -4
89	UI test server â€" full harness + independent assembly	Files: 96 0 +7 Lines: +1832 -241
90	UI test server — standalone repo + NuGet package	Files: 4 5 +0 Lines: +5 -1391
91	AppData: shared Backups folder replaces .bak + settings-backups	Files: 5 0 +0 Lines: +86 -112
92	Remove start_job.macro file + its seeded macro catalog entry	Files: 5 1 +0 Lines: +6 -87
93	Program view: fix Save for wizard previews + popup duplication	Files: 3 0 +0 Lines: +53 -65
94	Fixture definitions: named workholding positions replace the single-slot G28 combo	Files: 9 1 +3 Lines: +859 -337
95	Apply UiScale zoom to dialogs, not just MainWindow	Files: 10 0 +1 Lines: +42 -0
96	Shared program-comment parser; probe dialog tweaks; touch-plate diameter fix	Files: 7 0 +1 Lines: +165 -62
97	Fusion add-in: combine to one .nc; Load File gets outline parity; retire Load Folder	Files: 16 2 +0 Lines: +200 -848
98	Machinist Vise fixture: probe, Start Job generation, unit toggle, accurate drawing	Files: 13 0 +1 Lines: +1021 -342
99	pcorner.macro collision fixes; MacroProcessor truncates oversized comments	Files: 2 0 +0 Lines: +59 -29
100	WpfUiTestServer: /handoff releases control to the operator; /idle waits for a real paint	Files: 2 0 +0 Lines: +4 -4
101	Localization: locale rows for this round's new/removed UI strings	Files: 15 0 +0 Lines: +548 -321
102	Vise Start Job: error:71 root-caused, keep-out check	Files: 6 0 +0 Lines: +173 -21
103	Start Job: Touch Plate probing, vise partial Measure	Files: 22 0 +0 Lines: +513 -115
104	Settings restore redesign, G28 fixture picker, crash fix	Files: 18 0 +0 Lines: +327 -93

105	Vise touch-plate margin fix, first full run	Files: 1 0 +0 Lines: +94 -58
106	(TOOL) comment gains an L= tool-length parameter	Files: 3 0 +0 Lines: +88 -24
107	Probe definitions unified on an overall-length measurement, like (TOOL)'s L=	Files: 10 0 +0 Lines: +61 -30
108	Lathe mode auto-detected from the controller, manual toggle removed	Files: 4 0 +0 Lines: +26 -5
109	Start Job: align Probe/Stock-conductive rows, exact-size vise labels, tighten T LO-ref motion	Files: 3 0 +0 Lines: +55 -28
110	Reset-repro test-case generator (Help > Support)	Files: 9 0 +2 Lines: +308 -21
111	Durable console log; crash/debug logs consolidated	Files: 5 0 +1 Lines: +164 -16
112	Program view popup resized to exactly the right third of the client area	Files: 1 0 +0 Lines: +11 -4
113	Start Job Measure/units fit-and-finish	Files: 1 0 +0 Lines: +35 -9
114	Simulator settings tab + auto-launch	Files: 18 0 +2 Lines: +431 -21
115	Simulator settings: hardware match + ATC	Files: 25 0 +1 Lines: +1082 -65
116	Tear-off tabs (Job view)	Files: 1 0 +1 Lines: +164 -13
117	Vise corner 1/3 X-face probe, measured origin/skew, size_y fix	Files: 1 0 +0 Lines: +184 -23
118	Machine tab: firmware version + one-click update	Files: 12 0 +6 Lines: +1798 -0
Version 2.3		
119	One-line installer + rolling release CI	Files: 3 0 +3 Lines: +209 -37
Version 2.5		
120	One-click update now + rollback	Files: 9 0 +0 Lines: +54 -3
Version 2.6		
121	Jog buttons could send an overlapping unacknowledged $J, wedging the controller	Files: 1 0 +0 Lines: +44 -0
122	Fusion batch post failed on operations using Air/Mist coolant	Files: 1 0 +0 Lines: +35 -0
123	A zero-length/truncated ATC macro on the controller read as Installed	Files: 1 0 +0 Lines: +54 -4
124	Vise Set position could close mid-probe and silently keep the stale corner	Files: 3 0 +0 Lines: +9 -27

125	Start Job/Load Stock accuracy pass: jaw-origin schematic, safer corner travel, exact-size tight probing	Files: 11 0 +0 Lines: +309 -53
Version 2.7		
126	Hang-watchdog restart notice on reconnect	Files: 6 0 +1 Lines: +197 -4
127	Scaled message boxes (AppMessageBox)	Files: 3 0 +2 Lines: +323 -2
Version 2.8		
128	Analog joystick jog was jerky under continuous hold - moved off the UI thread	Files: 1 0 +0 Lines: +52 -13
129	Tooltips now scale with the app's UiScale zoom	Files: 1 0 +0 Lines: +15 -1
130	New -simulator command-line flag to auto-connect to the bundled simulator on startup	Files: 1 0 +0 Lines: +33 -1
Version 2.9		
131	-testserver launches skip single-instance; -simulator takes an explicit port	Files: 3 0 +0 Lines: +37 -9
132	-testserver launches no longer steal foreground focus	Files: 2 0 +0 Lines: +71 -4
133	Medium-priority tooltips: spindle, goto/outline, macro dialogs, network connection settings	Files: 13 0 +0 Lines: +177 -16
134	Dry run mode + eliminate the MDI bypass path	Files: 18 0 +0 Lines: +282 -207
135	Consolidated log file creation/rotation/retention into one shared LogFile primitive	Files: 4 0 +1 Lines: +171 -97
136	pcorner/pvisecorner: spoilboard probe was searching from the wrong Z	Files: 2 0 +0 Lines: +15 -3
137	Start Job: corner-2 travel, unit-toggle readout, dialog scaling	Files: 2 0 +0 Lines: +17 -3
Version 2.10		
138	Dry run skips the program's own tool changes instead of running them	Files: 11 0 +0 Lines: +58 -13
139	Start Job: touch-plate Z0 rise, corner 2 approach height	Files: 2 0 +0 Lines: +50 -17
140	Start Job: exact-size labels, touch-plate corner Z readout	Files: 1 0 +0 Lines: +29 -4
141	Dry run: Z clearance math fix + token-accumulation over-suppression fix	Files: 2 0 +0 Lines: +56 -18
Version 2.11		
142	OBS/Tapo camera recording control, independent of OBS's main Record button	Files: 6 0 +4 Lines: +473 -3
143	Fixture edit dialog: redesigned Corner Fence schematic + directly-editable X/Y/Z position	Files: 21 0 +0 Lines: +526 -108

144	Machine Setup tab could go permanently unresponsive after a mid-move tab switch	Files: 1 0 +0 Lines: +25 -2
145	Job tab workspace lost width priority to the Feed/Spindle/Outline column	Files: 1 0 +0 Lines: +6 -1
146	Message box text was too small	Files: 1 0 +0 Lines: +1 -1
147	-message launch flag: show a popup once the app is up	Files: 2 0 +0 Lines: +62 -0
Version 2.12		
148	Restart-to-apply fix; AppLaunch removed	Files: 5 8 +0 Lines: +32 -425
149	Generate-to-disk logging; Safe Z delta fix	Files: 7 0 +0 Lines: +48 -18
Version 2.13		
150	User-draggable GridSplitters: Start Job, Height Map, Probing/Tools sub-tabs, Job tab's 3 regions	Files: 10 0 +0 Lines: +120 -59
151	Start Job probes the fence's origin corner once instead of twice	Files: 7 0 +0 Lines: +276 -54
152	NumericField stores canonical mm internally; unit toggle is display-only	Files: 5 0 +0 Lines: +221 -54
153	pcorner.macro side probes at a fixed 5mm below top, not the stock midpoint	Files: 1 0 +0 Lines: +16 -13
154	Start Job stock schematic: bigger, clearer dimension/angle/thickness labels	Files: 1 0 +0 Lines: +39 -20
155	New Tools sub-tab: Stepper Calibration (Probe) + Start Job calibration-mismatch warning	Files: 7 0 +2 Lines: +683 -4
Version 2.15		
156	Versioned releases (2.N) with real changelog notes	Files: 5 0 +1 Lines: +217 -58
Version 2.19		
157	Install command CMD-compatible; window title version fix	Files: 3 0 +0 Lines: +13 -8
Version 2.20		
158	Macro WAITIDLE false-abort after a burst's trailing motion	Files: 2 0 +0 Lines: +130 -35
159	Stepper Calibration (Probe): parks at G30, Safe Z delta field, program view auto-hides on success	Files: 10 0 +0 Lines: +39 -12
160	Start Job: synthetic "G28 (loose probe)" fixture - no saved fixture required	Files: 2 0 +0 Lines: +83 -20
161	Cycle Start renamed to Run, with a proper run-mode dropdown	Files: 29 0 +0 Lines: +406 -184
Version 2.21		

162	Generate button folded into the shared Run bar (Start Job + 4 tools)	Files: 21 0 +0 Lines: +343 -47
163	Status bar: permanent large font + green/red flash instead of enlarge/shrink	Files: 3 0 +0 Lines: +81 -44
164	Stepper Calibration (Probe): Z axis via a 1-2-3 gauge block	Files: 14 0 +0 Lines: +640 -36
Version 2.22		
165	Check for Updates works on dev builds	Files: 2 0 +2 Lines: +241 -10
Version 2.23		
166	Apple-HIG visual redesign across all main tabs	Files: 32 0 +1 Lines: +352 -47
167	Machine Setup: optional steps (Fixtures, Simulator) always show green	Files: 1 0 +0 Lines: +16 -5
168	SD Card tab no longer shows "No card" while mount status is still resolving	Files: 1 0 +0 Lines: +7 -3
169	Silent update check in the title bar + live rebuild-status on Settings > Simulator	Files: 2 0 +0 Lines: +60 -0
170	Sidebar flyout panels no longer overlap the main-nav tab strip	Files: 3 0 +0 Lines: +76 -10
Version 2.24		
171	Offsets tab rework: inline-editable grid with change tracking, correct G28/G30/G92 semantics	Files: 3 0 +0 Lines: +433 -181
172	UI polish: oval checkboxes, unit-toggle peek, tab-drag/flyout fixes, stale status messages	Files: 19 0 +1 Lines: +601 -204
Version 2.25		
173	Generate button green cue restored; Z-gauge reuse WAITIDLE	Files: 3 0 +0 Lines: +27 -0
174	Tools-tab colon alignment; disambiguated .macro filenames	Files: 10 0 +0 Lines: +20 -16
Version 2.26		
175	ATC macro gate false-“Outdated” trip fixed	Files: 2 0 +0 Lines: +9 -2
176	Run dropdown: new Simulate mode	Files: 5 0 +0 Lines: +170 -0
177	Program-list explain fix; compact limits display	Files: 26 0 +0 Lines: +348 -105
Version 2.27		
178	ioSenderV2 Fusion Addin: Feeds & Speeds + Batch Post Process + installer	Files: 24 0 +8 Lines: +3275 -141
Version 2.28		
179	Tab reorder + Settings splitters/Simulator redesign	Files: 8 0 +0 Lines: +92 -40

180	3D View carve trail visibility fix	Files: 1 0 +0 Lines: +13 -6
181	Firmware update UI redesign + real changelog	Files: 6 0 +0 Lines: +149 -94
182	Env-var removal: OBS/AI key/GH token to Settings	Files: 11 0 +3 Lines: +513 -87
183	OBS Validate button	Files: 2 0 +0 Lines: +135 -0
184	AI review cancel actually cancels	Files: 2 0 +0 Lines: +50 -30
185	Exit-blocked message + search crash + tooltip order fixes	Files: 3 0 +0 Lines: +32 -2
186	Manual: per-screenshot breakdowns + reshoot tooling	Files: 6 0 +2 Lines: +244 -12
Version 2.29		
187	Firmware update-check: query real /releases/latest	Files: 1 0 +0 Lines: +9 -6
Version 2.30		
188	Configurable probe search distance on all 4 Probing tabs	Files: 11 0 +0 Lines: +88 -0
189	Crash fixes: null keymap load, cross-thread UI crash; console visibility restored at connect	Files: 6 0 +0 Lines: +67 -7
Version 2.31		
190	Phil Barrett's UI feedback pass	Files: 20 0 +4 Lines: +600 -152
191	Fix: tab drag could crash or freeze the window	Files: 3 0 +0 Lines: +44 -4
Version 2.32		
192	Odd Jobs tab — Setup + Work Order builder	Files: 48 0 +11 Lines: +6404 -125
Version 2.33		
193	Work Order tab, conductive-material rules, touch-plate TLO	Files: 32 2 +0 Lines: +251 -218
194	Work Order → Job tab handoff, load performance	Files: 17 0 +1 Lines: +440 -103
Version 2.34		
195	Work Order chamfer bit, cut direction, spindle-direction setting	Files: 22 0 +1 Lines: +424 -173
Version 2.35		
196	Setup cold-start crash; Work Order live status in the Job tab	Files: 5 0 +0 Lines: +100 -44
Version 2.36		

197	Stepper & squareness → Machine Setup: Calibration	Files: 10 4 +0 Lines: +250 -1123
198	Spoilboard surfacing → Work Order Surface toolpath	Files: 5 0 +0 Lines: +309 -28
199	User-addable custom tools (no more tool enum)	Files: 8 0 +3 Lines: +927 -396
200	App.config first-run processing & per-section isolation	Files: 2 0 +0 Lines: +115 -6
201	Fix batch — Go To, park height, dry run, alarm aborts, jog pad	Files: 14 0 +0 Lines: +632 -107
202	Touch-plate offsets vs conductive stock; Work Order WCS	Files: 4 0 +0 Lines: +98 -32
Version 2.37		
203	Feed rate readout width + unit in tooltip	Files: 10 0 +0 Lines: +31 -5
204	Tools tab consolidation - relocated tools removed, hides when empty	Files: 14 2 +0 Lines: +67 -1096
205	Jog back-pressure shared with the gamepad, and its ack made visible	Files: 4 0 +1 Lines: +122 -42
206	Fixture dialog: probe choice persisted, Test uses the live position, no more mi nimize	Files: 4 0 +0 Lines: +163 -29
207	Setup step timing, shorter probe retreats, and two upload/session hangs	Files: 7 0 +0 Lines: +174 -73
Version 2.38		
208	Settings & Machine Setup: searchable navigation tree replaces tab strips	Files: 45 0 +6 Lines: +1788 -309
Version 2.39		
209	Settings search names the matched control	Files: 3 0 +0 Lines: +125 -16
210	Main bar cut back; the rest opens from the File/Tools menus	Files: 12 0 +2 Lines: +590 -118
211	Interface preferences gather under User Interface → General	Files: 10 0 +2 Lines: +171 -24
212	Shortcuts follow a view; menu commands bindable; placement editor	Files: 15 0 +0 Lines: +621 -266
213	Jog pad optional; Job panels fill one column and clear the flyouts	Files: 15 0 +0 Lines: +323 -121
214	Fix batch: duplicate tabs, unsatisfiable setup prompt, Esc, scroll wheel	Files: 13 0 +0 Lines: +357 -58
Version 2.40		
215	Menu-hosted views open, and stop crashing, when you click them	Files: 7 0 +0 Lines: +948 -65
Version 2.41		

216	Large programs load 12× faster	Files: 3 0 +0 Lines: +87 -5
217	Data column no longer starts collapsed	Files: 2 0 +0 Lines: +45 -9
Pending 3.0 (PR 20)		
218	Portable core: CNC.Core runs without WPF, and builds on .NET 8	Files: 150 0 +4 Lines: +10527 -5440
219	Work Order runs in one press: one streaming engine for macros and programs	Files: 13 0 +3 Lines: +986 -940
220	The run strip: everything a running job needs, and the bottom status bar retired	Files: 31 0 +2 Lines: +1716 -391
221	Click the status line for every message since launch	Files: 11 0 +0 Lines: +263 -31
222	Text toolpaths: single-stroke engraving	Files: 17 0 +1 Lines: +894 -42
223	V-carving: TrueType outlines carved to shape	Files: 13 0 +2 Lines: +1305 -49
224	Shape text: text fitted inside Work Order geometry	Files: 12 0 +0 Lines: +476 -43
225	SAFETY: Feed Hold could be starved for the whole length of a dense job	Files: 1 0 +0 Lines: +17 -1
226	Safety batch: envelope, coordinate frames, and a Validate run that crashed a machine	Files: 3 0 +2 Lines: +360 -278
227	The link watchdog stops killing live operations	Files: 8 0 +1 Lines: +276 -34
228	Jogging: a cancelled jog no longer wedges the jog path	Files: 11 0 +1 Lines: +229 -1
229	Large programs load in seconds; the console stops starving the UI	Files: 4 1 +0 Lines: +137 -18
230	Generate says what it is doing, and what it will cost	Files: 17 0 +4 Lines: +765 -22
231	Probing and Setup: say what to fit, and stop parking over the spoilboard	Files: 19 0 +1 Lines: +319 -303
232	Fix batch: crashes, startup, and the connect path	Files: 4 0 +2 Lines: +236 -45
233	Fix batch: windows, dialogs and small corrections	Files: 39 0 +2 Lines: +864 -136
234	SAFETY: unparseable blocks were silently dropped from the program	Files: 1 0 +0 Lines: +75 -16
235	Connect: the false unexpected-response dialog, and the empty capability set	Files: 4 0 +0 Lines: +195 -19
236	Setup hands the program to the Job tab at Generate	Files: 3 0 +0 Lines: +373 -45
237	3D view: the stock block tells the truth	Files: 15 0 +1 Lines: +419 -56

238	Job tab: optional split screen	Files: 19 0 +0 Lines: +177 -6
239	V-carve Generate: 144 s down to 19.6 s, byte-identical output	Files: 2 0 +0 Lines: +190 -42
240	MDI dispatch unified with the streaming pump	Files: 3 0 +3 Lines: +941 -359
241	Jogging: g-code is refused while a jog is outstanding	Files: 1 0 +1 Lines: +61 -1
242	The simulator comes up as the machine actually is	Files: 4 0 +1 Lines: +431 -26
243	Controller filesystem: an unanswered query is unknown, not empty	Files: 2 0 +0 Lines: +183 -26
244	An error is an answer: no more waiting 36 seconds for an ok that never com es	Files: 2 0 +0 Lines: +94 -19
245	The app stops running out of memory during long macro runs	Files: 1 0 +0 Lines: +18 -0
246	Status pop-up: chosen dwell time, errors only, and focus handed back	Files: 12 0 +1 Lines: +208 -34
247	Configuration overlays	Files: 11 0 +1 Lines: +693 -13
248	Default layout back to the full tab bar	Files: 2 0 +0 Lines: +46 -37
249	First connect defaults to Serial	Files: 3 0 +0 Lines: +14 -8
250	Jog go-to buttons say why they refuse	Files: 2 0 +0 Lines: +28 -12
251	Settings changed elsewhere no longer go unnoticed	Files: 3 0 +0 Lines: +192 -0
252	The Job view stays on the tab bar, because moving it broke connecting	Files: 3 0 +0 Lines: +84 -22
253	A worker that threw left the app pumping for ever	Files: 15 0 +0 Lines: +169 -245
254	MDI and Status can be put on a key	Files: 3 0 +0 Lines: +29 -1
255	F12 opens the MDI console; Esc dismisses it, as everywhere else	Files: 14 0 +0 Lines: +124 -112
256	Console search now highlights every match	Files: 1 0 +1 Lines: +218 -1
257	Settings search points at the match on the page, not just at the page	Files: 1 0 +1 Lines: +321 -39
258	Bounded controller ack waits; an unanswered settings write reports instead o f hanging	Files: 8 0 +0 Lines: +153 -89
259	Corner reliefs (dogbones) on square and rectangular pockets	Files: 12 0 +0 Lines: +108 -6

260	Jog keys and keyboard shortcuts work in every window, including dialogs	Files: 7 0 +1 Lines: +214 -24
261	Bore feed compensated on wall passes only — a full-immersion helix keeps t he feed as entered	Files: 1 0 +0 Lines: +41 -2
262	SVG artwork as a Work Order toolpath	Files: 4 0 +4 Lines: +934 -14
263	The Status window is written to a log file, like the console	Files: 2 0 +1 Lines: +107 -0
264	Dry run's Z offset could outlive the job	Files: 2 0 +0 Lines: +139 -1
265	Three faults found running a real job	Files: 7 0 +0 Lines: +111 -32
266	A stuck probe is named, and the check waits for the machine	Files: 1 0 +0 Lines: +74 -3
267	The carve cache served the wrong artwork	Files: 1 0 +0 Lines: +86 -17
268	A depth cap for V-carve, so a narrow bit can cut fine detail	Files: 12 0 +0 Lines: +187 -19
269	Duplicate quietly dropped every field added to a toolpath since it was writte n	Files: 2 0 +0 Lines: +55 -32
270	An Indirect toolpath's X/Y are absolute, or relative to the toolpath it copies	Files: 11 0 +0 Lines: +151 -9
271	Group toolpaths, and switch a whole set on or off together	Files: 10 0 +0 Lines: +269 -41
272	Point an Indirect toolpath at a group and it copies the whole set	Files: 3 0 +0 Lines: +622 -257
273	Burn an SVG on a diode laser, straight from File > Open	Files: 9 0 +5 Lines: +902 -5
274	Depth of cut now drives the V-carve step, instead of cutting each groove six times over	Files: 10 0 +0 Lines: +45 -22
275	A disabled Generate now says why, and a 4.2 mm drill is a drill	Files: 4 0 +0 Lines: +108 -8
276	A fix could appear to do nothing: the program cache keyed on version, not b uild	Files: 1 0 +0 Lines: +17 -0
277	The connect handshake could read the wrong reply, and the controller's setti ngs came back empty	Files: 2 0 +0 Lines: +134 -3
278	A stored WCS origin is checked against the travel envelope before the job st arts	Files: 1 0 +0 Lines: +17 -5
279	A G53 move kept its axis words — the 3D view and the travel check were mix ing frames	Files: 3 0 +0 Lines: +58 -8
280	The status log now records what actually happened: connects, jobs, generate s and setting changes	Files: 11 0 +0 Lines: +225 -21
281	A connect announced the previous connect's result, and skipped its own rep ort	Files: 2 0 +0 Lines: +18 -2

282	Menu commands get their own keyboard group, apart from the tab destinations	Files: 4 0 +1 Lines: +116 -41
283	Height Map: probe the board, read the program before it moves, and see every reading	Files: 9 0 +0 Lines: +556 -62
284	Every probe on a height map aimed somewhere it could not reach	Files: 10 0 +0 Lines: +233 -49
285	The Surface Map had been empty since it was written, and its colours meant a different height on every board	Files: 12 0 +1 Lines: +303 -23
286	A height map run you can stop, resume, and not get stuck inside	Files: 10 0 +0 Lines: +424 -133
287	The step after a probe was silently dropped, and the run waited for ever	Files: 1 0 +0 Lines: +30 -0
288	The assert-the-probe dialog is gone — it asserted water is wet	Files: 19 2 +0 Lines: +10 -191
289	The spoilboard is not the travel envelope, and a cutter went there	Files: 13 0 +1 Lines: +335 -43
290	Spoilboard surfacing proves the depth on one plunge before committing the table	Files: 20 0 +0 Lines: +224 -10
291	A program bigger than the machine can travel is refused before it runs	Files: 1 0 +0 Lines: +75 -1
292	Machine Setup showed 0 for settings that had not arrived, and Apply would have written them back	Files: 1 0 +0 Lines: +102 -13
293	Two connects that could never succeed: a controller left in check mode, and a board slower than 2.5 seconds	Files: 3 0 +0 Lines: +81 -3
294	A coordinate system that was never reported is zero, not null — loading a file could take the app down	Files: 2 0 +0 Lines: +62 -9
295	A macro global documented “0 (default)” had no default, and aborted the probe	Files: 1 0 +0 Lines: +31 -1
296	Restore picks a moment, not a file — and says what that moment holds	Files: 13 0 +1 Lines: +383 -62
297	Feeds and speeds are held to what the machine can actually deliver	Files: 1 0 +0 Lines: +74 -1
298	Startup could die focusing a control that was never chosen	Files: 1 0 +0 Lines: +14 -2
299	Dialogs record what the operator was told, and what they answered	Files: 1 0 +0 Lines: +39 -12
300	SVG import reads the rendered tree, and applies transforms instead of just noting them	Files: 1 0 +0 Lines: +197 -22
301	Place the laser artwork, repeat it, and anchor it to the corner the machine actually homes to	Files: 10 0 +0 Lines: +344 -30
302	The laser dialog fits the screen, keeps its settings, and shows the exposure the S values hide	Files: 23 0 +0 Lines: +513 -126
303	“Save Drawing” — a work order as a dimensioned PDF sheet	Files: 9 0 +2 Lines: +1198 -14

304	Colour-coded, ballooned toolpaths on the stock diagram	Files: 2 0 +1 Lines: +303 -70
305	Envelope: contours draw a band, and text/artwork stop drawing a box nothin g set	Files: 1 0 +0 Lines: +143 -12
306	A work order records its own stock size	Files: 13 0 +0 Lines: +306 -21
307	Restart ioSender from the Help menu	Files: 10 0 +0 Lines: +48 -0
308	Connect prompt could hide behind the main window	Files: 1 0 +0 Lines: +7 -0
309	SVG laser exposure as named constants	Files: 3 0 +3 Lines: +551 -7
310	SVG laser power step per copy	Files: 10 0 +0 Lines: +183 -8
311	SVG laser placement checked against travel	Files: 2 0 +0 Lines: +170 -13
312	SVG laser header comment left unterminated	Files: 2 0 +0 Lines: +41 -2
313	SVG laser ramp: no redundant re-declaration	Files: 1 0 +0 Lines: +30 -2
314	SVG laser placement as x_org / y_org	Files: 3 0 +0 Lines: +151 -18
315	-enableSVGLaserJob replaces the committed flag	Files: 3 0 +0 Lines: +36 -11
316	Park lift: bare G53 G0 Z0, not #<_abs_x>/#<_abs_y>	Files: 4 0 +0 Lines: +44 -26
317	Stored-position reachability guard was dead code (HomedMask race)	Files: 2 0 +0 Lines: +52 -3
318	CNC.Svg - portable SVG import + laser emitter (for EngravingBox)	Files: 15 0 +10 Lines: +2348 -433
319	Console: an MDI command no longer resets the scrollback to line 0	Files: 1 0 +0 Lines: +25 -0
320	Laser skew compensation (D1 / D2 / Side from a test square)	Files: 11 0 +0 Lines: +194 -14
321	Parametric programs - PROMPT at load + bracket folding on stock Grbl	Files: 6 0 +0 Lines: +466 -28
322	SVG carve: Negative - the artwork stands proud of a carved panel	Files: 12 0 +0 Lines: +158 -10
323	V-carve: spine detail passes cut after the rings and flats, not first	Files: 2 0 +0 Lines: +15 -8
324	SVG carve: negative bounded by the artwork's own outline	Files: 12 0 +0 Lines: +170 -18
325	Work Order: Clear floor - an end mill flattens a V-carve's floor	Files: 4 0 +0 Lines: +348 -40

326	Clear floor: cuts a tenth below the carve floor, one pass by default	Files: 11 0 +0 Lines: +86 -10
327	Work Order: Mark - a V-bit traces the outline as a shallow groove	Files: 4 0 +0 Lines: +90 -8
328	Mark: Dashed lines	Files: 12 0 +0 Lines: +245 -20
329	FIX: Mark cut a 1 mm groove in five laps	Files: 1 0 +0 Lines: +12 -10
330	Stepper calibration (scratch) is back	Files: 15 0 +2 Lines: +983 -37
331	FIX: a disabled Generate button now says what it wants	Files: 6 0 +0 Lines: +68 -6
332	A view that needs the run strip opens as a TAB, not a window	Files: 16 0 +2 Lines: +362 -187
333	Closable tabs, and an opt-in ESC to close one	Files: 11 0 +0 Lines: +159 -11
334	CRASH FIX: corner 1 rapided into the touch plate	Files: 2 0 +0 Lines: +92 -7
335	CRASH FIX: no pause between corner 1 and corner 2	Files: 1 0 +0 Lines: +36 -12
336	Scratch calibration runs from the origin already set	Files: 9 0 +0 Lines: +198 -12
337	FIX: the Measured cell fought the caret	Files: 9 0 +0 Lines: +39 -5
338	FIX: changing an input left the OLD program loaded	Files: 4 0 +0 Lines: +114 -28
339	tlo.macro: one puck reference on the controller	Files: 13 0 +1 Lines: +376 -213
340	SAFETY: a TLO reference must hand back a parked machine	Files: 4 0 +0 Lines: +83 -25
341	FIX: the first TLO reference after a reset died	Files: 2 0 +0 Lines: +23 -2
342	Scratch calibration: a depth per mark	Files: 9 0 +0 Lines: +60 -8
343	One handoff for every Generate-first tab	Files: 7 0 +0 Lines: +622 -249
344	One prologue and one epilogue for nine program builders	Files: 6 0 +0 Lines: +115 -33
345	FIX: a nested paren in a message prompt ate every space	Files: 2 0 +0 Lines: +72 -21
346	FIX: the macro upload ran at the status-poll rate	Files: 2 0 +0 Lines: +51 -3
347	FIX: Clear floor defaulted to a ball nose	Files: 3 0 +0 Lines: +21 -3

348	Work Order: Mark only	Files: 11 0 +0 Lines: +491 -107
349	Save Drawing lists every instance with its X and Y	Files: 1 0 +0 Lines: +36 -0
350	FIX: two warnings the reader could do nothing about	Files: 3 0 +0 Lines: +60 -31
351	FIX: an assignment's RHS cannot be a bare binary expression	Files: 2 0 +0 Lines: +15 -3
352	Controller macros: send only the ones that changed	Files: 2 0 +0 Lines: +130 -14
353	FIX: WCS rotation was read once at connect	Files: 2 0 +0 Lines: +40 -2
354	WCS rotation: fix the one operation that breaks	Files: 2 0 +0 Lines: +33 -3
355	Run strip: hide Feed Hold and Stop when they mean nothing	Files: 2 0 +0 Lines: +55 -2
356	FIX: prefer network dropped serial onto a mute link	Files: 1 0 +0 Lines: +78 -6
357	FIX: WCS rotation pivots about MACHINE zero	Files: 1 0 +0 Lines: +55 -17
358	FIX: a G53 Z-only lift traversed the whole table	Files: 2 0 +0 Lines: +58 -7
359	FIX: a bare origin write turned a Z-only lift into a 662 mm rapid	Files: 5 0 +0 Lines: +71 -8
360	FIX: the console threw you to line 0 on every flush	Files: 2 0 +0 Lines: +45 -14
361	FIX: degree, delta and diameter signs encoded twice	Files: 2 0 +0 Lines: +5 -5
362	Verify skew: probe the whole descent, and a fly-over mode	Files: 9 0 +0 Lines: +191 -55
363	Persist the measure result across restarts	Files: 2 0 +0 Lines: +164 -10
364	FIX: a debug build logged only the status poll	Files: 1 0 +0 Lines: +22 -7
365	Verify skew fly-over: sight from 2 mm, inset, cross at height	Files: 10 0 +0 Lines: +185 -35
366	FIX: the V-bit dropdown was full of blank rows	Files: 3 0 +0 Lines: +43 -8
367	Scribe square: a verification you can measure	Files: 9 0 +0 Lines: +187 -3
368	Squareness (probe): measure against a reference square	Files: 12 0 +2 Lines: +1562 -68
369	Squareness (probe): the reversal test	Files: 9 0 +0 Lines: +502 -75

370	FIX: a setting the machine accepted was reported as failed	Files: 2 0 +0 Lines: +57 -6
371	FIX: a typed value was displayed, a different one written	Files: 6 0 +0 Lines: +144 -3
372	Peek - step away from a running job, and pick it up where you left it	Files: 12 0 +1 Lines: +748 -15
373	FIX: Machine Setup's Overview listed six steps for an eight-step wizard	Files: 8 0 +0 Lines: +59 -38
374	Entering a new stock size discards a measurement carried over from a previo us session	Files: 1 0 +0 Lines: +41 -2
375	FIX: an offset test that answered backwards, and a start-from-line that skippe d its prolog	Files: 2 0 +0 Lines: +27 -1
376	Toolpath outline on any program with tool changes, and run one toolpath fr om it	Files: 6 0 +1 Lines: +403 -7
377	Manual: the 21 missing topics from the audit	Files: 3 0 +0 Lines: +888 -72
378	Help > User manual on Shift+F1, landing on the right topic	Files: 14 0 +0 Lines: +258 -44
379	FIX: a message box could open BEHIND the window that raised it, which rea ds as a hang	Files: 2 0 +0 Lines: +26 -0
380	FIX: Set position captured the CACHED machine position, and could capture a stale one	Files: 2 0 +0 Lines: +113 -14
381	Height map: kept by Setup, applied by the Work Order at Generate	Files: 13 0 +2 Lines: +676 -27
382	FIX: the height map refused a map that matched the setup in front of it	Files: 5 0 +0 Lines: +99 -33
383	CHG: the height map is probed in its own tab, as a step you can see	Files: 14 0 +0 Lines: +336 -99
384	FIX: four faults in how the height map moves between points	Files: 11 0 +0 Lines: +97 -24
385	A Bluetooth shutter remote drives the machine from arm’s length	Files: 13 0 +2 Lines: +479 -55
386	FIX: applying a height map took thirty seconds and showed an empty progra m list while it worked	Files: 5 0 +0 Lines: +108 -8
387	FIX: a transformed program lost its parameter lines to the word “Parameter”	Files: 1 0 +0 Lines: +18 -0
388	The toolpath outline names each section after the TOOL, and survives a trans form	Files: 5 0 +0 Lines: +85 -5
389	FIX: a chamfer on a Line ran down the middle of the groove instead of along its edges	Files: 1 0 +0 Lines: +62 -6
390	FIX: a rotated WCS drew the toolpath outside the machine and tilted	Files: 4 0 +0 Lines: +88 -31
391	Shutter remote: binding + per-state button functions	Files: 13 0 +4 Lines: +1640 -24

392	Probe validity: seek and latch must agree	Files: 2 0 +0 Lines: +46 -0
393	Corner probe travel height clamped to the machine envelope	Files: 5 0 +0 Lines: +87 -9
394	Find a keyboard shortcut by pressing it	Files: 11 0 +0 Lines: +211 -3
395	Remote: one settings page, and the truth about the volume	Files: 18 0 +0 Lines: +26 -38
396	Height map: the hold follows the probe	Files: 2 0 +0 Lines: +101 -13
397	Step 1's machine pick stops overwriting real settings	Files: 10 0 +0 Lines: +103 -2
398	Step 1 stops pre-picking a machine you never chose	Files: 2 0 +0 Lines: +32 -2
399	FIX: Test position travelled diagonally to the fixture	Files: 2 0 +0 Lines: +36 -2
400	CHG: The probed corner IS the fixture position	Files: 7 0 +0 Lines: +167 -80
401	FIX: The tool-length probe asked for more travel than the machine had left	Files: 2 0 +0 Lines: +43 -6
402	NEW: `ALARM:2` now names the axis and shows the arithmetic	Files: 4 0 +1 Lines: +210 -3
403	FIX: The tool-length probe's seek distance is derived, not hardcoded	Files: 2 0 +0 Lines: +82 -5
404	NEW: Setup asserts a clean state before it runs, or refuses with the reason	Files: 3 0 +0 Lines: +77 -7
405	NEW: A run aborting with the probe still touching retracts Z	Files: 1 0 +0 Lines: +98 -0
406	A sound for a limit, a probe touch or an alarm	Files: 10 0 +4 Lines: +723 -0
407	A failed startup left a zombie that blocked every later launch	Files: 3 0 +0 Lines: +45 -6
408	Tool-length baseline stored in the wrong frame - derived seek silently off	Files: 2 0 +0 Lines: +136 -34
409	Reference TLO's seek stopped 2 mm short of the trigger	Files: 2 0 +0 Lines: +170 -62
410	Applied tool-length offset now logged as a number; console no longer eats a bracket	Files: 4 0 +0 Lines: +47 -7
411	Driver and driver version reported at connect	Files: 2 0 +0 Lines: +42 -0
412	Adapt to the firmware's git-tag version	Files: 7 0 +2 Lines: +284 -18
413	Version 3.0: unified assembly numbers, major bumps now cuttable	Files: 18 0 +1 Lines: +140 -35

414	The status log now says which build wrote it	Files: 1 0 +0 Lines: +49 -0
	Totals (414 changes)	Files: 3309 32 +305 Lines: +137093 -29793

Cross-repo: which ioSender features the `stevenrwood/Simulator` fork exercises.

No build dependency, and all-or-nothing — you run the whole `Simulator@integration` build against ioSender (not a chosen subset), so this table is informational rather than something to satisfy. The coupling is runtime / protocol: the simulator only exercises a feature when the ioSender change below drives it over the socket.

ioSender feature	Simulator feature it lights up	Coupling
#6 Load Folder	3D carve view	`PushHeaderToSimulator()` sends the program's leading `(STOCK …)` / `(TOOL …)` comments so the carve knows stock size + cutter geometry.
#11 macro pre-processor	littlefs macros	`O<name> CALL` forwarding resolves macros stored on the simulator's `/littlefs`.
#14 SD / filesystem browser	littlefs, YModem	File browser + YModem uploader that puts files onto the simulator filesystem.
#15 connect-to-simulator	3D carve view, settings persistence, sim core	Connects to the standalone sim; *always-send-comments* forces `(TOOL)`/`(STOCK)` through for the carve; "My Machine" EEPROM uses the sim's settings persistence.
#25 ATC macro provisioning	YModem, littlefs, probe surfaces	"Install ATC macros" uploads via YModem to `/littlefs`; the Start-Job / `probe_tfl` cycle drives the modelled probe surfaces.

For the full virtual-CNC experience, run **`ioSender@integration`** against **`Simulator@integration`** — one all-in-one bundle each (the firmware-submodule fixes ride along inside the sim). The matched-simulator feature (#38) builds an option-matched sim automatically.

#1 Bug Fixes for released ioSender

File	+	−
`.gitattributes`	3	2
`CNC Controls/CNC Controls/JobControl.xaml.cs`	7	0
`CNC Controls/CNC Controls/Widget.cs`	3	1
`CNC Core/CNC Core/CNC Core.csproj`	6	0
`CNC Core/CNC Core/Grbl.cs`	9	7

A bucket of fixes for bugs that exist in released ioSender, independent of the feature PRs. Each is small and self-contained:

- **RP.Math build fix.** CNC Core compiles `GCodeEmulator.cs`, which uses the `RP.Math.Vector3` type, but the project file declared no reference to RP.Math, so a clean build fails to resolve the type. Adds the `RP.Math` and `RP.Math.Vector3` references using the *same* `HintPath` convention already used by `CNC Controls.csproj` (`..\..\Vector-master\RP.Math.Vector3\bin\Release\`). Purely additive.
- **LF line-ending guard.** Adds `.gitattributes` (`* text=auto eol=lf`) so line endings are normalized.
- **CSV null-reference fix.** Avoids a `NullReferenceException` when an alarm/error/settings-group CSV resource is missing.
- **Boolean settings convention.** The settings editor checkbox initialised with `value == "1"`, but a boolean is true when `value ≠ "0"` everywhere else (the list, controller-value parsing). Booleans reported as a truthy value other than the literal `"1"` showed enabled in the list yet left the checkbox unchecked. Canonicalises boolean values to `"1"`/`"0"` in `GrblSettingDetails.Value` and reads the checkbox with the `≠ "0"` convention.
- **Null-comms streaming guard.** `JobControl.ResponseReceived` is raised by a specific comms instance but writes to the static `Comms.com`; during a reconnect/teardown (startup handshake or relaunch) that static can be null while an in-flight response from the old link still arrives, NREing in the SendMDI/Reset cases. Bails out early when `Comms.com` is null.

#2 Surface Motor Fault and Motor Warning signals in Signals display

File	+	−
`CNC Controls/CNC Controls/SignalsControl.xaml`	2	30

Removes the capability-gating `Styles` that prevented the Motor Fault (**F**) and Motor Warning (**W**) indicators from ever showing.

grblHAL does not advertise these as `NEWOPT` capability tokens, so the visibility gates were effectively dead code — the LEDs were permanently hidden. Motor Fault / Warning are now always displayed alongside the other signal LEDs (H / S / P), turning red when the corresponding signal triggers. This surfaces the Motor Fault signal as intended, without requiring the firmware to advertise a capability flag.

#3 Persist console state and add an in-app Key Mappings editor

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	14	0
CNC Controls/CNC Controls/AppConfigView.xaml	1	1
CNC Controls/CNC Controls/AppConfigView.xaml.cs	16	3
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Controls/CNC Controls/ConsoleControl.xaml	3	3
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	14	0
CNC Controls/CNC Controls/ConsoleWindow.xaml.cs	47	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	115	0
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	606	0
CNC Core/CNC Core/CNC Core.csproj	7	0
CNC Core/CNC Core/KeypressHandler.cs	111	0
CNC Core/CNC Core/ShortcutKey.cs	136	0
ioSender XL/ioSender XL/App.config	6	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	69	3
ioSender/ioSender/MainWindow.xaml.cs	69	3

Persists the console's state across sessions and replaces the old App Settings "*Save key mappings*" button (which only dumped the current bindings to `KeyMap<mode>.xml` for hand-editing) with a proper in-app **Edit Key Mappings** dialog. Applies to both **ioSender** and **ioSender XL**.

Console persistence: the three console checkboxes (*Verbose*, *Filter out realtime responses*, *Show all realtime responses*), whether the floating console window is open, and that window's size/position (clamped to the visible desktop so it can never restore off-screen) are all remembered. A new `ConsoleShortcut` setting (default `Esc`) toggles the console; it is now a first-class action inside the mappings editor rather than a separate, ad-hoc tab. A shared `ShortcutKey` helper (parse / storage / display) is used by both main windows, and an `AppConfig.ConsoleShortcutChanged` event re-registers the key live without a restart.

Key Mappings editor (`KeyMapEditor`, modal so jog/realtime dispatch can't fire mid-capture):

- One grouped, collapsible outline of every bindable action (grouped by function, like the Load Folder outline; groups remember their expansion per session). Click a row, press a combo to assign; right-click for *Reset to default* / *Clear*.
- Bindings changed from the factory default are highlighted; group headers flag a modified group and offer *Expand all / Collapse all / Reset group*.
- Per-axis jog keys are folded into the same list; rows and group headers carry tooltips.
- A *No-numpad keys* preset remaps the NumPad jog/step/feed actions to `Ctrl+1–8` and `[ ] − =` for keyboards without a numeric keypad (Windows exposes no reliable API to detect numpad presence, so this is a user-triggered preset).

The `KeypressHandler` gains an editor API (`GetActionBindings/GetJogBindings/Apply…`) over its existing function catalog, identifying bindings by reflection method name with a friendly-label map. Persistence still uses the existing `SaveMappings/LoadMappings`.

#4 Fix unhandled exceptions on connection loss (e.g. SD card swap)

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	5	0
CNC Controls/CNC Controls/SDCardView.xaml.cs	13	0
CNC Core/CNC Core/Comms.cs	74	0
CNC Core/CNC Core/EltimaStream.cs	6	0
CNC Core/CNC Core/Grbl.cs	13	4
CNC Core/CNC Core/GrblViewModel.cs	45	0
CNC Core/CNC Core/SerialStream.cs	179	42
CNC Core/CNC Core/TelnetStream.cs	105	7
CNC Core/CNC Core/WebsocketStream.cs	79	8

When the controller's serial/USB device disappears — for example a board that resets and re-enumerates when its SD card is removed or reinserted — the status-poll timer kept writing to the dead port. The write threw (an `IOException`, then an `InvalidOperationException`: "BaseStream only available when the port is open") on a `System.Timers.Timer` thread, and the unhandled exception terminated the whole process. Opening the SD card tab afterwards threw similarly from `PurgeQueue`.

Changes:

- Guard all stream writes (`IsOpen` + try/catch). A device-gone fault now closes the port and starts an auto-reconnect instead of crashing.
- Add a shared, transport-agnostic `Reconnector` (used by serial, network and websocket) exposing `ConnectionLost` / `Reconnected` events. `GrblViewModel` handles them (stop/resume polling, show status); it is wired up at connect time in `AppConfig`.
- **Serial:** refactor open into a reusable `OpenPort()` that probes `GetPortNames()` and re-opens; publish the port only once it is actually open, to avoid a non-null-but-closed race; make `PurgeQueue` tolerant of a closed port; harden the receive handler against a port torn down mid-read.
- **Telnet / websocket:** guarded writes, loss detection (read returning 0 / `OnClose`) and reconnect via the same helper.
- Wrap the poll-timer callback in try/catch as a last line of defence.
- Guard the SD card view against acting on a closed connection.

#5 Add Stepper calibration (scratch) tab to grbl:Settings

File	+	−
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Controls/CNC Controls/GrblConfigView.xaml	3	0
CNC Controls/CNC Controls/IGrblConfigTab.cs	1	0
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	92	0
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	486	0
.gitattributes	3	2
CNC Controls/CNC Controls/LibStrings.xaml	1	0
CNC Core/CNC Core/CNC Core.csproj	6	0
ioSender XL/ioSender XL/App.config	6	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	168	0

Adds a new tab to grbl:Settings for V-bit *scratch-line* steps/mm calibration over a long span — a more accurate alternative to the short jog-based calibration for the X and Y axes.

It generates an `.nc` program that scratches paired lines (perpendicular to the calibrated axis) for several candidate steps/mm values. Because GRBL cannot change `$100`/`$101` mid-program, each candidate is simulated using the *commanded-distance trick*: the commanded distance `C = span × candidate / current`, so the machine physically travels the distance that the candidate steps/mm value *would* produce.

The program is loaded into the job view (with an optional save to disk). After cutting, the user measures the actual spacings; the tab then computes and saves the corrected `$100`/`$101` from those measurements (an implied value per pair, averaged). The prolog mirrors the Load Folder convention. X/Y only — Z continues to use the existing jog wizard.

Includes a small follow-up commit with high-DPI layout tweaks for the new tab.

#6 Add "Load Folder": load a folder of per-toolpath .nc files as one outlined program

File	+	−
.gitattributes	3	2
CNC Controls/CNC Controls/CNC Controls.csproj	2	0
CNC Controls/CNC Controls/FolderPicker.cs	159	0
CNC Controls/CNC Controls/FusionFolderLoader.cs	353	0
CNC Controls/CNC Controls/GCode.cs	156	0
CNC Controls/CNC Controls/GCodeListControl.xaml	45	0
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	129	0
CNC Controls/CNC Controls/JobControl.xaml.cs	4	0
CNC Core/CNC Core/GCodeJob.cs	41	5
CNC Core/CNC Core/GrblViewModel.cs	7	1
Fusion Addin/README.md	116	0
Fusion Addin/install-macos.sh	39	0
Fusion Addin/install-windows.ps1	40	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.manifest	11	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.py	476	0
ioSender XL/ioSender XL/MainWindow.xaml	1	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	5	0
ioSender/ioSender/MainWindow.xaml	1	0
ioSender/ioSender/MainWindow.xaml.cs	5	0
CNC Controls/CNC Controls/LibStrings.xaml	1	0
CNC Core/CNC Core/CNC Core.csproj	6	0
ioSender XL/ioSender XL/App.config	6	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	21	0
Locale/*/csv/ioSender.resources.*.csv (7 locales)	7	0

File > Load Folder reads a folder of per-operation files named `<seq>_<name>_T<tool>.nc` (as produced by the companion Fusion 360 add-in) and assembles them in memory into a single program:

- strips each file's header and program-end footer;
- inserts a `G53 G0 Z0` safe-Z retract + `M6 T<n>` tool change before each toolpath, but only when the file does not already contain an `M6` (so any post works);
- optionally restores rapid moves that Fusion Personal Use downgraded to `G1`;
- shows the result as a collapsible per-toolpath outline (groups default collapsed, and stay expanded while scrolling);
- adds *Start from this toolpath* and *Run just this toolpath* (the latter bounded to that section), each preceded by the `G90 G94 / G17 / G21` prolog.

The new `FusionFolderLoader` is a faithful C# port of the add-in's combine / strip / rapid-restore rules; `FolderPicker` uses the modern `IFileOpenDialog` (`FOS_PICKFOLDERS`) for folder selection. `GCodeBlock` gains a `Section` tag for outline grouping and an optional `AddLineNumbers` gate; `GrblViewModel` gains `IsFolderView` and a one-shot `RunToBlock` end-bound consumed by `CycleStart`.

Companion Fusion 360 add-in (`ioSenderBatchPost`): a standalone add-in (no external framework) that posts every operation in every setup to its own `<seq>_<name>_T<tool>.nc` file in a chosen folder — the input for Load Folder. Combining, tool-change insertion and rapid restoration are intentionally left to ioSender, so the add-in only posts. A post-processor dropdown (default `grbl.cps`) selects which post Fusion runs per operation. Ships with `install-windows.ps1` / `install-macos.sh` and a README. Fusion auto-discovers it once copied, but it must be enabled once via *Scripts and Add-Ins*.

Tool table for the simulator: the add-in also writes a `0_ToolTable.nc` carrying `(STOCK X= Y= Z=)` and `(TOOL T= D= TYPE= [A=])` comment lines — the stock box size and each tool's diameter and shape (`FLAT`/`BALL`/`VBIT`, with the V-bit included angle) — read from the setup's stock parameters and tool definitions. Load Folder detects this file (name matches `tool table`), loads it first with comments preserved, and pushes the leading `(STOCK …)`/`(TOOL …)` lines to a connected grblHAL simulator, which uses them for its 3D material-removal carve (real controllers ignore the comments). A dialog checkbox toggles it; the output matches the SRWCommands PostProcess add-in byte-for-byte. See `TOOL_TABLE_FORMAT.md` in the simulator repo.

Note: the `Fusion Addin/` folder is an optional companion tool; the maintainer may prefer to host it separately. It can be dropped from the PR if so.

#7 Improve high-DPI / large-text-size layout across the UI

File	+	−
CNC Controls Camera/ … /ConfigControl.xaml	3	3
CNC Controls Lathe/ … /ConfigControl.xaml	6	6
CNC Controls Probing/ … /ProbingView.xaml	6	6
CNC Controls Probing/ … /ProbingView.xaml.cs	3	1
CNC Controls/ … /BasicConfigControl.xaml	4	4
CNC Controls/ … /CoordValueSetControl.xaml	6	6
CNC Controls/ … /FeedControl.xaml	5	5
CNC Controls/ … /GotoBaseControl.xaml	5	5
CNC Controls/ … /GotoControl.xaml	1	1
CNC Controls/ … /JogBaseControl.xaml	2	2
CNC Controls/ … /LEDControl.xaml	4	5
CNC Controls/ … /NumericField.xaml	6	3
CNC Controls/ … /NumericField.xaml.cs	3	1
CNC Controls/ … /OutlineBaseControl.xaml	8	4
CNC Controls/ … /OutlineControl.xaml	1	1
CNC Controls/ … /OutlineFlyout.xaml	1	1
CNC Controls/ … /OverrideControl.xaml	3	3
CNC Controls/ … /SpindleControl.xaml	8	8
CNC Controls/ … /StatusControl.xaml	3	3
CNC Controls/ … /StepperCalibrationWizard.xaml	2	2
CNC Controls/ … /StripGCodeConfigControl.xaml	15	9
CNC Controls/ … /ToolView.xaml	2	2
CNC Controls/ … /TrinamicView.xaml	1	1
CNC Controls/ … /WorkParametersControl.xaml	18	18
ioSender XL/ioSender XL/JobView.xaml	6	6
ioSender/ioSender/JobView.xaml	1	1

Makes the UI usable at 125–150% Windows text scaling by replacing fixed control heights/widths with auto-sizing plus `MinWidth/MinHeight`, so labels and fields grow with the font instead of clipping or overlapping. Changes are layout-only — no behavioural changes.

- **Shared controls:** the `NumericField` label column now auto-sizes (`MinWidth = ColonAt`) with a `SharedSizeGroup` so labels align in any container that sets `Grid.IsSharedSizeScope`. `CoordValueSetControl`, `StatusControl`, `LEDControl` (dropped a hard `MaxHeight`) and `OverrideControl` no longer pin `Height/Width`; the **Spindle** group switches its fixed `Width=250` to `MinWidth` so the RPM field is no longer clipped at larger text sizes.
- **grbl:Settings config controls** (Basic / StripGCode / Lathe / Camera) and both Stepper calibration tabs auto-size their rows and the Axis dropdown; the Probing view auto-sizes its left panel with shared-size label alignment.
- **Main window** (JobView XL + ioSender): right-panel grid / columns / panels auto-size instead of fixed 515 / 250 px, and each panel control (WorkParameters, Spindle, Coolant, Outline, Feed, Goto) auto-sizes.

#8 Native Xbox controller support (+ Key Mappings editor polish)

File	+	−
CNC Core/CNC Core/XInput.cs	160	0
CNC Core/CNC Core/ControllerService.cs	186	0
CNC Core/CNC Core/ControllerMapper.cs	408	0
CNC Core/CNC Core/CNC Core.csproj	9	0
CNC Core/CNC Core/GrblViewModel.cs	16	0
CNC Core/CNC Core/KeypressHandler.cs	23	0
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	5	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	222	79
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	958	606
.gitattributes	3	2
ioSender XL/ioSender XL/App.config	6	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	259	0

Adds first-class game-controller support with **zero new dependencies** — the project uses no NuGet, so the controller is read via native **XInput** P/Invoke (`xinput1_4.dll`, falling back to `xinput9_1_0.dll`). A ~60 Hz `DispatcherTimer` poll layer runs in parallel to the keyboard handler.

- **Interop / service:** `XInput.cs` wraps the gamepad state and vibration structs; `ControllerService.cs` scans all four slots, edge-detects button down/up, and exposes Connected/Disconnected events, deadzone/normalize helpers and rumble.
- **Button map:** `ControllerMapper.cs` dispatches a default, fully re-mappable button map to machine actions (A = Cycle Start, B = Feed Hold, X = Spindle Stop, Y = Home, Back = Reset, Start = Unlock, D-pad = step-jog X/Y). Actions route through the same `GrblViewModel.ExecuteCommand` path as the on-screen buttons and are gated to a live connection in Idle/Jog/Tool states (with a status message when ignored). A short rumble confirms connection.
- **Analog jog:** the left stick proportionally jogs X/Y and the triggers jog Z, via throttled overlapping `$J=` moves (jog-cancel on release) so motion stays smooth; speed scales with stick deflection. Enable, left-stick deadzone, max-feed multiple of the Jog panel feed, and per-axis invert are user-configurable. The button map and analog settings persist to `ControllerMap.xml`.
- **Controller tab** in the Key Mappings editor: live press indicator, per-button action dropdowns (each button's default marked with a leading `*`), *Restore Defaults* (enabled only when modified), machine-direction-aware tooltips, and the analog-jog settings — machine dispatch is paused while the dialog is open. New `JogDistanceProvider`/`JogFeedProvider` hooks let controller jog match the on-screen Jog panel's current step and feed.

The same branch also carries the keyboard-tab polish (changed-binding highlight with per-row/group reset, *Clear* moved into the row menu, column sizing, and the grouped outline now keeps a section expanded while scrolling instead of collapsing it). The branch is fully LF-normalized, so the counts above are the real diff — no line-ending noise.

#9 Configurable main page + flyouts + jog panels (ioSender XL)

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	141	20
CNC Controls/CNC Controls/AppConfigView.xaml	3	1
CNC Controls/CNC Controls/AppConfigView.xaml.cs	168	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	2	0
CNC Controls/CNC Controls/CNC Controls.csproj	65	0
CNC Controls/CNC Controls/Converters.cs	17	0
CNC Controls/CNC Controls/JogBaseControl.xaml	2	2
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	157	2
CNC Controls/CNC Controls/JogConfigControl.xaml	7	7
CNC Controls/CNC Controls/JogSlider.xaml	66	0
CNC Controls/CNC Controls/JogSlider.xaml.cs	51	0
CNC Controls/CNC Controls/JogUiConfigControl.xaml	18	9
CNC Controls/CNC Controls/KbdJogGridControl.xaml	46	0
CNC Controls/CNC Controls/KbdJogGridControl.xaml.cs	64	0
CNC Controls/CNC Controls/KeyboardJoggingControl.xaml	25	0
CNC Controls/CNC Controls/KeyboardJoggingControl.xaml.cs	34	0
CNC Controls/CNC Controls/LimitsControl.xaml	6	24
CNC Controls/CNC Controls/LimitsControl.xaml.cs	77	0
CNC Controls/CNC Controls/MachinePositionControl.xaml	69	0
CNC Controls/CNC Controls/MachinePositionControl.xaml.cs	21	0
CNC Controls/CNC Controls/MainPageEditor.xaml	108	0
CNC Controls/CNC Controls/MainPageEditor.xaml.cs	288	0
CNC Controls/CNC Controls/MainPanelRegistry.cs	111	0
CNC Controls/CNC Controls/NumericField.xaml	12	2
CNC Controls/CNC Controls/NumericField.xaml.cs	38	0
CNC Controls/CNC Controls/OffsetFlyout.xaml	41	0
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	133	0
CNC Controls/CNC Controls/PanelFlyout.xaml	43	0
CNC Controls/CNC Controls/PanelFlyout.xaml.cs	80	0
CNC Controls/CNC Controls/Properties/Settings.Designer.cs	36	0
CNC Controls/CNC Controls/Properties/Settings.settings	9	0
CNC Controls/CNC Controls/SidebarItem.cs	23	4
CNC Controls/CNC Controls/TabRegistry.cs	42	0
CNC Controls/CNC Controls/ToggleControl.xaml	20	2
CNC Controls/CNC Controls/UIJogGridControl.xaml	65	0
CNC Controls/CNC Controls/UIJogGridControl.xaml.cs	80	0
CNC Controls/CNC Controls/UIJoggingControl.xaml	44	0
CNC Controls/CNC Controls/UIJoggingControl.xaml.cs	29	0
CNC Core/CNC Core/KeypressHandler.cs	22	4
CNC Core/CNC Core/SerialStream.cs	1	1
ioSender XL/ioSender XL/JobView.xaml	23	14

`ioSender XL/ioSender XL/JobView.xaml.cs`	100	19
`ioSender XL/ioSender XL/MainWindow.xaml`	3	3
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	123	5
`Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)`	322	0

ioSender XL only. Makes the crowded right-hand panel area configurable instead of fixed. A new **Edit Main Page** dialog (beside *Edit Key Bindings*) is a shuttle editor with three buckets — *Available*, *Main page* (up to six slots), and *Flyouts* — so each named panel can be placed on the main page or as a pinnable right-edge flyout, or left off entirely to reclaim space. Nothing is auto-assigned.

- **Flyouts** share a flush, tab-height border and distribute their content vertically; each has a pin (stays open when another flyout opens, and persists across launches) and a close button.
- **Per-offset flyouts** (G28 / G30 / G54..G59.3 / G92) are tiny *Go* buttons (tooltip shows the live coordinates; G28/G30 also get a *Set* = `G28.1`/`G30.1`) so several can be pinned beside the jog panel.
- **Jog panels** — UI jogging (distance + speed) and keyboard default-speed — become assignable panels rendered as compact sliders with a read-only value; Settings:App stays the editor. The jog-arrow radios are hidden only when the UI jog panel is assigned.
- **Settings:App** gains opt-in autosave (with a discard prompt) and a *Reset to default* context-menu entry on numeric fields; the old *Edit Key Mappings* button is renamed **Edit Key Bindings**.

Layout choices persist (panels / flyouts / pins serialized as CSV to sidestep the `XmlSerializer` list-append quirk). Also includes robustness fixes: a resilient `SerialStream.OutCount` that no longer throws on a dropped port, and a deferred main-panel build so startup can't race the config load.

Extended: a fourth bucket places panels *left* of the 3D view (DRO, Program limits, Machine Position now in the registry) in a scroll region that never overlaps the fixed Signals/Status block. Per-offset *Set* now also handles G54–G59.3 (captures the current machine position as that coordinate system's origin via `G10 L2 P<n>`). The UI jog panel renders as a 2×4 distance/speed grid with green-click selection, a *Continuous* toggle and a *Remember distance & speed across restarts* option; *Link distance and speed to UI jogging* makes keyboard jogging follow the UI distance *and* speed (hiding the Keyboard Jogging panel). Coolant toggles render as a self-contained red/green pill (no theme dependency). A new **Tabs** tab in the editor reorders / hides the main TabControl tabs (Settings:App protected), applied on startup via `TabRegistry` / `Config.Tabs`. The *Restart* button moved to Settings:App (next to *Edit Main Page*), enabled only after a layout/tab change.

Panel refinements: the Machine Position panel shows MPos as a live `work + WorkPositionOffset` sum (tracking the DRO notifications); the limits panel stays visible and shows the machine soft-limit envelope from `$13x`/`$23` when no program is loaded (program bounding box when one is), retitling itself *Machine limits* / *Program limits* accordingly; the 2×4 jog grid keeps its green selection highlight using the retained discrete index (survives keyboard/continuous-mode flips); and the **Tabs** editor now also lists tabs that host an `IGrblConfigTab` (no `ViewType`) by falling back to the `TabItem` name.

#10 Macro manager + run-only flyout

File	+	−
CNC Controls/CNC Controls/MacroManagerDialog.xaml.cs	316	0
CNC Controls/CNC Controls/MacroManagerDialog.xaml	81	0
CNC Controls/CNC Controls/MacroExecuteControl.xaml.cs	19	8
CNC Controls/CNC Controls/AppConfig.cs	19	0
CNC Controls/CNC Controls/AppConfigView.xaml.cs	10	0
CNC Controls/CNC Controls/MacroExecuteControl.xaml	8	16
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Core/CNC Core/GCode.cs	5	0
CNC Controls/CNC Controls/JobControl.xaml.cs	3	2
CNC Controls/CNC Controls/MacroToolbarControl.xaml.cs	3	0
CNC Controls/CNC Controls/AppConfigView.xaml	1	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	126	0

Adds an **Edit Macros** button to the Settings:App page opening a macro manager — a DataGrid with one row per macro, in the spirit of Fusion's *Scripts and Add-Ins* dialog — and reworks the on-screen macro flyout for running.

- **Name** and a **Prompt** (confirm-before-run) flag are edited in-line; an assignable **Key** column (F1–F12) sets the function key (keys are unique; *Create* auto-assigns the first free one); a **File** column shows the referenced file for `@<path>` macros (full path on hover), blank for inline.
- **Edit** edits the macro's definition (inline G-code, or the `@<path>` reference line) and reads it back; **View** opens what it points to — the referenced file for an `@<path>` macro (created if missing, to author a new one), otherwise the code. **Create**/**Delete** add and remove rows.
- The **macro flyout** is now run-only: a macro-name dropdown plus a **Run** button (the Edit button is gone — editing lives in the manager). It pre-selects the most recently run macro, persisted in `Config.LastMacroId` and updated from every run path (flyout, toolbar, F-keys), falling back to the first.

The macro's function key is decoupled from its `Id` via a new `Macro.FKey` field (legacy configs migrated on load: `Id n` keeps `Fn`). The `@<path>` *run-time* resolution — loading and running the referenced file, with directive processing — is provided by **#11**; this PR adds the manager/flyout support for authoring, viewing, and repointing references.

#11 Macro Processor Enhancements

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	582	0
CNC Controls/CNC Controls/MacroToolbarControl.xaml.cs	1	1
CNC Controls/CNC Controls/MacroExecuteControl.xaml.cs	1	1
CNC Controls/CNC Controls/JobControl.xaml.cs	1	2
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
CNC Core/CNC Core/GCodeParser.cs	31	12
CNC Core/CNC Core/NGCExpr.cs	22	0
CNC Core/CNC Core/Grbl.cs	11	0
CNC Core/CNC Core/GrblViewModel.cs	9	2

A sender-side **macro pre-processor** (`MacroProcessor.Run`) that interprets parenthesised directives before the remaining lines are streamed to the controller. All macro entry points — the top toolbar, the on-screen macro flyout, and the F-key handler — now route through it (previously the toolbar called `ExecuteMacro` directly and silently ignored every directive).

Directives, each a no-op the controller would treat as a comment, interpreted by the sender instead:

- **`(PREREQ, cond, …)`** — require machine state *before* anything streams; if unmet the macro aborts with a message and nothing is sent. Conditions: `homed`, `idle`, `tlo`, `noalarm`, `connected`; stored positions / work offsets `g28`, `g30`, `g92`, `g54`–`g59.3` ("set" if any axis offset is non-zero, read fresh from `$#`); `nog92`, the inverse of `g92` — refuse when one IS set, reported with the offending offset, and never cleared automatically (`G92.1` would move every remaining cut of a job working under it); and any other token is required to be a controller `$I` build option (`NEWOPT`, e.g. `EXPR`), matched exactly and case-sensitively. `homed` reads grblHAL's live `sys.homed.mask` from `$#` rather than the change-based status `H:` cache, which can stay stale after a position-loss alarm.
- **`(MBOX, [buttons,] message)`** — modal hold; `OK`/`OKCANCEL`/`YESNO`, Cancel/No aborts the rest.
- **`(WAITIDLE)`** — hold streaming until the controller finishes what was sent and returns to Idle — needed after a controller-side job such as `$F=<file>`, which acks immediately and drops sender input while it runs.
- **`(PROMPT param, default [, label])`** — collect input values in a single up-front dialog and bind them two ways: assign the globals on the controller (so `$F=` files can read them) and substitute the references in the streamed body.
- **`@<path>`** — a macro whose body is a single `@<path>` line is a reference to an external file: that file's current contents are loaded and run in its place, re-read on every run — so a macro can be developed by editing the file directly. The loaded contents go through the same directive pipeline. (#10 adds the manager/flyout support for authoring and repointing these references.)

Supporting core changes so macros stream cleanly to an NGC-capable controller, all in the spirit of "don't reject what the controller can evaluate": `ExecuteMacro` now inherits the controller's expression support (as file streaming already did) and forwards verbatim any line its own parser cannot handle; `GCodeParser` recognises **named O-word subroutine calls** (`O<name> CALL […]`) and forwards them to the controller, which resolves the named sub from its filesystem; and the `$# [HOME:…:mask]` homed-axes mask is parsed for the authoritative `homed` check.

#12 Onboarding tooltips

File	+	−
CNC Controls/CNC Controls/Converters.cs	50	0
CNC Controls/CNC Controls/StatusControl.xaml	3	0
CNC Controls/CNC Controls/StripGCodeConfigControl.xaml	10	5
CNC Controls/CNC Controls/JogUiConfigControl.xaml	10	9
CNC Controls/CNC Controls/JogConfigControl.xaml	7	7
CNC Controls/CNC Controls/BasicConfigControl.xaml	3	3
CNC Controls/CNC Controls/FileActionControl.xaml	4	4
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	196	0

Tooltips aimed at users new to CNC, where ioSender's UI is otherwise terse.

- **State field** — a context-aware tooltip explains the current controller state in plain language (Idle, Run, Hold, Check, Tool, Door, …). For an **alarm** it appends the controller's specific message — e.g. `ALARM 4: Probe fail …`, pulled from the `GrblAlarms` enumeration loaded at connect — plus how to recover, so a bare `ALARM:4` is self-explanatory instead of a code to look up. (A new `GrblStateToTooltipConverter` drives a `ToolTip` binding on the State box.)
- **Settings:App** — a “what it does and why it’s useful” tooltip on each application setting (the grbl settings page already self-documents in its right-hand pane, so it’s left alone). Keyboard jogging (fast/slow/step feed & distance, link-step-to-UI), UI jogging (the four distance and four speed presets, noting distance and speed are selected *independently*), and GCode command stripping (M6/M7/M8/G61-G64, for machines lacking the matching hardware). Also fixes the “Keep MDI focus” tooltip (it had been copy-pasted from the buffering option) and the circular “Send comments” one.

#13 Build documentation: Mega-XL upgrade notes + repeatable tool-change guide

File	+	−
docs/Mega-XL-Upgrades.html	840	0
docs/Mega-XL-Upgrades.pdf	bin	0
docs/Repeatable-Tool-Change.html	419	0
docs/Repeatable-Tool-Change.pdf	bin	0

Two self-contained HTML documents (each rendered to a companion PDF), added under a new `docs/` folder. Purely additive; touches no source.

- **Repeatable-Tool-Change** — a turnkey, novice-friendly tool-change guide built around a **G59.3 fixed toolsetter** and a `T<n>/M6` flow, with a top-down machine travel diagram (color-coded paths). Aimed at the goal of repeatable, hands-off tool changes for users new to CNC.
- **Mega-XL-Upgrades** — build notes for an upgraded Kickstarter v1.0 Mega V XL: frame/spindle/motion/probing/power sections, a back-of-enclosure two-panel harness model (hand-built SVG diagrams), a cable & wiring schedule, and an **interactive bill-of-materials worksheet** — an in-page editor (double-click to edit Qty / Unit / Description / Source, live extended & category subtotals & grand total, URL shortening, Save-to-download) plus a linked table of contents.

The Mega-XL notes are specific to one machine and may be of limited upstream interest — they can be hosted separately or dropped from the PR. The repeatable tool-change guide is more generally applicable. The associated controller macros (`tc.macro`, `probe_tfl.macro`) and the ATC provisioning UI are intentionally *not* in this PR — they remain with the macro work, pending firmware support.

#14 SD + LittleFS file browser & file manager on the SD Card tab

File	+	−
CNC Controls/CNC Controls/SDCardView.xaml.cs	505	111
CNC Controls/CNC Controls/GrblFilesystems.cs	142	0
tests/GrblFilesystemsTests.cs	83	0
CNC Core/CNC Core/TelnetStream.cs	54	14
tests/README.md	22	0
CNC Controls/CNC Controls/SDCardView.xaml	19	6
CNC Core/CNC Core/YModem.cs	14	4
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	105	0

The SD Card tab listed only one filesystem (the SD card, or whatever was the current filesystem). On grblHAL builds that have both an SD card and a LittleFS store, the other filesystem — and any macros on it — were invisible.

This enumerates every mounted filesystem via `$FI` (`[FS:<name>@<path> size, free]`) and lists them together in one grid with a new **Location** column, plus a per-filesystem **free-space banner** above the list. The right-click menu is reworked from *Run* / *Download and run* / *Delete* into a small **file manager**, and file operations target the file's own filesystem — a bare name on the root volume (unchanged single-SD behaviour) or an absolute path on a sub-mount such as `/littlefs`.

- **File manager menu:** *View* / *Edit* / *Create* (new content, or from a local file) open the file in Notepad — Edit and Create write the result back to the controller when Notepad closes; *Load* and *Load and run* bring the program into ioSender; *Copy to* / *Move to* transfer a file to another mounted filesystem (submenus built live from the mounts); *Delete*. Actions are enabled only for a real selected file.
- **Empty filesystems are visible:** a mount with no files gets a placeholder row so it can still be selected as an upload / copy target; the listing's re-entrant `Load()` is guarded (it pumps the dispatcher while waiting on the controller, so an overlapping refresh could corrupt the shared table).
- **Correct attribution under nested mounts:** a root `$F` recurses into sub-mounts, so an SD-root listing also returned the `/littlefs/*` files; each result is now attributed to its deepest containing mount, so files are not duplicated under (and mis-pathed on) the parent filesystem.
- **Upload to any filesystem:** `UploadLocalFile` targets a chosen filesystem via `$CWD` (the FTP path is built from the explicit destination, not the possibly-stale `FsCwd`), and upload is allowed on a LittleFS-only controller (no SD card present, so no SD mount status to require).
- **Upload reliability:** `TelnetStream` now writes *synchronously* — overlapped `WriteAsync` calls silently dropped a YModem block's payload/CRC — and `PurgeQueue` clears the async-read buffer instead of racing a second synchronous read on the same stream (which lost per-block ACKs, producing 0-byte files and multi-minute uploads); plus a bounded connect timeout so a reconnect retries quickly. YModem per-block ACK timeouts are raised (10 s open / 5 s block) to ride out flash-filesystem GC stalls, and blocks are sent in **128-byte (SOH)** packets rather than 1024-byte (STX): grblHAL's YModem receive ring is 1024 bytes (usable 1023), so a full 1029-byte STX packet can't fit — over telnet it arrives in a TCP burst faster than the firmware drains it and overflows, stalling the transfer (a one-block file uploaded, larger ones hung). 128-byte blocks fit with margin on any transport.
- **Backward compatible:** builds that don't implement `$FI` (or report nothing mounted, `error:65`) return no `[FS:…]` lines, and the view falls back to the original single-filesystem mount + `$F` listing untouched. An SD card with files in its root works exactly as before, now also showing free space.

- **New `GrblFilesystems.cs`** holds pure, dependency-free helpers (mount-line parser, human-readable size, path qualification, free-space summary) and the `FsMount` model; the live `$FI`/`$F` querying lives in `GrblSDCard`.
- **Unit tested:** `tests/GrblFilesystemsTests.cs` exercises the pure helpers (29 assertions, run via the Framework `csc` — see `tests/README.md`; the app has no test project).

Builds clean (CNC Controls and the full ioSender solution). Also the groundwork (and the reliable LittleFS upload path) that the ATC macro-provisioning work reuses to target the `$FI`-discovered LittleFS path instead of a hardcoded one.

#15 USB / Ethernet / simulator connection support + Connect menu

File	+	−
CNC Controls/CNC Controls/SimulatorManager.cs	474	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	208	14
CNC Controls/CNC Controls/AppConfig.cs	165	28
CNC Controls/CNC Controls/BasicConfigControl.xaml	1	0
CNC Controls/CNC Controls/PortDialog.xaml.cs	56	2
CNC Controls/CNC Controls/PortDialog.xaml	43	8
ioSender XL/ioSender XL/MainWindow.xaml	39	2
CNC Controls/CNC Controls/GrblConfigControl.xaml.cs	22	0
simulator/README.md	18	0
CNC Core/CNC Core/TelnetStream.cs	14	3
CNC Controls/CNC Controls/JobControl.xaml.cs	14	2
CNC Controls/CNC Controls/UIUtils.cs	12	4
ioSender XL/ioSender XL/ioSender XL.csproj	12	0
CNC Core/CNC Core/HelperClasses.cs	8	2
ioSender XL/ioSender XL/JobView.xaml.cs	18	2
CNC Core/CNC Core/GrblViewModel.cs	7	0
readme.md	2	0
CNC Controls/CNC Controls/CNC Controls.csproj	2	0
CNC Controls/CNC Controls/Widget.cs	1	1
.gitignore	5	0
CNC Controls/CNC Controls/GrblConfigControl.xaml	1	0
simulator/sim-build.json	1	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	28	0

Adds first-class connection setup to the sender: pick USB (serial), Ethernet (`host:port`, now accepting hostnames as well as IPv4), or a local grblHAL **simulator** from a tab in the connection dialog, and (re)connect at any time from a menu. *(Updated 2026-06-20: the simulator is now run standalone and connected to over a port; the launch/download/My-Machine dialog was stripped out, and connection-loss + comment-forwarding fixes were folded in.)*

- **Simulator tab — connect-only:** the simulator now runs *standalone* — the user launches `grblHAL_sim.exe` directly (it opens its own 3D view + config window and listens on a TCP port), and ioSender simply connects to `127.0.0.1:<port>`. The tab is reduced to a single *Port* field ("Launch grblHAL_sim.exe first, then connect"); ioSender no longer launches, downloads or manages the simulator process.
- **Connect / Reconnect:** a `File > Connect…` item opens the dialog; while connected it becomes *Reconnect…* (disconnects, then re-opens the dialog so a different target can be chosen). The app now stays open when started without a connection instead of exiting, so the menu is reachable.
- **Remembered network host:** the dialog's network tab defaults to the most recent IP that connected successfully (persisted as `Config.NetworkHost`) rather than always the mDNS name `grblHAL.local`. It starts empty; on a serial connection it is seeded from the controller's `$I`-reported IP if still unset — so after one USB session a later reconnect already offers the controller's real address. Empty falls back to `grblHAL.local`; the bundled simulator (`127.0.0.1`) is excluded.

- **Prefer network connection (opt-in):** a Settings:App checkbox (`Config.PreferNetwork`, default off). After a serial/USB connection whose `$I` reports an IP, ioSender probes `<ip>:23` and, if it answers, automatically switches the live link to the network — status line: *"Connection migrated to network (ip:23)"* — falling back to the serial port if the network connect fails. The probe runs off the UI thread; the switch is deferred and guarded against re-entry, and only fires from a serial link with a known IP.
- **Target status & cues:** the status bar shows the current connection target (`GrblViewModel.ConnectionTarget`), green when connected and red when not. The connect dialog opens on the tab matching the active target (Serial / Network / Simulator, disambiguated via the `StartSimulator` flag) and green-bolds that header; the XL window background tints pale yellow while connected to the **simulator** (restored to grey on a real target or when disconnected), so a virtual machine is unmistakable.
- **"Not connected" on a dropped socket:** when the link drops (e.g. the simulator window is closed) the reconnect loop keeps the target name set, so the target box would otherwise still read connected. An `IsConnectionLost` trigger now turns the XL target box red and shows *Not connected* until the connection is re-established.
- **Stop the job on disconnect:** if the controller connection drops *during a running job*, the **client-side** stream is torn down (job state reset to idle) so the UI no longer hangs waiting on `ok` responses that will never arrive — previously this was handled cleanly only while idle. This is a Stop, not a feed-hold, and because the link is already gone it cannot command the controller; what the machine does physically on comms loss is up to the firmware.
- **Always send comments to the simulator:** g-code comments are forwarded to the simulator regardless of the app's *Send comments* setting — the simulator reads them for tool/stock metadata (`(TOOL …)`, `(STOCK …)`) that drives its 3D view and material-removal carve.
- **Startup ordering:** `AppConfig.SetupAndOpen` is split into `LoadConfig` (run synchronously at construction, so config is available before any control's `Loaded` handler reads it) and `OpenConnection` (deferred to `ApplicationIdle` so the main window paints before the — possibly blocking — connection dialog). Transport is chosen by target string (`COMx` vs `host:port`) rather than a leading-digit IPv4 check.
- **Copy current settings to the simulator:** a *Copy to simulator* button on the grbl:Settings tab (next to Save) mirrors the live controller settings into the bundled simulator's "My Machine" EEPROM via `SimulatorManager.BuildMyMachineEeprom`, so a later simulator connection boots with this machine's configuration.

Builds clean (CNC Controls and the full ioSender XL + classic solutions) off `master`. The "My Machine" EEPROM builder is now wired to the *Copy to simulator* button (above); the `SimulatorManager` launch / web-builder-download machinery is no longer wired to the (now connect-only) dialog — a candidate for removal in a follow-up. No simulator binary is committed; see `simulator/README.md`.

#16 Validate controller (check-mode g-code conformance test)

File	+	−
CNC Controls/CNC Controls/ValidateProcessor.cs	1091	0
CNC GCodeViewer/CNC GCodeViewer/Renderer.xaml.cs	81	16
ioSender XL/ioSender XL/MainWindow.xaml.cs	3	21
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
CNC Controls/CNC Controls/LibStrings.xaml	3	0
ioSender XL/ioSender XL/MainWindow.xaml	1	1
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	21	0
Locale/*/csv/ioSender.resources.*.csv (7 locales)	7	0

Replaces the placeholder *Validate controller* action (#15) with a real, in-app conformance test: it streams a g-code program tailored to the connected controller's reported capabilities and reports which commands it accepts — without moving the machine.

- **Capability-tailored test:** generated from `$I` build options, axis count, `$32` mode, tool count, aux-I/O counts and the work coordinate systems present, so a feature the firmware was never built with is not tested (and cannot be reported as a false failure).
- **Check mode, no motion:** streamed in `$C` check mode and parsed only — the machine never moves, so homing and a clear envelope are not required. One feature per line, so an `error:N` pinpoints exactly which command the controller rejected.
- **Robust against grbl's check-mode behaviour:** a check-mode error *latches* the parser (every later line then errors), so the run recovers on each error — reset, re-enter check mode (unlocking first if the reset re-alarmed a homed machine), re-apply the modal set-up — then resumes, always re-confirming check mode before sending another line so a test line can never reach the controller as real motion.
- **Non-destructive:** check mode *commits* `G10/G92` writes, so the run snapshots and restores the work offsets / tool table and avoids the un-restorable writes (`G28.1/G30.1`, and `G92` when an offset is active). Machine settings (`$$`) are never written.
- **Progress & results:** a small non-modal panel shows a live pass/fail tally and the current test as it streams; on completion a *View Summary* button opens the full report (grouped by category, error code/message per failure, the pre-run parameter snapshot, Copy-to-clipboard). The controller is left in `Idle` with check mode off.
- **3D visualization:** the passed bounded-motion features are assembled into a runnable program (absolute `G90` moves at a fast feed) and loaded into the buffer, so the user can press Cycle Start and watch the toolpath run in the 3D view via the normal job path. The viewer also gains an *always-active machine scene* (work envelope, axes and a live tool cone shown when homed even with no program loaded), plus fixes to the tool/executed-trail animation subscription and a bounded executed-trail advance (an out-of-range block was an unbounded spin).

Right-click the **Target** status item → *Validate controller*. Builds clean; the only failures on a stock grblHAL are commands it genuinely does not implement (e.g. `G90.1`, `G61.1`, `G64`, `M52`; `M56` needs parking-override enabled).

#17 Restore main window position across launches

File	+	−
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	58	7
`CNC Controls/CNC Controls/AppConfig.cs`	2	0

The "Restore last window size on startup" option restored only the window *size*; the position was discarded (clamped to the top-left). This persists the position too.

- **Position persisted:** new `WindowLeft`/`WindowTop` settings, saved on close (using `RestoreBounds` when maximized so the un-maximized rectangle is stored) and restored on startup — but only when the saved rectangle lands on a currently-connected monitor (checked against the whole virtual desktop), so a window saved on a since-removed screen can't open off-screen.
- **Takes effect immediately:** the live "save" flag was only set at startup, so toggling the option mid-session did nothing until a restart. It now updates live and captures + persists the current placement the instant the option is enabled.

Builds clean. The size/position restore builds on the connection support's `CompleteStartup`; the underlying `KeepWindowSize` option is from the ioSender baseline.

#18 grbl:Settings — live edits, change highlight, revert & snapshots

File	+	−
CNC Core/CNC Core/Grbl.cs	356	15
CNC Controls/CNC Controls/GrblConfigControl.xaml	49	6
CNC Controls/CNC Controls/GrblConfigControl.xaml.cs	39	8
CNC Controls/CNC Controls/Widget.cs	25	14
CNC Controls/CNC Controls/RestorePointDialog.xaml	41	0
CNC Controls/CNC Controls/RestorePointDialog.xaml.cs	65	0
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	77	0

Orca/PrusaSlicer-style indication of which controller $ settings you have changed, on the grbl:Settings **Basic** tab, plus live editing feedback, granular revert and automatic per-Save restore points. Applies to both the grblHAL settings tree and the classic-grbl grid.

- **Live edits in the left-hand list:** edits stage in a pending-changes map (`GrblSettings.PendingEdits`) that the list reads via a new `EditValue`, so the value and its highlight update *immediately* without mutating the controller value. Checkboxes/radios commit on change; text/numeric editors commit on lost focus. Pending edits are flushed to `Value` and cleared on Save; discarded on Reload/Restore.
- **Two highlight states:** **orange/bold** = unsaved edit (`IsEdited`); blue = saved to the controller this session but still different from the session-start value (`IsSavedModified`). Groups roll up the same way.
- **Revert from a right-click menu:** *Revert to value at startup* on a setting, and *Revert all in group to value at startup* on a group node (staged as a pending edit, written on next Save).
- **Snapshot on Save + Restore picker:** every Save writes a timestamped restore point (full settings dump in the Backup format, led by a `; changes (N)`: old→new comment block), per controller identity, keeping the last 20. The **Restore** button opens a picker listing restore points newest-first with a changed-count column and a per-row tooltip showing exactly what each Save changed; *Browse…* falls back to any backup file.
- **Baselines, cleanly separated:** `GrblSettingDetails` tracks the current value, the frozen *startup* value (highlight) and the *loaded* controller value (drives `IsDirty`). Making `IsDirty` mean "differs from the controller" rather than "changed" fixes a spurious "settings changed, save now?" prompt when reverting an edit back to the controller value.
- **Bitfield flag tooltip:** `BITFIELD`/`XBITFIELD` settings still display numerically, but hovering the value shows the set flags decoded to their labels (one per line, `N/A` entries skipped) via `GrblSettingDetails.BitfieldLabels`.

Builds clean off `master` (CNC Controls and the full ioSender solution); verified against the bundled grblHAL simulator. Per-setting "revert to factory default" is intentionally deferred — grblHAL exposes no per-setting default (only the global $RST=$ reset-all), so factory revert would need a future firmware-side snapshot of as-flashed values.

#19 Console — inline terminal-style command prompt

File	+	−
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	117	0
CNC Controls/CNC Controls/ConsoleControl.xaml	28	4
CNC Core/CNC Core/GrblViewModel.cs	20	0
CNC Core/CNC Core/KeypressHandler.cs	7	0
CNC Controls/CNC Controls/MDIControl.xaml.cs	5	16
CNC Controls/CNC Controls/MDIControl.xaml	1	1
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	42	0

Makes the **Console** tab read like a traditional terminal by adding a command input line at the bottom, while keeping the global single-line MDI strip for the other tabs (3D View, Program). A natural answer to "why is MDI a separate strip instead of a console prompt?" — now it can be both.

- **Reuses the MDI plumbing:** Enter routes the line through the existing `MDICommand`, which already echoes the command into the console's `ResponseLog` — so the typed command and its response read inline like a shell. No `JobView` changes.
- **Shared command history:** a single `GrblViewModel.MDIHistory` (newest first) is fed by both the MDI strip and the console prompt and surfaced in both — the strip's dropdown binds to it and the prompt recalls it with Up/Down (Esc clears). A command entered in either place is recallable in the other. (The view-model `CommandLog` is not populated for these entries, so it is not used.)
- **Same guards, auto-focus:** the prompt is locked out while a job runs (same as the MDI strip) and grabs focus when the console becomes visible (tab switch or the pop-out console window) so it's ready to type into.
- **Pop-out for free:** the prompt is self-contained in `ConsoleControl`, so the detachable console window (which hosts the same control) gets it too.
- **Output context menu:** right-click the response log for *Clear all*, *Clear up to here* (drops the clicked line and everything above it), and *Save…* (writes the in-memory log to a text file).
- **No jog conflict:** the keyboard-jog handler jogs whenever a key is *mapped* to jog (Up/Down are by default) *before* its per-key text-box guard, so the prompt's arrows would otherwise move the machine. `ProcessKeypress` now forces no-jog when the focused element is a `TextBox` tagged `"MDI"`, and the prompt is tagged accordingly — uniformly exempting both the MDI strip's edit box and the console prompt.

Builds clean off `master` (CNC Controls and the full ioSender solution); verified against the bundled grblHAL simulator: commands echo + execute, auto-focus, history recall (Up/Down step through prior commands), and an A/B test confirming Up jogs the DRO when a non-input element is focused but does not when the console prompt is focused. Keys are handled in `PreviewKeyDown` so the arrows reach the prompt rather than being consumed by the TextBox.

#20 3D view — Ctrl+click to jog + per-axis orientation

File	+	−
CNC GCodeViewer/CNC GCodeViewer/Renderer.xaml.cs	164	52
CNC Core/CNC Core/Grbl.cs	26	1
CNC GCodeViewer/CNC GCodeViewer/RenderControl.xaml	11	7
CNC Core/CNC Core/GCodeEmulator.cs	3	0
CNC Controls/CNC Controls/AppConfig.cs	2	0

Ctrl+left-click in the 3D or top-down (XY) view jogs the machine to the clicked XY position — a quick coarse-positioning aid that complements the jog buttons (park over a corner to set work zero, check alignment, touch-screen rigs).

- **Safe by construction:** the move always retracts **Z to the top of travel first**, then rapids to the new XY, so the traverse can never plough through stock or fixtures. Only X/Y move; plain left-drag still rotates the camera.
- **How it maps:** the click ray is intersected with the work Z=0 plane (HelixToolkit `UnProject`) to recover an XY target, converted to machine coordinates (`MPos = WPos + WCO`), clamped to the soft-limit travel exactly as the jog limiter does, and emitted as two queued `$J=G53` jogs at the controller's max rate (`$110`–`$112`) so it moves at rapid speed.
- **Guarded:** requires homed + idle + max travel set + soft limits (`$20`) on, with a status-bar reason when a prerequisite is missing. Gated by a `GCodeViewerConfig.ClickToJog` setting (default on).
- **Orientation fix (prerequisite):** the rendered machine frame — work envelope, floor grid and crosshair — now honors the `$23` homing-direction mask + force-set-origin *per axis*, so the view matches any home corner (e.g. top-back-left, X+/Y−/Z−) instead of assuming +X/+Y. Standard +X/+Y/−Z machines render identically to before. `GrblInfo.ForceSetOrigin` now reads the live `$22` bit 3 rather than the `$I OPT` 'Z' flag (absent on some builds, e.g. the simulator), so the orientation is correct even when the option string omits it. The camera bounding box in the always-on machine scene is derived from the same per-axis envelope, fixing a mirrored home corner in the 3D view.
- **Toolbar tidy-up:** the 3D-view toolbar tooltips now show on disabled buttons and are reworded for consistency, the *save view* button is no longer needlessly gated, and the *show job envelope* button enables only with a file loaded. A null-tokens guard in `GCodeEmulator` avoids a crash when the machine scene renders with no program loaded.

Builds clean off `master` (CNC GCodeViewer chain). Verified on the author's machine: Ctrl+click retracts Z then rapids to the clicked XY at the configured max rate, and the envelope/grid orient to the real home corner.

#21 grbl:Settings — Machine Setup Wizard

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	795	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml	278	0
CNC Controls/CNC Controls/MachineCatalog.cs	194	0
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	30	0
CNC Controls/CNC Controls/CNC Controls.csproj	8	0
CNC Controls/CNC Controls/GrblConfigView.xaml	4	0
CNC Controls/CNC Controls/AppConfig.cs	2	0
CNC Controls/CNC Controls/IGrblConfigTab.cs	2	1
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	427	0

A guided, first-run **Machine Setup Wizard** tab in grbl:Settings that configures the machine-description settings a fresh controller needs — so a new machine can be described without hunting through the $ list. Auto-selected when the machine looks unconfigured (no remembered machine, or `$130–$132` all zero).

- **Single page, nothing written until applied:** the whole machine description is laid out on one page — machine selector, a clickable home-position picture, a per-axis table, and the remaining homing & limits questions — so it reads at a glance instead of an eight-step click-through. *Apply* writes the changes via `GrblSettings.Save()`.
- **Apply only when there's something to write:** Apply is disabled while the wizard's values already match the controller; the pending-change set recomputes on every edit (machine pick, field, reload). Numeric settings are compared by value, so a formatting-only difference (`1` vs `1.000`) is not flagged as a change. Hovering Apply lists the exact pending writes (`$id name: old → new`).
- **Out of the way:** sits as the *last* grbl:Settings tab (rarely needed after initial commissioning), so Basic settings is the default tab.
- **Machine catalog:** a built-in catalog (`MachineCatalog.cs`) of common hobby machines as manufacturer → product → model drop-downs (or a custom X×Y), each carrying a sensible home corner and force-set-origin default; the first entry is a generic XYZ machine with limit switches. Picking a model pre-fills the page, which the user then confirms or tweaks.
- **Axis table:** an editable per-axis grid — travel limit (`$130–$132`), max rate (`$110–$112`), steps/mm (`$100–$102`), invert direction (`$3`) and normally-closed limit (`$5`) — edited in place.
- **Home corner:** a clickable isometric box — click the corner where the machine homes — sets the X/Y homing directions (`$23`) and auto-enables force-set-origin (`$22` bit 3) on grblHAL; Z is fixed to the top of the gantry. This is the same convention that orients the 3D view (#20). The axis table also carries a per-axis **Home min ($23)** checkbox kept in sync with the picture, so the homing direction is visible/editable per axis (Z's box is locked to "top").
- **Homing & limits:** hard limits (`$21`) default on, with the remaining global questions as a short list and the explanatory help moved into per-input tooltips. A right-justified *Firmware Info* button shows the live `$I` build output in a non-modal dialog.
- **Remembered & forgettable:** the chosen machine is saved (`AppConfig.LastMachine`) and restored next launch — keeping the live controller values over the catalog preset on restore — and a *Forget machine* button clears it so the wizard reappears next time.
- **Correct write order & honest errors:** grblHAL rejects soft limits (`$20=1`) unless homing is enabled (`error:10`). Apply now enables homing by the user's intent (writing `$22` even on a fresh controller where it is currently off, instead of gating on the live state) and writes soft limits in a second pass — after `$22`

— since `GrblSettings.Save()` writes in ascending id order. Soft limits are only requested when homing will be on. On failure it lists the exact rejected $ settings and the controller's reason rather than a generic “failed to write settings”.

Builds clean off `master` (CNC Controls and the full ioSender solution). UI verified by the author against a live controller and the bundled simulator; interactive `$3` direction-verify and homing-cycle-order editing were intentionally left out of this first cut. The first-run gate that forces the wizard to the foreground until a machine is applied — and its skip-when-connected-to-the-simulator behaviour — depend on connection-side plumbing and ride along in the integration branch rather than this stand-alone PR. The “copy this machine into the simulator’s My Machine profile” action now lives on the grbl:Settings tab (#15).

#22 Keyboard jog — keep active when Job-view focus drifts

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	19	0
ioSender XL/ioSender XL/JobView.xaml.cs	4	1

Keyboard jogging only fired through `JobView.OnPreviewKeyDown`, which needs keyboard focus inside the Job view's tree. Interacting with a flyout, side panel or menu moves focus out, so the arrow keys stopped jogging until the view was re-focused (e.g. by switching tabs). This keeps jog alive on the Job page wherever focus has drifted.

- **Window-level forwarding:** the existing `MainWindow` tunneling `PreviewKeyDown` (always fires) forwards jog keys to `JobView.ProcessKeyPreview` when the Job view is the current view but is not keyboard-focused. The focused case is unchanged — `JobView` still handles it, including its MDI/DRO/spindle/work-params jog gates.
- **Skips text input:** forwarding is suppressed when focus is in any `TextBoxBase`, so typing in the MDI strip or console prompt never jogs.
- **Clean stop:** a mirrored `PreviewKeyUp` forwards the key-up too, so a continuous jog started while the view was unfocused still receives its release and stops (no text-input guard here — an in-progress jog must always cancel).

Builds clean off `master` (the full ioSender solution). Two-file change reusing the console-shortcut `PreviewKeyDown` hook already on `master`; `ProcessKeyPreview` is promoted to `public` so the window can forward into it.

#23 Go To — optional safe-Z (lift, traverse, descend)

File	+	−
`CNC Controls/CNC Controls/GotoBaseControl.xaml.cs`	34	8
`CNC Controls/CNC Controls/AppConfig.cs`	5	0
`CNC Controls/CNC Controls/BasicConfigControl.xaml`	1	0
`Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)`	14	0

A **Go To** a work-coordinate origin (G54–G59.3), `G28` or `G30` was a single coordinated rapid — a straight diagonal in machine space that can drag Z through clamps, fixtures or tall stock on the way. This adds an optional safe-Z decomposition so the traverse always happens clear of anything standing up in Z.

- **Three-phase move:** with *Safe Z on Go To* enabled, each Go To becomes `G53 G0 Z0` (retract to machine top), then `G53 G0 X…Y…` (traverse at the safe height), then `G53 G0 Z…` (descend to the target). Machine `Z0` is the homed top — already top-minus-pull-off with force-set-origin — so it never re-trips the Z limit, and it needs no configured clearance height (it clears anything that physically fits under the gantry).
- **All Go To targets:** applies uniformly to the selected work origin and to G28/G30, read from the same `$#`-populated coordinate table the work-origin button already used (so G28/G30 get true X/Y-then-Z, not a firmware diagonal). Rotary axes travel with X/Y; only Z is separated.
- **Guarded, with a clean fallback:** gated on the *Safe Z on Go To* option (Settings:App → Main, default on), the machine being **homed** (machine coordinates valid) and **soft limits** (`$20`) on so the three rapids are envelope-checked. If any is missing it falls back to the original single coordinated move — no behaviour change for unhomed/soft-limits-off setups.

Builds clean off `master` (both apps). The setting lives in the Base app config so it shows in the always-present Main settings panel regardless of jog mode; grbl's junction deviation (~0.01 mm) means the corners are effectively full stops, so no dwell is needed between phases.

#24 comms — fix telnet/websocket UI deadlock

File	+	−
`CNC Core/CNC Core/TelnetStream.cs`	17	6
`CNC Core/CNC Core/WebsocketStream.cs`	12	4

Over a telnet or websocket connection the UI could freeze whenever a modal message box (e.g. a macro `M0`/MBOX hold) was open while polling continued — a lock-ordering **deadlock** between the read callback and the UI thread.

- **The deadlock:** `ReadComplete` ran a synchronous `Dispatcher.Invoke(DataReceived)` *while holding* `lock(input)`. The UI thread's `PurgeQueue` also takes `lock(input)`, so once the dispatcher was busy (a modal pumping its own message loop) the read callback waited on the UI thread while the UI thread waited on the lock — a classic cyclic wait that hung the app.
- **The fix:** extract the complete replies from the buffer *under* the lock, then release it *before* dispatching them to `DataReceived`. The serial and Eltima streams already dispatched after releasing the lock (via `BeginInvoke`); this brings telnet/websocket in line.

Builds clean off upstream `master` (2.0.47); a pure comms-layer fix, no UI or feature change. Surfaced while testing the ATC Start Job sequence (#25), whose install-probe MBOX hold kept an active telnet poll — exactly the deadlock window.

#25 ATC tool-change macro provisioning (Install ATC + Start Job)

File	+	−
CNC Controls/CNC Controls/AtcMacros.cs	369	0
CNC Controls/CNC Controls/SDCardView.xaml.cs	80	3
CNC Controls/CNC Controls/SDCardView.xaml	4	1
CNC Controls/CNC Controls/CNC Controls.csproj	11	0
CNC Core/CNC Core/Grbl.cs	14	1
CNC Core/CNC Core/YModem.cs	14	5
ioSender XL/ioSender XL/JobView.xaml.cs	6	0
ioSender/ioSender/JobView.xaml.cs	6	0
macros/tc.macro	105	0
macros/probe_tfl.macro	140	0
macros/start_job.macro	52	0
macros/cal.macro	3	0

Turnkey **repeatable tool change** for ATC controllers: ioSender ships the controller-side tool-change macro set and installs it to the controller's persistent **LittleFS**, then seeds an ioSender-side "Start Job" macro that drives the home → set-WCS → reference-toolsetter → run sequence. The novice only sets `G28/G30/G59.3` and supplies a 3D probe + toolsetter.

- **Embedded macros + provisioning:** `AtcMacros.cs` carries `cal/probe_tfl/tc.macro` as embedded resources and uploads them to `/littlefs` over **YModem** (absolute `/littlefs/<name>` path so they land on the right filesystem even with an SD card mounted at root; a `atc.sum` SHA-256 sidecar detects drift / re-installs).
- **Explicit "Install ATC macros" button** on the SD Card tab is the *only* provisioning trigger — user-initiated, re-verifying the filesystem each click. Auto-on-connect/auto-on-show were removed after proving fragile against controller-I/O timing (could wedge the link or hang on close).
- **`$I` capability parsing** (`Grbl.cs`): `ATC=1` → `HasATC`, `ATC=0` → `AtcMacrosRequired` (an ATC is configured but its `tc.macro` is missing — prompt to install + reboot), `YM` → `HasYModem`. A 2-arg `YModem.Upload(path, remoteName)` overload targets an absolute remote path.
- **Start Job macro** (both apps): seeded on connect to an ATC controller (`@start_job.macro`, run line-by-line through the macro pre-processor). The controller-side macros it `CALL`s stay on LittleFS, resolved by `O<cal> CALL` / `M6`. The Start Job go-to `G30`/`G28` moves are written as an explicit safe-Z sequence (lift to machine top, traverse X/Y, descend) rather than a bare diagonal rapid; the Z probes descend to `−140` (below the spoilboard, within `$132`).
- **Embed-on-Build:** `CNC Controls.csproj` disables VS's fast up-to-date check so edits to the external `..\..\macros\*.macro` embedded resources are actually re-embedded on a normal Build (the fast check ignores external files and would otherwise ship a stale copy).

Builds clean in Release for both apps; the full Start Job sequence was verified end-to-end on hardware (2026-06-18). Kept self-contained on PR 14 by seeding the Start Job macro without an auto function-key (that nicety needs PR 10's `Macro.FKey`); assign one in the Macro manager if wanted. The `O<cal> CALL` error:81 fix lives in PR 11 (label/keyword gap-close) plus a firmware-side macro-search-path fix.

#26 Offsets — editable G28/G30 (Set rapids & stores)

File	+	−
`CNC Controls/CNC Controls/OffsetView.xaml.cs`	21	2

The predefined offsets `G28` and `G30` were read-only in the Offsets view — you could only store the *current* machine position. This makes them **editable**: type a target and *Set* now honours it.

- **Set rapids, then stores:** because `G28.1`/`G30.1` can only capture the live position, *Set* first rapids the machine to the edited target with `G53 G0`, then stores it with `G28.1`/`G30.1`. A confirmation prompt spells out the move first (it physically moves the machine); a pending unit change is honoured around the move.
- **Everything else unchanged:** non-predefined work offsets keep their existing behaviour.

Builds clean off `master` (CNC Controls and the full ioSender solution). One-file change to `OffsetView`.

#27 Dialogs — center owner-less modal dialogs

File	+	−
`CNC Controls/CNC Controls/MPGPending.xaml`	1	0
`CNC Controls/CNC Controls/AppConfig.cs`	1	1
`CNC Controls Probing/CNC Controls Probing/ProbeVerify.xaml`	1	0
`CNC Controls Probing/CNC Controls Probing/ProbingViewModel.cs`	1	1

Two modal dialogs — **MPGPending** (the "MPG mode active" prompt shown during the connect handshake) and **ProbeVerify** (probe-connection check) — set neither `Owner` nor `WindowStartupLocation`, so they opened as independent OS-placed top-level windows with their own taskbar button instead of centering over ioSender.

- **Owner + CenterOwner:** each now sets `Owner` to the main window at the call site and `WindowStartupLocation=CenterOwner` + `ShowInTaskbar=False` in XAML, so they center over the client area and stay attached to the app — matching every other dialog in the sender.

Builds clean off `master`. The other modal dialogs already set an owner; the owned-but-not-centered ones (g-code transform / importer dialogs) were left as-is for now.

#28 Machine Setup restructure + Tools tab + guided setup gate

File	+	−
CNC Controls/ToolsView.xaml	33	0
CNC Controls/ToolsView.xaml.cs	135	0
CNC Controls/GrblConfigView.xaml.cs	65	20
CNC Controls/IGrblConfigTab.cs	4	1

Reorganises the top-level UI into a clearer commissioning flow.

- **Settings → Grbl + App sub-tabs:** the controller ($) settings and the sender's own preferences are split into two sub-tabs instead of one long page.
- **New Machine Setup top-level tab:** an overview plus six colour-coded step-tabs, with a startup *gate* that leads a new user to the first incomplete step (step 6 queries the controller filesystem via `AtcMacros.GetStatus` for macro status). Reworks the Machine Setup Wizard (#21) into the step-tab form.
- **New Tools container:** a single home (`ToolsView`) for the commissioning/tuning tools — Stepper calibration & scratch, Surface Spoilboard (#29), Squareness (#31), Trinamic and PID — via the `IGrblConfigTab` contract; top-level Trinamic/PID dropped.
- **Inline grids:** a probe-definition grid (step 5) and a macro-status grid (step 6) surface state without leaving the wizard.

In the integration build. The bulk of the wizard body lives in #21; this entry covers the hosting/restructure (Tools container + sub-tabs + gate). Localization CSVs for the new/renamed tabs are still pending.

#29 Surface Spoilboard — machine-referenced facing tool

File	+	−
CNC Controls/SurfaceSpoilboardWizard.xaml	84	0
CNC Controls/SurfaceSpoilboardWizard.xaml.cs	589	0

Flattens the spoilboard with a boustrophedon raster, generated to the homed machine envelope.

- **Machine-referenced:** the operator touches off *Z only*; XY is referenced to the homed envelope (less an edge margin), so the raster can never overrun a soft limit no matter where Z was zeroed.
- **High-point touch-off:** jog anywhere to the board's highest point and set Z0 there; the cut still references XY to the machine corner automatically.
- **Reuse Z0 + Finish pass + abbreviated perimeter Dry run:** re-cut at the same Z0 (with a cutter/dust-boot prompt), reserve a light final pass, or trace just the perimeter to check extents.
- **Outline = leveling check:** pauses at each corner on the Z0 plane for a dial-gauge reading.
- Runs through the flow-controlled job streamer (Feed Hold / Stop stay live).

In the integration build; hardware-tested.

#30 Load Stock — corner-probe stock measurement

File	+	−
ioSender XL/LoadStockView.xaml	102	0
ioSender XL/LoadStockView.xaml.cs	721	0
ioSender XL/MachineControlPanel.xaml(.cs)	63	0
ioSender XL/MachineControlWindow.xaml(.cs)	96	0
macros/pcorner.macro	143	0

Measures real stock by probing its corners, then sets the work origin and reports geometry.

- **Corner probe via `pcorner.macro`:** an inline grblHAL NGC program calls the controller-side corner-probe macro per corner; probe feeds come from the selected 3D probe definition (no hardcoded feeds).
- **Outputs:** XY footprint, per-corner Z (flatness), squareness (skew + diagonal Δ), and stock thickness (top − spoilboard reference). A results popup draws the probed quad with corner angles.
- **Copy size:** one click puts `X Y Z` on the clipboard to paste into the Fusion `ioSenderBatchPost` add-in (#6) — round-trips the measured stock into the simulator `(STOCK)` line.
- **Floating run-control panel** (`MachineControlWindow`) so the probe run can be driven without leaving the tab.

In the integration build; hardware-tested (427×427 mm stock).

#31 Squareness (Auto Square) — ganged-gantry offset tool

File	+	−
CNC Controls/AutoSquareWizard.xaml	97	0
CNC Controls/AutoSquareWizard.xaml.cs	589	0

Facilitates Phil Barrett's auto-square *offset* method for a ganged, auto-squared gantry.

- **Drill an L:** peck-drills a reference L (corner + far-X + far-Y holes, at the framing-square arm lengths, centred on the bed with X/Y nudge to dodge dog/insert holes).
- **Measure & apply:** register a framing square on the pins, enter the gap; the tool converts it (gap ÷ lever × rail span) to a squaring-offset change, writes `$170–$172` and re-homes.
- **Ganged-axis selector:** grblHAL reports the offset for all of `$170–$172` but only the ganged axis accepts a write — the selector targets the right one.
- **Measure-only gauge:** on a build without the offset setting it still drills + measures to show how far out of square the gantry is (correct mechanically). Dry run + Reuse Z0.

In the integration build. Note: the squaring offset is fine-trim only — a grossly racked gantry must be corrected mechanically first.

#32 Program view as a reusable object + background loading

File	+	−
CNC Controls/ProgramView.xaml(.cs)	129	0
CNC Core/GCodeJob.cs	108	49
CNC Controls/GCode.cs	99	33
ioSender XL/MainWindow.xaml(.cs)	79	44
ioSender XL/LoadStockView.xaml(.cs)	68	16
CNC Controls/GCodeListControl.xaml(.cs)	29	3
CNC Controls/{AutoSquare,SurfaceSpoilboard,StepperCalibration}Wizard.xaml.cs	57	9
CNC Controls/JobControl.xaml.cs	13	7
CNC Controls/MacroProcessor.cs	0	14
macros/pcorner.macro	13	8
docs/ (2 design/session notes)	224	0
CNC Core/GrblViewModel.cs, CNC Core.csproj	11	1

“A program is a program”, taken further: the single shared program overlay becomes a standalone, streamer-connectable `ProgramView` object, and the loader stops blocking the UI.

- **Reusable program view:** Load File, Load Folder and each wizard create their own `ProgramView` and `Connect()` it to a push/pop stack — the overlay hosts whichever is active, and the loaded job is no longer a special “null” case. One Cycle Start runs the active view.
- **Continuous block numbering:** an explicit `N` word is shown in the Block column and hidden from the G-code column (Data stays intact for streaming); unnumbered lines (O-word calls, comments) continue previous + 1.
- **Background loading:** Load File / Load Folder parse on a worker thread — the per-line parse and the end-of-load bounding-box emulation run off the UI thread, blocks flush to the bound list in batches (rows appear as files are read), and a program-view-scoped wait cursor replaces the global one. A 322k-line folder combine no longer freezes the app.
- **Also shipped here:** Load Stock “set tool-length reference” option (Start Job parity) and the absolute-seek `pcorner.macro` (workaround for the G53-then-G91-on-inverted-axis firmware bug, filed upstream).

In the integration build; hardware-tested (Load Stock, single-file and folder loads, background load).

#33 Explain G-code on hover (novice tooltips)

File	+	−
CNC Core/GCodeExplainer.cs	634	19
CNC Controls/GCodeListControl.xaml(.cs)	170	20
CNC Core/CNC Core.csproj	1	0

Makes the program view teach: hovering a line shows a plain-language explanation of what the G-code does — aimed at novices.

- **Token-based:** builds the explanation from the parser's own semantic tokens (keyed to a block by line number), so it inherits the parser's modal state — a bare `X10 Y0` continuation still reads correctly as a rapid or a cut.
- **Line or selection:** hovering one line explains it ("Cutting move to X 125.4, Y 0; feed 800 mm/min"); hovering a line within a multi-row selection gives a line-by-line breakdown.
- **Full coverage:** a complete G/M-code dictionary plus a raw-line lexer fallback explain bare modal codes (e.g. `G54`) and views without parsed tokens. Computed lazily per row; skipped (thread-safe) while a background load is still parsing.
- **Reads real programs:** a bracket/parameter-aware lexer explains expression operands (`G53 G0 X[#5181]`), decodes the well-known `#5xxx` parameters (G28/G30 positions, probe result, WCS offsets), names O-word CALLs by the `/<name>.macro` file they run + their arguments, and reads G10 (WCS origin/rotation), `G65 P5` (probe-input select) and `#`-assignments plainly.
- **Click to copy:** the hover balloon is an interactive popup — click it to copy the explanation to the clipboard (a plain tooltip can't be moused into).
- **Localizable:** English for now, but every phrase is produced in one place so it can move to LibStrings later.

In the integration build; hardware-verified.

#34 Probe definition diagrams

File	+	−
CNC Controls/ProbeDefinitionEditDialog.xaml	136	22
CNC Controls/ProbeDefinitionEditDialog.xaml.cs	6	0

The probe definition editor now shows a labelled schematic beside the fields, so a novice can see what each measurement means.

- **Per-type drawing:** switched with the Type dropdown — 3D touch probe, touch plate, tool setter and edge finder each draw their shape and dimension the key measurements the user enters (tip / ball / bit diameter, plate thickness, lip width, setter height, spin speed).
- **Touch-plate lip:** drawn as a down-protruding hook whose inner face registers against the stock side (aluminium).
- **Pure vector** (Canvas + Viewbox), toggled with the same show/hide as the fields — no image assets, theme-friendly.

In the integration build.

#35 Jog distance +/− controller actions

File	+	−
CNC Core/ControllerMapper.cs	11	1
CNC Core/GrblViewModel.cs	2	0
CNC Controls/JogBaseControl.xaml.cs	1	0
CNC Controls/KeyMapEditor.xaml.cs	3	1

Two new assignable controller actions that step the on-screen jog *distance* preset (the 2×4 grid), distinct from the existing *Jog speed +/−* which step the feed preset.

- **Jog distance +/−** appear in the Controller-tab action list; assign them to any button (no default binding).
- Adds `GrblViewModel.CycleJogDistance`, wired by the jog panel to the distance index (clamped to the four presets).

In the integration build.

#36 Probe input follows the selected probe type

File	+	−
CNC Core/Grbl.cs	4	0
ioSender XL/LoadStockView.xaml.cs	6	0
ioSender XL/HeightMapView.xaml.cs	6	0

The program-generating tools now select the controller probe input from the chosen probe's type — the same rule the Probing page already uses — so a stale selection can't send a 3D-probe descent to the wrong input and drive into the work.

- **Rule:** Load Stock and Height Map emit `G65 P5 Q<n>` for the selected probe — tool setter → `Q1`, else the main probe → `Q0`.
- **Why:** an interrupted tool-setter cycle (`tc.macro`) can leave `G65 P5 Q1` selected; a later probe would then watch the tool-setter line and stab the spoilboard.
- Gated on new `GrblInfo.HasToolSetter` (only meaningful when a tool setter exists to select between).

In the integration build; hardware-verified (Load Stock).

#37 One in-place streaming path (stayPut removed)

File	+	−
`ioSender XL/MainWindow.xaml.cs`	49	66
`CNC Controls/MacroProcessor.cs`	12	43
`ioSender XL/LoadStockView.xaml.cs`	1	1

With each program view self-contained, the old `stayPut` flag and the "stream-in-place vs load-into-the-job" fork were a relic of the single shared job view. Collapsed to one path.

- **One streaming path:** every streamed macro/wizard run streams a transient with full flow control (Feed Hold/Stop stay live) and **never touches the loaded job** (`GCode.File`) or the Job tab. Before, wizards other than Load Stock replaced your loaded program when they ran.
- **Own view per run:** a tool that owns a program view (a wizard) marks its own; a plain streamed macro gets a dedicated run view (titled with the macro name) that auto-dismisses ~20 s after it finishes.
- **Multi-burst safe:** a run flushes several bursts (a park move, then each `O< … > CALL`); the streamer source reverts per burst, guarded so a finishing burst can't clobber the next, and the view stays connected across all of them.
- Deletes the `stayPut` param, the `RunStreamedJob` job-clobber path + its wiring, and the orphaned tab-restore helper — a net code removal.

In the integration build; hardware-verified via a clean Load Stock run (multi-burst / directives / O-words). Not yet exercised on hardware: the plain-macro run view (only a large *streaming* flyout macro reaches it).

#38 Match the bundled simulator to the connected controller

File	+	−
`CNC Controls/SimulatorManager.cs`	309	0
`ioSender XL/JobView.xaml.cs`	46	0

Keeps the bundled grblHAL simulator in step with whatever controller you connect to, so a probe / rotation / multi-axis feature behaves on the sim exactly as on the real firmware.

- **Options → signature:** on connect, the controller's behaviour-affecting build options (axis count, probe, WCS rotation) are read from `GrblInfo` into a set of compile symbols and a short 12-hex signature.
- **Cache ladder:** already-active build → locally-cached `grblHAL_sim-<sig>.exe` → a `sim-<sig>` release on the fork (public download, no token) → otherwise dispatch the parameterized `build-matched-sim` CI and pick up the result. The set of `sim-<sig>` releases acts as a shared remote build cache.
- **Token only to dispatch:** a GitHub token (`GH_TOKEN`, `workflow` scope) is needed only to trigger a build — every download is public. Runs on a background thread off `InitSystem`, never blocks connect, is skipped when talking to our own simulator, and won't re-dispatch an in-flight signature.

In the integration build; the Simulator-fork CI (`build-matched-sim.yml`) validated green end-to-end (dispatch → per-signature release → public download). The real on-connect round-trip is pending a hardware spot-check.

#39 Re-establish modal state on a mid-program start

File	+	−
`CNC Controls/StreamPump.cs`	22	0

“Start from this toolpath” queues a `G90 G94 / G17 / G21` prolog to re-establish modal state (the folder combiner strips each per-op file’s header, where `G21` lived). The legacy streamer drained that queue, but the `StreamPump` streams `source.Data` directly and never touched `source.Commands` — so the prolog was silently dropped on a mid-program start.

- **Symptom:** a mid-program start inherited whatever units the controller was left in; in `G20` the toolpath’s literal-mm coordinates read as inches → targets off the table.
- **Fix:** the pump now sends the queued `source.Commands` lines first, ahead of the first block, and swallows their acks so they don’t advance job-line accounting. Full runs from the top were unaffected (there the prolog is prepended as real blocks).

In the integration build. A latent correctness fix for any pump-based run-from-block.

#40 Load Stock “apply work rotation” defaults off

File	+	−
`ioSender XL/LoadStockView.xaml.cs`	6	1

Setting a WCS rotation via `G10 L2 R` armed a grblHAL rotation-transform bug (a rotated `G54` move with both X and Y double-applied the second axis’s offset) that soft-limited the *next* program’s first cut rapid. Reproduced on hardware: a −0.08° measured skew crashed the following job; clearing the rotation let it run.

- Default the skew→WCS rotation **off** so Load Stock never silently arms it; the checkbox stays for opt-in.
- The real fix is in the firmware rotation transform (`gcode.c`: `bit(idx)→bit(idx_0)`), now fixed and flashed; once hardware-verified the default can go back on.

In the integration build. A guard against the firmware rotation bug until the firmware fix is hardware-verified.

#41 Live 3D toolpath + material-removal carve view

File	+	−
CNC GCodeViewer/CarveView.xaml.cs	850	0
CNC GCodeViewer/Renderer.xaml.cs	190	54
CNC GCodeViewer/CarveView.xaml	36	0
CNC GCodeViewer/ConfigControl.xaml.cs	13	1
CNC GCodeViewer/RenderControl.xaml	11	7
CNC GCodeViewer/CNC GCodeViewer.csproj	7	0

A live 3D **carve view** in the toolpath viewer — not just lines on screen, but the stock being *machined away* as the program runs, so you can see the finished shape before you cut it.

- **Machine scene, always on:** the work envelope, axes and a live tool cone are shown whenever the machine is homed, even with no program loaded — a persistent sense of where the tool is in the work area.
- **Program-sized stock:** the stock block is sized to the program (its real footprint read from a `(STOCK X Y)` comment), drawn solid with the correct top, in work coordinates aligned to the toolpath.
- **Material-removal carve:** a dexel / height-field carve removes material as the tool moves, using the *real cutter bottom profile* (flat / ball / V-bit) on a fine grid, so the carved surface matches the actual tool. Toolpath lines hide during playback to reveal the cut and restore on stop.
- **Play a preview:** press Play for a machine-free preview that animates the program and builds the carve; Reset view restores the default orientation, and a pale stock theme keeps the cut visible.
- **Real machine vs. simulator:** streaming to a *real* machine **disables** the carve — the view freezes so it never competes with motion-critical streaming. To watch the carve run in step with execution, connect to the **simulator** and Cycle Start there.
- **Performance:** in-place dirty-cell mesh updates (no per-frame realloc) and the scene build deferred off the layout pass.

In the integration build. Built up over several phases (envelope / stock / tool cone → toolpath + Play → dexel carve); the validate-controller 3D visualization (#16) uses the same scene.

#42 Settings redesign — flat tab strip + searchable Grbl settings

File	+	−
`CNC Controls/GrblConfigView.xaml.cs`	359	56
`CNC Controls/GrblConfigControl.xaml.cs`	156	26
`CNC Controls/KeyMapEditor.xaml.cs`	66	40
`CNC Controls/GrblConfigView.xaml`	41	8
`CNC Controls/BasicConfigControl.xaml`	27	14
`CNC Controls/GrblConfigControl.xaml`	27	3
`CNC Controls/MainPageEditor.xaml.cs`	19	3
`CNC Controls/MacroManagerDialog.xaml.cs`	14	9
`CNC Core/Grbl.cs`	10	0
`CNC Controls/SettingsPanelRegistry.cs`	8	0
`CNC Controls/AppConfig.cs`	5	0
`CNC Controls/AppConfigView.xaml(.cs)` *(retired)*	0	373
`Camera / Probing / Lathe / Jog config panels (widen + wrap)`	19	10
`Locale/*/csv (7 locales)`	184	0

The Settings area was two tabs — **Grbl** and an **App Settings** page that was a wall of GroupBoxes plus a row of buttons that each opened a modal dialog. It's now a single **flat tab strip**:
Grbl · App · Jogging · G Code · Keyboard & Controller · Macros · Main Page.

- **Free-text Grbl search:** a leading `$` means "setting id" — `$53` jumps to it, `$5?` expands all of `$50`–`$59`; anything else (a word like `Jog` or a bare number like `10`) matches every setting whose *name or value* contains it. Matching groups auto-expand and matches are highlighted; an **"n of m"** readout plus up/down arrows (and **F3**/**F4**, Enter) step through them.
- **Grbl-settings autosave:** an opt-in toggle (default **off** — these are `$` writes to hardware) with a companion "prompt before auto-saving"; when on, the manual Save button on the Grbl tab hides, mirroring the App-settings autosave.
- **Editors folded in as tabs:** the Key Bindings, Macros and Main Page editors were modal `Windows` launched by buttons — they're now inline tabs (converted to `UserControl`s) that *save on leave* instead of OK/Cancel. The Key Bindings tab only pauses controller dispatch while it is actually shown.
- **App/Jogging/G Code:** the config panels are bucketed by type into the three tabs; App & Jogging columns are 25% wider, Camera + Probing share one column, and panels stretch to fill their column so long labels wrap instead of clipping.
- **Under the hood:** `AppConfigView` is retired — its Save/Restart footer, restart-hooking and auto-save-on-leave moved into the settings host. New strings localized across all seven locales.

In the integration build. Builds clean (Release, EXIT 0); on-hardware review of the high-DPI arrow buttons and panel wrapping pending.

#43 Per-user config folder + standalone config folded into App.config

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	258	1
CNC Controls/CNC Controls/ConfigPanel.cs	97	0
CNC Controls/CNC Controls/LoadStockConfig.cs	37	0
ioSender XL/ioSender XL/Default-App.config	64	0
CNC Core/CNC Core/ControllerMapper.cs	82	27
CNC Core/CNC Core/KeypressHandler.cs	102	63
CNC Controls/CNC Controls/ProbeDefinition.cs	32	4
CNC Controls/CNC Controls/SettingsPanelRegistry.cs	43	0
Surface/AutoSquare/StepperScratch Wizard .xaml(.cs) — ConfigPanel conversion (3 wizards)	121	187
CNC Controls/CNC Controls/ConsoleControl.xaml(.cs)	162	6
CNC Controls/CNC Controls/GrblConfigControl + GrblConfigView .xaml(.cs)	143	63
Basic/StripGCode/Jog/JogUi ConfigControl + KeyMapEditor + JobControl	126	16
CNC Core/CNC Core/Grbl.cs, GrblViewModel.cs	2	2

App.config now lives per-user in `%AppData%\ioSender`: on first run the existing app-folder config (plus key maps, probe defs, macros and settings backups) is moved there, and the app-folder copy ships read-only as the reset-to-default template. The standalone `.xml` files each feature used to write next to it are folded in as `App.config` sections.

- **Section fold-in:** ProbeDefinitions, the game-controller map, key mappings, and the Surface Spoilboard / Auto Square / Stepper calibration / Load Stock parameter blobs become sections; a one-time importer reads any legacy file then deletes it. A new `ConfigPanel<T>` base gives the wizards save/restore of their section from a single DTO.
- **Curated defaults:** a shipped `Default-App.config` is the factory template (auto-saves default off) and the source for "Reset to default".
- **Settings/console polish:** the console gains an output search bar + a font-size control; the Settings tabs share one Save/Restart/Reset footer (Grbl sub-row for reload/backup/restore/copy-to-sim); panels opt into "Reset to default"; *ResetAndUnlock* is a bindable keyboard action; a key binding reads as "modified" only when it differs from its value at startup.

In the integration build. Builds clean (Release, EXIT 0); first-run migration is best-effort and idempotent. On-hardware migration spot-check pending.

#44 Start Job workflow + Verify skew, flyout & offset fixes

File	+	−
ioSender XL/ioSender XL/LoadStockView.xaml(.cs)	229	127
ioSender XL/ioSender XL/MainWindow.xaml(.cs)	88	12
ioSender XL/ioSender XL/JobView.xaml.cs	3	3
CNC Controls/CNC Controls/AtcMacros.cs	6	19
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	7	3
CNC Controls/CNC Controls/OffsetView.xaml.cs	26	9
CNC Controls/CNC Controls/LayoutModel.cs	2	1
CNC Controls/CNC Controls/LibStrings.xaml + Locale/*/csv (7 locales)	8	8

Load Stock is reworked into **Start Job** — the first tab, the guided front end for setting a run up before the Job tab cuts it.

- **Workflow:** the operational tabs are disabled until a controller connects; startup lands on the Job tab so its handshake reads `$I`. Start Job auto-selects the configured 3D probe (no picker), zeroes at the front-left corner, and writes `start_job.macro` so the seeded macro/F-key runs the same sequence.
- **Verify skew:** a new pass re-touches each corner in the rotated work frame — the back corners twice (ideal rectangle, then the probed corner) so the gap reads out the stock's out-of-square amount — then parks at G30. The measured-skew rotation sign is corrected (grblHAL `G10 L2 R` aligns with −`atan2(dy,dx)`).
- **Flyout & offset fixes:** pinned sidebar flyouts stay open across tab switches; a flyout *Set* writes the offset directly (it was silently dropped by the streamer-state gate); the Offsets tab *Set All* skips the rapid-to-target confirmation when the machine is already there.

In the integration build. Builds clean (Release, EXIT 0). Start Job macro run + Verify skew validated on hardware; a separate 3D-view stock-orientation issue is tracked.

#45 Run-bar jog preset spinners + startup splash on top

File	+	−
CNC Controls/CNC Controls/JogPresetSelector.xaml(.cs)	106	0
ioSender XL/ioSender XL/MainWindow.xaml	11	2
ioSender XL/ioSender XL/SplashWindow.xaml	1	1
CNC Controls/CNC Controls/MDIControl.xaml(.cs)	14	6

Small quality-of-life fixes to the always-visible run bar.

- **Jog preset spinners:** a new compact selector fills the space on the *State / Check* line — *Dist [▼ value ▲] Jog [▼ value ▲] Feed*. Each field is read-only; the down/up arrows step through the four jog distance/feed presets and **wrap around** at the ends. It binds the shared `JogData` singleton, so the selection stays in sync with the jog pad, keyboard and controller (the preset values are still edited on Settings:App / the Jog tab). Right-aligned to the Program button's edge above.
- **Startup splash stays on top:** the progress splash was `Topmost="False"` and got buried by the main window's black frame almost immediately; it is now `Topmost="True"`, so the whole init sequence (connecting → reading settings → validating setup) is actually visible during the ~10–15 s startup.
- **MDI Send button re-aligned:** the MDI line was rebuilt as a single-row grid (command box + *Send*) that share one row height, and the run bar's *Bare* mode no longer pins Send to its own fixed height — so Send stays vertically centred against the editable command box (it had drifted after the high-DPI pass dropped its fixed height).

In the integration build. Builds clean (Release, EXIT 0). Splash confirmed on hardware; spinner layout and Send-button alignment confirmed in the running app.

#46 High-DPI audit — text no longer clips at 125–150% scaling

File	+	−
`CNC Controls/CNC Controls/*.xaml (38 files)`	114	115
`CNC Controls Probing/CNC Controls Probing/*.xaml (10 files)`	35	35
`ioSender XL/ioSender XL/*.xaml (4 files)`	18	18
`CNC Controls Camera/CNC Controls Camera/*.xaml (2 files)`	8	8
`CNC Controls Lathe/CNC Controls Lathe/*.xaml (4 files)`	7	7
`CNC Converters/JobParametersDialog.xaml`	6	6
`CNC GCodeViewer/CNC GCodeViewer/*.xaml (3 files)`	4	4
`CNC Controls Dragknife/DragKnifeDialog.xaml`	4	4

A reviewed, case-by-case scrub of hard-coded pixel `Width=/Height=` across the whole UI, so controls size to their content and no longer clip text when Windows text scaling is set to 125–150%. Roughly 250 attributes across 63 XAML files were classified one by one (not a blind find/replace):

- **Drop fixed `Height`** on text-bearing controls (buttons, labels, text boxes, combos, radio buttons, group boxes, and the rows that hold them) so they auto-size to the scaled font instead of cropping glyphs.
- **Convert fixed `Width` to `MinWidth`** (and floor heights to `MinHeight`) where the value was only a minimum — text buttons, combos, fields and container panels can now grow.
- **Convert fixed-pixel grid rows to `Height="Auto"`** where the row holds text.
- **Deliberately kept** genuine fixed geometry that must *not* scale with text: icons and images, LED/signal indicators, probe/lathe schematic diagrams, 3D render surfaces, colour swatches, icon-only glyph buttons, data-grid column widths, colon-aligned label columns, load-bearing scroll viewports and fixed dialog chrome.

In the integration build. Builds clean (Release, EXIT 0). High-traffic controls confirmed on hardware (“no more clipped text”). Known follow-ups: the five absolute-`Margin` lathe wizards (ThreadingWizard, ProfileDialog, Turning/Parting/Facing) need a real re-layout rather than an attribute scrub, and are intentionally left for a later pass.

#47 Lathe mode enable/disable checkbox

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	41	0
CNC Controls/CNC Controls/BasicConfigControl.xaml.cs	14	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	1	0

A *Lathe mode* checkbox on the App › Main settings panel, so lathe mode can be turned on or off without hand-editing the config. Previously the only lathe-enable control lived on a settings panel that a lathe view had to load first — and that view/tab was itself gated on lathe already being enabled, a catch-22 that made it unreachable when off.

- **Simple on/off facade** — `LatheConfig.LatheEnabled` maps to the existing `XMode` (Radius when on, Disabled when off) and persists through the same `Lathe` config section.
- **Shows/hides the Lathe wizards tab** — a new `AppConfig.SetTabPresent` adds or removes the tab in both the persisted layout tree and the legacy flat tab list, inserted after the SD-card tab.
- **Restart to apply** — the tab is only built at startup from the controller's reported lathe mode, so toggling raises `RestartRequired` (see #48).

In the integration build. Builds clean (Release, EXIT 0). Restores an implementation that had been prototyped then reverted; hardware-confirmed working.

#48 Restart button pulse + restart-on-leave prompt

CNC Controls/CNC Controls/GrblConfigView.xaml	43	1
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	15	0

Makes a pending restart impossible to miss. Some settings (buffer/reset delay, 3D-view enable, main-page layout, and now lathe mode) only take effect at startup; when one changes, the shared Settings footer reveals a *Restart* button. That button now behaves like an alert:

- **Slow red pulse** — while enabled the Restart button animates between bright and dark red (custom template so the theme chrome doesn't paint over it), driven by its `IsEnabled` state so it keeps pulsing across inner-tab switches until the app is restarted.
- **Restart-on-leave prompt** — leaving the Settings area with a restart still pending asks "Restart ioSender now?" (Yes relaunches). Switching between the inner Settings tabs does not prompt.
- **Shared restart path** — the relaunch logic is factored into one `DoRestart()` used by both the button and the prompt.

In the integration build. Builds clean (Release, EXIT 0). Applies to every restart-only setting, since all route through the one shared Restart button. Hardware-confirmed ("works great").

#49 Lathe wizards rebuilt to the app layout idiom (high-DPI)

CNC Controls Lathe/CNC Controls Lathe/ThreadingWizard.xaml	275	235
CNC Controls Lathe/CNC Controls Lathe/TurningWizard.xaml	50	42
CNC Controls Lathe/CNC Controls Lathe/FacingWizard.xaml	49	41
CNC Controls Lathe/CNC Controls Lathe/PartingWizard.xaml	47	39
CNC Controls Lathe/CNC Controls Lathe/ProfileDialog.xaml	40	30

The five lathe wizards were the last upstream code still laid out on an absolute-`Margin` canvas — fixed pixel `Width/Height` and hard-coded positions — so their text clipped and controls overlapped once Windows text scaling was set to 125–150% (they were deliberately skipped in the #46 attribute scrub because they needed a real re-layout, not a mechanical edit). Each wizard body was rebuilt with the app's normal patterns so it flows and scales:

- **Standard idiom** — vertical `StackPanel/Grid` with `Auto` sizing, `NumericField` + `ColonAt` label alignment, `MinWidth/MinHeight` floors, a `ScrollViewer` for overflow, and grouped `GroupBox` sections.
- **Threading** — a two-column responsive grid (selection / Dimensions / Tool on the left; Thread values / Options / Cut on the right), with the linuxCNC and Mach3 option boxes sharing one slot to preserve the code-behind's draw-over behaviour, and the Tool schematics + shape radios re-hosted without absolute margins.
- **Turning / Parting / Facing** — a single scrolling input column with Calculate and the generated g-code pinned at the bottom.
- **Profile dialog** — the fixed-size modal became a `SizeToContent` window; the CSS control and fixed-RPM field share one slot, and it follows the app theme instead of a hard-coded grey.

In the integration build. Builds clean (Release, EXIT 0). Every `x:Name`, binding, converter and event handler the code-behinds rely on was preserved — no code-behind changes. The rebuilt layouts are functional and scale correctly; some aesthetic cleanup (spacing/grouping refinement) is still wanted at some point, ideally by someone who runs a lathe.

#50 Tab strips fill their width (StretchTabControl)

CNC Controls/CNC Controls/StretchTabControl.cs	119	0
ioSender XL/ioSender XL/MainWindow.xaml	6	4
CNC Controls Lathe/CNC Controls Lathe/LatheWizardsView.xaml	4	3
CNC Controls/CNC Controls/GrblConfigView.xaml	2	2
CNC Controls/CNC Controls/MachineSetupWizard.xaml	2	2
CNC Controls Probing/CNC Controls Probing/ProbingView.xaml	2	2
CNC Controls/CNC Controls/ToolsView.xaml	1	1
CNC Controls/CNC Controls/CNC Controls.csproj	1	0

The tab headers used to sit bunched at the left of each strip with dead space to the right, so tabs ran together and were hard to pick out. A new `StretchTabControl : TabControl` spreads them to fill the strip: each tab keeps its natural width (plus one space character each side) scaled up proportionally, so a wider label gets a wider tab and the row fills the width.

- **Assigns each header a width** — the default `TabControl` template hosts its headers in a fixed `TabPanel` (`IsItemsHost`), so a replacement `ItemsPanel` is ignored; setting each `TabItem.Width` is honoured instead.
- **Recomputed off-layout** — widths are recalculated only on the discrete tab add/remove event and, on resize, from a deferred (`DispatcherPriority.Background`) callback that runs *after* the layout pass. Doing it synchronously from inside layout (`SizeChanged/LayoutUpdated`) re-enters the layout system and mis-measures content; the deferred approach sets the widths and lets one ordinary follow-up pass consume them, with no feedback loop. The trade is that during a drag-resize the tabs show old widths for a frame and snap on release.
- **Applied to** the main tab bar and the Settings, Tools, Lathe, Probing and Machine Setup sub-tab strips.

In the integration build. Builds clean (Release, EXIT 0). Confirmed on the running app: strips fill, first tab flush, resize re-spreads cleanly, and tab content still fills/left-aligns (the label-centering was pulled back out after it leaked into content alignment on content-sized views).

#51 SD Card first-line cleanup (overlap + free-space banner)

`CNC Controls/CNC Controls/SDCardView.xaml`	6	2
`CNC Controls/CNC Controls/GrblFilesystems.cs`	3	2

Two small fixes to the SD Card tab's top line.

- **No more overlap** — the "List CNC files only" checkbox and the filesystem-status text were absolutely positioned at fixed x offsets, so the status ran under the checkbox label at larger fonts. They now flow in a horizontal `StackPanel`.
- **Banner hides when uninformative** — `FreeSpaceSummary` only lists a filesystem that actually reports free space; the bare "*name*: mounted" fallback carried no information (the file list already shows the filesystem is present) and just added clutter, so the whole banner disappears when the controller reports no sizes.

In the integration build. Builds clean (Release, EXIT 0). Confirmed on the running app.

#52 Per-tool guidance text on the Tools tabs

File	+	−
CNC Controls/CNC Controls/ToolView.xaml	3	0
CNC Controls/CNC Controls/SurfaceSpoilboardWizard.xaml	11	5
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	7	1
CNC Controls/CNC Controls/StepperCalibrationWizard.xaml	8	2
CNC Controls/CNC Controls/AutoSquareWizard.xaml	4	4
CNC Controls/CNC Controls/TrinamicView.xaml	8	2
CNC Controls/CNC Controls/PIDLogView.xaml	5	0

Most of the Tools tabs showed a form with no explanation of what the tool did or how to run it. Each now carries a short “About…” heading + wrapped explainer in the tab’s right-hand pane (matching Auto Square and Stepper Calibration, which already had step text).

- **Covered** — Tool table, Surface spoilboard, Stepper calibration (scratch), Trinamic tuner, PID tuner.
- **Layout tidy** — the Stepper-calibration collision warning is contained to the right column (instructions box fills the height); the Surface-spoilboard result sits in its own light panel distinct from the description; Squareness puts the description above the diagram.
- **Localizable** — every string carries an `x:Uid` so it feeds the locale CSVs; English inline is the fallback.

In the integration build. Builds clean (Release, EXIT 0). Confirmed on the running app.

#53 Headless build script + crash logging

File	+	−
`build.ps1`	105	0
`.vscode/tasks.json`	69	0
`ioSender XL/ioSender XL/App.xaml.cs`	72	7
`CLAUDE.md`	9	1

Build and run the WPF app entirely from the command line / VS Code, without opening the Visual Studio GUI. (PlatformIO can't drive a .NET Framework build — it only builds the C/C++ firmware/sim — so a plain MSBuild wrapper is the right tool.)

- **`build.ps1`** — kills a running `ioSender.exe`, builds via MSBuild (located with `vswhere`, Enterprise-path fallback), optionally launches. `-Configuration Debug|Release|Both`, `-Launch`, `-Headless`, `-NoKill`.
- **VS Code tasks** — *Build Debug + Launch* (Ctrl+Shift+B), *Build Release*, *Build Both*, all delegating to the one script.
- **Crash logging** — the global handlers (Dispatcher / AppDomain / TaskScheduler) now dump a timestamped, versioned, full-stack entry to `%AppData%\ioSender\ioSender.crash.log` and exit `0xFA11` (64017). Fixes a real bug where a background-thread exception showed a box but never exited, wedging the process. Interactive runs still show a dialog; `IOSENDER_HEADLESS=1` suppresses it so an unattended crash can't hang on an un-clicked box. VS's JIT debugger does not auto-attach.

In the integration build. Debug + Release build clean; both crash paths verified live (log written, exit 0xFA11).

#54 Rename LoadStock → Start Job (code matches the UI)

File	+	−
`ioSender XL/ioSender XL/StartJobView.xaml(.cs)` *(was LoadStockView)*	9	9
`CNC Controls Probing/…/StartJobControl.xaml(.cs)` *(was LoadStockControl)*	9	9
`CNC Controls/CNC Controls/StartJobConfig.cs` *(was LoadStockConfig)*	10	10
`CNC Controls/CNC Controls/AppConfig.cs`	6	6
`CNC Controls/CNC Controls/LayoutModel.cs`	3	3
`CNC Controls/CNC Controls/{ICNCView,LibStrings,StreamPump}`	4	4
`ioSender XL/…/{MainWindow,JobView}.xaml.cs + IProbeTab.cs`	3	3
`Locale/*/csv (7 locales, 2 files each)`	126	126
`*.csproj (3 projects)`	7	7

The feature is "Start Job" everywhere in the UI, but the code was still `LoadStock*`. Full rename — there is no back-compat to preserve, so no migration code.

- **Files & classes** — `LoadStockView` / `Control` / `Config` / `Settings` → `StartJob*`.
- **Tokens** — `ViewType.LoadStock` and `ProbingType.LoadStock` → `StartJob`; `LayoutKeys` value, the folded App.config section (`LoadStock` / `LoadStock.xml`), the `str_tabLoadStock` resource key and its locale-CSV baml paths, and the StreamPump temp-log name all follow.
- **Persisted config** — the three tokens saved in `%AppData%\ioSender\App.config` (tab name, layout component, section key) were patched out-of-band to match.

In the integration build. Debug + Release build clean; headless startup smoke test passes on the renamed config.

#55 Localization sweep — V2 strings into all 7 locales

File	+	−
`Locale/*/csv/CNC.Controls.WPF.resources.*.csv` *(7 locales)*	1300	0
`Locale/*/csv/CNC.Controls.Probing.resources.*.csv` *(7 locales)*	147	0
`Locale/*/csv/ioSender.resources.*.csv` *(7 locales)*	63	0
`Locale/*/csv/CNC.Controls.Viewer.resources.*.csv` *(7 locales)*	35	0
`tools/locadd.py`	36	1

Later V2 features were `x:Uid`'d in XAML but never had LocBaml rows added, so they fell back to English in the other six languages. The self-deriving `tools/locadd.py` was pointed at the current view set and backfilled every missing row.

- **+1545 rows across 7 locales** — 159 new keys in en-US plus the backfills for keys that had reached en-US but not the other locales, so all seven are consistent again.
- **Covers** — Start Job (renamed view + the probing control), the probing-redesign dialogs (`ProbeMotionParams` / `ProbeDefinition*`), the main-page editor, console search, the Tools tab guidance (#52), the carve view, and assorted flyouts/config controls.
- **Baseline, not machine-translation** — each new row carries the English text for a translator to replace; the app already rendered English via fallback, so nothing changes on screen. Comma values RFC4180-quoted; re-running the tool adds nothing.

In the integration build. Lathe wizards are excluded (no `CNC.Controls.Lathe` locale CSV exists — never LocBaml-localized).

#56 Online user manual + in-app context help (F1)

File	+	−
`CNC Controls/ManualHelp.cs` *(new)*	112	0
`ioSender XL/MainWindow.xaml.cs`	13	2
`CNC Controls/CNC Controls.csproj`	1	0

A new task-oriented **online user manual** — a single self-contained HTML page hosted on GitHub Pages at `iosenderv2.github.io/ioSender` — plus an in-app hook that opens it *in context*.

- **Context help** — `F1` opens the manual at the page for whatever tab is current (a `ViewType`→anchor map in `ManualHelp`, resolving a bundled copy next to the exe or the online copy). The Help menu's *Usage tips* and *A brief tour* are repointed from the upstream wiki to the V2 manual.
- **The manual** (in `docs/manual/`, published from an isolated `gh-pages` branch) — 18 topics threaded into three reading tracks (novice / intermediate / machinist), an A–Z subject index, live search, three hand-built inline SVG diagrams (MCS-vs-WCS, machine anatomy, tool-change flow), 13 screenshots, and a lazy "Watch on the machine" video hook for future walkthroughs.

In the integration build. The manual site + its authoring tools (`docs/manual/*`) are not enumerated above (self-referential docs).

#57 Fixes: Start Job tab strand + program-view overlay width

File	+	−
`ioSender XL/MainWindow.xaml.cs`	21	0
`ioSender XL/JobView.xaml.cs`	4	0
`ioSender XL/MainWindow.xaml`	1	1

Two rough edges surfaced while documenting the Start Job flow.

- **Stranded tab** — tab enablement was driven by fragile `Hold` / `IsGCLock` event pairs (Start Job's auto-probe locks G-code); a mis-paired clear could leave a connection-gated tab (e.g. Start Job) disabled with no way back. New `RefreshTabsIdle()` reconciles every tab to its correct connection + capability state whenever the controller is idle and nothing is streaming — idempotent, and it never runs during a real job.
- **Program-view overlay** — the generated-program overlay had no maximum width, so it grew to cover the whole tab and hid the Start Job inputs; capped with `MaxWidth="720"` so it stays a right-hand column.

In the integration build. Both regressions trace to the run-bar / StretchTabControl era (#45, #50).

#58 Numeric fields select their value on focus

File	+	−
CNC Controls/NumericTextBox.cs	35	0

Tabbing or clicking into any numeric field now selects the whole value, so you can just type the replacement instead of clearing the old number first.

- Applied once in the shared `NumericTextBox`, so it covers every numeric field app-wide (DRO, Settings, Probing, Start Job, jog step, the wizards…).
- Keyboard / tab focus selects immediately; a focusing click lets WPF place the caret natively then selects on mouse-up (plain click only, so a click-drag still keeps your partial selection). Once the field already has focus, later clicks fall through untouched — so editing a single digit still lands the caret where you click.

In the integration build.

#59 Connect: Network default + LAN controller discovery

CNC Core/NetworkScanner.cs	349	0
CNC Controls/PortDialog.xaml.cs	127	3
CNC Controls/PortDialog.xaml	32	6
Locale/*/csv (7 locales)	19	0

The Connect dialog now opens on the **Network** tab by default (most grblHAL controllers are networked), and a new **Scan network** button finds controllers on the LAN so you don't have to know the IP.

- **Discovery by protocol, not open ports.** An async subnet sweep (up to a /24 per interface, 64 concurrent) probes each host and confirms it's really a grbl controller by sending the real-time status request `?` and requiring a valid `<State|…>` report back — then reads `$I` to show "grblHAL <version>". Random devices with telnet open are rejected. mDNS (`grblHAL.local`) is tried first but not relied on: consumer WiFi routers routinely drop mDNS multicast between the wireless and wired segments, so the subnet scan is the real workhorse.
- **Editable host picker.** The IP field is now a combo — type a host, or pick one the scan found (the dropdown shows "host:port — grblHAL x.x", the edit box shows just the host, and OK connects to whatever is current).
- The dialog also no longer hides behind the startup splash (it docks to the top of the screen while the splash is up), and its fields no longer clip at high DPI.

In the integration build.

#60 Drag to reorder tabs (persisted across restarts)

CNC Controls/StretchTabControl.cs	178	0
CNC Controls/TabOrderConfig.cs	48	0
CNC Controls/AppConfig.cs	56	0
ioSender XL/MainWindow.xaml.cs	13	1
CNC Controls/ToolsView.xaml.cs	5	1
CNC Controls Probing/ProbingView.xaml	5	5
CNC Controls/GrblConfigView.xaml	1	1

Grab any tab **header** and drag it left or right to change tab order — on the main bar and on every sub-tab strip (Settings, Tools, Probing). A plain click still just selects the tab; the drag only starts once the pointer passes the system drag threshold, and the cursor switches to a ↔ while dragging. The new order now **survives a restart**.

- **Persisted to whichever store already governs a bar — never a second source of truth.** The Probing and Settings strips (which had no order config before) are saved by the control itself into a new `TabOrder` section of `App.config`, keyed per strip. The main bar and the Tools sub-tabs are already backed by the layout tree, so a drag there writes into `Config.Tabs` / the layout tree — the *same* store the **Edit Main Page** → **Tabs** editor uses, so dragging and the editor always agree (and drag takes effect immediately, no restart needed).
- **Capability-hidden tabs stay put.** Tabs hidden for the current controller (e.g. Lathe Wizards outside lathe mode, Trinamic / PID when unsupported) keep their stored position — only the on-screen tabs reorder among themselves, so a hidden tab can never be dropped from the saved layout.

In the integration build.

#61 grbl:Settings tree populates when opened after connecting

`CNC Core/Grbl.cs`	12	1
`CNC Controls/GrblConfigControl.xaml.cs`	25	3

Opening **Settings › Grbl** after connecting to a grblHAL controller could show an empty settings tree — no settings at all, and a `$`-search found nothing. The flat-tab Settings redesign builds the settings control eagerly at startup, and during the connect handshake the Grbl tab gets an early activation *after* the controller answers `$$` but *before* `$I` establishes that it reports setting enums. The settings load then took the plain `$$` path and populated the list group-less, skipping the setting-details/group query that builds the tree — and because the list was now non-empty, every later load skipped that query too, so the group tree was never built.

- **Build the metadata when it's finally available.** The settings load now also fetches the setting-enum details and group tree when the controller reports enums but the tree hasn't been built yet, dropping the stale group-less entries first so they're rebuilt with full name/type/group metadata (no duplicates).
- **Reach the load on entry.** The Grbl tab no longer short-circuits its activation when it was pre-activated (empty) during the disconnected startup pass, so opening the tab after connecting always (re)builds the tree.

In the integration build. Fixes a regression from the Settings flat-tab redesign (#42); supersedes an earlier partial fix that only re-bound the (still-empty) tree.

#62 Probing & Tools sub-tabs share one layout

CNC Controls Probing/EdgeFinderControl.xaml	10	8
CNC Controls Probing/EdgeFinderIntControl.xaml	10	8
CNC Controls Probing/CenterFinderControl.xaml	10	8
CNC Controls Probing/ToolLengthControl.xaml	6	4
CNC Controls Probing/ProbingView.xaml	1	1
CNC Controls/StepperCalibrationWizard.xaml	1	1

A consistency pass so the descriptive/instructions area and the results area look the same across the four Probing sub-tabs (Tool length offset, Edge finder external/internal, Center finder) and the Stepper calibration tool, matching the newer Tools tabs: **instructions on the ambient window grey, results in a white panel**, with clear separation.

- **Instructions on the ambient grey, not a near-white box.** The instruction text sits directly on the window grey (as the Tools tabs do), and each tab's output (position / message / preview) is a bordered white panel.
- **The real culprit was the tab host.** The Probing sub-tabs render inside a `StretchTabControl` that — unlike the Tools one — never set `Background="Transparent"`, so the stock TabControl template painted its content area white behind the transparent instructions. Making the Probing host transparent lets the ambient grey show through, so all four tabs finally match.

In the integration build.

#63 Check mode ($C) moved to a Cycle Start context-menu item

CNC Controls/JobControl.xaml.cs	22	0
CNC Controls/JobControl.xaml	11	1
CNC Controls/StatusControl.xaml.cs	1	11
CNC Controls/StatusControl.xaml	0	10
Locale/*/csv (7 locales)	14	7

The **Check** checkbox next to the *State* field on the status bar was misplaced and rarely used. It's gone from the status bar; the grbl check-mode toggle is now a checkable **"Check mode ($C)"** item on the **Cycle Start** button's right-click menu — a dry-run / validate toggle now lives with the button that starts a run.

- **Same semantics.** The menu item mirrors the live check-mode state, is enabled only when not running a job and not asleep, sends `$C` from Idle to enter check mode, and soft-resets to leave it — exactly as the old checkbox did.
- **Localized.** The new menu string (header + tooltip) is added to all 7 locale CSVs; the retired checkbox label rows are removed.

In the integration build.

#64 Opt-in diagnostic trace log (`-debuglog`)

`CNC Core/DebugLog.cs`	136	0
`ioSender XL/App.xaml.cs`	36	5
`CNC Controls/GrblConfigControl.xaml.cs`	6	0
`CNC Core/CNC Core.csproj`	1	0

A lightweight, always-compiled-in diagnostic logger — `CNC.Core.DebugLog`. It is **off by default** and costs a single boolean test per call site when disabled, so `DebugLog.Write(…)` statements can be left permanently at the tricky points in the code rather than being hand-added and torn down each time a layout / flow bug needs tracing. Diagnosing the earlier empty-settings-tree regression (#61) meant hand-rolling a throwaway logger; this makes that capability standing.

- **How it's turned on.** Launch with `-debuglog` (all categories) or `-debuglog=settings,comms` (an allow-list); an `IOSENDER_DEBUGLOG` env var does the same for unattended/headless runs (mirrors `IOSENDER_HEADLESS`).
- **Where it writes.** Timestamped lines (`HH:mm:ss.fff [category] message`) to `%AppData%\ioSender\ioSender.debug.log`, next to the crash log, with a per-run banner (version + categories) and automatic roll at 8 MB. Thread-safe append, and never throws — logging can't take the app down.
- **First call sites.** The fragile grbl:Settings activation path (`Activate` / `ConfigureView`) now traces its `HasEnums` / `IsLoaded` / setting & group counts, so a recurrence of the empty-tree symptom is visible in the log immediately.

In the integration build. Available in every configuration (not Debug-only), gated by the flag.

#65 grbl:Settings premature activation prevented at source

CNC Controls/GrblConfigView.xaml.cs	19	0

The root trigger behind the empty-settings-tree saga (#61), now closed off at the source. The Settings view's inner tab strip raises an initial `SelectionChanged` for its default tab during eager startup layout — before the view is ever shown and, on a network connect, mid-handshake. That fired a premature `Activate(true)` on the Grbl tab while `$I` hadn't yet reported enum support (`HasEnums == false`), which is exactly what loaded the settings group-less.

- **The fix.** A `_viewActive` flag, set only by the top-level `Activate(bool, ViewType)`. `tab_SelectionChanged` now ignores inner selection churn until the view is genuinely active — the top-level `Activate(true)` already enters the current tab itself, so real user tab switches still work, but the spurious startup fire is dropped.
- **Belt-and-suspenders** over the #61 `Load()` guard (which rebuilds the tree metadata whenever it's missing): even if an early activation slipped through, the tree would still build — but now it can't fire disconnected in the first place. Verified with the #64 trace log: the mid-handshake `Activate` line is gone; the first and only activation lands post-`$I` with the groups already populated. Also removes wasted startup work.

In the integration build.

#66 Startup flag `-forgetNetwork`

`CNC Controls/AppConfig.cs`	13	0

A small startup/testing aid: launch with `-forgetNetwork` to **ignore the saved connection target for that run** and open the connection dialog instead of auto-connecting the remembered host — from there you can scan the network or pick any transport (serial / network / simulator).

- **Non-destructive.** Nothing is written to `app.config` unless you then choose a target, so the remembered value survives; on the next serial connect it's re-learned from the controller's `$I`-reported IP anyway. It reuses the existing `-selectport` path to force the dialog.
- **Why.** The modal connect dialog delays the handshake, which makes it a clean, repeatable way to reproduce startup-timing bugs (such as the #65 activation race) without hand-editing the config. The connect path also gained a `[connect]` trace line (see #64) reporting the saved target and flags.

In the integration build.

#67 Demo-video capture markers (`-demomarker`)

CNC Core/DemoMarker.cs	94	0
CNC Core/GrblViewModel.cs	1	0
CNC Core/CNC Core.csproj	1	0
ioSender XL/App.xaml.cs	14	0
ioSender XL/MainWindow.xaml	32	1
ioSender XL/MainWindow.xaml.cs	76	0

Production aids for recording the manual's demo videos — opt-in via `-demomarker` (or the `IOSENDER_DEMOMARKER` env var) and invisible in normal use. A sibling of the #64 trace log; the full capture-and-composite procedure lives in `docs/demo-videos/`.

- **Marker log.** `CNC.Core.DemoMarker` writes a small CSV (`%AppData%\ioSender\ioSender.demo-markers.csv`, started fresh each launch) of timestamped events: `SESSION_START`, `RUN_START`/`RUN_END` (from the single `IsJobRunning` edge transition), and `TIMELAPSE_ON`/`TIMELAPSE_OFF`. A video compositor keys off these to *sync* the app screen-recording to the machine-camera footage and to speed up the timelapsed stretch — no hand-keyframing.
- **Timelapse toggle.** A run-bar toggle (shown only under `-demomarker`, beside the State field) marks the timelapse window live — flip it as you switch the camera to / from timelapse. Traffic-light coloured: green = off (press to start), red = running (press to stop). Fallbacks if you never flip it: auto-on a few minutes after Cycle Start, forced off at job end.
- **Near-free when off:** a single boolean test per call site — nothing is written and the toggle is collapsed on normal runs.

In the integration build. Builds clean (Debug + Release).

#68 OBS auto-record bridge + tool-change cut markers

CNC Core/ObsBridge.cs	168	0
CNC Core/GrblViewModel.cs	13	0
CNC Core/CNC Core.csproj	1	0
ioSender XL/App.xaml.cs	14	0
ioSender XL/MainWindow.xaml.cs	2	0
build.ps1	13	3

Extends the #67 demo facility so a shoot can be hands-off. When armed with `-demomarker`, ioSender connects to **OBS Studio**'s built-in **obs-websocket** server (v5 handshake, optional auth via `IOSENDER_OBSWS_PASSWORD`, defaults to `localhost:4455`) and drives recording from job state — no manual Record button. Reuses the websocket-sharp dependency already in the build; never throws and no-ops if OBS isn't reachable.

- **Auto record. Program load** → `StartRecord`; **program end** (unload) → `StopRecord`. The recording spans the whole session (setup, probing, every cut, tool changes), so it doesn't stop between phases.
- **Cut markers refined.** `RUN_START/RUN_END` now mark the *actual cutting*. Entering the `Tool` state (an M6 tool change) ends the current cut and leaving it starts the next — i.e. `…RUN_END <tool-change> RUN_START…` — so the compositor can treat cutting spans and tool-change gaps differently.
- **Launch pass-through.** `build.ps1` now forwards any trailing tokens to the launched exe, e.g. `.\build.ps1 -Launch -demomarker -forgetnetwork`.

In the integration build. Builds clean (Debug + Release). Camera-agnostic — drives OBS, so it works with any source (RTSP cams, screen capture). See `docs/demo-videos/`.

#69 Per-Monitor V2 DPI awareness

`ioSender XL/app.manifest`	33	0
`ioSender XL/MainWindow.xaml.cs`	40	0
`ioSender XL/App.config`	5	0
`ioSender XL/ioSender XL.csproj`	1	0

ioSender was **System-DPI-aware** (no manifest) — it used a system-DPI value cached at logon, so a resolution / scale change made in the current session (or a stale per-monitor override) could leave it rendering at the wrong scale until the next sign-out. It is now **Per-Monitor V2** aware, reading the *live* per-monitor DPI like Edge and VS Code.

- **How.** An `app.manifest` declaring `dpiAwareness = PerMonitorV2` (referenced from the csproj), plus the WPF `Switch.System.Windows.DoNotScaleForDpiChanges=false` app-config switch so the visual tree re-scales on a live DPI change.
- **Diagnostics.** A `-debuglog=dpi` trace (see #64) logs the resolved `DpiScale` / device transform / screen & window metrics at startup and on every `DpiChanged`, so a "wrong size" report can be pinned to a scale value in seconds.
- **Note.** Interop surfaces (3D carve view, camera) should be spot-checked across a live DPI change; the switch can be flipped back to keep launch-time correctness without live re-scaling if needed.

In the integration build. Builds clean (Debug + Release). Verified at 100% / 175% / 300% — the app tracks the live monitor scale.

#70 Named G28 fixtures

CNC Controls/NamedPositionConfig.cs	77	0
CNC Controls/OffsetFlyout.xaml.cs	215	3
CNC Controls/GotoBaseControl.xaml.cs	24	0
CNC Controls/OffsetFlyout.xaml	3	0
CNC Controls/AppConfig.cs	3	0
CNC Controls/CNC Controls.csproj	1	0
Locale/*/csv/CNC.Controls.WPF (7 locales)	13	0

A saved library of **named machine-coordinate positions** (“fixtures”) recalled from the **G28** sidebar flyout — for recurring jigs like a 90° sheet-goods fence origin or a machinist-vice corner. grblHAL stores only a single G28 slot, so the library lives app-side (in machine coordinates) and never clobbers it.

- **In the flyout.** An editable, alphabetically-sorted name dropdown sits left of the *Go* button (G30 / G59.3 flyouts are unchanged — those are deliberate out-of-envelope park spots, not fixtures). **Set** stores the current machine position under the box’s name; **Go** rapids to the selected fixture with the same Safe-Z lift as the G28 button (a new `GotoBaseControl.SafeGotoMachine`, app-side `G53`), falling back to the firmware G28 when no fixture is named.
- **Position-aware.** On focus the box reconciles the name to where the machine actually is — the matching fixture, else the next `Fixture-N` default — so Set saves the right thing without hunting.
- **One name per position.** Set renames / dedupes / updates in place, and duplicates left by earlier sessions self-heal on launch (keeping your custom name over an auto `Fixture-N`).

In the integration build. Builds clean (Debug + Release). Hardware-verified. Loc: `cbo_fixtures` tooltip in all 7 CNC.Controls.WPF CSVs.

#71 Smoother, faster corner probing

`macros/probe_tfl.macro`	18	11
`macros/pcorner.macro`	12	11

Two refinements to the Start Job corner-probe macros (`probe_tfl` / `pcorner`).

- **No more table shudder.** Every post-trigger move — the pull-off between the coarse and latch probes, and the retract after the latch — was a `G0`, so it ran at max rapid *acceleration*: a violent jerk on a 4–10 mm move that shook the whole gantry. They are now `G1 F1000`: identical distance and accuracy (the measurement is the slow `G38.2` latch), but a bounded peak velocity so the ramp is gentle.
- **≤ 1" stock optimization.** The spoilboard probe now rapids straight down to the stored G28 Z (the same sequence as the G28 flyout button) and seeks only a short capped distance; the stock-top probe starts just above the tallest possible 1" top instead of machine-top, cutting its seek from ~100 mm to ~32 mm. Documented as preconditions: stock ≤ ~25 mm and G28 Z set just above the spoilboard.

In the integration build. Hardware-verified ("smooth as silk"). On-controller macros — re-uploaded to littlefs by Start Job's checksum provisioning.

#72 Start Job: estimated stock size & Z-thickness check

ioSender XL/StartJobView.xaml.cs	13	0
ioSender XL/StartJobView.xaml	5	2
CNC Controls/StartJobConfig.cs	1	0
Locale/*/csv/ioSender.resources (7 locales)	28	14

The Start Job setup fields are relabelled **Est.** (they are an estimate a measure run refines), and a new **Est. thickness (Z)** field is added. Since the probe macros now assume stock ≤ 1″ (#71), a red warning appears under the field when the Z estimate exceeds 25.4 mm — live and on load. The estimate persists with the other Start Job inputs; it is advisory only.

#73 Playbooks: one-stop procedure library

`docs/playbooks/*.md (14 files)`	456	0

The repeatable development procedures — previously scattered through dev notes — are collected into a single checked-in `docs/playbooks/` folder, **one procedure per file**, each with the ordered steps plus a ready-to-run command or copy-paste fragment (substitute the parameters and go). `README.md` indexes the set.

- **Docs & release:** add a changelog entry (the four places to touch + the detail/TOC HTML skeletons), regenerate `Overview.pdf`, publish the manual site, re-import a manual screenshot, wire a video into the manual.
- **Build & ship:** the build/commit/test loop, the headless `build.ps1`, the Teensy firmware build, the end-of-session wrap-up sequence and conversation-log capture.
- **Cross-cutting:** the localization catch-up pass, running `gh` / the GitHub API on this box, and compositing a demo video.

Documentation only — no code change. Each playbook cites the repo script it drives (`build.ps1`, `tools/locadd.py`, `docs/manual/publish-pages.ps1`, …) as the authoritative implementation.

#74 Startup dialogs no longer hidden behind the splash

ioSender XL/SplashWindow.xaml(.cs)	14	1
CNC Controls/PortDialog.xaml.cs	4	16

The startup progress splash was `Topmost` and centred, so any dialog that opened during startup was mostly hidden behind it. That was previously worked around for the connect dialog alone by docking it to the top of the screen — but other startup popups (message boxes, machine-setup prompts, the "controller never reported" safety net) still landed behind the splash.

- **Fixed at the source:** the splash now docks itself near the top of the work area, leaving the centre clear for *every* dialog. The connect-dialog special-case is removed — it simply centres on screen again (still owns to the splash so it wins z-order).

#75 Status-bar messages briefly shown at double height

ioSender XL/MainWindow.xaml.cs	37	0

The status line at the bottom of the window is short and easy to miss. When a new message is written to it, it is now shown at **twice its normal height for 10 seconds** (or until the next message re-triggers the enlarge), then shrinks back — so an important message stands a chance of being noticed. An empty/cleared message reverts immediately, and the base size tracks the theme/DPI font scaling.

#76 Program view: 3-line run view on Cycle Start

CNC Controls/GCodeListControl.xaml.cs	90	0
CNC Controls/ProgramView.xaml(.cs)	66	2
ioSender XL/MainWindow.xaml.cs	14	0

When a wizard *Generate* pops its program view open and you press **Cycle Start**, the view now collapses to a compact **3-line run view** — previous line, current executing line (green), next line — kept centred on the executing line as the run advances, and dropped to the **bottom** of the tab content area (flush above the run bar) so a running program takes little space and stays in the line of sight.

- **Toggle:** click anywhere on the title bar to switch between the compact run view and the full program (hand cursor + hint text make it discoverable).

#77 Selected tab stands out — bold, slightly larger label

`CNC Controls/StretchTabControl.cs` 62 1

The active tab was distinguished only by its white background, easy to lose track of on the wider strips. The selected tab's **label** now also goes **bold and ~18% larger** so the current tab reads at a glance. Applies to every bar built on `StretchTabControl` (main bar + Settings / Tools / Lathe / Probing / Machine Setup) with no per-view change — see #50.

- **Header only:** the emphasis is set on the tab's header presenter, not the `TabItem` — a `TabControl` re-parents inheritance so font set on the selected `TabItem` would bold the whole page; confining it to the header leaves the page content untouched.
- **No reflow:** each tab keeps its already-assigned width, so enlarging the selected label doesn't retrigger the stretch recompute — the bolder text simply fills its fixed slot (a touch cramped when selected, by design).

#78 Run-bar jog pad restored & the button row rebalanced

ioSender XL/MainWindow.xaml	34	33
CNC Controls/StatusControl.xaml	5	3
CNC Controls/JobControl.xaml	3	3

The shrunk jog pad in the bottom run bar had disappeared: it lived on the leftover space right of the run buttons, and the **Program** toggle added to that row (the program-overlay work) widened it and consumed the leftover, collapsing the pad to zero width.

- **Four-column run bar:** *button row · jog pad · signals+feed · overrides*. The button row is the flexible (star) column and stretches to fill the slack up to the jog pad — its three groups spread across so the run buttons span the width instead of huddling left, with the MDI and State lines following so *Send* stays aligned with *Program*. The jog pad, signals and feed/rapids overrides are content-sized and ride at the right edge, adjacent.
- **Reclaimed the width:** run buttons are now content-sized (one space of padding each side) instead of a fixed 66 / 95 px, and the Signals + override clusters were shrunk ~25%, so the jog pad has room.

#79 Bind any tab to a key — right-click Bind-to-Key + header badges

File	+	−
CNC Controls/TabKeyBinder.cs	259	0
ioSender XL/MainWindow.xaml.cs	122	4
CNC Controls/KeyMapEditor.xaml.cs	142	2
CNC Controls/GrblConfigView.xaml.cs	46	1
CNC Controls/MachineSetupWizard.xaml.cs	35	0
CNC Controls Lathe/LatheWizardsView.xaml.cs	27	1
CNC Controls Probing/ProbingView.xaml.cs	27	1
CNC Controls/ToolsView.xaml.cs	27	2
CNC Controls/AppConfig.cs	19	0
CNC Controls/MachineSetupView.xaml.cs	7	1
CNC Controls/KeyMapEditor.xaml	2	3
CNC Controls/ComponentAvailability.cs	1	1
CNC Controls/LibStrings.xaml	1	1
CNC Controls/CNC Controls.csproj	1	0

Every main-page tab *and* its second-level sub-tabs can now be bound to a keyboard shortcut, so you jump straight to a tab from anywhere instead of hunting through the strip. Builds on the in-app Key Mappings editor (#3).

- **Right-click any tab → Bind to Key** (or *Change Key* / *Clear Shortcut*): a small capture modal takes one combination — no need to open the editor and find the action. The current shortcut shows as a dim **badge in the tab's upper-right corner**, live-updating.
- **Full coverage:** the 10 main tabs plus the inner tabs of *Settings* (7), *Probing* (4), *Tools* (7), *Machine Setup* (Overview + 6 steps) and *Lathe Tools* (4). Nested `Tab.<Parent>.<Sub>` ids select the parent tab then the inner one through a shared `ITabBindingHost`; bindings persist in `Config.TabShortcuts` and dispatch at the window level so they fire regardless of focus.
- **Smart no-ops:** a stale binding to a tab that has since become unavailable (capability removed, disconnected, mode disabled) does nothing at all rather than half-switching to the parent. Text-producing combos (no modifier, or Shift only) are never stolen from a focused text box.
- **Editor parity:** the Key Mappings list groups every tab by parent; tab rows highlight when bound (changed from the unbound default), the same whether set via right-click or in the editor, and re-sync on show.
- Also fixed a pre-existing key-capture bug in the editor (a stray `FocusManager.IsFocusScope` swallowed keypresses) and switched its list to line-by-line scrolling. Renamed the *Lathe Wizards* tab label to **Lathe Tools**.

#80 Capability-gated tabs own their own “unavailable” reason

File	+	−
CNC Controls/IAvailabilityGated.cs	26	0
CNC Controls/StretchTabControl.cs	23	0
CNC Controls/ComponentAvailability.cs	21	32
CNC Controls/ToolsView.xaml.cs	7	33
ioSender XL/JobView.xaml.cs	7	21
CNC Controls/AutoSquareWizard.xaml.cs	9	1
CNC Controls Probing/ProbingView.xaml.cs	8	1
CNC Controls/SDCardView.xaml.cs	6	1
CNC Controls/ToolView.xaml.cs	6	1
CNC Controls/PIDLogView.xaml.cs	5	1
CNC Controls/TrinamicView.xaml.cs	5	1
CNC Controls Lathe/LatheWizardsView.xaml.cs	5	1
CNC Controls/CNC Controls.csproj	1	0

When the connected controller lacks a capability, the tab that needs it is removed and listed — with the reason — under *Edit Main Page › Unavailable*. That reason used to be a hand-maintained switch kept *separately* from the code that actually removes the tab, so the two drifted: the list claimed SD Card needs an SD card while the real test was “any filesystem” (SD *or* LittleFS).

- **One source of truth:** each gated view now owns its own prerequisite + message through a new `IAvailabilityGated` interface (Probing, Lathe Tools, SD Card, Tool table, Trinamic, PID, Auto Square). The removal and the listing read the same property, so they can’t disagree.
- **No central list:** `StretchTabControl.PruneUnavailable()` — used by both the main tab bar and the Tools sub-bar — drops the unavailable tabs and records their reasons; the old string switch is gone.
- Fixes the SD Card mismatch (both now agree on “has a filesystem”), and Auto Square stays as a squareness gauge when the offset setting is absent, with the limitation noted rather than the tab removed.

Internal refactor; no visible behaviour change beyond the corrected SD Card reason. Release build verified.

#81 No more restart prompt hiding behind the relaunch splash

File	+	−
CNC Controls/GrblConfigView.xaml.cs	18	1

Applying a restart-only change (e.g. toggling lathe mode) and relaunching could leave a “Restart ioSender now?” message box stranded *behind* the new instance’s splash screen — it only surfaced after the freshly launched copy went away, looking like the app prompted *after* it had already restarted.

- **Cause:** the relaunch launches the replacement (splash on top) and then shuts down, and the shutdown path re-enters the Settings “on leave” logic — which, seeing a restart still pending, popped the prompt a *second* time from the dying instance.
- **Fix:** once a relaunch is under way the shutting-down instance suppresses all further on-leave prompts (it has already saved), so no stray dialog is left behind the splash.

Long-standing glitch, unrelated to #80. Release build verified.

#82 Dev playbooks distilled to one-command scripts

File	+	−
`tools/add-changelog-entry.ps1`	171	0
`tools/push-all.ps1`	79	0
`tools/regen-overview-pdf.ps1`	54	0
`tools/gh.ps1`	38	0
`docs/playbooks/*.md (5 files)`	42	11

The mechanical playbook procedures — the ones that are a fixed command sequence rather than judgement — are now small scripts in `tools/`. The value is retrieval over re-derivation: one path plus args instead of reconstructing the command (and its failure traps) every session. The playbooks stay as the invocation note and keep the gotchas.

- **tools/push-all.ps1** — pushes the work all the way out: `origin/integration` *and* `v2/master`, with an ahead/behind report, a clean-tree guard, and a verify that both refs actually landed (`-DryRun` to preview).
- **tools/regen-overview-pdf.ps1** — regenerates `Overview.pdf` via headless Edge with a fresh `--user-data-dir` each run (without it Edge silently no-ops) and a LastWriteTime check that fails loudly if it did.
- **tools/gh.ps1** — runs the portable `gh.exe` with `GH_TOKEN` injected from the registry (harness shells do not inherit it), args passed straight through.
- **tools/add-changelog-entry.ps1** — adds a `#N` entry to this changelog from a JSON spec: it derives the number, computes every count from the file rows and re-sums the grand totals, inserts the detail block, TOC row and at-a-glance row, bumps the header count, and self-checks that all four places agree. You author only the content — this very entry was generated by it.

Tooling only; no application change, so nothing to build.

#83 Relaunch supervisor, true single-instance & crash reporting

File	+	−
ioSender XL/ioSender XL/CrashReporter.cs	584	0
CNC AppLaunch/CNC AppLaunch/AppLaunch.cs	106	33
CNC Controls/CNC Controls/PipeServer.cs	51	25
ioSender XL/ioSender XL/App.xaml.cs	48	0
CNC Core/CNC Core/RelaunchSupervisor.cs	38	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	13	0
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	10	0

Three linked robustness features around the app's launch, restart and crash lifecycle. `AppLaunch.exe` — historically the single-instance file-open forwarder — is repurposed as a small, application-agnostic **relaunch supervisor**: `AppLaunch.exe -ExitCodeToRelaunch 240 -- <exe> [args]` starts the child with an `EXITCODE_TO_RELAUNCH` environment variable set and relaunches it whenever it exits with exactly that code (any other code propagates and ends the loop). The env var is the whole handshake; ioSender knows nothing about AppLaunch beyond it.

- **True single instance.** On startup ioSender probes its named pipe: if another instance is already running it forwards the file argument (if any), raises that window, and exits — a second launch never opens a second window. This moved out of AppLaunch and into ioSender, where the server already lived.
- **Clean supervised restart.** When supervised, the in-app "restart to apply" simply exits with code 240 and lets the supervisor relaunch it — no in-process re-spawn, so the restart prompt can no longer strand itself behind the relaunch splash. Unsupervised runs (development, headless) keep the in-process relaunch as a fallback.
- **Opt-in crash reporting, no backend.** A fatal unhandled exception now writes a real minidump plus a text summary to `%AppData%\ioSender\crashes`. The next launch shows a consent dialog with the summary; "Send report" bundles the dump into a `.zip` (GitHub rejects raw `.dmp`), opens a pre-filled GitHub issue, and docks the file-manager window to a strip across the top of the screen with the zip selected, ready to drag onto the issue — sized in physical pixels and raised past the foreground lock so it works correctly on high-DPI displays. A topmost instruction banner spells out the one step to take.

Builds clean (Debug + Release). Wiring the desktop shortcut / file association to AppLaunch is a packaging step (no installer in this repo). Hidden `-crashtest` flag exercises the crash → report path.

#84 Main-menu overhaul: dissolve the File menu, slim the menu bar

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	47	37
CNC Controls Camera/CNC Controls Camera/ConfigControl.xaml.cs	43	0
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	29	0
CNC Controls/CNC Controls/AppConfig.cs	27	1
ioSender XL/ioSender XL/ProgramPanel.xaml	26	6
ioSender XL/ioSender XL/ProgramPanel.xaml.cs	19	0
CNC Controls/CNC Controls/Converters.cs	15	0
CNC Controls Camera/CNC Controls Camera/ConfigControl.xaml	14	0
ioSender XL/ioSender XL/MainWindow.xaml	13	20
ioSender XL/ioSender XL/JobWorkspace.xaml.cs	12	0
CNC Controls Camera/CNC Controls Camera/Camera.xaml.cs	11	0
CNC Controls/CNC Controls/GCode.cs	7	0
CNC Controls/CNC Controls/GCodeListControl.xaml	6	1
CNC Controls/CNC Controls/ICamera.cs	1	0
Locale/*/csv (7 locales x 3 assemblies)	188	0

The menu bar is slimmed to essentials and the File menu is dissolved, with each item rehomed to where it is actually used.

- **Program-view header** â€" with no file loaded it offers Load File / Load Folder; once loaded it shows the program source with a close (Ã—). Save and a Transform submenu move to the program list's right-click menu â€" Save works on any program view, Transform is rebuilt per menu-open.
- **File menu gone** â€" Connect/Reconnect is a top-level item, Open Console is a double-click on the Console tab (Esc still toggles it), Open Application data folder moves to a new Help â†' Support submenu, and Exit is dropped (the window close / Alt+F4 quit).
- **Toolbar row removed** â€" the file-action icons and quick-run macro buttons beneath the menu bar are gone; macros still run by F-key.
- **Camera is opt-in** â€" instead of always showing (and opening a laptop's built-in webcam), the Camera menu appears only when a video device is bound via a new Device picker + Connect/Disconnect button in Settings â†' App â†' Camera.

Builds clean Debug + Release; user-verified. Localized across all 7 locales. Commits 52b51d1 / 54ebcdd / 9511d3e.

#85 Start Job corner probe: one macro + sacrificial spacer/backer support

File	+	−
macros/pcorner.macro	21	14
macros/probe_tfl.macro	0	147
macros/cal.macro	0	3
macros/start_job.macro	8	49
ioSender XL/ioSender XL/StartJobView.xaml.cs	9	3
ioSender XL/ioSender XL/StartJobView.xaml	2	0
CNC Controls/CNC Controls/StartJobConfig.cs	5	0
CNC Controls/CNC Controls/AtcMacros.cs	6	6
CNC Controls/CNC Controls/CNC Controls.csproj	0	2
SDCardView / MachineSetupWizard / JobView / GrblViewModel (string + comment cleanup)	8	8
Locale/*/csv (7 locales)	21	0

Start Job's corner probe is now a single macro, and it understands a sacrificial spacer/backer board under the stock.

- **One probe macro.** Dropped `cal.macro` and `probe_tfl.macro`; the sole corner probe is `pcorner.macro` — a superset (all four corners, tip radius and feeds from the 3D probe definition, Z soft-limit clamp, per-corner flatness). The live Start Job program already emitted `pcorner`, so nothing changed at run time; the old pair survived only as a pre-Generate seed and is gone, leaving fewer files to install on the controller.
- **Sacrificial spacer/backer.** A new optional *Spacer/backer (Z)* field covers the thin-sheet case — e.g. a 2.5 mm aluminium sheet taped to a 1/4 in MDF backer of the same footprint. The probe finds the real spoilboard in the clear air beside the stock, so `pcorner` now derives an effective floor = spoilboard + spacer and starts the face probe from there: the 2 mm tip lands on the metal instead of dropping into the backer/tape band. A spacer of 0 reproduces the previous spoilboard-only behaviour exactly, so bare-fence jobs are unaffected.

Hardware-validated on a grblHAL Teensy 4.1 rig: the 2 mm probe tip lands on a 2.5 mm sheet edge sitting on a 1/4 in backer, and a spacer of 0 leaves normal-stock probing unchanged.

#86 Start Job: fail fast when a corner is over bare board

File	+	−
macros/pcorner.macro	12	4

With *Measure stock size* on, corners 2–4 are positioned from the estimated stock size. If that estimate is too large a corner lands over bare spoilboard — and previously the top probe there sought all the way down to the board, mistook it for the stock top, then drove a side probe into empty air that only faulted at the search limit. Worse, a Feed-Hold then Stop during that latch could leave the controller needing a reset.

Those reuse corners now cap the top probe at the effective floor (spoilboard + spacer): a genuine corner still triggers on the material above it, but a corner over bare board reaches the floor with nothing there and the probe aborts immediately — no side search into air, and nothing to wedge. The first corner, positioned by hand from G28, still discovers the board as before, so flatness/skew measurement is unaffected.

#87 In-app UI test server (flag-gated developer / QA harness)

File	+	−
`ioSender XL/UiTestServer.cs`	472	0
`ioSender XL/App.xaml.cs`	23	0
`ioSender XL/MainWindow.xaml.cs`	10	1

A small in-process HTTP server, **off by default** and started only with `-testserver[=PORT]` or `IOSENDER_TESTSERVER=1|PORT` (default port 8760), that lets an external script drive the running UI and read state back — so a change can be exercised in the real app instead of always being handed off for someone to click. A normal user run never opens a socket.

- **Addressing by x:Uid.** Localization already put a unique `x:Uid` on nearly every interactive control; the server surfaces those at runtime (`UIElement.Uid`), walks the visual tree of the open windows, and acts through each element's *AutomationPeer* — the same machinery real UI-automation uses, so it respects enabled/disabled state and fires the real handlers.
- **Routes (JSON):** `GET /ping`, `/idle`, `/tree`, `/state/{uid}`; `POST /invoke/{uid}`, `/set/{uid}?value=`.
- **Nav tabs are addressable too.** The top-level tabs are built in code, so they carried no `x:Uid`; they now get one from the tab registry, and selecting a tab realizes its (lazily-created) content.
- **No new dependencies, no admin.** A raw loopback `TcpListener` (not `HttpListener`, which would need a `netsh` URL reservation) keeps it zero-config for unattended runs.

Debug and Release verified; the server is inert unless explicitly enabled.

#88 Machine Setup no longer false-flags “install ATC macros” on connect

File	+	−
CNC Controls/AtcMacros.cs	9	1
CNC Controls/SDCardView.xaml.cs	8	3

On connecting to an ATC controller, the Machine Setup gate could jump straight to step 6 (“install ATC macros”) even though the macros were already present — the step then painted green a moment later. A timing race between the setup check and the first controller-filesystem listing.

- **Cause.** The macro-status check lists the controller filesystem, which pumps the dispatcher and has a re-entrancy guard. When another listing was already in flight (just after it cleared the shared file table), the status check's own listing was skipped and it read a half-cleared table, so the required macros looked absent — and an absent-looking result was reported as *Missing*, which the gate read as “not installed.”
- **Fix.** The filesystem loader now reports whether it actually performed a listing; when it did not (skipped, or the link was down) the status check returns *unknown* rather than fabricating *Missing* rows. The gate already treats unknown as “no action,” and a genuinely-empty filesystem still lists a placeholder, so a real first-run *missing* case still routes to step 6 correctly.

#89 UI test server matured into a full automation harness

File	+	−
WpfUiTestServer/ (new assembly: engine, status seam, HTTP spec)	1371	0
CNC Core/AppDialogs.cs	92	0
ioSender XL/GrblStatusProvider.cs	53	0
ioSender XL/App.xaml.cs	63	3
MessageBox.Show → AppDialogs (7 projects, ~130 sites)	135	130
x:Uid on named controls (52 XAML files)	112	105
ioSender XL/MainWindow.xaml.cs	6	3

The flag-gated UI test server introduced in #87 was built out into a capable automation harness and lifted into its own **app-agnostic assembly** (`WpfUiTestServer`, no ioSender references). It still only exists behind `-testserver` / `IOSENDER_TESTSERVER`; a normal run never opens a socket. Full HTTP reference in `WpfUiTestServer/README.md`.

- **Drive & assert.** Address any control by `x:Uid` (with `?index=` for duplicates); `invoke/set/select`; read `/state`, `/tree`, and a `/uids` discovery list (uid + type + count). App-level `/status` (connection, controller state, streaming/job, DRO) via a host-supplied provider seam, and `/waitfor` to block on a control or status condition rather than guessing at timing.
- **See it.** `GET /screenshot[/{uid}]` returns a PNG of the window or an element.
- **Dialogs no longer block automation.** Every message box now routes through `AppDialogs` (a `MessageBox.Show`-shaped wrapper — identical for a real user); the harness can pre-arm/answer prompts and read their text back. All ~130 `MessageBox.Show` sites across seven projects were migrated (the fatal crash box excepted).
- **Input.** Keyboard injection (`/key`) and context-menu open/invoke (`/menu`).
- **Failure visibility.** `GET /exceptions` surfaces unhandled/route exceptions the app used to swallow into a crash-and-exit, and a small `ioSender.exit.json` reports the exit code + reason after the app goes away.
- **Addressability sweep.** Added `x:Uid` to every named, addressable control that lacked one (105 across 52 files), plus the generated nav tabs and Machine-Setup step tabs, so the whole UI is reachable.
- **Operator safety.** A pulsing red banner across the top marks when the app is under test-server control.

Off by default; loopback only, no auth — keep it test/dev-gated. New x:Uids on a few text-bearing controls await a localization CSV pass (English fallback works). Debug + Release verified.

#90 WpfUiTestServer extracted to a standalone NuGet package

File	+	−
`WpfUiTestServer/* — extracted to standalone repo/package`	0	1371
`ioSender XL/ioSender XL/ioSender XL.csproj`	1	4
`CNC Core/CNC Core/CNC Core.csproj`	2	5
`ioSender XL/ioSender XL.sln`	0	10
`build.ps1`	2	1

The app-agnostic UI test server assembly introduced in #89 now lives in its own public repository (`github.com/stevenrwood/WpfUiTestServer`) and ships as a NuGet package. ioSender references it like any third-party dependency; the in-repo copy is gone. No user-facing behaviour change — it remains off by default, behind `-testserver` / `IOSENDER_TESTSERVER`.

- **Package, not project.** Both consumers (`ioSender XL` and `CNC Core`) swapped their `ProjectReference` for `<PackageReference Include="WpfUiTestServer" Version="1.0.0">`; the project was dropped from the solution and the folder removed.
- **Published via Trusted Publishing.** The standalone repo packs and publishes to NuGet from GitHub Actions using OIDC (no stored API key), converted to an SDK-style `net462` project with package metadata.
- **Build now restores.** These projects had no prior NuGet dependencies, so `build.ps1` gained `-restore` on the MSBuild call. Building in Visual Studio restores automatically; a raw `msbuild` of the solution now needs a restore step too.

Dev/QA infrastructure only. Debug + Release both build-verified against the published package.

#91 AppData cleanup: a shared Backups folder replaces App.config.bak + settings-backups

File	+	−
CNC Core/Grbl.cs	37	81
CNC Controls/AppConfig.cs	46	21
CNC Controls/RestorePointDialog.xaml	1	9
CNC Controls/RestorePointDialog.xaml.cs	1	1
ioSender XL/JobView.xaml.cs	1	0

`%AppData%\ioSender` collected two separate backup mechanisms over time: a single rolling `App.config.bak` rewritten on every save, and a `settings-backups` folder taking a Grbl settings snapshot on every $-setting save (count-retained, 20 deep). Both are replaced by one shared `Backups` folder with day-based retention (10 days).

- **App.config** is snapshotted once at startup, before any in-session edits, to `Backups\<datetime>_App.config`; `Load()`'s crash-recovery fallback now reads the newest such file instead of a single `.bak`.
- **Grbl settings** are snapshotted once per successful *connect* (not on every settings save) to `Backups\<datetime>_Grbl.txt`, since a fresh connect-time snapshot is the restore point that matters - a per-edit changelog header is no longer written.
- The Restore-settings picker (`RestorePointDialog`) is updated to the simplified filename format and shared folder.

Verified live: startup and connect-time snapshots both confirmed written correctly; the old .bak/settings-backups artifacts removed from a real AppData folder without disturbing the live App.config.

#92 Remove start_job.macro file + its seeded macro catalog entry

File	+	−
CNC Controls/AtcMacros.cs	3	51
ioSender XL/StartJobView.xaml.cs	2	17
ioSender XL/JobView.xaml.cs	0	6
macros/start_job.macro	0	11
CNC Controls/CNC Controls.csproj	0	1
ioSender XL/StartJobView.xaml	1	1

Clicking *Generate* on the Start Job tab also wrote the generated program to `%AppData%\ioSender\start_job.macro` and, on the next ATC connect, seeded a “Start Job” entry in the macro catalog referencing it - so it could be re-run from the Macro list or an F-key outside the tab. But Start Job already runs entirely in-memory via `MacroProcessor.ActiveRun`, the same paradigm as the Auto Square, Stepper Calibration, and Surface Spoilboard wizards (none of which write a file); the disk copy and catalog entry were redundant with that and just left an extra file behind. Removed `WriteStartJobMacro`, `AtcMacros.SeedStartJobMacro`, and the embedded `macros/start_job.macro` template.

#93 Program view: fix Save for wizard previews, stop the popup duplicating the Job tab

File	+	−
`CNC Controls/GCodeListControl.xaml.cs`	36	6
`ioSender XL/MainWindow.xaml.cs`	11	41
`ioSender XL/MainWindow.xaml`	6	18

Two gaps left over from the ProgramView refactor (each Generate->Cycle Start wizard - Start Job, Auto Square, Stepper Calibration, Surface Spoilboard - now owns its own transient `ProgramView` instead of sharing one overlay):

- **Save program... was permanently disabled** for a wizard's own preview - its enablement still checked only the loaded-job singleton (`GCode.File.IsLoaded`), which is never true for a wizard's separate, transient program. It now also enables for a view showing its own generated program, and writes that program's lines to a file the operator picks.
- **The floating “Program” popup duplicated the Job tab.** With no wizard connected it fell back to showing the loaded job a second time - identical content to the always-docked Job-tab program list (`ProgramPanel`). The popup now hosts a view only when it is genuinely transient (`ProgramView.AutoShow` - a wizard preview or a plain macro run); the “Program” button disables itself whenever there is nothing to show there, since the loaded job already has a persistent home.

Verified live via the UI test server: Program button starts/stays disabled with nothing loaded, enables and shows the generated program on Start Job's Generate, and disables again on leaving the tab.

#94 Fixture definitions: named workholding positions replace the single-slot G28 combo

File	+	−
CNC Controls/CNC Controls/Fixture.cs	192	0
CNC Controls/CNC Controls/FixtureEditDialog.xaml	176	0
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	181	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	103	11
ioSender XL/ioSender XL/StartJobView.xaml.cs	108	14
macros/pcorner.macro	26	15
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	2	211
CNC Controls/CNC Controls/NamedPositionConfig.cs	0	77
CNC Controls/CNC Controls/MachineSetupWizard.xaml	37	2
ioSender XL/ioSender XL/StartJobView.xaml	20	1
CNC Controls/CNC Controls/AppConfig.cs	6	2
CNC Controls/CNC Controls/CNC Controls.csproj	8	1
CNC Controls/CNC Controls/OffsetFlyout.xaml	0	3

A new **Fixture definitions** library: named, typed workholding fixtures (Corner fence to start) each with their own saved machine position, edited from a new Machine Setup wizard step and consumed by Start Job's fixture picker - replacing the old single-slot firmware G28 named-position combo (retired here, folded into fixture instances).

- **pcorner.macro generalized** to read a fixture instance's saved position instead of the firmware's single G28 slot, so multiple named fixtures can each have their own reference.
- **Corner-probing schematic.** The fixture editor draws a clearance circle sized to the live 3D probe's body diameter as the jog target, instead of manually-entered offset fields.

Only Corner fence is wired to a working macro this round.

#95 Apply UiScale zoom to dialogs, not just MainWindow

File	+	−
`CNC Controls/CNC Controls/DialogScaling.cs`	32	0
`9 dialog .xaml.cs files (+1 line each, apply DialogScaling)`	9	0
`CNC Controls/CNC Controls/CNC Controls.csproj`	1	0

Dialog windows (probe/tool editors, job-parameter prompts, port picker, pending-changes, etc.) rendered at 100% regardless of the configured UiScale zoom. A shared `DialogScaling.Apply(this)` call, added to each dialog's constructor, fixes that.

#96 Shared program-comment parser; probe dialog tweaks; touch-plate diameter fix

File	+	−
CNC Controls/CNC Controls/GCodeProgramComments.cs	100	0
CNC GCodeViewer/CNC GCodeViewer/CarveView.xaml.cs	24	49
CNC Controls/CNC Controls/ProbeDefinitionEditDialog.xaml.cs	11	6
CNC Controls Probing/CNC Controls Probing/ProbingViewModel.cs	10	1
CNC Controls/CNC Controls/ProbeDefinitionEditDialog.xaml	11	2
CNC Controls/CNC Controls/ProbeDefinition.cs	7	3
CNC Core/CNC Core/Grbl.cs	1	1
CNC Controls/CNC Controls/CNC Controls.csproj	1	0

`GCodeProgramComments` parses a loaded program's `(TOOL T=n D=d TYPE= … )/(STOCK X=.. Y=.. Z=..)` comment lines (the Fusion ioSenderBatchPost add-in's format) once, shared by every consumer instead of duplicated regex logic - touch-plate probing's edge-radius compensation and CarveView's 3D carve simulation both switched to it. Probe definition dialog tweaks and a touch-plate diameter fix alongside.

#97 Fusion add-in: combine to one .nc; Load File gets outline parity; retire Load Folder

File	+	−
CNC Controls/CNC Controls/GCode.cs	13	126
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.py	78	88
Fusion Addin/README.md	29	62
CNC Core/CNC Core/GCodeJob.cs	58	2
CNC Controls/CNC Controls/FusionFolderLoader.cs	0	353
CNC Controls/CNC Controls/FolderPicker.cs	0	159
CNC Controls/CNC Controls/AppConfig.cs	2	16
CNC Controls/CNC Controls/JobControl.xaml.cs	3	7
ioSender XL/ioSender XL/ProgramPanel.xaml.cs	3	6
ioSender XL/ioSender XL/ProgramPanel.xaml	4	6
CNC Core/CNC Core/GrblViewModel.cs	5	4
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	5	5
ioSender XL/ioSender XL/MainWindow.xaml.cs	0	9
5 small config/xaml cleanup files	0	5

The Fusion `ioSenderBatchPost` add-in now **always** combines every posted operation into a single `<folder>.nc` (STOCK/TOOL header, per-op `(--- seq: name (Tn) ---)` section markers, tool changes/rapids restored inline) instead of leaving per-op files for Load Folder to stitch afterward.

- **Load File gets outline parity.** `GCodeJob.ParseFileLines` now recognizes those section markers and tags sections the same way Load Folder did, so a plain File > Load of the combined output shows the same collapsible, groupable outline.
- **Load Folder retired.** `FusionFolderLoader.cs` and `FolderPicker.cs` are gone; `GrblViewModel.IsFolderView` renamed to `HasOutline` (now set by either load path).

Phase 1 (the Python add-in) is code-review-verified only - no Fusion 360 available to run a real post in this environment.

#98 Machinist Vise fixture: probe, Start Job generation, unit toggle, accurate drawing

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	438	116
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	256	78
CNC Controls/CNC Controls/FixtureEditDialog.xaml	60	84
macros/pvisecorner.macro	100	0
CNC Controls/CNC Controls/Fixture.cs	44	30
CNC Controls/CNC Controls/ProgramView.xaml.cs	52	7
CNC Controls/CNC Controls/GCodeProgramComments.cs	29	6
CNC Controls/CNC Controls/AtcMacros.cs	20	6
ioSender XL/ioSender XL/StartJobView.xaml	10	12
CNC Controls/CNC Controls/StartJobConfig.cs	5	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	4	1
MachineSetupWizard.xaml(.cs), CNC Controls.csproj	3	2

Machinist Vise moves from a defined-but-unwired fixture kind to a fully working one, end to end:

- **pvisecorner.macro** - a dedicated macro, not a variant of pcorner.macro: the reference sits *over* the jaw interior (pcorner needs its reference *outside* the corner), so the face-probe logic is a mirror image - back off outward, then seek inward.
- **Set/Test position** in the Fixture dialog now actually probes the jaw's front-left corner and stores the resolved position, with async-completion tracking that fixed two real staleness bugs found on hardware: a jogged position captured before a blocking confirm dialog let a hardware MPG pendant move the machine before the probe ran, and Test position was reading controller state before its own async probe had finished.
- **Start Job's BuildViseProgram** sets origin X/Y from the probed jaw corner directly and finds Z by probing the centre of the entered stock footprint.
- **Stock size fields** gained an mm/in unit toggle (display-only - everything downstream still works in mm), are disabled until a fixture is chosen, and Spacer/backer (a Corner-Fence-only concept) is hidden for vise.
- **The selected fixture now persists** across restarts, not just the entered sizes.
- **The schematic drawing tracks the selected fixture's kind**: a vise's origin dot sits on the jaw's own corner (not always front-left), and the fixed/moving jaw bars are sized/spaced from the fixture's own Jaw width and Max opening - a dashed line marks the max-opening limit, turning red if the entered stock won't fit.
- **ProgramView.Stock** (declared-or-envelope stock size, scoped only to the loaded job) lets Start Job auto-fill from a loaded program's own (STOCK) comment and sanity-check an oversized/stale entry against it.

Corner Fence remains hardware-validated; vise Set/Test/Start-Job-generation self-verified on hardware this round, drawing changes UI-test-server-verified.

#99 pcorner.macro collision fixes; MacroProcessor truncates oversized comments

File	+	−
`macros/pcorner.macro`	43	28
`CNC Controls/CNC Controls/MacroProcessor.cs`	16	1

Two distinct collision bugs, both found probing a fence fixture on real hardware: a stale prior-corner safe-Z used for the pre-top-probe descent on REUSE corners, and the Y-face section descending onto the exact point the top probe had just touched instead of backing outside the face first. Every traverse/retract in the macro also switched from a bare rapid to a controlled feed, so a still-undiscovered logic bug produces a gentle push instead of a full-speed crash.

Separately: grblHAL rejects a line over its receive-buffer size outright ("Max characters per line exceeded") and the stream never recovers - an auto-generated comment with interpolated names (a probe/fixture name) had grown past that unnoticed and derailed an entire Start Job run. `MacroProcessor.SanitizeComment` now shortens an oversized pure-comment line's interior instead of sending it whole.

#100 WpfUiTestServer: /handoff releases control to the operator; /idle waits for a real paint

File	+	−
CNC Core.csproj, ioSender XL.csproj (PackageReference bump 1.0.0 to 1.2.0)	4	4

Both new capabilities live in the standalone `WpfUiTestServer` package (`github.com/stevenrwood/WpfUiTestServer`), bumped here from 1.0.0 to 1.2.0:

- **POST /handoff** (v1.1.0) - for an agent that has been driving the UI unattended up to a point where a human needs to take over. Shows a non-modal popup with caller-supplied instructions (so the operator has full control while reading it, never blocked), removes the "under test-server control" banner, and stops the server - a real stop, not a pause; a fresh `Start()` works again if needed later in the same run.
- **GET /idle waits for a real paint** (v1.2.0) - draining the Dispatcher to Background priority only proves layout/arrange ran; WPF's actual pixel composition happens on a separate compositor thread not synchronized with that queue at all. `/idle` now also waits (bounded) for a `CompositionTarget.Rendering` tick - the real "has this painted" signal.

Dev/QA infrastructure only, off by default behind -testserver. No ioSender-side behaviour change beyond the version bump.

#101 Localization: locale rows for this round's new/removed UI strings

File	+	−
`Locale/*/csv (7 locales x 2 files) + tools/locadd.py`	548	321

English-baseline locale rows for every new or retired `x:Uid` introduced this round, across all 7 locales via `tools/locadd.py` - Fixture definitions, the retired G28 named-position combo, Fusion add-in outline parity, Machinist Vise fixture (Set/Test, schematic, Jaw width/Max opening), and Start Job's mm/in unit toggle.

#102 Machinist Vise Start Job: fixed a parser-corrupting comment + an MDI queue pacing bug, added a jaw keep-out check

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	115	17
CNC Controls/CNC Controls/JobControl.xaml.cs	13	1
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	19	0
CNC Controls/CNC Controls/FixtureEditDialog.xaml	1	1
CNC Core/CNC Core/TelnetStream.cs	17	1
build.ps1	8	1

The Machinist Vise's Start Job program (jaw-corner origin + stock-top Z probe) has been failing with a storm of `error:71 – Unknown operation found in expression` since it shipped - first hardware run to actually reach the probe traced it down through a genuinely long chase: a suspected firmware issue, a suspected TCP/streaming issue, and finally two real, distinct bugs.

- **Root cause:** `BuildViseProgram` split one comment across two generated G-code lines without closing the parenthesis on the first - grblHAL has no multi-line comment continuation, so the dangling `(` corrupted the parser's state for every line that followed, until a lone blank line happened to reset it. Fixed by closing/reopening the comment per line.
- **Found along the way:** the shared MDI/macro command queue (`JobControl.SendCommand/ResponseReceived`) flipped back to Idle the instant its local queue emptied, right after writing a line, instead of waiting for that line's real controller ack - so a macro sending several non-motion setup lines in a tight loop fired all of them within milliseconds, zero flow-control pacing. Confirmed via a new opt-in raw comms trace (`-debuglog=comms-tx,comms-rx`). Affects any macro streaming multiple lines, not just the vise.
- **New:** a keep-out check warns (never blocks) if the loaded job's toolpath enters the vise jaws' footprint, tool-radius aware via the grblHAL tool table; the Spacer/backer field is no longer hidden for the vise kind and now feeds the stock-top probe's safe descent height.
- **Also:** the Fixture definition dialog's Cancel button now issues a Feed Hold (never Stop/Reset) if closed mid-probe, instead of leaving the corner probe running unsupervised behind a closed window.

First real end-to-end hardware validation of the vise Start Job program: jaw-corner Set, stock-top Z probe, TLO-reference tool change, and final work-origin set all confirmed on real hardware.

#103 Start Job: Touch Plate probing, vise partial Measure, TLO-ref toolsetter fix

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	273	54
macros/pcorner.macro	76	38
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	50	10
macros/pvisecorner.macro	14	11
CNC Controls/CNC Controls/Fixture.cs	14	0
CNC Controls/CNC Controls/FixtureEditDialog.xaml	10	1
ioSender XL/ioSender XL/StartJobView.xaml	9	0
CNC Controls/CNC Controls/StartJobConfig.cs	4	1
Locale/*/csv (14 files, 7 locales)	63	0

Touch Plate wired end-to-end as an alternative to the 3D probe for Start Job's own corner/stock probing: a new Stock Size panel selector (Probe: 3D Probe/Touch Plate + Stock conductive), `pcorner.macro` `_ls_mode/_ls_plateoffset` globals, and a vise-only Probe: picker in Fixture definitions' Set/Test position (the jaw is bare conductive metal, so no plate-thickness math is needed there — `pvisecorner.macro` needed zero changes).

- **TLO-reference fix (hardware near-miss).** `M6 T8` hardcodes the main probe input (tc.macro's own "T8 = 3D probe in spindle" convention) — wrong for Touch Plate, which has no self-triggering probe, and drove the tool into the toolsetter puck without stopping on the first real test. A new `EmitTloReference` helper bypasses `M6 T8` for touch-plate mode and explicitly selects the toolsetter input instead. Confirmed working on real hardware.
- **Partial Measure for the Machinist Vise** (previously hidden entirely for this fixture kind). Corners 2/4 get a real `pcorner.macro` face+Z probe; corners 1/3 (blocked by the jaw) get a Z-only probe for flatness data. Width/height computed against the known jaw origin; skew is deliberately not computed — there is no independently-verified 4th point to check squareness against.
- **`_floor` renamed `_bottom`** throughout `pcorner.macro`, and its value grounded in real physical geometry (the vise's 1" frame recess) instead of an arbitrary buffer, since it also drives the REUSE rapid-approach height.
- **Rapids restored** in `pcorner.macro`/`pvisecorner.macro`/`BuildViseProgram` for moves at an already-verified-safe height, undoing an earlier blanket "everything G1" defensive sweep — only genuine near-material descents stay controlled-feed.
- A vise-must-be-empty confirmation before Set position, a 200 mm/min search-feed floor, and an explicit rapid to the approximate corner position before each `pcorner.macro` call.

Builds clean (Debug+Release). Vise origin Set-via-Touch-Plate and the TLO-reference fix are hardware-confirmed. The vise descent-height formula, partial Measure, search-feed floor, and corner-approach fix are UNVERIFIED on hardware — session paused for a suspected Teensy fault requiring a firmware reflash before further testing.

#104 Settings restore/rollback redesign, G28 fixture-position picker, generated-macro persistence, GCodeListControl crash fix

File	+	−
CNC Controls/CNC Controls/GrblConfigControl.xaml.cs	101	75
CNC Controls/CNC Controls/PendingChangesDialog.xaml.cs	79	1
CNC Controls/CNC Controls/PendingChangesDialog.xaml	23	1
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	25	4
CNC Controls/CNC Controls/MacroProcessor.cs	17	0
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	18	1
CNC Controls/CNC Controls/OffsetFlyout.xaml	3	0
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	8	2
CNC Controls/CNC Controls/GrblConfigControl.xaml	2	0
CNC Core/CNC Core/Grbl.cs	6	0
Locale/*/csv (7 locales)	42	7
tools/run-iosender.ps1	3	2

Bundle of settings/UX groundwork carried from a long session: a Restore-from-file redesign, a named-fixture G28 picker, permanent macro output, and a real crash fix.

- **Restore-from-file redesign:** preview-first (only rows that would actually change, reusing `MachineSetupWizard.TargetDiffers` for the numeric-aware comparison), an OK("Restore settings")/Cancel confirm, and **live per-row coloring** as each setting applies - `PendingChangesDialog` grew an optional `ApplyOne/RollbackOne` delegate mode plus a `SettingChange.Status` enum and matching `DataGrid.RowStyle` triggers, with automatic rollback-with-coloring offered on a mid-run failure. `GrblConfigControl.SetSetting` was reworked from "ask interactively per-setting" to "return success/error, let the bulk-apply caller decide" - grblHAL has no bulk/atomic settings apply (each `$id=value` is flashed individually), so rollback is the best available mitigation.
- **Machine Setup stays in sync:** new `MachineSetupWizard.NotifySettingsChangedExternally()` re-checks the setup gate after a Restore, so an already-open wizard reflects restored values instead of showing stale state.
- **G28 flyout: named-fixture picker.** A new read-only ComboBox over `Fixtures.Items` - Go navigates to the selected fixture's saved `Coords` via `GotoBaseControl.SafeGotoMachine` instead of the firmware G28 slot when one's selected, falling through to the normal G28 behavior otherwise. Machine Setup still owns Set.
- **Real crash fixed:** `GCodeListControl.grdGCode_SelectionChanged` had an unguarded `(DataContext as GrblViewModel).StartFromBlock` that crashed reliably when switching to the Start Job tab, because `SetProgram`'s own `DataContext` reassignment fires `SelectionChanged` mid-transition. Pre-existing bug, unrelated to anything else in this batch.
- **Generated macros persisted:** every `MacroProcessor.Run()` call now writes its code to `%AppData%\ioSender\Generated\<name>.macro` - the streamed program itself was never saved to disk before, so this gives a permanent, inspectable copy for diagnosing "why did it stop early" without a live trace.
- **`tools/run-iosender.ps1` fix:** a PS7-expression-style `$x = (if(){} else{})` that Windows PowerShell 5.1 rejects ("term 'if' not recognized"); fixed to statement-form if/else in both spots.

Builds clean Debug+Release. Carried over from an earlier session's feature work; not independently re-verified in this session beyond the build.

#105 Vise touch-plate corner-probe margin fix; corners 1/3 dropped; first clean end-to-end Start Job run

File	+	−
`ioSender XL/ioSender XL/StartJobView.xaml.cs`	94	58

Closes out the Machinist Vise touch-plate corner-probe thread: root-caused why the corner 4/2 face-probe margin kept flip-flopping between two wrong extremes, fixed it, dropped a pair of probes that weren't earning their keep, and ran the whole thing clean on real hardware for the first time.

- **Root cause:** `rx`/`ry` (pushed outside the exact edge, the face-probe's outward retreat/descent point) and `_topx`/`_topy` = `rx − topClearance` (pulled back inside, the initial top-probe point) are computed from the SAME reference in series inside `pcorner.macro`, not two independent points - and entered vise stock sizes are precision-exact, so the margin was never covering size uncertainty, only physical tip clearance so a vertical descent doesn't land exactly on the corner.
- **Fix:** both margins now derive from the ACTUAL tip radius currently in the collet (`refMarginMm = radius + 5`, `topClearance = refMarginMm + 10`, so the top-probe point always nets 10mm inside the true edge regardless of radius - previously `topClearance`(10) < `refMarginMm`(15), guaranteeing an overshoot).
- **Live tip diameter, no firmware tool table (N_TOOLS=0):** new `StartJobView.ActiveOrFallbackProbeDiameter()` prefers the loaded program's own `(TOOL T=n D= … )` comment for the currently loaded tool (`GCodeProgramComments.DiameterFor`, the same source the general Probing tab already uses) over the touch-plate probe definition's fallback diameter field.
- **Corners 1/3 dropped entirely** (jaw-covered edge, Z-only readout) - they never fed size (corners 2/4 vs. the known jaw origin only), origin, or skew (deliberately never computed), just an extra flatness data point, while carrying the same corner-catch risk 2/4 just got fixed for.

Hardware-verified 2026-07-14: corners 2/4 probe cleanly with real working clearance, and a full Start Job run (probe -> size -> park -> set origin -> M2) completed end-to-end for the first time. Builds clean Debug.

#106 (TOOL) comment gains an L= tool-length parameter

File	+	−
CNC Controls/CNC Controls/GCodeProgramComments.cs	53	18
CNC Controls/CNC Controls/ProgramView.xaml.cs	10	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.py	25	6

Extends the Fusion add-in's `(TOOL T=n D=d TYPE=..)` comment syntax with an optional `L=<mm>` tool-length parameter, so a program can declare each tool's overall length the same way it already declares diameter and shape.

- **Fusion add-in:** reads the CAM tool's `tool_overallLength` parameter (cm -> mm) and emits `L=<mm>` alongside `D=/TYPE=`; falls back to a 40mm default when a tool has no usable length.
- **GCodeProgramComments:** parses the new key into `GCodeToolInfo.Length` (same 40mm default for programs generated before this change, or when a comment omits it).
- **ProgramView.DeclaredTools:** new property (mirrors `DeclaredStock`) exposing every tool's parsed diameter/shape/angle/length for the current program instance - available on any `ProgramView`, not just the loaded job.

Verified end-to-end against a real Fusion CAM project: debug dump confirmed tool_overallLength is the correct parameter, and the generated .nc file carried correct non-default L= values. Builds clean Debug.

#107 Probe definitions unified on an overall-length measurement, like (TOOL)'s L=

File	+	−
CNC Controls/CNC Controls/ProbeDefinition.cs	8	7
CNC Controls/CNC Controls/ProbeDefinitionEditDialog.xaml	13	5
CNC Controls/CNC Controls/ProbeDefinitionEditDialog.xaml.cs	5	4
Locale/*/csv (7 locales)	35	14

The 3D probe dialog's "Probe length" field/schematic measured stylus-only (body-bottom to tip) - a measurement that doesn't compose with anything else measured from a known datum. Renamed to `OverallLength` (top of body to end of tip) so it reads the same way a CAM tool's overall length does, and updated the label, tooltip, and schematic dimension line to match.

- **Touch plate:** added a matching `BitLength` field ("Overall length of bit", default 40mm) plus a schematic dimension line, since the plate probe also needs to know how far the bit's tip sits below the collet.

UI-verified via the WpfUiTestServer: both dialogs' schematics and field labels checked by screenshot for the 3D probe and touch plate types.

#108 Lathe mode auto-detected from the controller, manual toggle removed

File	+	−
CNC Core/CNC Core/Grbl.cs	8	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	14	2
CNC Controls Lathe/CNC Controls Lathe/LatheWizardsView.xaml.cs	4	2
CNC Controls/CNC Controls/BasicConfigControl.xaml	0	1

grblHAL's own `$I` NEWOPT reply already reports a `LATHE` capability flag when the controller's Mode of operation setting (Settings > Grbl) is set to Lathe, so the separate Settings: App checkbox was redundant - and could disagree with the controller's actual mode.

- **Grbl.cs** pushes `LatheModeEnabled` through `GrblViewModel` on every NEWOPT parse (firing `PropertyChanged`, mirroring `DetectNumAxes`'s own pattern).
- **MainWindow** persists whatever the controller reports, so next startup's tab-availability check (which runs before a connection exists) is right without waiting on a fresh connection.
- Lathe Wizards' unavailable reason now points at Settings > Grbl > Mode of operation instead of the removed App-tab checkbox.

Unverified on real lathe-enabled hardware - the detection wiring hasn't been exercised against a controller actually reporting LATHE.

#109 Start Job: align Probe/Stock-conductive rows, exact-size vise labels, tighten TLO-ref motion

File	+	−
`ioSender XL/ioSender XL/StartJobView.xaml`	3	3
`ioSender XL/ioSender XL/StartJobView.xaml.cs`	27	17
`macros/pcorner.macro`	25	8

Left-align the `Probe:` row and `Stock conductive` checkbox to match the flush-left fields below them instead of a right-aligned label column.

- **Machinist vise fixture:** stock size is measured directly off the vise jaws, not estimated - the size hint text and per-field labels now say "Exact width/height/thickness" instead of "Est. ..." when that fixture is selected.
- **Generated Start Job code:** after setting the TLO reference and pausing at G30, the vise corner-probe sequence used to bounce through several higher-than-necessary moves (safe Z, over to corner 4, back up to Z0, over the stock, down to Z0 again) before probing. It now goes straight from G30 to over the corner-4/2 approach position, drops to Safe Z once, and probes - `pcorner.macro` takes an explicit `#<_ls_maxz>` clearance height instead of hardcoding `Z0/#<_bottom>+30`.

Hardware-verified: Start Job now runs the vise flow start to finish without the extra moves.

#110 Reset-repro test-case generator (Help > Support)

File	+	−
CNC Controls/CNC Controls/ResetReproDialog.xaml	24	0
CNC Controls/CNC Controls/ResetReproDialog.xaml.cs	173	0
ioSender XL/ioSender XL/MainWindow.xaml	1	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	5	0
Locale/*/csv (7 locales)	105	21

Writes a .nc file with an O-word subroutine, called several times, that dwells then moves - a controlled way to reproduce the controller-wedge-on-Reset bug under investigation. grblHAL only allows O-word flow control from a file-backed stream, so the file must be uploaded to SD Card / the on-board filesystem and run from there rather than loaded as a normal job.

- Lets Reset be tested mid-call, and the same file compared over both Serial and Telnet transports.

Tool built and verified to generate a valid file; the actual hardware repro run against it is next session's work.

#111 Durable per-run console log; crash/debug logs consolidated under logs\

File	+	−
CNC Core/CNC Core/ConsoleLog.cs	118	0
CNC Core/CNC Core/DebugLog.cs	1	5
CNC Core/CNC Core/Grbl.cs	27	0
CNC Core/CNC Core/GrblViewModel.cs	11	0
ioSender XL/ioSender XL/App.xaml.cs	6	10
ioSender XL/ioSender XL/MainWindow.xaml	1	1

Every console line the on-screen Console tab shows is now also mirrored to `%AppData%\ioSender\logs\console_<run-timestamp>.log` - a durable record that survives the 2000-line in-memory trim and, more importantly, survives a UI freeze (the failure mode this was built to diagnose: a firmware debug flood once froze the WPF UI for the length of a job, with no record afterward of what had actually happened). The write runs on a dedicated background thread fed by a queue, so a traffic burst can't freeze the UI a second way. Files are timestamped per run (not a single ever-growing file) and pruned after 10 days.

- **Crash log and debug log consolidated** - both moved from directly under `%AppData%\ioSender\` into the same new `logs\` subfolder, sharing one resolution helper (`Resources.ResolveLogsDirectory()`) instead of duplicated fallback logic.
- **Help > Support > "Generate reset test case..."** is now hidden from the menu (a diagnostic tool, not something a normal user needs) - still reachable via UI Automation for test tooling.

Grew out of a multi-session firmware controller-wedge investigation (Reset during a named G-code macro call could leave the controller needing a power-cycle) - this logging groundwork is what made the rapid flash/reproduce/read-log iteration loop possible. Root cause found and fixed upstream in grblHAL core (PRs #987, #988), hardware-verified.

#112 Program view popup resized to exactly the right third of the client area

File	+	−
`ioSender XL/ioSender XL/MainWindow.xaml`	11	4

The on-demand Program view overlay was a `Border` anchored right with a fixed `MinWidth="560" MaxWidth="720"` - on the app's minimum window width that could occupy up to ~58% of the client area. Replaced with a proper 3-equal-column `Grid.ColumnDefinitions` layout, with the overlay in the rightmost column (`Grid.Column="2"`, `HorizontalAlignment="Stretch"`) - it now always occupies exactly one third of the client width, regardless of window size.

#113 Start Job: touch-plate-safe Measure gating + unit-aware measured-size readout

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	35	9

Two Start Job fit-and-finish fixes, found while HW-testing the Machinist Vise fixture:

- **Measure gating.** A separate touch plate against non-conductive stock only closes the circuit at the plate itself - the stock's own edges never trigger it, so the 4-corner edge probing Measure depends on can't work. The Measure checkbox now auto-disables (and unchecks, so a stale checked-but-disabled state can't reach Generate) whenever Touch Plate + unchecked Stock conductive is selected, with a tooltip explaining the fix (switch to 3D Probe, or check Stock conductive). Re-enables live when either control changes, and is applied correctly on load from a persisted config.
- **Unit-aware readout.** The post-run "Measured stock" summary (X/Y size, flatness, squareness/diagonal delta) and the drawing overlay's edge dimension labels were always shown in mm regardless of the mm/in radio selection. Both now convert through the existing FromMm() unit helper and label the correct unit, matching the input fields above them. The Fusion-add-in clipboard copy (Copy size) and the internal g-code emitter intentionally stay in mm - those are machine-consumed contracts, not display output.

#114 Settings > Simulator tab: pick options, build via CI, auto-launch from Connect dialog

File	+	−
CNC Controls/CNC Controls/SimulatorConfigView.xaml	48	0
CNC Controls/CNC Controls/SimulatorConfigView.xaml.cs	114	0
CNC Controls/CNC Controls/SimulatorManager.cs	160	10
CNC Controls/CNC Controls/PortDialog.xaml.cs	20	0
CNC Controls/CNC Controls/PortDialog.xaml	5	5
CNC Controls/CNC Controls/AppConfig.cs	14	3
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	6	0
CNC Controls/CNC Controls/GrblConfigView.xaml	3	0
CNC Controls/CNC Controls/IGrblConfigTab.cs	2	1
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
Locale/*/csv (7 locales)	49	0
tools/locadd.py, csproj version bumps	3	2

The Connect dialog's Simulator tab used to just be a port field with a comment saying the user launches grblHAL_sim.exe by hand - ioSender no longer managed the process at all. This closes that loop:

- **Settings > Simulator tab** (new, SimulatorConfigView) - pick axes (3-7), probe input, and coordinate-system rotation (WCSROT), then Build. Reuses SimulatorManager's existing build-matched-sim CI workflow dispatch + sim-<sig> release cache (the same one the auto-matched-to-controller flow already uses) but always installs a plain, unsuffixed grblHAL_sim.exe to %AppData%\Simulator - a fixed path independent of the auto-matched system's app-relative simulator\ folder, so the two coexist without interfering.
- **Connect dialog gating.** The Simulator tab is only enabled once that exe exists; when disabled its hint text points at Settings > Simulator. When enabled, clicking Ok launches the exe (if not already running, checked by process name so it also catches an instance started outside this session) before connecting to 127.0.0.1 on the chosen port.
- **Startup auto-reconnect.** EnsureSimulatorRunning() was a stubbed no-op ("the simulator runs standalone now") - un-stubbed to apply the same start-if-not-running logic when auto-reconnecting to a previously-chosen simulator target, so a saved simulator connection still works after an app restart.

SimulatorManager.cs additions are structured as thin new entry points (AppDataSimulatorDir/ExePath/Present, BuildManualOptionSymbols, EnsureAppDataSimulator, PollAndInstallAppData) parallel to the existing EnsureMatchedSimulator/PollForMatchedRelease, sharing a refactored-out DownloadReleaseBytes helper - the auto-matched flow's own code path is unchanged.

#115 Simulator settings: match connected hardware, full ATC support, one-button Machine Setup step

File	+	−
CNC Controls/CNC Controls/SimulatorManager.cs	322	21
CNC Controls/CNC Controls/SimulatorConfigView.xaml.cs	218	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	146	0
CNC Controls/CNC Controls/SimulatorConfigView.xaml	72	0
tools/gen-sim-macros.py	70	0
CNC Controls/CNC Controls/PortDialog.xaml.cs	35	1
simulator/README.md	23	16
CNC Controls/CNC Controls/AppConfig.cs	22	5
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	20	2
CNC Controls/CNC Controls/MachineSetupWizard.xaml	20	0
Locale/*/csv (7 locales)	119	0
PortDialog.xaml, GrblConfigView.xaml, IGrblConfigTab.cs, csproj x3, .gitignore, locadd.py, sim-build.json	15	20

Session-long arc built on the previous Settings > Simulator / Connect-dialog work (#114): closes the gap between "a simulator exists" and "a simulator that actually behaves like the connected machine."

- **Hardware-matched defaults, every time.** Settings > Simulator now re-syncs its picks from the connected controller's live $I on every visit (was a one-shot-ever bug - fixed), not just the first time the tab loaded. Machine Setup Wizard gains step 8, "Build simulator matching this machine": one button, no picks, same status readout as the Settings tab (build id + up-to-date/rebuild-needed/not-built messaging), hidden automatically while the active connection IS the simulator (nothing to match against).
- **8 compile-time options**, up from 3: axes/probe/rotation plus Toolsetter, Lathe U/V/W, Safety door, E-stop, and Ganged+Auto-square Y - the full set of genuine, $I-detectable, simulator-relevant grblHAL build toggles (confirmed against grbl core and the Simulator driver source; lathe MODE and spindle variability turned out to be runtime $-settings, not compile options).
- **Full ATC support in every built simulator.** Root-caused a real M6T1 "error:20": machines using macro-driven ATC ($341=Ignore) left the simulator's M6 completely unhandled since it had no tc.macro. Fixed in the Simulator repo: driver.c now bakes ioSender's canonical ATC macros (generated from macros/*.macro by new tools/gen-sim-macros.py) into littlefs at boot, with a non-obvious mount-then-remount fix (found via a local MinGW build, not guessed) so grblHAL's own ATC-detection hook sees them in time. CI now builds from the Simulator repo's `integration` branch (which has this + littlefs/YModem support) instead of `master` (which never had any of it) - both ioSender's dispatch ref and the repo's own release workflow repointed.
- **Live machine settings + NVRAM copy.** Build now also replays the connected controller's $ settings into the simulator's EEPROM.DAT, same headless-replay mechanism the Grbl tab's existing "Copy to simulator" button uses, generalized to share one implementation.
- **Location fix:** the manually-built simulator lives at `%AppData%\ioSender\Simulator` (alongside App.config/logs/Backups), not a stray top-level `%AppData%\Simulator`.
- **Two IP-clobbering bugs found and fixed** on the Connect dialog: the remembered real-machine IP could get silently overwritten by the simulator's own loopback address on the write side

(CaptureConnectedIp) and separately shown incorrectly on the read side (the dialog's own pre-fill logic) - both guarded now.

- **Build-hang fix:** installing a freshly-built simulator over a currently-running instance of itself silently failed (locked file) for the CI-dispatch flow's full 10-minute poll window before giving a misleading "still building" message; now stops the running instance first.

Fully HW-verified end to end multiple times this session: a real CI-dispatched build downloaded and run standalone confirmed $I reports ATC=1 and M6T1 enters the macro flow; the hardware-default/status/hide-tabs/IP fixes were all verified live against a real controller + the real simulator via the UI test server, not just build-checked.

#116 Tear-off tabs: detach Program/3D View/Console into their own window

File	+	−
`CNC Controls/CNC Controls/TearableTab.cs`	161	0
`ioSender XL/ioSender XL/JobWorkspace.xaml.cs`	3	13

New generic `CNC.Controls.TearableTab.Attach`, built as the foundation for embedding the simulator's own window as a future Job tab, but immediately useful on its own: double-click a center tab's header to detach its content into a standalone custom-chrome window (no native title bar - so double-click is free for a custom gesture instead of the OS's maximize), double-click that window's title bar to dock it straight back into the tab strip at the same spot. No close button and no close/hide affordance on the tab itself - re-docking is the only way out, matching how these tabs already behaved.

- Replaces the Console-specific double-click-to-open-a-duplicate-floating-window special case with the same generic mechanism all three center tabs now share (Program, 3D View, Console) - the old `OpenConsoleWindow`/`ConsoleWindow` duplicate-window path stays only for its own separate Esc-shortcut toggle feature, untouched.
- **Ctrl+Alt+=** / **Ctrl+Alt+-** zoom the floating window's content in/out (0.5x-3x), **Ctrl+Alt+0** resets - via a LayoutTransform so content actually re-measures/reflows at the new size rather than just stretching pixels, applied uniformly regardless of which tab was torn off.
- Fixed same session: a detached window has no ancestor to inherit DataContext from, so content relying on it (Console's bound GrblViewModel) went blank and silently stopped responding to input until this was propagated explicitly.

HW-verified: tab strip renders correctly post-refactor (screenshot + no exceptions via the UI test server); the double-click detach/redock/zoom gestures themselves needed the user's own hands, since the automated harness can't synthesize real multi-click mouse input - confirmed working, including the DataContext fix.

#117 Machinist Vise: corners 1/3 gain a real left-edge X-face probe, measured origin + skew, size_y bug fix

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	184	23

Follow-on to the Machinist Vise Measure work (#105/#113): corners 1 and 3 sit on the jaw-covered edge, so they only ever got a Z-only top probe, dropped from size/origin/skew entirely. Live HW testing this round found they can do more, and turned up two real bugs along the way.

- **Corner 1/3 X-face probe.** The fixed jaw's own body only blocks the Y face at these corners (it sits at the BACK, at/near the origin's Y) - it never blocks an X approach at either corner. Each now backs off 15mm outward to open air, drops to the material midpoint (this corner's own just-probed top minus half the entered thickness, same reasoning as pcorner.macro's start_z), and seeks back in to find the true left edge, radius-compensated. First attempt probed straight into the jaw at the true corner Y - fixed by running the X-face probe at the SAME 5mm-inset Y the Z probe already uses to clear it.
- **Measured origin.** Corner 3 *is* the jaw origin corner, so its own probed X/Z now set the WCS origin directly (G10 X/Z), replacing the fixed fixture-saved X and the footprint-centre Z probe - a real measurement instead of an assumption, when Measure is on.
- **Left-edge skew.** Corner 1 vs. corner 3's probed X, over the entered (exact) height as the trusted Y separation, gives a real squareness check for the vise - something Corner Fence's own skew math couldn't do here (it needs a full 4-corner XY quad the vise never has).
- **size_y bug fix.** The existing corner 2/4 width/height calc averaged `(origin.Y − corner2.Y)` with `(origin.Y − corner4.Y)` for height - but corner 4 shares the origin's own Y (it's only the X-neighbour), contributing ~0 to that average and silently halving size_y. Caught on a 12″×3″ stock reading back 3″×1.49″. Y now comes from corner 2 alone; X still correctly averages 2/4 (both differ from the origin by the true width).
- **Schematic Z-label bug fix.** The existing per-corner Z/angle label loop needs both a full 4-corner XY quad and a spoilboard reference (only ever printed by pcorner.macro's DISCOVER pass) - the vise has neither, so its probed corners never showed a Z label at all. Added a vise-specific label path showing each probed corner's Z relative to the lowest one (a flatness map), independent of those two gates.

Hardware-verified this round: the size_y fix, schematic Z labels, and the corner 1/3 X-face probe (after the Y-inset fix) all confirmed on a real run. Builds clean Debug.

#118 Machine Setup > Machine tab: grblHAL version display + one-click firmware update

File	+	−
CNC Core/CNC Core/Grbl.cs	29	0
CNC Controls/CNC Controls/FirmwareUpdateManager.cs	238	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml	14	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	195	0
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
ioSender XL/ioSender XL/ioSender XL.csproj	9	0
Locale/*/csv (7 locales)	21	0
tools/teensy_loader_cli/* (vendored PJRC source + build.ps1)	1277	0
firmware-tools/* (bundled teensy_loader_cli.exe + README)	14	0

The Machine Setup > Machine tab now detects a connected grblHAL controller and shows its firmware, version, build, and (for the SRW fork's own CI builds) the exact driver commit it was built from - parsed from a new `[BUILD: … ]` line already emitted by `report.c` on this fork.

- **Version display** - `GrblInfo.BuildStamp/DriverRef/DriverSha` (Grbl.cs) parse the driver's own build-provenance stamp out of `$I`; the Machine tab renders firmware/version/build plus the driver ref, and self-heals if the active connection changes (e.g. simulator to real board) while the tab stays open.
- **Check for firmware update** - queries a new `fw-latest` GitHub Release on `stevenrwood/iMXRT1062` (published by an addition to `firmware.yml`'s CI, mirroring the Simulator's existing public-release pattern) and compares its driver commit against the connected board's.
- **Update firmware** - when a newer build is found, downloads its `.hex` and flashes it via a newly bundled `teensy_loader_cli.exe` (built from PJRC's open-source loader, vendored + a small build script under `tools/teensy_loader_cli/`, shipped in `firmware-tools/`). ioSender disconnects first; because Windows builds of `teensy_loader_cli` have no automatic reboot-into-bootloader path, the confirmation prompt tells the user to press the board's physical RESET/PROGRAM button once it says it's waiting.

HW-verified end-to-end on a real Teensy 4.1 board over its network connection: detected the running build, found and downloaded a newer `fw-latest` release, flashed successfully after a manual reset-button press, and reconnected showing the new build stamp.

#119 Zero-friction installer — one-line install/update, rolling release CI

File	+	−
`.github/workflows/release.yml`	53	0
`install.ps1`	97	0
`ioSender XL/ioSender XL/BuildInfo.cs`	10	0
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	40	36
`ioSender XL/ioSender XL/ioSender XL.csproj`	1	0
`readme.md`	8	1

A new user previously had to clone the repo and build it in Visual Studio just to try ioSender. Now: open PowerShell, run `irm https://raw.githubusercontent.com/ioSenderV2/ioSender/master/install.ps1 | iex`, done — downloaded, installed, shortcut on the desktop, launched.

- **Rolling release CI** — `.github/workflows/release.yml` builds the Release config on every push to `master` and republishes a single rolling `latest` GitHub Release (zipped `bin\Release`), so there's no version-tag ceremony to keep the installer fed.
- **install.ps1** — installs per-user to `%LocalAppData%\Programs\ioSender` (no admin rights, no UAC prompt), creates the desktop shortcut, launches. Re-running it is the update path: the current install is moved to `ioSender\previous` first, so `-Rollback` can swap back to the prior build if a new one misbehaves — one level deep, not a full history.
- **Fixed Check for Updates** — the existing Help → Support → Check for updates feature compared semver against the rolling release's tag, which is literally the string `latest`; parsed as a version it silently degraded to `0.0.0` and would always have claimed "up to date" even when stale. Replaced with a commit-SHA compare: `BuildInfo.cs` (new, stamped by the release workflow's build step) holds the commit the running binary was built from, matched against a `build:master@<sha>` marker in the release body — the same convention `FirmwareUpdateManager` already uses for firmware's `drv:<ref>@<sha>`. A local dev build (no embedded commit) short-circuits with a message pointing at `install.ps1` instead of comparing against GitHub.

Builds clean in Release. First rolling release published and round-trip verified: CI 403'd on its first run (the `ioSenderV2` org locks default Actions workflow permissions to read-only and changing it needs org-admin, unavailable here) — fixed by routing the release step through a PAT stored as the `RELEASE_TOKEN` repo secret instead. Installed via the one-liner and confirmed Check for Updates correctly reports the CI build's commit as current.

#120 One-click Update Now + Support > Roll back to previous version

File	+	−
`Locale/*/csv (7 locales)`	14	0
`ioSender XL/ioSender XL/MainWindow.xaml`	1	0
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	39	3

Follow-up to #119: Check for Updates found a newer build but could only open the GitHub releases page, leaving the user to figure out there's no install button there and re-run `install.ps1` by hand.

- **Update now** — the "A newer build is available" prompt now asks "Update now?". Yes fetches `install.ps1` fresh from GitHub (not a possibly-stale local copy) into a detached, visible PowerShell process and closes ioSender so the file swap can happen; `install.ps1` downloads, installs and relaunches as usual.
- **Help > Support > Roll back to previous version...** — same mechanism with `install.ps1 -Rollback`, reusing that one engine instead of re-implementing the `ioSender\previous` folder swap in C#. Only goes back one version, matching `-Rollback`'s own limit.

Builds clean in Release. New menu item's 2 strings backfilled to all 7 locale CSVs via `tools/locadd.py`. Round-trip verified: installed the prior build, confirmed Check for Updates now offers Update now, and confirmed the composed PowerShell command parses clean.

#121 Jog buttons could send an overlapping unacknowledged $J, wedging the controller

File	+	−
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	44	0

Confirmed on real hardware via console log: a fast burst of jog-button clicks fired a new `$J=` command before the controller had acknowledged (`ok`'d) the previous one. grblHAL then stopped responding to *anything* - not just jog commands, but plain status queries too - with no error reported, until the controller was power-cycled.

- **Jog-ack gate** - `JogCommand()` now tracks whether a previously-sent `$J` is still outstanding and refuses to send another until it's acknowledged, dropping the extra click rather than risking an overlap.
- **Safety timeout** - the gate auto-clears after 2 seconds even without an `ok`, so a single dropped response (unrelated comms hiccup) can't permanently lock jogging out for the rest of the session.
- **Cancel is exempt** - the jog-cancel realtime byte always goes through immediately and clears the gate, since it's not a queued `$J` line.

This doesn't fix the underlying firmware behavior (grblHAL arguably shouldn't be able to wedge on overlapping jog commands at all), it stops ioSender's own jog buttons from being able to trigger it. Hardware-verified: the same rapid-click pattern that previously wedged the controller now sends cleanly, each $J waiting for its own ok.

#122 Fusion batch post failed on operations using Air/Mist coolant

File	+	−
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.py	35	0

`cam.postProcess` raised `3 : Unsupported 'air' coolant` on every operation using Fusion's Air or Mist coolant setting - `grbl.cps` only understands Flood and disabled. `_force_flood_coolant()` reads each operation's coolant parameter and force-remaps Air/Mist to Flood before posting, logged per-op to `_batchpost.log` when it fires. Default behavior, no opt-in checkbox.

#123 A zero-length/truncated ATC macro on the controller read as Installed

File	+	−
CNC Controls/CNC Controls/AtcMacros.cs	54	4

Found chasing a hardware bug where a Start Job's tool-length reference (`M6 T8`) never actually probed the toolsetter puck: `tc.macro` on the controller had been truncated to 0 bytes by an earlier interrupted write, but the Machine Setup gate and Controller Macros tab both reported it **Installed** anyway - `GetStatus`/`EnsureProvisioned` only checked whether the file NAME was present in the listing, never its size, and the aggregate `atc.sum` checksum sidecar wasn't rewritten by the corruption either.

- **Zero-length = Missing** - so the Machine Setup gate still fires and routes to the Controller Macros step, same as a genuinely absent file.
- **Any other size mismatch against the embedded copy = Outdated** - a cheap, no-extra-round-trip signal (the file listing already includes size) that catches partial truncation too, not just total.
- **EnsureProvisioned** (the self-healing re-upload path) gets the same checks, so it can actually catch and fix the exact failure it exists to catch.

#124 Vise Set position could close mid-probe and silently keep the stale corner

File	+	−
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	8	2
CNC Controls/CNC Controls/MachineSetupWizard.xaml	0	2
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	1	23

Chased a hardware report of "Set position didn't move anything" that turned out to be two separate bugs:

- **OK-button race** - the vise corner probe (`pvisecorner.macro`) streams asynchronously, so `btnOk` (`IsDefault="True"`, fires on Enter too) being clickable mid-probe let the dialog close and copy the CLONE's still-stale `Coords` back to the real fixture before the probe's own result was ever written - the fresh corner was silently lost. `SetBusy()` now disables OK for the same window it already disabled Set position.
- **Bogus quick-access button** - the Fixtures-tab list's own "Set position" button just captured the raw jogged position for every fixture kind, which is correct for corner-probing kinds but silently wrong for a Machinist Vise (no vise-empty confirmation, no `pvisecorner.macro` call at all). Removed; the Add/Edit dialog's Set position (which branches correctly per fixture kind) is now the only path.

#125 Start Job/Load Stock accuracy pass: jaw-origin schematic, safer corner travel, exact-size tight probing

File	+	−
CNC Controls/CNC Controls/StartJobConfig.cs	10	0
ioSender XL/ioSender XL/StartJobView.xaml	9	0
ioSender XL/ioSender XL/StartJobView.xaml.cs	208	37
macros/pcorner.macro	33	16
Locale/*/csv (7 locales)	49	0

A cluster of related fixes and one opt-in feature, developed and hardware-verified together against a real Corner Fence Measure run:

- **Jaw-origin schematic labels** - each vise corner's + the centre-footprint probe's Z label on the Start Job schematic now reads as height above the fixture's true bare-jaw-top (backing the Set-position park margin out of the stored fixture Z), directly comparable to the descent math Start Job prints during a run - previously it read relative to the lowest probed corner (a flatness-only number, unrelated to the jaw).
- **No more silent STOCK overwrite** - Start Job used to overwrite the entered Width/Height/Thickness from a loaded program's `(STOCK ...)` comment on every tab activation, even if the operator had already set them for a different job that happened to still be loaded. It now detects a mismatch (0.05mm tolerance) and reveals a "Copy from loaded program's STOCK" button instead of applying anything automatically.
- **Safer, faster corner-to-corner travel** - Corner Fence's corners 2-4 now travel at corner 1's own measured stock top plus an operator-set "Corner travel margin (Z)" field (new, persisted), instead of always retracting fully to machine Z0 or using a generic floor+30 estimate before each corner's own top probe - reusing the `#<_ls_maxz>` mechanism the vise path already had. The optional TLO-reference step's G30 detour gets its own explicit full retract, decoupled from that trusted height so it doesn't also starve the corner that follows it.
- **pcorner.macro sign bug fix** - its `#<_ls_maxz>` "is this set" check used `GT 0`, but maxz is a machine Z coordinate and every legitimate value is negative on a machine homing Z to the top - the check silently failed for every real maxz ever passed (both Corner Fence's and the vise's own), always falling back to the slower/more conservative path with no error. Fixed to `NE 0`.
- **New opt-in "Stock size is exact" mode** - when the entered size is trusted exact rather than a padded estimate, corners (including a tight re-probe of corner 1 itself, using its own first-pass result as reference) probe within a small, hand-tuned distance of their computed true position instead of a generous safety margin. Off by default; unchecked behavior is unchanged.
- **Tidier face-probe repositioning** - pcorner.macro's X/Y-face retreats now prefer the caller's trusted safe height over recomputing one per corner, and the Y-face probe moves diagonally straight to its start position instead of routing back through the top-probe's own XY (over the stock) first.

Hardware-verified end-to-end: all 4 corners of a Corner Fence Measure run, exact-size mode enabled.

#126 Surface the firmware hang-watchdog restart notice + firmware-change detection on reconnect

File	+	−
CNC Core/CNC Core/Grbl.cs	38	0
CNC Controls/CNC Controls/AppConfig.cs	15	0
ioSender XL/ioSender XL/HangWatchdogReporter.cs	129	0
ioSender XL/ioSender XL/CrashReporter.cs	1	0
ioSender XL/ioSender XL/App.xaml.cs	6	0
ioSender XL/ioSender XL/JobView.xaml.cs	7	4
ioSender XL/ioSender XL/ioSender XL.csproj	1	0

The iMXRT1062 driver grew a hardware hang watchdog (WDOG1) this session: a stuck g-code/$-command dispatch (main loop never returns to feed it) now resets the board instead of staying wedged, and the line it was stuck on is retained across the reset and reported via `$I`/`$I+` as `[MSG:Restart after controller hang processing: <line>]` - see the firmware repos' own history for that half.

- **Console surfacing on reconnect.** `GrblInfo.Get()` (already queried automatically on every connect) now forwards any `[MSG: … ]` line straight to the console, bypassing the `Silent` flag that normally suppresses connect-time chatter - the restart notice (and any other controller message) shows up without a manual `$I`.
- **Reconnect no longer waits for Alarm to clear.** `JobView.OnReconnectInit()` used to skip the `$I`/handshake re-query entirely while the controller was in Alarm (common right after any reset - homing required) and defer it until a later unlock. `$I` is explicitly Alarm-safe on the firmware side, so this ran immediately instead - the exact case this feature needs to catch.
- **Firmware-change detection.** The connected firmware's build identity is compared against the one seen on the previous connection (persisted in `App.config`); a change logs `[MSG:Firmware changed since last connection — was: X, now: Y]`.
- **Report-it dialog.** New `HangWatchdogReporter.cs`, modeled on the existing crash-reporter pattern (no backend, no auto-send) - pops up when the restart notice is detected, showing the stuck line and a button that opens a pre-filled GitHub issue against the firmware repo.
- Both the new dialog and the existing crash-reporter dialog now call `DialogScaling.Apply()`, which they'd been missing - they were rendering at 100% regardless of the configured UiScale zoom.

Firmware-side half (WDOG1 arm/feed, retained-line record, $HANG test command) lives in the core/iMXRT1062 driver repos, not this repo. Builds clean.

#127 AppMessageBox: message boxes now scale with the app's UiScale zoom

File	+	−
CNC Controls/CNC Controls/AppMessageBox.xaml	22	0
CNC Controls/CNC Controls/AppMessageBox.xaml.cs	131	0
CNC Core/CNC Core/AppDialogs.cs	13	2
ioSender XL/ioSender XL/App.xaml.cs	1	0
docs/playbooks/drive_ui_test_server.md (new)	156	0

The native `System.Windows.MessageBox` is an OS dialog outside the WPF visual tree, so unlike every other ioSender dialog it never picked up the app's UiScale zoom (Ctrl+Alt+Plus/Minus) - on a high-DPI laptop it stayed small and hard to read while the rest of the app scaled with it.

- **New `CNC.Controls.AppMessageBox`** - a WPF window mirroring `MessageBox.Show`'s API (message/caption/buttons/icon/default result), scaled the same way every other dialog is (`DialogScaling.Apply`), so it grows with the rest of the UI instead of staying fixed size.
- **Drop-in for existing call sites** - `AppDialogs.Show` (the app's one message-box funnel, ~146 call sites) now prefers a registered `CustomMessageBox` delegate over the native `MessageBox.Show` for the real-user fallback path; `AppMessageBox.Register()` wires it in once at startup. No call site changed.
- **New playbook** - `docs/playbooks/drive_ui_test_server.md` documents launching ioSender under `--testserver` and driving/verifying controls by `x:Uid` over HTTP, plus the gotchas hit building a standalone in-process render harness to verify this feature's scaling (stale satellite resource DLL, missing app.config, dispatcher shutdown mode, layout-settling races).

Verified via an in-process RenderTargetBitmap capture harness at 1.0x/1.8x/2.5x UiScale (icon, text, and buttons all scale together, full text visible with no clipping) - caught and fixed a real bug along the way, a stray `Window MaxWidth` that clipped scaled-up content instead of letting it grow.

#128 Analog joystick jog was jerky under continuous hold - moved off the UI thread

File	+	−
CNC Core/CNC Core/ControllerMapper.cs	52	13

Keyboard's continuous jog sends one `$J` command with an effectively-infinite distance and lets grblHAL run it until jog-cancel on key-up, so it's smooth by construction. Analog stick jogging instead re-sent a short `$J` segment roughly 15x/sec, relying on each new segment landing before the previous one finished so grblHAL could blend them into continuous motion - and that resend loop ran on `ControllerService`'s UI-thread `DispatcherTimer`. Any UI-thread hitch (layout, rendering, other UI-bound work) could delay a tick past that window: the current segment ran dry, motion decelerated to a stop, and the next segment's arrival read as a jerk. Same bug class as the job-streaming stutter fixed earlier via the background `StreamPump`.

- **Background analog-jog thread.** The stick/trigger poll-and-send loop now runs on its own dedicated background thread with a precise sleep-based cadence, independent of the WPF dispatcher.
- **Direct low-level writes.** Jog commands are written straight to `Comms.com.WriteCommand` - the same path keyboard jogging already uses - instead of `GrblViewModel.ExecuteCommand`, which touches UI-bound `ObservableCollections` and isn't safe to call off the UI thread.
- **Button mapping unaffected.** Discrete button-press actions (step jog, cycle start, home, etc.) stay on the UI thread as before - only the continuously-resent analog stream needed to move.

Hardware-verified: user reported motion "smooth as silk" holding the stick over a real connection, versus visibly jerky before.

#129 Tooltips now scale with the app's UiScale zoom

File	+	−
`ioSender XL/ioSender XL/App.xaml`	15	1

ToolTips render in their own `Popup`, outside `MainWindow`'s `LayoutTransform`-scaled visual tree - the same gotcha `DialogScaling.cs` (#127) already documents for dialog windows. Without accounting for it, tooltip text stayed fixed at 1x regardless of the app's UiScale zoom (Ctrl+Alt+Plus/Minus) - readable at the default size but nearly illegible once the rest of the UI was zoomed up.

- **App-wide implicit `ToolTip` style** (`App.xaml`) - a `LayoutTransform` bound to `Base.UiScale`, the same scaling pattern `MainWindow` uses for its own content and `DialogScaling` uses for other top-level windows. No `x:Key`, so it applies automatically to every tooltip app-wide, including the common `ToolTip="…"` string shorthand - no per-control changes needed anywhere.

Verified by hand: hover tooltip text now grows in step with Ctrl+Alt+Plus zoom.

#130 New -simulator command-line flag to auto-connect to the bundled simulator on startup

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	33	1

Serial and network already had a scriptable startup path via the existing `-port`/`-baud` flags (e.g. `-port COM3 -baud 115200`, `-port 192.168.1.50:23`) - both wired straight into the same startup connect path the Connect dialog uses, bypassing it entirely. The bundled simulator had no equivalent: a raw `-port 127.0.0.1:23` would only work if the simulator process happened to already be running, since launching it first requires setting `Base.StartSimulator` and the simulator exe/args - fields only the Connect dialog's Simulator tab populated.

- **New `-simulator` flag** - rewrites the startup target to `127.0.0.1:<port>` and marks it as a simulator connection (the same fields the Connect dialog's Simulator tab sets), so the existing `EnsureSimulatorRunning` launches the bundled sim before connecting, exactly mirroring a manual Simulator-tab connect. Reuses the last simulator port if the saved target was itself already a simulator connection, otherwise defaults to 23 (the Connect dialog's own default).
- **Graceful no-op if nothing's built yet** - shows a message and falls through to the normal startup path instead of trying to connect to a nonexistent exe, if nothing has been built to `%AppData%\Simulator` yet (same precondition the Connect dialog's Simulator tab already enforces).

Verified end-to-end: `build.ps1 -Launch -simulator` launched the bundled simulator and connected automatically with no dialog, confirmed via the console log's boot banner + handshake.

#131 Parallel agent-driven UI testing: -testserver launches no longer single-instance, -simulator takes a port

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	20	5
ioSender XL/ioSender XL/App.xaml.cs	7	1
ioSender XL/ioSender XL/MainWindow.xaml.cs	10	3

Agent Cockpit-style parallel UI testing (several automated sessions each driving their own ioSender instance via the WpfUiTestServer) hit two collisions:

- **Hard single-instance lock** - `PipeServer.cs`'s fixed named pipe (`"ioSender"`) meant a second launch always forwarded its args into the first and exited immediately, and the first instance's window got forcibly foregrounded via `ActivateRequested/BringToForeground` - stealing focus from whatever the operator was doing. Skipped both sides (the check in `App.xaml.cs`, becoming a pipe listener in `MainWindow.xaml.cs`) whenever `-testserver` is active - an automation-driven instance was never meant to be a singleton. A normal interactive launch is untouched; a `-testserver` instance now coexists with a real instance, or with other `-testserver` instances, without colliding or force-activating anything.
- **`-simulator` now takes an optional explicit port** (`-simulator 8901`) that drives both the bundled simulator's own `-p` launch argument and the TCP address ioSender connects to - the same `simPort` variable already did double duty here, just newly exposed on the command line. Bare `-simulator` is unchanged (reuses the last saved simulator port, else defaults to 23), so two parallel test agents can each point at their own simulator instance instead of colliding on one port.

Self-verified end-to-end: built Debug, launched a normal instance (A) then a `-testserver=18762` instance (B) - both stayed alive simultaneously (previously B would have forwarded into A and exited), A was never force-activated (foreground window confirmed via `GetForegroundWindow` before/after), and B's own test server answered independently on its port. Known separate, still-open issue: a fresh launch with no other instance running can still take initial OS foreground focus on its own (confirmed in the same test) - a window-creation behavior, not a single-instance collision, and not addressed by this fix.

#132 -testserver launches never steal OS foreground focus, screenshots stay intact

File	+	−
ioSender XL/ioSender XL/App.xaml.cs	7	1
ioSender XL/ioSender XL/MainWindow.xaml.cs	64	3

Even with the single-instance collision fixed (#131), a fresh `-testserver` launch with nothing else running still grabbed OS foreground focus purely from becoming visible - confirmed via `GetForegroundWindow` before/after, independent of `Activate()`/`ShowInTaskbar`/window position.

- **WS_EX_NOACTIVATE, applied at the raw window-handle level** as soon as the HWND exists (`SourceInitialized`), gated on `-testserver` - a documented, first-party Win32 extended window style that makes the window permanently non-activatable at the OS level, regardless of any later `Show()`/`Opacity` transition.
- **Screenshots stay fully intact** - the first attempt (leaving the window at `Opacity=0` forever instead) did stop the focus steal, but broke `WpfUiTestServer`'s `/screenshot`: it renders the window via `RenderTargetBitmap` (an in-memory Visual render, not a screen grab), which honours the Window's own Opacity, so every whole-window screenshot came back blank - unacceptable, since screenshots are the essential tool for automated visual verification. `RevealMainWindow` now always sets `Opacity=1` again; the window is parked off-screen (`Left`/`Top` = -32000) and out of the taskbar purely for tidiness, with WS_EX_NOACTIVATE doing the actual focus protection.

Hit two real P/Invoke bugs along the way, both confirmed via the crash log's `EntryPointNotFoundException` rather than guessed: `GetWindowLongPtr`/`SetWindowLongPtr` are C preprocessor macros, not real DLL exports, and this build actually runs 32-bit (confirmed via `IsWow64Process`, despite `AnyCPU`) where `GetWindowLongPtrW` doesn't exist at all - only `GetWindowLongW` does. Used the standard `IntPtr.Size`-dispatched 32/64-bit pattern. Self-verified end-to-end multiple times: foreground window confirmed unchanged across a fresh launch with nothing else running, WS_EX_NOACTIVATE confirmed actually set on the live window handle, full UIAutomation tree functional (204 elements), and `/screenshot` returns a complete, correctly-rendered ~600KB PNG of the live UI - not the blank 35KB frame from the Opacity=0 attempt.

#133 Medium-priority tooltips: spindle, goto/outline, macro dialogs, network connection settings

File	+	−
CNC Controls/CNC Controls/SpindleControl.xaml	3	3
CNC Controls/CNC Controls/GotoBaseControl.xaml	3	3
CNC Controls/CNC Controls/OutlineBaseControl.xaml	1	1
CNC Controls/CNC Controls/MacroManagerDialog.xaml	2	2
CNC Controls/CNC Controls/MacroEditor.xaml	3	3
CNC Controls/CNC Controls/PortDialog.xaml	5	4
Locale/*/csv (7 locales)	160	0

Fills in the next tier of secondary controls left without hover guidance: the spindle direction radio buttons, the Goto/Outline action buttons, the Macro Manager/Macro Editor dialogs, and PortDialog's network tab (WebSocket, port, host, Scan network) - the last of these connection-critical settings had no guidance at all.

- **SpindleControl, GotoBaseControl, OutlineBaseControl, MacroManagerDialog, MacroEditor** - added `ToolTip` to each control listed above; none of these five files had ever been registered with `tools/locadd.py`, so they were also missing rows for their *existing* ToolTips (e.g. Macro Manager's View/Edit/Prompt/Key column tooltips) - backfilled along with the new ones.
- **PortDialog.xaml** - added a `x:Uid` to the network-port `NumericTextBox` (previously un-addressable) and tooltips to the WebSocket checkbox, port field, host combo, and Scan network button.

Verified: Debug build clean; test-server run confirmed all Goto/Outline/Spindle controls realize on the main window, and the PortDialog Network tab (WebSocket/Port/IP address/Scan network) renders correctly via `/screenshot/dlg_connection`. `tools/locadd.py` backfilled 160 rows across all 7 locale CSVs and is idempotent on a second run.

#134 Dry run mode: sender-side spindle/coolant-off run, and the MDI path is now typed-text only

File	+	−
CNC Core/CNC Core/GrblViewModel.cs	7	112
CNC Controls/CNC Controls/JobControl.xaml.cs	96	1
CNC Controls/CNC Controls/JobControl.xaml	9	0
CNC Controls/CNC Controls/StreamPump.cs	12	0
CNC Controls/CNC Controls/GCode.cs	13	0
CNC Core/CNC Core/GCodeJob.cs	39	7
CNC Controls/CNC Controls/MacroProcessor.cs	65	77
ioSender XL/ioSender XL/MainWindow.xaml.cs	15	7
CNC Controls Probing/CNC Controls Probing/ProbingView.xaml.cs	2	1
CNC Controls Probing/CNC Controls Probing/Program.cs	2	1
ioSender XL/ioSender XL/StartJobView.xaml.cs	1	1
Locale/*/csv (7 locales)	21	0

A dry run offsets Z clear of stock and forces the spindle/coolant off for the whole run, regardless of what the loaded program contains, so a toolpath can be watched without cutting or spinning anything. Built directly in response to a real incident: an incomplete, uncommitted "dry run" implementation offset Z but never suppressed spindle/coolant commands, so a loaded program's own M3 turned the spindle on at 10,000 RPM with a 3D probe installed, tearing the USB cable out of its connector.

- **Per-block spindle/coolant detection** (`GCodeBlock.HasSpindleOrCoolantOn`) computed once at load time from the real G-code parser, consumed by both streaming paths (`StreamPump`/legacy `SendNextLine`) to neutralise M3/M4/M7/M8 as they're sent; M5/M9 are also queued as a defensive preamble in case the spindle/coolant was already on.
- **Auto-clears** when a run ends or aborts, or a different program loads - and is now correctly scoped to the loaded job only, never a macro/tool-generated run streamed via `RunStreamedJobInPlace` (`GCode.IsTransient`), which was leaking the Z offset into unrelated macro runs.
- **Refused outright** (not silently skipped) for SD-card jobs and generator/wizard runs where the sender can't filter individual commands, rather than running them unprotected.
- **The MDI bypass is gone**: every macro/wizard/probing call used to fall back to `GrblViewModel.ExecuteMacro` (an unprotected, non-flow-controlled send) for content that merely looked "small" or non-motion. `MacroProcessor.Flush` now always routes through the flow-controlled job streamer, so dry-run's filtering (and Feed Hold/Stop) can't be bypassed by a burst that happens to look too small to matter - MDI is reserved exclusively for text the operator actually typed into the MDI box.

Two follow-on bugs surfaced by that transition, both fixed: a stray line-number prefix on a bare `#<name≥value` line broke the controller's parameter-assignment routing (GCodeJob now exempts those lines from numbering, same as O-word lines); and an intermediate macro burst could race a later one for a shared streaming hand-off (RunStreamedJobInPlace only kicks off a burst asynchronously, sharing one RunControl.Source field) - intermediate flushes now block until their burst genuinely finishes before the next one starts.

#135 Consolidated log file creation/rotation/retention into one shared LogFile primitive

File	+	−
CNC Core/CNC Core/LogFile.cs	141	0
CNC Core/CNC Core/ConsoleLog.cs	13	53
CNC Core/CNC Core/DebugLog.cs	10	36
ioSender XL/ioSender XL/App.xaml.cs	6	8
CNC Core/CNC Core/CNC Core.csproj	1	0

Housekeeping found while chasing an unrelated bug: three separate log writers (`console_<timestamp>.log`, `ioSender.debug.log`, `ioSender.crash.log`) each hand-rolled the same 8MB-to-".1" rotation logic, and only the console log had any age-based pruning.

- **`CNC.Core.LogFile`** - one shared primitive: logs-directory resolution (with the app-folder fallback everyone needed), size-based rotation, and - only for a fresh-file-per-run log - per-run timestamped naming, age-based pruning, and a "latest" alias. `ConsoleLog`/`DebugLog`/the crash logger now all call into it instead of three copies of the same code.
- **`ioSender.crash.log` had no size cap at all** - unbounded growth over the life of an install - and now gets the same rotation as the others.
- **`latest_console.log`** - an NTFS hard link (not a symlink, so no elevation/Developer Mode needed) to the current run's console log, updating live since it's the same underlying file, so it never has to be hunted down by timestamp.

#136 pcorner/pvisecorner: spoilboard probe was searching from the wrong Z

File	+	−
macros/pcorner.macro	9	2
macros/pvisecorner.macro	6	1

Both macros clear G54 (`G10 L2 P1 X0 Y0 Z0`) so their "absolute" Z moves act as machine coordinates - but grblHAL's WCO is G5x offset + G92 offset + tool length offset (`gc_get_offset`, `gcode.c`), and G10 L2 P1 only zeros the first term. A real Start Job run left a non-zero G92 offset active (confirmed via `$G` and grblHAL's own `is_g92_active()` in `report.c`), which the macros never cancelled - so a machine-Z reference like the fixture's saved spoilboard position got driven as if it were a work-Z offset, landing the search roughly the G92 offset's own magnitude away from the real surface, well outside its 12mm travel cap.

- **`G92.1` added** alongside the existing G10 L2 P1 in both macros, so they're robust to whatever modal state preceded them - matching what their own "run in machine coordinates" comment already claimed to guarantee. `G49` also added defensively (tool length offset was already clean in this repro, but WCO includes it too).

Confirmed fixed on real hardware after re-uploading both macros.

#137 Start Job: corner-2 travel height after TLO ref, unit-toggle readout, dialog scaling

File	+	−
`ioSender XL/ioSender XL/StartJobView.xaml.cs`	15	3
`CNC Controls/CNC Controls/MacroProcessor.cs`	2	0

Three small issues found testing the pcorner G92 fix above:

- **Corner-2 travel height** - after setting the TLO reference the machine parks at G30 and explicitly retracts to Z0, but pcorner.macro's own internal logic then drops straight to the trusted "Safe Z" height *before* traveling XY to corner 2 - undoing that retract and crossing the unverified G30-to-fixture territory at a lower height than intended. Now crosses to corner 2's approach XY while still at Z0, scoped to just this TLO-ref detour (normal corner-to-corner travel elsewhere is already validated safe at Safe Z height).
- **Zstock label always in mm** - the schematic's per-corner `t=` label formatted the raw mm value directly instead of the same unit-aware `FormatLen` helper the bottom-left readout uses; the in/mm toggle also relabelled the input fields and redrew the schematic but never re-ran that readout, leaving it stale.
- **Dialog scaling** - the modeless "install probe" MBOX prompt and the PROMPT-directive input dialog (`MacroProcessor.ShowHoldPrompt/ShowPromptDialog`) built their own raw `Window` and never called `DialogScaling.Apply` like every other dialog in the app, so - unlike the "Run macro?" confirm shown right before them - they didn't pick up the UiScale zoom.

#138 Dry run skips the program's own tool changes instead of running them

File	+	−
CNC Controls/CNC Controls/JobControl.xaml	8	4
CNC Controls/CNC Controls/JobControl.xaml.cs	8	0
CNC Controls/CNC Controls/StreamPump.cs	13	0
CNC Core/CNC Core/GCodeJob.cs	22	2
Locale/*/csv (7 locales)	7	7

There's no safe way to suspend/restore a mid-stream `G92` around a tool change - G-code is locked out during the Tool state a change parks in. Simpler and more robust: dry run never cuts, so which tool is actually in the spindle doesn't matter.

- **`GCodeBlock.HasToolChange`** (detected from the real parser's M6 tokens, same pattern as the existing spindle/coolant suppression) - `M6` lines are now neutralised the same way `M3/M4/M7/M8` already are, so the program's own tool changes never run during a dry run at all. The program just runs straight through without pausing for a swap.

#139 pcorner.macro: touch-plate DISCOVER now rises to Z0 first; corner 2's approach height fixed

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	15	14
macros/pcorner.macro	35	3

Two related corner-travel issues found testing the pcorner G92 fix from the previous entry:

- **Touch-plate DISCOVER rise to Z0** - the Z0-rise-before-travel was written inside the same conditional block as the 3D-probe-only spoilboard search, so touch plate mode (which skips that block - a continuity probe can't detect a non-conductive spoilboard) silently skipped the rise too, going straight from wherever the caller left the machine to an ESTIMATED safe height with no verified-clear waypoint first. Split into its own unconditional, mode-independent step.
- **Corner 2's approach height** - reached via the TLO-reference detour (parks at G30, an arbitrary point not verified clear of any fence/clamp hardware), used to get a bespoke extra waypoint that made its structure diverge from every other corner. Replaced with pcorner.macro's new `#<_ls_appz>`: an optional override for ONLY that corner's first height decision (Z0 instead of the footprint-scoped `#<_ls_maxz>`), leaving every later height in that same call on the trusted value, same as every other corner. Sentinel is 9999, not 0 - Z0 is itself a legitimate override value here.

#140 Start Job: exact-size labels/hint follow the checkbox, touch-plate corners show a real Z reading

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	29	4

Two small polish items:

- **Exact-size labels** - checking "Stock size is exact" now switches the Width/Height/Thickness field labels to plain "Width (X):"/etc. and updates the hint text, matching the vise fixture's always-exact treatment, instead of staying on "Est. ..." regardless.
- **Touch-plate corner readout** - the schematic's per-corner thickness label ("t=...") only ever had a value in 3D-probe mode, since touch plate has no spoilboard reference to subtract (continuity probing can't detect a non-conductive spoilboard) - the value was silently omitted every time. Touch plate corners now show "z=<measured top>" instead: a genuine reading (useful for spotting unevenness across corners), not mislabelled as a thickness the mode can't measure.

#141 Dry run's Z clearance was computed wrong - two separate real bugs, both found on real hardware

File	+	−
CNC Controls/CNC Controls/JobControl.xaml.cs	30	2
CNC Core/CNC Core/GCodeJob.cs	26	16

Two independent bugs, both surfaced chasing the same symptom (dry run's Z clearance reading wildly different from the intended 10mm + stock thickness) on real hardware, with the operator's own live measurements pinning down the exact arithmetic.

- **G92 is relative to current position, not absolute.** `G92 Zk` makes WHEREVER THE MACHINE CURRENTLY IS read as work-Z=k - it has no notion of "the stock" at all. The previous code sent a fixed `G92Z−<offset>` from wherever Cycle Start happened to be pressed (typically wherever the last job/macro parked, e.g. Start Job's G30 - nowhere near the stock), so the resulting clearance was "however far the machine already was from the stock" plus the intended offset, not the offset alone. Now computes the actual `G92` value needed to put work-zero exactly `offset` mm above the TRUE stock top, using the machine's live current position and the live (pre-offset) work-coordinate-offset: `k = MachinePosition.Z − (WorkPositionOffset.Z + offset)`.
- **Token-accumulation over-suppression, found along the way.** `GCodeBlock.HasSpindleOrCoolantOn/HasToolChange` scanned `Parser.Tokens` - a CUMULATIVE, whole-file list (cleared only once per file load, not per line; several other features - the 3D carve view, rotate/wrap, etc. - all rely on it staying that way) - so once the first `M3/M4/M6/M7/M8` appeared anywhere in a file, every line after it was flagged too, silently turning the entire rest of the program into no-op comments during a dry run. Confirmed on real hardware: hundreds of instant acks, zero motion, ending in a Hold then a reset. Both helpers now only scan the token slice added by that specific line's own parse call.

Confirmed fixed on real hardware after both fixes landed - dry run's Z clearance now matches the intended offset regardless of where the machine starts.

#142 OBS/Tapo camera recording control, independent of OBS's main Record button

File	+	−
CNC Controls/CNC Controls/RtspCamerasControl.xaml.cs	247	0
CNC Controls/CNC Controls/RtspCamerasControl.xaml	15	0
CNC Controls/CNC Controls/ScenarioNameDialog.xaml.cs	61	0
CNC Controls/CNC Controls/ScenarioNameDialog.xaml	20	0
CNC Core/CNC Core/ObsBridge.cs	70	2
ioSender XL/ioSender XL/App.xaml	28	0
CNC Controls/CNC Controls/ActionKeyBinder.cs	8	0
CNC Controls/CNC Controls/MainPanelRegistry.cs	5	1
ioSender XL/ioSender XL/MainWindow.xaml	5	0
CNC Controls/CNC Controls/CNC Controls.csproj	14	0

New "OBS Control" panel (RtspCamerasControl) starts/stops recording on the 2 Tapo RTSP cameras (Front Left/Front Right) plus the app's own screen-capture source, independently of OBS's main Record button - via obs-websocket v5's `SetSourceFilterEnabled` against each camera's own Source Record filter instance (not `TriggerHotkeyByName`, whose hotkey names collide across filter instances).

- **Per-camera + "All" toggles** - red circle (idle) swaps to a red square (recording); a matching "All" toggle also lives on the main run bar next to Timelapse.
- **F7-F12 hotkeys** registered per camera/app via ActionKeyBinder, driving the same ObsBridge entry point as the panel buttons.
- **"All" stop** prompts for a take name and files the 3 resulting recordings into a named subfolder (matched by source name or camera label, since Source Record's actual output filename uses the label).

Gated behind `-demomarker` (same visibility rule as Timelapse) - a demo-shoot-only affordance, hidden in normal use. Built for filming ioSender manual videos.

#143 Fixture edit dialog: redesigned Corner Fence schematic + directly-editable X/Y/Z position

File	+	−
CNC Controls/CNC Controls/FixtureEditDialog.xaml	135	51
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	116	27
CNC Controls/CNC Controls/Fixture.cs	33	1
tools/render-harness/ (new)	114	0
docs/playbooks/drive_ui_test_server.md	37	15
Locale/*/csv (7 locales)	91	14

The dialog is restructured to match ProbeDefinitionEditDialog's pattern (fields on the left half, schematic on the right half). The Corner Fence schematic is now a real 3-zone diagram - red keep-clear fence rails, a green near-corner reference zone, a cream far zone - all sized off the active 3D probe's body diameter (radius = D/2), replacing the old single dashed clearance circle. (The real bug behind "the green area looks too small" turned out to be an SVG arc sweep-flag silently picking the wrong of the two circles through the given endpoints/radius - not a colour or semantics issue.)

- **Position row** is now Set/Test buttons + a checkmark (green once Test position succeeds) inline with the label, followed by **directly-editable X/Y/Z Axis fields** - an alternative to re-jogging for a small nudge or a known-good value.
- **Test position now rapids** (G53G0) through its retract/traverse/drop-to-target travel, like every other Goto button, instead of crawling at G1 F1000.
- Fixed a real hardware bug where Test position's completion callback could silently never fire (and the checkmark never turn green) if the app's StreamingState already happened to equal Send/SendMDI from unrelated prior activity - the completion watcher is now armed directly from the macro runner's own "did it start" signal instead of inferring it from a property-change event that could be silently suppressed. An Alarm mid-probe no longer leaks that watcher's subscription forever, either.
- Added a warning note across the dialog's bottom: the 3D probe must be installed before Set/Test, and the saved position must be within ~10 mm of the spoilboard.

`tools/render-harness/` (new, kept permanently): a standalone WPF tool that renders the real dialog class against the real on-disk AppConfig, for verifying modal dialogs the UI test server can't reach - built after two rounds of a hand-copied XAML mock silently drifting from what the real dialog actually rendered. Fully HW-verified end to end.

#144 Machine Setup tab could go permanently unresponsive after a mid-move tab switch

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	25	2

MachineSetupWizard.Activate(true) called RefreshMacroStatus() synchronously - but that queries the controller's filesystem listing via a call that pumps the WPF dispatcher, and tab activation runs from MainWindow's tab-switch handler *during* that switch's own layout pass. A pumped-dispatcher wait colliding with an active layout pass throws mid-switch, after the new tab was already marked current but before the switch actually finished - leaving the tab looking selected but frozen, permanently (a mid-layout-pass exception doesn't self-heal the visual tree). A second call site in the same file already deferred the identical call for this exact documented reason; this one just never got the same treatment.

Reproduced and root-caused via targeted diagnostic logging against a live G28+G30-then-click-Machine-Setup repro; fixed by deferring the call, matching the existing pattern. Confirmed fixed on real hardware.

#145 Job tab workspace lost width priority to the Feed/Spindle/Outline column

File	+	−
ioSender XL/ioSender XL/JobView.xaml	6	1

The right-hand slot panel (Feed rate/Spindle/Outline) only had a MinWidth, so it could stretch as wide as its content wanted at the expense of the Program/3D View/Console workspace, which only got whatever width was left over. Capped at MaxWidth=260 - that column's content wraps to more lines instead, which its own ScrollViewer already handles.

#146 Message box text was too small

File	+	−
CNC Controls/CNC Controls/AppMessageBox.xaml	1	1

AppMessageBox is the one funnel every message box in the app (146+ call sites, via AppDialogs.Show) routes through, so bumping its text size here fixes all of them at once.

#147 -message launch flag: show a popup once the app is up

File	+	−
ioSender XL/ioSender XL/App.xaml.cs	11	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	51	0

Dev/testing convenience, same idiom as the existing `-testserver`/`-debuglog` flags: `-message="text"` shows a plain informational popup once the main window is up and painted, skipped for a `-testserver` launch since no one is watching that window interactively.

Also folds in MainWindow's run-bar copy of the OBS Control "All" recording toggle (see the camera-control entry above) and standing diagnostic logging for an intermittent, still-unreproduced ATC-macro-gate false positive.

#148 "Restart to apply" silently closed the app instead of relaunching; dead AppLaunch supervisor removed

File	+	−
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	15	10
CNC Controls/CNC Controls/PipeServer.cs	1	1
ioSender XL/ioSender XL/App.xaml.cs	8	2
tools/run-iosender.ps1	8	8
ioSender XL/ioSender XL.sln	0	10
CNC Core/CNC Core/RelaunchSupervisor.cs	0	38
CNC AppLaunch/ (whole project, 7 files)	0	356

Clicking **Restart** on any Settings tab that flags a restart-only change just closed ioSender and never reopened it - the new process's own single-instance startup probe found the still-shutting-down old instance still listening on the singleton pipe, forwarded into it, and exited immediately.

- **Root cause** - a race between `GrblConfigView.DoRestart()`'s `Process.Start`-then-`Shutdown` fallback and `PipeServer`'s single-instance check in `App.xaml.cs`. An earlier attempt at disposing the old listener before relaunching looked correct in isolation but failed under real timing (`NamedPipeServerStream.Dispose()` doesn't reliably cancel a pending `WaitForConnection()` on .NET Framework).
- **Fix** - the child process is now launched with `IOSENDER_SELF_RELAUNCH=1`, and `App.xaml.cs` skips the single-instance probe for that one launch, since it's a known self-relaunch rather than a genuine second instance. Verified end-to-end via UIAutomation: old instance closes, new instance comes up and responds, and a genuine third launch still correctly single-instance-forwards afterward.
- **AppLaunch.exe removed entirely** - the `CNC AppLaunch` project and `RelaunchSupervisor.cs` implemented an exit-code relaunch-supervisor protocol that turned out to be dead code: `install.ps1`'s desktop shortcut always launched `ioSender.exe` directly, so the supervisor path was never actually wired into the real product.

#149 Generate buttons now save their program to disk; Start Job Corner 2 ALARM:2 fixed (Safe Z delta)

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	21	2
ioSender XL/ioSender XL/StartJobView.xaml.cs	20	3
ioSender XL/ioSender XL/StartJobView.xaml	2	2
CNC Controls/CNC Controls/AutoSquareWizard.xaml.cs	1	3
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	1	3
CNC Controls/CNC Controls/SurfaceSpoilboardWizard.xaml.cs	1	3
Locale/en-US/csv/ioSender.resources.en-US.csv	2	2

Two related fixes from a real hardware debugging session.

- **Generate-to-disk logging** - every tab with a Generate button (Start Job, Auto square, Stepper calibration, Surface spoilboard) now writes its generated program to `%AppData%\ioSender\Generated\<name>.macro` at generate-time via a new shared `MacroProcessor.PublishGenerated` helper, instead of only when the program actually streams via `MacroProcessor.Run`. This makes the exact generated program inspectable for post-mortem even if the run alarms out immediately or Run is never pressed.
- **Start Job Corner 2 ALARM:2, hardware-confirmed root cause** - the field controlling the safe crossing height between corners (`c1_maxz` = corner 1's measured stock top + this value) had silently persisted at 0mm, leaving zero clearance for corner-to-corner travel. Renamed the confusingly-named "Corner travel margin (Z)" field to **"Safe Z delta"**, and Generate now warns (not blocks) if it's set below the 10mm default - confirmed on real hardware that 5mm was still not enough clearance and broke a probe tip; 1" (25.4mm) ran Start Job through to completion.

#150 User-draggable GridSplitters: Start Job, Height Map, Probing/Tools sub-tabs, Job tab's 3 regions

File	+	−
ioSender XL/ioSender XL/JobView.xaml	58	31
ioSender XL/ioSender XL/HeightMapView.xaml	11	3
Probing sub-tabs (Center/Edge ext/Edge int/Tool length)	26	12
Tools sub-tabs (Auto Square/Stepper cal/Stepper cal scratch/Surface spoilboard)	25	13

Every 2-column "setup inputs | drawing" view now has a real draggable divider instead of a fixed split. First attempt put the splitter on the shared column boundary (behind the input panel's own content, which swallowed its hit-test area - no resize cursor ever appeared on most views, and one that did show a cursor still didn't drag); fixed by giving each splitter its own dedicated column, the same approach already proven on the Job tab's two dividers.

- **Job tab** - left sidebar / Program-3D View-Console tabs / right sidebar converted from a DockPanel's fixed Left/fill/Right docking to a Grid with two GridSplitters. The right sidebar's own internal Feed/Spindle/Outline column split keeps its existing MaxWidth=260 cap untouched.
- **Start Job / Height Map** - left column also made proportional (min original px width) instead of a hardcoded pixel width, so it no longer gets squeezed as the window widens.

User-verified on real hardware across all views.

#151 Start Job probes the fence's origin corner once instead of twice

File	+	−
CNC Controls/CNC Controls/Fixture.cs	52	3
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	104	4
ioSender XL/ioSender XL/StartJobView.xaml.cs	48	42
ioSender XL/ioSender XL/StartJobView.xaml	24	3
CNC Controls/CNC Controls/AppConfig.cs	24	0
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	17	2
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	7	0

Fixture.CornerOffsetX/Y and Fixture.SpoilboardZ (new): captured once by FixtureEditDialog's "Test position" via a real pcorner.macro probe against the physical fence, instead of Start Job re-locating the corner and re-probing the spoilboard every single job. StartJobView.BuildProgram's corner-1 probe now runs in REUSE mode using these cached values, landing directly on the tight 5mm-inset anchor in one probe instead of a loose locate pass followed by a tight re-probe (the old "exact size" corner-1 block, now removed).

- **Fixtures.SetItems/Load** now mutate the existing collection in place instead of replacing the reference - MachineSetupWizard binds its fixture grid before AppConfig.LoadConfig() runs, so a reference swap was silently orphaning that binding (only fixtures added/edited in-session ever showed).
- **Fixture.Coords** setter no longer clears CornerOffsetX/Y/SpoilboardZ itself - that fired during XML deserialization too (property setters run in declared order), silently zeroing a just-loaded valid offset on every app load.
- **OffsetFlyout's** G28 fixture picker was filtering the SHARED default CollectionView of the app-wide Fixtures.Items, which leaked its "validated only" filter into every other control bound directly to that same collection (Machine Setup's own fixture grid showed only one fixture, looking like the rest had been deleted). Now uses its own private ListCollectionView.
- **StartJobView** - Width/Height/Thickness/units used to be the only fields gated on a fixture being selected; Probe type, Set origin, Measure/Rotate/Exact-size/TLO-ref and Safe Z delta were left fully interactive with none chosen. Gated as one block now. Left column was also a hardcoded 360px width, squeezing text as the window widened - now proportional.

Two real hardware bugs found and fixed along the way: the corner-locate probe was silently inheriting a stale DISCOVER flag (probed the spoilboard twice), and it parked 10mm outward instead of at the true corner, landing 5mm outside the corner instead of inside it.

#152 NumericField stores canonical mm internally; unit toggle is display-only

File	+	−
CNC Controls/CNC Controls/NumericTextBox.cs	86	2
CNC Controls/CNC Controls/UIUtils.cs	62	0
CNC Controls/CNC Controls/NumericField.xaml.cs	31	1
ioSender XL/ioSender XL/StartJobView.xaml.cs	40	50
CNC Controls/CNC Controls/NumericField.xaml	2	1

Closes the long-standing improvement-backlog item: NumericField.Value is now ALWAYS canonical mm for any field whose Unit is "mm"/"in" - every other Unit ("mm/min", "s", "deg", "%", ...) is completely untouched, so this is a no-op for the ~286 existing NumericField uses that never touch inches.

- **NumericField.IsImperial** (new, inheritable) - set ONCE on a container (a tab's root panel), every NumericField underneath picks it up automatically via WPF property inheritance and re-displays its already-canonical Value in the new unit - no per-field looping, no external code touching Value at all.
- **Input parsing** moved from strict per-keystroke rejection to commit-time (blur/Enter) for length-unit fields, so it can accept free typing including an inline unit suffix that overrides the field's own displayed unit for that one entry.
- **StartJobView migrated** as the proof of concept: every ToMm/FromMm wrapper around a NumericField.Value read/write is gone; the unit-toggle handler is now a single SetIsImperial call instead of manually converting 5 fields.

AutoSquareWizard/SurfaceSpoilboardWizard/StepperCalibrationScratchWizard still have their own pre-existing manual ToMm/FromMm wiring - not yet migrated.

#153 pcorner.macro side probes at a fixed 5mm below top, not the stock midpoint

File	+	−
macros/pcorner.macro	16	13

Was top - thickness/2 (self-safe for any thickness, but dependent on the caller's estimated stock thickness, which is only ever a guess before Measure has actually run) - now a fixed 5mm inset per user request. The thickness global is no longer read by this macro at all (still accepted from callers, kept in case it's wanted again).

Not self-safe for very thin stock the way the midpoint approach was - anything under ~5mm has its side face searched below its own bottom face. Documented minimum stock thickness precondition bumped from 3mm to 5mm.

#154 Start Job stock schematic: bigger, clearer dimension/angle/thickness labels

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	39	20

Per-corner angle + thickness/Z used to be one combined label outside the corner; split into two - angle (a property of the drawn box shape) now sits INSIDE toward the centre, thickness/Z (a material reading at that point, not the shape) stays OUTSIDE past the corner. Offset bumped 22px -> 30px so the larger text doesn't crowd the corner.

#155 New Tools sub-tab: Stepper Calibration (Probe) + Start Job calibration-mismatch warning

File	+	−
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	497	0
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml	64	0
CNC Controls/CNC Controls/ToolsView.xaml.cs	39	1
CNC Controls/CNC Controls/AppConfig.cs	33	0
ioSender XL/ioSender XL/StartJobView.xaml.cs	37	0
CNC Controls/CNC Controls/LayoutModel.cs	4	2
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Controls/CNC Controls/LibStrings.xaml	1	1
CNC Controls/CNC Controls/IGrblConfigTab.cs	1	0

New Tools sub-tab "Stepper Calibration (Probe)": calibrates steps/mm by probing a reference block of ALREADY-KNOWN true size against a validated Corner Fence fixture, instead of V-bit scratch marks measured by hand with calipers - removes the manual-measurement step that was the likely source of an existing bad calibration (a probed touch is far more repeatable than reading a scratch mark with calipers). Reuses the corner-1 single-probe optimization (#151). Parks at G30 and prompts to install the probe before touching anything, same as Start Job; the tab itself is disabled whenever no 3D probe is configured. Shows the actual `new = current × measured ÷ true` calculation next to the measured values, plus an Undo button that reverts to the steps/mm value from right before the last Save.

- **Start Job** - a one-shot warning dialog fires when "Stock size is exact" is checked and the measured X/Y differs from the entered exact size by more than 0.5mm once all 4 corners are in - suggests running Stepper calibration, since that mismatch is exactly what motivated building this tool.
- **Old "Stepper calibration"** tool renamed to "Stepper calibration (Manual)" to disambiguate. Tools tab reordered (Probe, Squareness, Spoilboard, Scratch, Manual, then Tool table/Trinamic/PID) via a one-time layout-tree fixup.

User-verified on real hardware: converged from a ~1.5mm measured error to sub-0.01mm over two calibration passes. An inverted first attempt at the steps/mm formula made the error WORSE (~1.5mm -> ~4mm) before landing on the correct `current × measured ÷ true` direction.

#156 Versioned releases (2.N) replace the rolling "latest" build

File	+	−
`tools/cut-release.ps1`	130	0
`.github/workflows/release.yml`	40	24
`tools/add-changelog-entry.ps1`	2	1
`ioSender XL/ioSender XL/BuildInfo.cs`	2	1
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	40	30
`install.ps1`	3	2

The rolling "latest" release (one unversioned prerelease, overwritten on every push, with a body no more informative than a commit SHA) is gone. `tools/cut-release.ps1` now runs on every push to master: it looks up the previous published release, diffs `Overview.html`'s changelog entries against that release's embedded `changelog-through:N` marker, and publishes a real numbered release (`v2.1`, `v2.2`, ...) whose notes list exactly the #N entries that shipped in it. The Features & Fixes TOC gains a **Version** column, stamped by the same script, so every entry shows which release it first shipped in.

`install.ps1` and Check for Updates both switched from GitHub's by-tag/prerelease workaround (comparing commit SHAs) to the real `releases/latest` endpoint, comparing version numbers numerically instead.

The 13 rolling builds published since 2026-07-15 were backfilled as real historical releases (v2.1-v2.13, each tagged at its actual commit) so the version history matches what actually shipped, rather than starting cold.

#157 Install command now runs from CMD too; window title showed the wrong version

File	+	−
`tools/cut-release.ps1`	3	3
`install.ps1`	6	2
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	4	3

The release-notes install command used bare `irm … | iex`, which only works in PowerShell - pasting it into cmd.exe just prints an error. It's now wrapped as `powershell "irm … | iex"`, which runs correctly from either shell.

Separately, the main window's title bar (and splash screen) showed a hardcoded `2.0.1` left over from before the versioned-release pipeline existed, instead of the actual installed release version (e.g. 2.18). `MainWindow.Version` now reflects `BuildInfo.Version` for real published builds.

#158 Macro WAITIDLE false-abort after a burst's trailing motion

File	+	−
`CNC Controls/CNC Controls/MacroProcessor.cs`	56	8
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	74	27

A macro's `(WAITIDLE)` directive could false-abort right after a G30 park move, even though the move completed cleanly - hit intermittently on both Start Job and Stepper Calibration (Probe).

- **Root cause** - the streamer's own "burst complete" signal could fire a beat early (a transient Idle status blip between the last line being accepted and the controller actually starting to execute the queued motion), so `WAITIDLE`'s guard walked straight into the real motion resuming a moment later and aborted.
- **Fix** - removed the immediate-fail guard and let `WAITIDLE` fall into its own already-robust, report-driven completion loop (two consecutive real Idle reports, stall/alarm detection), which correctly waits out the trailing motion instead of racing it.

Confirmed via targeted diagnostic tracing + the raw comms log, then verified fixed on real hardware across repeated Start Job and Stepper Calibration (Probe) runs.

#159 Stepper Calibration (Probe): parks at G30, Safe Z delta field, program view auto-hides on success

File	+	−
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml	3	0
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	19	6
ioSender XL/ioSender XL/StartJobView.xaml.cs	1	6
Locale/*/csv (7 locales)	16	0

Two small usability fixes to Stepper Calibration (Probe), plus a general Generate/Run polish item that applies to every tool tab.

- **Parks at G30 when done** - was just lifting to Z0 above the last probed corner, leaving the probe sitting mid-air; now uses the same shared G30-park helper Start Job already relies on (extracted into `MacroProcessor.EmitGotoG30` so both tools share one tested implementation).
- **Safe Z delta field** - the corner-to-corner travel height was hardcoded (15 mm); it's now an editable field matching Start Job's own "Safe Z delta", persisted per session.
- **Program view auto-hides on success** - any tool's Generate'd program preview used to sit in its shrunk 3-line "run view" state after a clean finish; it now fully hides once the run completes with no error, instead of lingering until a 20-second timer or the next run reused it. On an actual error it's still left up so the operator can see what failed.

#160 Start Job: synthetic "G28 (loose probe)" fixture - no saved fixture required

File	+	−
`CNC Controls/CNC Controls/MacroProcessor.cs`	6	2
`ioSender XL/ioSender XL/StartJobView.xaml.cs`	77	18

Start Job's fixture dropdown always has a working option now, even with no fixture library set up: a synthetic **"G28 (loose probe)"** entry, always present (never saved to the fixture library), that recreates the original pre-Fixture-library Start Job behavior.

- Corner 1 probes in a loose DISCOVER pass anchored live at the controller's own G28 stored position, instead of a real Fixture's tight cached corner offset - read at run time (grblHAL's G28 named parameters), never baked into the generated program.
- If G28 isn't set yet, Generate offers to set it on the spot: jog to the position you want to probe the spoilboard from, click OK to store it there (`G28.1`), or Cancel to back out.
- Everything after corner 1 - Measure, rotation, exact size, tool-length reference, origin set - runs exactly like a real Corner Fence fixture, since it only ever consumes corner 1's measured result, not how it was obtained.

User-verified on real hardware, both paths (G28 already set, and G28 unset via the jog-and-confirm dialog).

#161 Cycle Start renamed to Run, with a proper run-mode dropdown

File	+	−
CNC Controls/CNC Controls/JobControl.xaml	90	25
CNC Controls/CNC Controls/JobControl.xaml.cs	179	85
Wording touch-ups, "Cycle Start" → "Run" (13 files: AutoSquareWizard, Converters, FixtureEditDialog, JobView, KeyMapEditor, MainWindow, StepperCalibration{Probe,Scratch}Wizard, SurfaceSpoilboardWizard)	18	18
Locale/*/csv (7 locales x 2 resource files)	119	56

"Cycle Start" never said what it actually does - renamed to **Run** throughout the run bar, its tooltips, and everywhere else the button is mentioned.

- **New mode dropdown** replaces the old right-click context menu: a small arrow to Run's right opens a list of **Run / Dry Run / Check Run** - picking one relabels the button to match, so the current mode is always visible at a glance instead of hidden behind a right-click.
- **Disabled-state tooltip** - hovering Run while it's greyed out now explains why (load a file or generate a program first) instead of showing nothing.
- **Mode-aware tooltip** - when enabled, the tooltip describes exactly what THIS press will do (plain run, Dry Run's Z-offset/spindle-off behavior, or Check Run's $C validation), not just a static "Alt+R".
- **Check Run no longer sends $C until you press Run** - selecting it from the dropdown only arms the intent; previously it activated check mode immediately on selection, which greyed out Home and other Idle-gated controls before the operator had even started anything.
- **Always reverts to Run when a job ends** - normal finish, error, or stop/abort all now leave the button back on plain Run (previously Check mode could linger indefinitely, since grblHAL has no auto-exit for $C - it needed an explicit reset, which now happens automatically).

User-verified on real hardware across all three modes.

#162 Generate button eliminated on 5 tool tabs – folded into the shared Run bar

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	43	0
CNC Controls/CNC Controls/JobControl.xaml.cs	66	12
CNC Controls/CNC Controls/JobControl.xaml	13	5
ioSender XL/ioSender XL/StartJobView.xaml.cs	35	2
ioSender XL/ioSender XL/StartJobView.xaml	0	1
ioSender XL/ioSender XL/MainWindow.xaml.cs	9	0
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	41	3
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml	0	2
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	38	4
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	1	3
CNC Controls/CNC Controls/AutoSquareWizard.xaml.cs	39	1
CNC Controls/CNC Controls/AutoSquareWizard.xaml	0	2
CNC Controls/CNC Controls/SurfaceSpoilboardWizard.xaml.cs	36	8
CNC Controls/CNC Controls/SurfaceSpoilboardWizard.xaml	1	4
Locale/*/csv (7 locales)	21	0

Every tool tab that generated an in-memory probe/test program before running it (Start Job, Stepper Calibration Probe, Stepper Calibration Scratch, Auto Square, Surface Spoilboard) had its own standalone Generate button, separate from the shared Run bar pinned at the main-window bottom. That's gone now – the shared Run bar itself takes over both roles.

- **MacroProcessor** gains four new statics alongside the existing `ActiveRun`/`ActiveProgramName`: `SupportsGenerateMode`, `IsGenerateReady`, `IsProgramGenerated`, `ActiveGenerate`, and `DiscardGenerated` – a Generate-capable tab registers these on `Activate(true)` and clears them on `Activate(false)`, same lifecycle as `ActiveRun`.
- **JobControl**'s Run bar reads "Generate" (disabled until that tab's own live readiness check passes) while focused on one of these tabs and nothing has been built yet; pressing it only builds the program (no run). Once built the button reads plain "Run" and a press streams it, same as any other wizard-driven run. The Dry Run/Check Run mode dropdown is hidden outright for the whole time such a tab is focused – neither applies to these tools.
- After a run that tab owned finishes cleanly (the same isFinalBurst/Idle hook that already auto-hides the program view), **MainWindow** calls the tab's `DiscardGenerated`, dropping the in-memory program and reverting the bar to "Generate" for the next job. Left alone on error/halt so the operator can still inspect or re-run the same generated program after fixing whatever interrupted it.
- Each tab wires its existing readiness checks (fixture selected, travel legs known, current steps/mm valid, no outstanding warning, etc.) into `IsGenerateReady` instead of a local button's `IsEnabled` – no new validation was invented, the same preconditions just now drive the shared bar. A new `isActiveTab` guard on every migrated tab stops a stale event (Model_PropertyChanged stays subscribed after a tab is left) from stomping whichever OTHER Generate-capable tab is now focused, since these statics are global.
- `JobControl.UpdateStartButtonLabel` renamed to **UpdateRunButtonLabel** (there is no "Start" button in this UI, only Run) – naming-only, no behavior change.

Debug build verified clean; all 5 tabs driven end-to-end via `-testserver` (label/enable/dropdown-visibility state on each tab, plus one live Generate-press round trip confirming the button flips Generate to Run with the dropdown staying hidden). The discard-after-run revert-to-Generate path shares the same isFinalBurst hook as the existing program-view auto-hide (verified on real hardware last session) but was not itself re-verified against a live/simulated controller this session.

#163 Status bar: permanent large message font, flash-color notice instead of enlarge/shrink

File	+	−
CNC Core/CNC Core/GrblViewModel.cs	30	2
ioSender XL/ioSender XL/MainWindow.xaml	20	13
ioSender XL/ioSender XL/MainWindow.xaml.cs	31	29

The status line's old "enlarge to 2x for 10s then shrink back" notice mechanism shifted the whole window's layout up and down every time a message arrived - distracting, and easy to miss anyway since it reverted itself. Replaced with a permanently large font (doubled once at startup from whatever the ambient default is) and a background color flash that carries the notice instead, with no layout movement.

- **GrblViewModel** gains `IsMessageError`, set by a new `SetErrorMessage` helper used by the two real error sources - `SetGrblError` and the Alarm-state transition - so the status bar can distinguish an actual error/alarm from a plain status update. Every other `Message` assignment (~24 call sites, untouched) resets the flag to false, so nothing needed to opt in.
- **MainWindow**'s status message is now wrapped in a `Border` (`msgFlashBorder`) whose background flashes light green for a normal message or light red for an error, fading to transparent over 5s via a one-shot `ColorAnimation`.

Debug build verified clean; visually confirmed on real hardware (red flash on a real controller error/alarm, permanent large font, stable layout). Green flash for a normal message not independently re-confirmed after testing was cut short by an unrelated MDI-safety incident during this session (see the testserver-real-hardware-safety note) - low risk, same code path as the error flash with only the color swapped.

#164 Stepper Calibration (Probe): Z axis via a 1-2-3 gauge block

File	+	−
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	457	17
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml	45	19
CNC Controls/CNC Controls/MacroProcessor.cs	24	0
5 Generate-first tabs (tab-leave discard fix)	30	0
Locale/*/csv (6 non-English locales)	84	0

A new **Calibrate: XY steppers / Z stepper (1-2-3 block)** toggle on the Stepper Calibration (Probe) tab adds a Z-axis calibration path alongside the existing XY one, using a standard 1-2-3 gauge block instead of a corner-fence reference square.

- Enter the block's three known dimensions (default 1/2/3 in, mm/in toggle). Generate builds a program that parks at G30, probes the spoilboard once as a shared baseline, then prompts three times to present the block's small/middle/large side up in turn (only one orientation fits at each prompt's retract height) and probes each.
- All three measured-vs-true deltas and errors are shown, then combined into one corrected Z steps/mm via a least-squares fit through the origin (`k = Σ(true·measured) / Σ(true²)`) rather than trusting any single probe - Save/Undo now handle `$102` alongside the existing `$100`/`$101`.
- **Reuse last saved starting position** checkbox skips the manual jog prompt on repeat runs, rapiding to the saved XY at machine top first, prompting to confirm clear, then descending. The position is only persisted once the spoilboard probe actually TRIGGERS (not merely on arrival) and is invalidated - checkbox auto-unchecked and disabled - whenever a steps/mm Save would make the cached position physically stale (a saved mm value is `step count / current steps_per_mm`; changing steps_per_mm changes what that mm value means).

Also fixes two bugs found while testing this feature, both general rather than Z-tool-specific: MacroProcessor's macro loop now aborts immediately if a flushed G-code burst alarmed, instead of continuing on to the next prompt as if nothing had gone wrong (same check `(WAITIDLE)` already had); and all 5 Generate-first tool tabs now discard their generated program on tab-leave, not just after a run finishes, so returning to a tab always starts fresh at "Generate". Debug + Release both build clean; verified end-to-end on real hardware across several iterations (sign-inversion fix, retract-ordering fix, reuse-position staleness/corruption fixes).

#165 Check for Updates now works on local dev builds – pick a release to install over it

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	131	2
CNC Controls/CNC Controls/ReleasePickerDialog.xaml.cs	62	0
CNC Controls/CNC Controls/ReleasePickerDialog.xaml	20	0
install.ps1	28	8

Previously, Check for Updates on a local `build.ps1` build just refused ("no embedded version to compare against a release"). It now queries GitHub for every published release that carries an installable `ioSender.zip` asset, offers a pick-a-release dropdown, and on OK installs the chosen release **over this dev build's own bin folder** (not the normal `%LocalAppData%` install location) — useful for quickly comparing a real published build against in-progress dev work without disturbing a separate install.

- **New `ReleasePickerDialog`** — OK/Cancel dropdown of every release with an installable asset, newest first.
- **`install.ps1` gains `-Tag <version>` and `-InstallDir <path>`** — install a specific release into an arbitrary folder instead of always the latest into the default location; skips the desktop shortcut when a custom install dir is given.

User-verified end-to-end: installed 2.15 from a dev build, then updated 2.15 → 2.21 via the normal Check for Updates path. Builds clean Debug/Release.

#166 Apple-HIG visual redesign: design system + all main tabs

File	+	−
CNC Controls/CNC Controls/AppleStyles.xaml	190	0
CNC Controls/CNC Controls/NumericField.xaml	52	17
ioSender XL/ioSender XL/MainWindow.xaml	58	2
ioSender XL/ioSender XL/App.xaml	5	0
CNC Controls/CNC Controls/JobControl.xaml	5	1
CNC Controls/CNC Controls/MacroManagerDialog.xaml	5	2
CNC Controls/CNC Controls/JogBaseControl.xaml	7	1
CNC Controls/CNC Controls/OffsetView.xaml	3	1
Remaining tab/control XAML (implicit-style adoption, 25 files)	27	23

Replaces the flat, monochrome control styling with a small, deliberate design system (`AppleStyles.xaml`) rolled out across Job, Settings, Probing, Height Map, SD Card, Offsets, Machine Setup and Tools tabs:

- **One primary button per screen** - a single solid-accent CTA (Start/Run/Upload/Set/Apply) per tab; every other button stays neutral gray, including when disabled.
- **Color-by-function panel dots** - GroupBox headers pick up a small colored dot (position/teal, jogging/purple, feed & spindle/ochre, coolant/steel-blue) via a `HeaderTemplate` that wraps the existing localized header text without touching localization.
- **Pill-style main-nav tabs** - the top-level tab strip gets rounded, spring-animated selection with a light-accent fill, scoped to the top-level `StretchTabControl` only so it doesn't leak into nested sub-tab strips (Settings, Probing, Tools).
- **Recessed numeric fields** - `NumericField.xaml` switched to `TemplateBinding` so per-instance background/border overrides (e.g. the merged-unit suffix trick) keep working.

Built via parallel worktree agents, one per tab, then merged; every tab reviewed live by the user before merge.

#167 Machine Setup: optional steps (Fixtures, Simulator) always show green

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	16	5

The Fixtures and Simulator steps in the Machine Setup wizard are optional - there's no gate to fail - but were left with no step color at all, which read as "not checked yet" rather than "passed". They now always color green. Separately, the step-7 (ATC macros) tab color now reads from the same `AtcMacros.GetStatus()` query the gate-check itself just ran, instead of issuing a second, potentially-stale query.

#168 SD Card tab no longer shows "No card" while mount status is still resolving

File	+	−
CNC Controls/CNC Controls/SDCardView.xaml.cs	7	3

The tab treated an `Undetected` mount status as an immediate "No card" - but that status just means nothing's been reported yet, not that there's no card. It now calls the same file-list query the ATC macro status uses first, and only falls back to "No card" once the status explicitly resolves to `Unmounted`.

#169 Silent update check in the title bar + live rebuild-status on Settings > Simulator

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	48	0
CNC Controls/CNC Controls/SimulatorConfigView.xaml.cs	12	0

On launch, a silent background check against the GitHub releases API appends "(update available)" to the window title if a newer release exists (dev builds and any network/API failure are silently ignored - separate from, and doesn't touch, the existing interactive Check for Updates flow).

Settings > Simulator's "up to date" / "click Build to update" status text previously only refreshed on tab-show or after a build, so toggling an option checkbox mid-visit left it showing stale info until you navigated away and back. It's now wired to every option control's change event.

#170 Sidebar flyout panels no longer overlap the main-nav tab strip

File	+	−
`ioSender XL/ioSender XL/MainWindow.xaml`	7	7
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	35	0
`CNC Controls/CNC Controls/SidebarItem.cs`	34	3

The sidebar's flyout panels are positioned two ways that both needed fixing: each flyout's own vertical slot was previously driven by a `static` running counter that accumulated across the whole app session in item-registration order, unrelated to the actual on-screen tab order - replaced with a `TranslatePoint` alignment against each flyout's own tab. The container's top margin (clearing the main-nav tab strip) was a hardcoded pixel guess that had to be re-tuned by hand every time the tab strip's rendered height changed (font, DPI, the Apple-HIG pill redesign's `MinHeight` bump) - two such guesses both proved wrong on live testing. It's now measured from the tab control's real `TabPanel` template part and kept in sync via `SizeChanged`, so it self-corrects if the template ever changes again.

#171 Offsets tab rework: inline-editable grid with change tracking, correct G28/G30/G92 semantics

File	+	−
CNC Controls/CNC Controls/OffsetView.xaml	153	72
CNC Controls/CNC Controls/OffsetView.xaml.cs	197	108
CNC Core/CNC Core/Grbl.cs	83	1

The Offsets tab's old side-panel editor is gone - X/Y/Z are now inline `NumericField` grid cells, with a Clr (zero + commit) and Get MPos (stage current machine position) column per row. A row commits when focus leaves it (or the tab is switched away from); G54-G59.3 save silently, while G28, G30 and G92 confirm first.

- **Correct G28/G30/G92 semantics.** G28.1/G30.1 capture wherever the machine physically is right now - no app-driven rapid move to a typed target any more. G92 has no equivalent "capture current position" primitive - it only knows how to declare "the machine's current position is this work coordinate", so re-declaring the row's own Get-MPos snapshot was silently collapsing the offset toward zero. Row-leave on G92 now always sends the conventional "zero the work origin here" (`G92 X0 Y0 Z0`), which is what actually leaves the controller reporting back the captured position.
- **Change tracking.** Each row carries the same orange (unsaved edit) / blue (saved, different from session startup) convention as the Grbl settings tree, plus a right-click "Restore to value at startup". A write now waits for the controller's "ok" before marking a row committed, and the modified-value comparison tolerance was widened to absorb normal machine settling jitter between a Get MPos snapshot and the value echoed back by the next status query - both were previously showing a just-saved row as still edited for one extra tab round-trip.
- **Sign-constrained X/Y/Z** per this machine's home-corner convention (X ≥ 0, Y/Z ≤ 0), with inline validation.

Verified interactively against real hardware, including the G92/G28/G30 write-and-reread round trip.

#172 UI polish: oval checkboxes, unit-toggle peek, tab-drag/flyout fixes, stale status messages

File	+	−
CNC Controls/CNC Controls/AppleStyles.xaml	41	0
CNC Controls/CNC Controls/NumericField.xaml	17	0
CNC Controls/CNC Controls/NumericField.xaml.cs	55	0
CNC Controls/CNC Controls/UnitToggleMenu.cs	107	0
CNC Controls/CNC Controls/SharedStyles.xaml	11	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	12	8
CNC Controls Probing/CNC Controls Probing/ConfigControl.xaml	11	7
CNC Controls/CNC Controls/StretchTabControl.cs	51	6
CNC Controls/CNC Controls/TabKeyBinder.cs	10	1
ioSender XL/ioSender XL/MainWindow.xaml.cs	24	1
CNC Controls/CNC Controls/JobControl.xaml.cs	20	3
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	6	2
ioSender XL/ioSender XL/StartJobView.xaml	104	75
ioSender XL/ioSender XL/StartJobView.xaml.cs	111	91
Machine Position/Limits/Spindle/Lathe DRO fields (unit-toggle + minor)	21	10

A grab-bag of visual and small-behavior polish that came out of hands-on testing this session.

- **Oval checkboxes app-wide.** The stock square+tick was reported nearly invisible at normal viewing distance - replaced with a small oval indicator (outlined at rest, solid accent blue with a check glyph when checked) via one keyless style in AppleStyles.xaml.
- **NumericField:** read-only fields get muted styling instead of looking editable; a new `ShowUnit` flag hides a redundant unit label (e.g. Limits' Min/Max pair) without opting the field out of unit conversion; and a new right-click "Show in inches/millimeters" gives a one-time tooltip peek at the value in the other unit, without a persistent per-field toggle that could drift out of sync with the app-wide metric/imperial setting.
- **Tab-header drag no longer leaves debris.** MainWindow's pinned sidebar flyout icons (docked as a sibling of the tab strip, outside what StretchTabControl can Clip) now hide for the duration of a drag instead of repainting alongside the clipped tab content; fixed a null-ref when a live reorder's own SelectionChanged re-entrantly cleared the dragged-tab field mid-drag. TabKeyBinder's "Bind to Key" context menu is now scoped to the tab header element, not the whole tab body.
- **Two stale-message fixes:** JobControl's "<name> ready - press Run to run" line no longer appears before Generate has actually been pressed on a Generate-first tab, or lingers on screen after a run has started; FixtureEditDialog's "home first" warning now shows in a visible dialog instead of a status message hidden behind the modal that triggered it.

#173 Generate button loses its green “ready” cue on a fresh tab; Z-gauge reuse descend hardened

File	+	−
CNC Controls/CNC Controls/JobControl.xaml	5	0
CNC Controls/CNC Controls/JobControl.xaml.cs	14	0
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	8	0

Two small fixes found while exercising Stepper Calibration (Probe):

- **Generate button green cue** — when the shared Run bar folded the standalone Generate button into itself (#162), the green “ready to press” highlight was tied to a flag that's deliberately false while the button still reads “Generate” (it also drives the “ready – press Run to run” status text, which must stay quiet until a program actually exists). That correctly kept the status text quiet but also left Generate looking dead on tab entry. A separate flag now drives just the button color, decoupled from that status text.
- **Z-gauge reuse-saved-position hardening** — added a `(WAITIDLE)` gate after the rapid descend to the saved Z in the “reuse last saved starting position” path, before the automated probe search that immediately follows it with no operator click in between to absorb settle time – the same class of burst-trailing-motion race already fixed for the G30 park move.

Builds clean Debug. Generate-button fix user-confirmed visually. The Z-gauge reuse descend issue did not reproduce on retest (with or without this change) - kept as a defensive hardening, not a confirmed root-cause fix.

#174 Tools-tab inputs colon-align; generated .macro filenames no longer collide across a tab's own variants

File	+	−
CNC Controls/CNC Controls/NumericField.xaml	6	2
Tools-tab XAML (Auto Square, Stepper Cal Probe/Scratch, Surface Spoilboard)	4	4
ioSender XL/ioSender XL/StartJobView.xaml	1	1
Tools-tab code-behind (Auto Square, Stepper Cal Probe/Scratch)	7	7
ioSender XL/ioSender XL/StartJobView.xaml.cs	2	2

Two related cleanups to the 5 Tools tabs (Start Job, Stepper Calibration Probe/Scratch, Auto Square, Surface Spoilboard):

- **Colon-aligned inputs** — each row's value box used to stretch from right-after-its-own-label all the way to the tab's right edge, so a short label (“Feed rate:”) sat next to a huge empty-looking box while a long one (“Gauge size 2 (middle):”) sat right against it — and labels of different lengths didn't line up. `Grid.IsSharedSizeScope="True"` on each tab's root now makes every row's label column match the tab's longest label, so labels right-align on a shared colon and each box grows to fill the space after it.
- **Disambiguated generated filenames** — the diagnostic copy every Generate/Run writes to `%AppData%\ioSender\Generated\*.macro` was named only for the tool (e.g. `stepper_calibration__probe_.macro`), so Stepper Cal Probe's XY vs Z modes, Scratch/Auto Square's per-axis runs, and Start Job's per-fixture programs all landed on one shared file – whichever ran last silently overwrote the others. The distinguishing top-level selection (axis, or fixture name) is now folded into the name passed to both the Generate-time save and Run's own internal save. Surface Spoilboard has no such variant and is unchanged.

Builds clean Debug. User-verified the colon alignment visually on Stepper Calibration (Probe) Z mode.

#175 ATC macro gate false-trips “Outdated” when a poll response lands mid-checksum-read

File	+	−
CNC Controls/CNC Controls/AtcMacros.cs	5	1
CNC Controls/CNC Controls/SDCardView.xaml.cs	4	1

Reproduced and fixed a long-standing intermittent bug: the Machine Setup wizard's ATC-macro-provisioning gate would sometimes report a freshly-uploaded macro as “Outdated” and re-trip step 7, even though the controller's filesystem held the exact right bytes.

- **Root cause** — not a race condition as first suspected, but a response-filtering bug: `AtcMacros.ReadControllerFile` and `SDCardView.ReadFileContent` both read a controller file via `$F≤` (dump) while `GrblViewModel.SuspendProcessing` is set, which reroutes every incoming serial line to the caller unfiltered — but the poll thread keeps running and sending `?` the whole time. Neither dump reader excluded realtime status lines (`<Idle|…>`) from the captured text, so a poll reply landing mid-read got appended straight into the checksum, corrupting it against the freshly-computed embedded checksum.
- **Fix** — both readers now exclude lines starting with `<`, same as they already excluded `ok`/`error`/`[`.

User-verified: launching with the controller already in an unhomed Alarm state — the condition that most reliably surfaced the race — no longer trips the gate.

#176 Job tab Run dropdown gains a fourth mode: Simulate

File	+	−
`CNC Controls/CNC Controls/JobControl.xaml.cs`	63	0
`ioSender XL/ioSender XL/MainWindow.xaml.cs (Simulate switch/restore)`	63	0
`CNC Controls/CNC Controls/AppConfig.cs`	29	0
`CNC Controls/CNC Controls/SimulatorManager.cs`	11	0
`CNC Controls/CNC Controls/JobControl.xaml`	4	0

Alongside Run / Dry Run / Check Run, the Job tab's Run dropdown now offers **Simulate**: picking it and pressing Run disconnects from whatever's currently live, connects to the bundled `%AppData%\Simulator` simulator, streams the loaded job there, and automatically reconnects to the original target once the job finishes, errors, or is aborted.

- **AppConfig.ConnectToSimulator** — connects straight to the bundled simulator without showing the Connect dialog and without persisting it as the new saved startup target (session-only, unlike the existing Connect/ConnectTo paths).
- **MainWindow.SwitchToSimulatorForRun / RestoreConnectionAfterSimulate** — remembers the pre-Simulate target and restores it; a failed restore is left as the normal “Not connected” state rather than a popup.
- **Cross-project hook** — `CNC Controls` (where the Run button lives) can't reference `MainWindow` directly, so the switch/restore functions are wired through two new `SimulatorManager` static hooks (`SwitchToSimulatorForRun/RestoreConnectionAfterSimulate`), the same pattern already used for `AppConfig.DeviceEnumerator`.
- Precondition: only works once a simulator has been built via Settings > Simulator — Run just shows a message and leaves the connection untouched otherwise.

User-verified end to end against a real controller.

#177 Program-list hover explanations were matching the wrong line; Machine/Program limits compacted to one line per axis

File	+	−
CNC Core/CNC Core/GCodeJob.cs	15	2
CNC Core/CNC Core/GCodeExplainer.cs	18	5
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	43	44
CNC Controls/CNC Controls/GCodeListControl.xaml	1	1
CNC Controls/CNC Controls/LimitsBaseControl.xaml.cs	62	3
CNC Controls/CNC Controls/LimitsBaseControl.xaml	16	13
CNC Controls/CNC Controls/LimitsControl.xaml	5	3
CNC Controls/CNC Controls/OffsetView.xaml	18	12
CNC GCodeViewer/CNC GCodeViewer/CarveView.xaml.cs	8	3
CNC Controls/CNC Controls/ActionKeyBinder.cs	7	0
ioSender XL/ioSender XL/MainWindow.xaml.cs (debug screenshot hotkey)	84	0
ioSender XL/ioSender XL/JobView.xaml	13	19
Locale/*/csv (7 locales)	58	0

A grab-bag of program-view and layout fixes from the same session:

- **Hover-explain was matching the wrong line** — the popup matched a hovered block's tokens via `GCodeToken.LineNumber == block.LineNum`, but those are two unrelated numbering schemes: `LineNum` is ioSender's own sequential count, while a token's `LineNumber` comes from the file's own (author-chosen, often sparse) `N`-word and just carries over from the last line that had one. The two spaces can coincidentally collide anywhere in a file that mixes explicit and unnumbered lines. `GCodeBlock` now carries its own `Tokens` list, sliced directly from the exact range the parser generated for that line — no line-number matching left at all.
- **Explanation bullets now follow source word order** — the parser emits tokens in a fixed per-modal-group order, not text order, so "G90G94" was explaining G94 before G90; bullets are now sorted by where each token's own word is actually found in the line.
- **Program-list Data column** no longer starts collapsed on file load (a WPF `DataGrid` quirk where a `Width="*"` column can under-compute on a layout pass with few realized rows) and no longer has an artificial `MinWidth` floor cutting off long lines.
- **Machine/Program limits** (`LimitsBaseControl`) — one line per axis, e.g. `X: [0 .. 860) mm`, instead of two full-precision fields; trailing zeros dropped; bracket convention is closed at the home/zero end (always exactly reachable), open at the far/travel end (a soft-limit boundary). Also fixed a pre-existing bug where `MaxValue`'s getter read `MinValueProperty` by mistake. `UnitToggleMenu` extended to recognize the new no-`NumericField` control directly.
- **Offsets tab** gains a draggable splitter between the grid and the usage-notes panel.
- **Carve (3D) view** — front/back material contrast increased; the near-flat single-tone stock made pocket walls hard to read.
- **Debug-only screenshot hotkey** (Keyboard & Controller > UI zoom group, Debug builds only) — renders the main window to a PNG and prompts to save it under `~/Downloads/ioSenderV2/Screenshots`, filename defaulted from the active tab.
- An experimental Job-tab splitter (jog pad vs. the right-hand panel column) was tried and reverted back to the simpler Auto-sized layout after it fought the panels' own content sizing.

#178 ioSenderV2 Fusion Addin: Feeds & Speeds analysis + Batch Post Process, plus a one-click installer

File	+	−
Fusion Addin/ioSenderV2/ioSenderV2.py	113	0
Fusion Addin/ioSenderV2/batchPostProcess.py	164	96
Fusion Addin/ioSenderV2/feedsAndSpeeds.py	820	0
Fusion Addin/ioSenderV2/ioSenderV2.manifest	11	0
Fusion Addin/README.md	62	22
Fusion Addin/install-windows.ps1	35	12
Fusion Addin/install-macos.sh	11	7
CNC Core/CNC Core/FeedsSpeedsAdvisor.cs	543	0
CNC Core/CNC Core/FeedsSpeedsAiReview.cs	298	0
CNC Core/CNC Core/FeedsSpeedsExport.cs	105	0
CNC Controls/CNC Controls/FeedsAndSpeedsView.xaml	105	0
CNC Controls/CNC Controls/FeedsAndSpeedsView.xaml.cs	611	0
CNC Controls/CNC Controls/AppConfig.cs	30	0
CNC Controls/CNC Controls/LayoutModel.cs	3	2
CNC Controls/CNC Controls/ICNCView.cs	2	1
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	39	1
ioSender XL/ioSender XL/MainWindow.xaml.cs	224	0
ioSender XL/ioSender XL/MainWindow.xaml	1	0
Locale/*/csv/CNC.Controls.WPF.resources.*.csv (7 locales)	84	0
Locale/*/csv/ioSender.resources.*.csv (7 locales)	14	0

The Fusion 360 add-in (renamed `ioSenderBatchPost` → `ioSenderV2`) now puts a single `ioSenderV2` panel directly on the Manufacture workspace toolbar with two commands, instead of one button buried in the Actions panel:

- **Batch Post Process** — the existing post-every-operation-and-combine flow, unchanged, plus a new option to split output by material (grouping setups by a name-prefix convention, e.g. `MDF_TopSetup`) into one `.nc` per material instead of one combined file.
- **Feeds and Speeds** — exports every Setup/Operation's current feeds, speeds, tool and geometry data to `~/Downloads/ioSenderV2/<doc>.json` as soon as it opens, and applies ioSenderV2's recommended adjustments from a companion `-apply.json` when you press OK. Cleans up both files on open (a stale apply file from an abandoned cycle must never silently apply) and on close (nothing lingers).

On the ioSenderV2 side, a new **Feeds & Speeds** tab (Intro / Load-Import / Results sub-tabs, each shown only when there's actually something to do there):

- A ported material chip-load/surface-speed reference table (Hardwood, Softwood, Plywood, MDF, Aluminum, Brass, Steel) cross-checked against the *connected controller's actual* grblHAL limits (`$30` max RPM, `$110`–`$112` max feed rates per axis) — a check the Fusion side could never do on its own. Each Setup's material can be derived automatically from its name, matching the same convention Batch Post Process now uses.
- An optional second-opinion AI review pass (model picker: Sonnet 5/Opus 4.8/Fable 5/Haiku 4.5), only offered when an `ANTHROPIC_API_KEY` is configured. Structured per-parameter responses populate an "AI Says" column and a "Prefer AI" override checkbox — never applied automatically. Cancellable mid-run

(Esc/Ctrl+C), resilient to one operation's response failing, with a running transcript and live token/cost totals.

- A "Write apply file" action that archives the export + apply JSON pair into the app's logs folder, since Fusion deletes its own copies once the cycle closes.

Also new: **Help > Support > Install ioSenderV2 Fusion Addin...** symlinks Fusion's AddIns folder straight to the copy shipped alongside `ioSender.exe`, so a future ioSenderV2 update refreshes the add-in in place — no reinstall, just a Stop/Run in Fusion's Scripts and Add-Ins (which ioSenderV2 now detects and reminds you to do, if the add-in's code changed since Fusion last loaded it).

This was a full end-to-end build in a single working session, not just a features list: an interactive design pass settled the naming (`ioSenderV2 Addin`, a top-level `ioSenderV2` panel with two direct commands - not the nested dropdown the first draft produced), the decision-engine architecture (table-driven by default, an LLM pass as an optional second opinion rather than a replacement for the arithmetic), and an explicit acknowledgment that this app now has real users beyond its author - so a persisted `App.config`/layout represents someone else's data, and needs migrating forward, not silently dropped. A string of real bugs surfaced only through live testing against the actual Fusion add-in and a real CAM document, not from a clean compile:

- Fusion keeps already-imported Python modules in memory across a plain Stop/Run of the add-in - edits to `batchPostProcess.py`/`feedsAndSpeeds.py` silently kept running stale code until an explicit `importlib.reload()` was added, matching the pattern SRWCommands' own entry point already used for this exact reason.
- The apply-file writer serialized JSON with C#'s default PascalCase property names, while Fusion's reader expects lowercase `ops`/`id`/`set`, case-sensitively - silently produced "apply file listed no ops" despite a fully populated file, until `JsonNamingPolicy.CamelCase` was set explicitly on the writer.
- An early toolbar layout nested a dropdown labeled "ioSenderV2" inside a panel ALSO labeled "ioSenderV2" - a narrow ribbon collapsed both into their own same-named flyouts, one extra level of clicking to reach either command. Fixed by putting both commands directly on the panel, no nested dropdown.
- Adding a brand-new top-level tab to the main window needed TWO separate one-time `AppConfig` fixups, not one: the newer layout-tree structure AND the legacy flat `Config.Tabs` list, since a later reconciliation step (`TabOrder.Apply`) resyncs the tree to exactly what's in the flat list and silently drops anything that only exists in the tree - confirmed missing from both the tab bar and the "Edit Main Page" editor before this was caught.
- A model returning bare JSON `null` for one operation (instead of the requested object shape) crashed the whole review batch; the parser and per-operation loop are now resilient to any single response failing - note-and-continue rather than aborting the rest of the run.
- An AI-recommended value that matched CURRENT (not the table's own recommendation) displayed as a bare number indistinguishable from a genuine proposed change - it now reads "keep as-is, disagrees w/ table" so a considered "no change needed" opinion can't be misread as a fresh suggestion.

Built and verified against local Debug builds across the session; Fusion-side behavior verified interactively by the user against a real CAM document, including Text Commands log output, apply-file round-trips, and a live AI review run.

#179 Top-level tab reorder + Settings tab splitters and Simulator redesign

File	+	−
CNC Controls/CNC Controls/LayoutModel.cs	6	6
ioSender XL/ioSender XL/MainWindow.xaml.cs	8	8
CNC Controls/CNC Controls/GrblConfigView.xaml	30	3
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	16	12
CNC Controls/CNC Controls/SimulatorConfigView.xaml	26	5
CNC Controls/CNC Controls/SimulatorConfigView.xaml.cs	4	4
CNC Controls/CNC Controls/AppConfig.cs	1	1
ioSender XL/ioSender XL/Default-App.config	1	1

Top-level tab order changed to match how a session actually flows: **Settings, Feeds & Speeds, Start Job, Job, Offsets, SD Card, Probing, Tools, Machine Setup, Height Map, Lathe Tools** — both `LayoutModel.DefaultLayout.Build()` (the real runtime order) and `MainWindow`'s `TabRegistry.Register` calls (kept in sync for consistency, though not itself load-bearing).

- **Settings → App / Jogging / G Code** were a plain `WrapPanel` that stretched its second column to fill all remaining width once a splitter was added — now a 3-column `Grid` (left panel, a real `GridSplitter`, right panel) with the right column's `ColumnDefinition` changed from `Width="*"` to `Width="Auto"` so it hugs its own content instead of stretching full-width.
- **Settings → Simulator** redesigned: removed a fixed `MaxWidth` that was clipping content on wide/high-DPI displays, and replaced several small status lines with one combined sentence ("Up to date with the following build options: ..." / "Will build with the following options...") at a larger, more readable font size; the built simulator's executable path moved into a read-only text box next to the Open Folder button instead of sitting on its own tiny line.
- **Prefer network if available** now defaults ON for new installs (separate small change, bundled here since it's the same Settings > App area).

Verified via Debug build + launch; UndoLimit/searchbox crash and env-var work from the same session are tracked separately.

#180 3D View: carve trail was never actually visible + lighter stock color

File	+	−
`CNC GCodeViewer/CNC GCodeViewer/CarveView.xaml.cs`	13	6

`Play_Click` unconditionally called `SetToolpathVisible(false)` the instant playback started. Trail points sit at the actual (below-surface) cut Z, so before pressing Play they were buried under the flat, uncut stock top and invisible; the moment Play ran, they were removed from the scene entirely by that call — the trail was never both present in the scene AND unoccluded, in any state. Removed the hide call so the trail actually draws as the cut progresses.

- Stock top-face color lightened from `(230,193,138)` to `(237,205,176)` for better contrast against the darker cut trail (a first attempt at `(214,133,100)` was rejected on review — it was actually more saturated/darker than the original, not lighter).
- Cut-trail line color changed from `(20,90,210)` to a brighter `(0,174,239)`.

Verified visually against a sample stock file with the color/visibility change confirmed on screen.

#181 Firmware update UI redesign + real, regex-parsed release changelog

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml	9	8
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	65	56
CNC Controls/CNC Controls/FirmwareUpdateManager.cs	36	11
CNC Controls/CNC Controls/AppMessageBox.xaml.cs	25	11
CNC Core/CNC Core/AppDialogs.cs	13	7
Locale/en-US/csv (1 file)	1	1

Machine Setup's firmware panel was a cluttered mix of a version line, an update-status line and an Update Firmware button that didn't clearly say what pressing it would do. Trimmed to **Firmware version** + **Build** info and one **Check for Updates** button; the result (up to date / update available, with real changelog text) now shows in a message box with custom-labelled **Flash Firmware** / **Cancel** buttons instead of a generic Yes/No.

- **Real changelog, not hardcoded line indexing** — the previous approach sliced the release-notes text by fixed line offsets; rewritten with two `Regex` patterns (`DriverRefPattern`, `ChangelogPattern`) that find the `drv:<ref>@<sha>` marker and capture everything after it as the actual changelog, robust to the notes format shifting.
- **Custom message-box buttons** — `AppDialogs.Show/AppMessageBox` gained optional `yesText/noText` parameters. Buttons now carry their logical `MessageBoxResult` on `Tag` instead of being parsed back out of their (now customizable) `Content` text, which the old `Enum.Parse(Content)` approach would have broken.

Verified via Debug build + launch against Machine Setup's Machine tab.

#182 Remove env-var reliance: OBS settings, AI review key and GitHub token move into Settings/registry

File	+	−
CNC Controls Camera/CNC Controls Camera/ConfigControl.xaml	84	1
CNC Controls Camera/CNC Controls Camera/ConfigControl.xaml.cs	113	1
CNC Controls/CNC Controls/AppConfig.cs	21	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	9	0
CNC Controls/CNC Controls/BasicConfigControl.xaml.cs	35	0
CNC Controls/CNC Controls/SecretPromptDialog.xaml	20	0
CNC Controls/CNC Controls/SecretPromptDialog.xaml.cs	69	0
CNC Core/CNC Core/SecretStore.cs	62	0
CNC Core/CNC Core/FeedsSpeedsAiReview.cs	20	12
CNC Controls/CNC Controls/SimulatorManager.cs	10	14
ioSender XL/ioSender XL/App.xaml.cs	38	44
build.ps1	17	8
tools/run-iosender.ps1	5	6
tools/cut-release.ps1	10	1

ioSender itself now reads **zero** environment variables - every launch-time behavior is a real CLI flag, and every persistent setting lives in Settings or the registry instead of hidden env state that made `build.ps1 -Launch` hide what was actually about to run.

- **OBS demo-recording config** (WebSocket host/port/password, camera + app-capture source/filter names) moves from `IOSENDER_OBS*/IOSENDER_OBSWS_*` env vars into a real **Settings > App > Camera > "Demo recording (OBS)"** panel, backed by new `AppConfig.CameraConfig` fields. `ObsBridge.Cameras` changed from env-var-populated-at-class-load to a `ConfigureCameras( … )` call the app makes at startup once settings are loaded.
- **AI review key** - a new registry-backed `SecretStore` (`HKCU\Software\ioSenderV2\Secrets`) plus a minimal `SecretPromptDialog` replace the `ANTHROPIC_API_KEY` env var. Settings > App gets a "Set AI key" button that relabels itself "Update AI key" once one is stored.
- **GitHub token** for building the matched simulator is now a compiled-in, fine-grained PAT (`Actions: write` only, scoped to the `ioSenderV2/Simulator` repo only) instead of requiring a user-supplied `GH_TOKEN` - worst case of embedding it is CI spam on one low-value repo, not real compromise, and it means building the simulator works out of the box with no setup step.
- `-self-relaunch` and `-headless` become real CLI flags (were `IOSENDER_SELF_RELAUNCH/IOSENDER_HEADLESS` env vars); `IOSENDER_DEMOMARKER/_DEBUGLOG/_TESTSERVER` env fallbacks removed from `App.xaml.cs` entirely - CLI-only now.
- `build.ps1/run-iosender.ps1` no longer translate legacy env vars to CLI flags on your behalf - the exact command line printed is always the exact command line that ran, nothing hidden.

Built and launched via Debug across the session; the OBS panel and AI key button were interactively exercised, not just compiled.

#183 OBS "Validate" button - check WS connection and source names before a shoot

File	+	−
`CNC Core/CNC Core/ObsBridge.cs`	91	0
`CNC Controls Camera/CNC Controls Camera/ConfigControl.xaml.cs`	44	0

A typo in an OBS source name previously stayed silent until the middle of an actual recording, when nothing showed up. **Validate** connects to OBS right now with whatever's currently typed (host/port/password, not necessarily saved yet), fetches OBS's real source list, and reports which of the three configured names (Front Left / Front Right / App-screen) actually exist.

- Uses its own short-lived WebSocket connection (`ObsBridge.ValidateConnection`) - separate from the shared demo-recording connection, so validating never disturbs an already-armed recording session.
- Checks both **inputs** (`GetInputList`) and **scenes** (`GetSceneList`): the "App (screen)" capture is commonly a whole OBS Scene rather than a plain input, and checking inputs alone always reported a scene-based capture as "NOT FOUND" even when correctly named.
- Name matching is case/whitespace-insensitive, since the field is free text typed by hand to match whatever the source happens to be named in OBS - a harmless casing difference shouldn't read as "this source doesn't exist".

Verified interactively against a real OBS instance: confirmed a genuine name mismatch correctly reported NOT FOUND, and reported found once the name was corrected in OBS.

#184 AI review Cancel actually cancels - and says so immediately

File	+	−
`CNC Controls/CNC Controls/FeedsAndSpeedsView.xaml.cs`	16	5
`CNC Core/CNC Core/FeedsSpeedsAiReview.cs`	34	25

Pressing Esc/Ctrl+C during an AI review only stopped the loop from starting the *next* operation - an already-in-flight request (a real ~20-30s Anthropic API call) ran to completion regardless, with no visible sign the keypress had registered until it eventually finished. Two real gaps, both fixed:

- **The cancellation token never reached the network call.** `Task.Run(() ⇒ Review(...), token)` only honors its token before the delegate starts - once `Review`'s synchronous `HttpWebRequest` was already executing, cancelling had no effect on it. `Review` now accepts a `CancellationToken` and registers `req.Abort()` against it, so cancelling now genuinely interrupts a live request instead of waiting it out.
- **No feedback that the keypress landed.** The status transcript now logs "Cancelling..." the instant Esc/Ctrl+C is pressed, instead of staying silent until the (now much faster) cancellation actually completes.

Verified interactively: pressing Esc mid-review now shows "Cancelling..." immediately, followed promptly by the cancelled summary, instead of a long silent wait.

#185 Small fixes: exit-blocked explanation, Grbl search-box crash, T1M6 tooltip word order

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	14	1
CNC Controls/CNC Controls/GrblConfigControl.xaml	8	0
CNC Core/CNC Core/GCodeExplainer.cs	10	1

- **Exit blocked with no explanation** — `Window_Closing` silently refused to close the window while a job was running or paused (e.g. in Hold), leaving no indication why the X button appeared to do nothing. Now shows a message box explaining that a job is running/paused and must be stopped or finished first.
- **Grbl settings search-box crash** — typing fast enough in Settings > Grbl's search box could hit a real WPF framework bug, `InvalidOperationException: Cannot Reopen undo unit while another unit is already open`, a reentrancy issue in WPF's own undo stack when a synchronous `TextChanged` handler (re-filtering the settings tree on every keystroke) hadn't finished before the next keystroke arrived. Undo/redo was never meaningful for a search box anyway, so `UndoLimit="0"` sidesteps the buggy code path entirely rather than trying to debounce typing.
- **Tool-select hover tooltip in the wrong order** — hovering a line like `T1M6` explained M6 ("Tool change") before T1 ("Select tool 1"), the reverse of how the words actually appear. Every `GCodeToken` subclass declares its own `ToString()` with `new`, not `override`, so a lookup written against the base-typed `GCodeToken` reference always resolved to the base class's own `ToString()` - harmless for token types whose `Command` enum name happens to match its raw g-code word, but `GCToolSelect`'s `Command` is the generic marker `Commands.ToolSelect` (renders as "ToolSelect", never found in the raw line), so a tool-select word always sorted last regardless of where "T1" actually sits. Fixed via dynamic dispatch to the token's real, most-derived `ToString()`.

All three verified interactively via Debug build + launch: the exit dialog now shows its message during a Hold; the search box no longer crashes on fast typing; T1M6's tooltip now reads tool-select before tool-change.

#186 Manual: "What you're looking at" breakdown for every screenshot + reshoot tooling

File	+	−
docs/manual/index.html	119	3
tools/copy-latest-screenshot.ps1	39	9
tools/gen-image-review.ps1	56	0
docs/manual/_image-review.html	30	0
docs/manual/img/*.png (4 reshot)	0	0

Every one of the manual's 14 real app screenshots (connect dialog, Job screen, Start Job, Machine Setup, Probing, Tools, Offsets, Grbl settings, Feeds & Speeds, SD Card, 3D viewer, Height Map, Lathe wizard, Errors/alarms) now has a "What you're looking at" bullet list right under its caption, walking through every labeled control/region in plain language for a first-time reader instead of leaving the picture to speak for itself.

- Reshot four screenshots against the current 2.28 UI: Job screen (now also demonstrates the corrected T1M6 tooltip word order, #185), Grbl settings, Tools tab, and Feeds & Speeds Results.
- `tools/copy-latest-screenshot.ps1` gained `-Rank`/`-BatchSize`: shooting several screenshots in a row and asking for "the latest" after each one breaks once more than one new capture has landed since the last copy - `-Rank` indexes into the recent batch by age (1 = oldest of the batch, 2 = next, etc.) instead.
- New `tools/gen-image-review.ps1` + tracked `docs/manual/_image-review.html`: a quick local dark-themed gallery of every file in `docs/manual/img` with its filename and last-modified time, for eyeballing a whole reshoot batch at a glance instead of opening the full manual page by page.

Two screenshots (Lathe wizard, Errors/alarms) still show an older app build (2.0.1) and are flagged for a future reshoot; the Feeds & Speeds shot happens to show an AI review mid-cancel rather than a completed run, described honestly as such rather than staged.

#187 Firmware Check for Updates could perpetually offer a build it had already flashed

File	+	−
CNC Controls/CNC Controls/FirmwareUpdateManager.cs	9	6

Flashing the firmware update Check for Updates offered would still show the same update available afterward - not because flashing failed, but because of how the firmware CI published its release. The build-side `fw-latest` tag was a single ROLLING release, overwritten in place on every push, which turned out to have two real bugs: two close-together pushes could interleave their publish steps (the release body ending up describing a newer commit than the `.hex` asset actually attached), and re-uploading uniquely-named assets to the same release just accumulated old `.hex` files forever instead of replacing them - `FindHexAssetUrl` grabs the first `.hex` found, so a stale leftover asset could get downloaded and flashed instead of the current one.

- **Firmware CI** (`stevenrwood/iMXRT1062`, not in this repo) now publishes a real, immutable release per build (tag `fw-<short-sha>`, `prerelease:false`) instead of overwriting a shared tag - nothing is ever mutated in place, so neither bug class can happen again.
- **ioSender**'s `FirmwareUpdateManager.GetLatestRelease` now queries GitHub's real `/releases/latest` resolution instead of a fixed `/releases/tags/fw-latest` - the exact same call shape `MainWindow.checkForUpdates_Click` already uses for the app's own updates.

Verified end-to-end: flashed the newly-published build, confirmed GitHub's /releases/latest resolves to it with exactly one matching asset, and confirmed a local build with this fix correctly reports up to date afterward.

#188 Configurable probe search distance on all 4 Probing tabs

File	+	−
`CNC Controls Probing/CNC Controls Probing/CenterFinderControl.xaml`	1	0
`CNC Controls Probing/CNC Controls Probing/EdgeFinderControl.xaml`	1	0
`CNC Controls Probing/CNC Controls Probing/EdgeFinderIntControl.xaml`	1	0
`CNC Controls Probing/CNC Controls Probing/ToolLengthControl.xaml`	1	0
`Locale/*/csv/CNC.Controls.Probing.resources.*.csv (7 locales)`	84	0

Reported (GitHub #10): probing a Tool Length Offset gave no way to shorten the probe's search distance, risking overtravel damage on a broken/ungrounded probe with no warning.
`ProbingViewModel.ProbeDistance` was already wired end-to-end - validated on input and used to build the `G38.2` probe move - but was only settable via Machine Setup > Probe definitions > Edit > Motion parameters, with no control on the tab you're actually about to probe from.

- Added a "Max search distance" field (mm) directly to all 4 Probing tabs, reusing the exact wording from the existing `ProbeMotionParamsDialog` for consistency.
- `CenterFinderControl.xaml` was missing entirely from `tools/locadd.py`'s localization TARGETS list (a pre-existing gap) - added it per the project's x:Uid convention.

User-verified interactively on all 4 tabs.

#189 Crash fixes: null keymap load, cross-thread UI crash; console visibility restored at connect

File	+	−
CNC Core/CNC Core/KeypressHandler.cs	3	0
CNC Core/CNC Core/HelperClasses.cs	13	1
ioSender XL/ioSender XL/MainWindow.xaml.cs	10	0
CNC Core/CNC Core/Grbl.cs	11	4
CNC Core/CNC Core/GrblViewModel.cs	23	1
CNC Controls/CNC Controls/SDCardView.xaml.cs	7	1

- **Null keymap crash on startup** (GitHub #12) — a saved `KeyMap` config entry (App.config or legacy `KeyMap0.xml`) missing its `Method` element deserializes with a null `method` field; `KeypressHandler.ImportMappings` called `.StartsWith()` on it with no null check, crashing every startup for an install with a stale/hand-edited keymap. Guarded.
- **Cross-thread UI crash** (GitHub #7) — `ProbingViewModel.WaitForResponse` (and similar helpers, plus `StreamPump.Run`'s job-streaming thread) call into `GrblViewModel` directly from a worker thread, bypassing the normal `Dispatcher`-marshaled response path. A manually-wired UI subscriber touching a `DependencyObject` in response then throws `"the calling thread cannot access this object"` - originally reported against a since-renamed method (`EmphasizeMessage` → `FlashMessage`), same live bug either way. Fixed at the single choke point every property-change notification funnels through: `ViewModelBase.OnPropertyChanged` now marshals onto the UI thread when raised off it, a no-op for the normal case already on that thread.
- **Console showed nothing from the controller** (GitHub #9) — four related console-visibility gaps: a typed `$$`/`$?`/`$pins` command's reply was silently dropped unless "Verbose" was checked, even though typing it is an explicit request to see it; the controller's reset/boot banner could be swallowed by an internal "quiet" handshake window; the `$I` "who am I" response at connect was deliberately hidden as boilerplate in an earlier change, removing the only visible confirmation a connection came up correctly; and, in the opposite direction, internal `$CWD` directory-navigation traffic (checking/provisioning ATC macros against littlefs) was leaking into the console as noise whenever Verbose was on. All four addressed without reintroducing the others.

All three fixes user-verified interactively: keymap crash confirmed closed via code inspection (exact stack-trace line unchanged since v2.0.1); cross-thread fix verified via the original Probing-tab repro plus a normal job run; console fixes verified via $?/$$/$pins/$1 replies and $CWD noise gone during an ATC macro check.

#190 Phil Barrett's UI feedback pass

File	+	−
CNC Controls/MacroPinnedListControl.xaml	82	0
CNC Controls/MacroPinnedListControl.xaml.cs	141	0
CNC Controls/MacroCreateDialog.xaml	37	0
CNC Controls/MacroCreateDialog.xaml.cs	148	0
CNC Controls/MacroManagerDialog.xaml.cs	81	76
CNC Controls/MacroExecuteControl.xaml	2	2
CNC Controls/MacroProcessor.cs	4	0
CNC Controls/MainPanelRegistry.cs	8	2
CNC Controls/AppleStyles.xaml	12	0
CNC Controls/AppConfig.cs	4	6
CNC Controls/FeedsAndSpeedsView.xaml	10	9
CNC Controls/MachineSetupWizard.xaml	6	11
CNC Controls/JobControl.xaml	3	2
CNC Controls/OffsetView.xaml	3	3
CNC Controls/MDIControl.xaml	1	1
CNC Controls/StatusControl.xaml	3	1
CNC Controls/GrblConfigView.xaml.cs	1	0
CNC Core/GCode.cs	5	2
ioSender XL/MainWindow.xaml + .xaml.cs	33	12
ioSender XL/ProgramPanel.xaml + .xaml.cs	8	17
Locale/en-US/csv (2 files)	8	8

Phil Barrett — a long-time ioSender user, and the first person outside the project to run V2 in anger — sent a long and generous set of UI observations after his first sessions with it. This entry works through that list. His framing was that most CNC applications gain comfort from the even grid of a physical operator's panel, and that V2 was losing some of that to details.

- **Uniform button sizes.** His lead point: text-driven sizing made clusters look ragged — the Job tab's *Run* button changed width as its label cycled through Run / Dry Run / Check Run / Simulate, and sat beside a wider *Feed Hold*. Fixed widths now apply across the Job run bar, MDI *Send*, the Offsets *Clr*/*Get* pair and Feeds & Speeds.
- **Macros out of the flyout.** They were reachable only from the flyout sidebar, which he found hard to get to for something like a Go Home macro used many times a session. Macros now have a pinned list in the main panel and a proper *Create/Edit* dialog — name, prompt, add-to-menu, and an explicit F-key assignment in place of the automatic key allocation he rated bogus — plus an `@<path>` reference so a macro can live in its own `.macro` file instead of inline.
- **Tooltips on disabled controls.** "I prefer to see what even the greyed buttons do" — and a disabled control is precisely when you want to know why. Tooltips now appear regardless of enabled state, app-wide.
- **Loading a program was obscure.** It meant navigating to the Job panel's Program sub-panel first. *Load File* is now a main-menu item, alongside Connect.
- **Machine Setup's stray text bar.** He noticed only that tab carried a band of text between the tab rows. It has moved into the Overview tab, where it belongs.

- **The square Program button.** “I’ve never seen it active” — it toggled the generated g-code view and earned its keep rarely. Removed.

Also from his list, and fixed: the Reset button’s square corners, which sat oddly beside a rounded Home and Unlock — unintentional rather than deliberate, and now rounded to match. Still outstanding: acknowledging V1 and Terje Io’s work in the docs, and filling the wider tooltip gaps (an audit found 500+ controls without one). The bottom-mounted Program / 3D View / Console tabs were considered and deliberately left alone.

#191 Fix: dragging a tab could crash ioSender or leave the window unusable

File	+	−
CNC Controls/StretchTabControl.cs	26	4
ioSender XL/MainWindow.xaml.cs	11	0
CNC Controls/SDCardView.xaml.cs	7	0

Tabs can be dragged to reorder them. Doing so could either crash ioSender outright or leave the window looking frozen — page contents gone, the cursor stuck as a horizontal arrow, and no apparent response to input until the app was closed and restarted. Reported as issue #17, present since the drag-to-reorder feature was added.

Both faults came from the same thing: the reorder rearranged the live tab list while the drag was still in progress.

- **The crash.** Moving a tab in the list makes the app select a different one *immediately*, while the dragged tab is momentarily detached — and a detached tab has no data behind it. Whichever page was activated at that instant then read data that was not there. The page shown is now left alone until the drag finishes, at which point the settled selection activates normally.
- **The stuck window.** The tab strip never claimed the mouse for the duration of the drag, so releasing the button anywhere except back over the tab strip — over the page, another panel, or outside the window — meant the drag never formally ended. The two things the drag switches on, an application-wide cursor override and the clipping that hides the page while tabs shuffle, were left switched on permanently. The app was never actually hung; it was clipped down to its tab strip. The drag now claims the mouse, and both effects are cleared unconditionally when it ends.

Ordinary clicking to change tabs is unaffected: the mouse is only claimed once a drag has genuinely started.

#192 Odd Jobs — a Work Order builder for jobs that don't deserve a CAM round-trip

File	+	−
CNC Controls/WorkOrderView.xaml.cs	1336	0
CNC Controls/WorkOrderCompiler.cs	805	0
CNC Controls/WorkOrderModel.cs	466	0
CNC Controls/WorkOrderView.xaml	208	0
CNC Controls/OddJobsFeedsSpeedsDialog.xaml + .xaml.cs	391	0
CNC Controls/OddJobsView.xaml + .xaml.cs	166	0
CNC Controls/OddJobsGeometry.cs	153	0
CNC Controls/OddJobsStockCanvas.cs	139	0
CNC Controls/OddJobsToolMemory.cs	82	0
ioSender XL/StartJobView.xaml + .xaml.cs	243	23
CNC Controls/ProgramView.xaml + .xaml.cs	122	1
CNC Controls/FixtureEditDialog.xaml.cs	103	2
CNC Controls/JobControl.xaml + .xaml.cs	72	23
macros/tc.macro	56	3
CNC Controls/StartJobConfig.cs	53	0
CNC Controls/AppConfig.cs	42	0
macros/pcorner.macro	41	12
CNC Controls/AppleStyles.xaml	40	0
CNC Controls/NumericField.xaml + .xaml.cs	39	7
CNC Controls/MacroProcessor.cs	34	5
CNC Controls/TabKeyBinder.cs	34	2
ioSender XL/MainWindow.xaml.cs	24	4
CNC Controls/StretchTabControl.cs	20	0
CNC Core/GCodeJob.cs	12	3
CNC Controls/SignalControl.xaml.cs	12	2
CNC Controls/KeyMapEditor.xaml.cs	10	0
CNC Core/GrblViewModel.cs + Grbl.cs	12	0
CNC Controls/LayoutModel.cs	8	1
CNC Controls/StepperCalibrationProbeWizard.xaml.cs	6	1
ioSender XL/App.xaml.cs	6	0
CNC Controls/ICNCView.cs	2	1
XAML x:Uid / style touch-ups (7 files)	8	7
Locale/*/csv (7 locales, 2 files each)	1659	28

The idea came from Phil Barrett, who said in passing that ioSender could use a *workbench* tab — somewhere to machine a hole or a pocket without going out to CAD/CAM and back for a job that takes two minutes at the machine. This is that tab.

Odd Jobs has two sub-tabs. **Setup** is the familiar Start Job flow, constrained to the `G59` fixture, so the stock is measured and the origin and tool length reference are established the usual way. **Work Order** is where the job is described.

- **Geometry belongs to the toolpath; operations act on it.** Add a toolpath — line, circle, oval, square or rectangle — then add operations under it: pocket, contour, drill, bore, side finish, bottom finish, chamfer. That split is why nothing needs a special case: a counterbore is one circle carrying a shallow bore, a through drill and a chamfer, all on one centreline, and the compiler knows the recess is already open so the drill rapids through it.
- **One program, not a pile of them.** *Generate* compiles every enabled row into a single g-code program, with per-row enable checkboxes for partial runs (ticking a toolpath cascades to its operations, and back).
- **Tool order is chosen, not stumbled into.** Ordering the work by tree position stranded a tool — a drill wedged between two chamfers meant loading the chamfer tool twice. Each candidate is now scored by simulating it to completion and counting the operations that would still need it, preferring a tool that won't have to be revisited.
- **No redundant tool changes.** The compiler tracks what is already in the spindle, so a job with fifteen sections and five real tool changes now does five — not fifteen park/prompt/re-probe cycles.
- **Feeds and speeds are advised, not invented.** A dialog proposes chipload-based numbers per tool and material, and the tab remembers what was actually used.

There is no completion gate. An earlier build made you certify the setup before generating, its own watches false-positived, and it was simply annoying — it is replaced by one Yes/No at *Generate*: this is built on the cached G59 origin and tool length reference, proceed?

Tool numbers are preassigned from the Feeds & Speeds list position, and **T8 is reserved** — `tc.macro` uses it as the "3D probe already fitted" sentinel, so the numbering runs T1–T7, T9, T10. This machine runs grblHAL with `N_TOOLS 0`, i.e. no controller-side tool table, so a T-number means only what the macros make it mean. Cut so far in plywood and MDF only; feeds for other materials are untested advisor numbers. Hardware verification of bore clearing, counterbore→through-drill on one centreline, tabs on the last op to depth and patterned bolt circles is still in progress.

#193 Work Order promoted to its own tab; Setup gains conductive-material rules and touch-plate TLO support

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	22	0
CNC Controls/CNC Controls/ICNCView.cs	1	1
CNC Controls/CNC Controls/LayoutModel.cs	11	5
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	24	1
CNC Controls/CNC Controls/OddJobsSettingsControl.xaml	1	1
CNC Controls/CNC Controls/OddJobsView.xaml	0	12
CNC Controls/CNC Controls/OddJobsView.xaml.cs	0	153
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	1	1
CNC Controls/CNC Controls/LibStrings.xaml	1	0
ioSender XL/ioSender XL/MainWindow.xaml	4	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	23	2
ioSender XL/ioSender XL/StartJobView.xaml	6	4
ioSender XL/ioSender XL/StartJobView.xaml.cs	26	12
CNC Controls/CNC Controls/StartJobConfig.cs	2	2
CNC Core/CNC Core/FeedsSpeedsAdvisor.cs	4	0
CNC Controls/CNC Controls/AtcMacros.cs	35	9
ioSender XL/ioSender XL/App.xaml.cs	8	0
macros/tc.macro	40	15
CNC Controls Probing/Resources/CenterO.png, centerI.png (redrawn)	0	0
Locale/*/csv (14 files, 7 locales)	42	0

Continues the Job/Setup/Odd-Jobs flow redesign: the Work Order composer is now a top-level tab in its own right (Odd Jobs' shell, which only ever hosted this one sub-tab, is retired), Setup's probing rules now follow the selected stock Material, and the ATC tool-change macro can use a fixed touch-plate block for tool-length reference instead of a dedicated toolsetter.

- **Work Order tab** - promoted from an Odd Jobs sub-tab to a top-level tab; "New Work Order..." / "Load Work Order..." main menu items added (same idea as Load File) alongside its existing toolbar buttons. An existing saved layout's tab order is preserved by a one-time migration.
- **Conductive-material probing rules** - the "Stock conductive" checkbox is gone; conductivity is now read from the selected Material (Aluminum/Brass/Steel are conductive, wood-based materials aren't). Touch Plate probing of Internal or External-Is-Circle geometry on non-conductive stock shows a warning instead of a hard block, since a custom-shaped plate can still reach those points.
- **Touch-plate TLO block** - tc.macro can now use a fixed block in place of a toolsetter puck (new # <_tc_touchplate> flag): every tool probes the fixed position via the main probe input instead of switching to a dedicated toolsetter input. The TLO math is unaffected since it was already purely differential.
- **Setup Geometry preview images** - the Is-Circle preview images were redrawn to match the External/Internal picker's thin-line style (the first pass had a heavy dark-filled circle and thick arrows that didn't match).

Debug build verified after each change; touch-plate TLO path awaiting a hardware test pass.

#194 Work Order runs from the Job tab; Load File performance

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	116	23
CNC Controls/CNC Controls/GCode.cs	96	26
CNC Core/CNC Core/GCodeJob.cs	65	1
CNC Core/CNC Core/BulkObservableCollection.cs	66	0
CNC Controls/CNC Controls/ProgramView.xaml.cs	32	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	21	0
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	15	0
CNC Controls/CNC Controls/MacroProcessor.cs	10	0
CNC Controls/CNC Controls/WorkOrderView.xaml	4	11
CNC Controls/CNC Controls/ProgramView.xaml	4	0
CNC Controls/CNC Controls/AppConfig.cs	4	0
Locale/*/csv (7 locales)	7	42

Work Order's Run button now hands its generated program to the Job tab as the actual loaded job, not a floating preview overlay - it shows in the Job tab's real docked program list (the same one a loaded .nc file uses), streams through the existing run pipeline, and pops back to whatever was loaded before once the run ends (or is stopped, or a prerequisite check fails), switching the operator back to the Work Order tab.

- **GCode.File.Push()/Pop()** - a real push/pop against the loaded job. Push snapshots the current program (in memory, not a filename to re-read); LoadText loads generated G-code through the same completion pipeline Load File uses; Pop restores the snapshot.
- **ProgramView** gained a Source (File/Generated) and an Edit button that jumps back to the source tab for a generated program's own preview.
- **Load File performance** - parsing now writes into a private buffer instead of the live, DataGrid-bound collection, and binds once via a new BulkObservableCollection (a single Reset notification instead of one per line). Fixes a real UI freeze restoring a large loaded job (confirmed "Not Responding" on a 220k-line file); the originally-reported Load File slowness itself timed out at ~44s of pure parsing, not binding.

Built and tested on real hardware across several iterations this session; Load File's own parse-time cost (not binding) is a separate, not-yet-started investigation.

#195 Work Order: 4-flute chamfer bit, Climb/Conventional cut direction, spindle-direction setting

File	+	−
CNC Core/CNC Core/RotatingFileStore.cs	118	0
CNC Core/CNC Core/LogFile.cs	19	76
CNC Core/CNC Core/ConsoleLog.cs	7	6
CNC Core/CNC Core/DebugLog.cs	6	4
CNC Core/CNC Core/Grbl.cs	20	40
ioSender XL/ioSender XL/App.xaml.cs	7	5
CNC Controls/CNC Controls/FeedsAndSpeedsView.xaml.cs	13	12
CNC Controls/CNC Controls/AppConfig.cs	23	13
CNC Controls/CNC Controls/WorkOrderModel.cs	24	0
CNC Controls/CNC Controls/WorkOrderCompiler.cs	74	10
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	7	1
CNC Controls/CNC Controls/OddJobsFeedsSpeedsDialog.xaml.cs	24	5
CNC Controls/CNC Controls/OddJobsFeedsSpeedsDialog.xaml	9	0
CNC Controls/CNC Controls/OddJobsSettingsControl.xaml.cs	15	0
CNC Controls/CNC Controls/OddJobsSettingsControl.xaml	9	0
CNC Controls/CNC Controls/SpindleControl.xaml.cs	21	1
Locale/*/csv (7 locales)	28	0

Follow-on fit-and-finish for the Work Order tab, plus a shared file-rotation scheme for logs and backups that had each been reimplementing their own ad hoc retention.

- **4-flute chamfer bit** - a dedicated 1/4" 90° chamfer bit alongside the existing V-bit, same cone geometry for feeds/speeds and TOOL declarations.
- **Climb/Conventional cut direction** - a per-operation choice on the Feeds and Speeds dialog, defaulting to Conventional. Climb/Conventional isn't a fixed CW-or-CCW answer: it flips depending on whether the feature is internal (a bore/pocket, where climb orbits the SAME direction as the spindle) or external (a Contour cutting a part free, where climb orbits OPPOSITE it) - WorkOrderCompiler now derives the correct raw arc/winding direction per operation instead of hardcoding one.
- **Spindle-direction-capability setting** (Settings:App > Work Order) - for a spindle/VFD that can only physically run one direction regardless of the M3/M4 label (found on real hardware: grblHAL's own capability report said reversible, the VFD's documented behavior says otherwise). WorkOrderCompiler's direction math accounts for it, and the Job tab's Spindle panel now hides whichever of CW/CCW can't actually run.
- **Fixed a real crash** - the Spindle panel's RPM field threw a NullReferenceException on Enter (its key handler cast the wrong control type).
- **Shared day-of-week log/backup rotation** - new RotatingFileStore generalizes the scheme the console/debug/crash logs already used (a fresh timestamped file per run inside a `logs\<DayOfWeek>\` folder, capped and pruned per folder, with a `latest_*` alias at the top level) to App.config/Grbl settings backups and the Feeds & Speeds Fusion-archive copies, replacing several flat, unbounded-growth folders.

Debug build verified after each change; the new Work Order direction/tool options and the spindle-direction setting have not yet had a real-hardware cutting test pass.

#196 Fix: Setup's cold-session crash, and Work Order Run's live status now shows in the real Job tab list

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	26	19
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	15	5
ioSender XL/ioSender XL/MainWindow.xaml.cs	30	12
ioSender XL/ioSender XL/StartJobView.xaml.cs	20	6
macros/tc.macro	9	2

Two real bugs found and fixed the same session while sourcing manual screenshots for the job-flow redesign (#190–195) and driving it on real hardware for the first time end to end.

- **Cold-session crash fixed.** `#<_tlo_ref>` is only ever assigned by a prior `tc.macro`/Setup run — on a genuinely fresh session (first-ever run, or right after a firmware/NVS reset) it's undefined, and Setup's TLO-baseline save (`#<_tlo_saved> = #<_tlo_ref>`) errored reading it (grblHAL error 2), confirmed on real hardware. A first attempt guarded the read with an `EXISTS[]`-checked O-word `IF/ELSE/ENDIF` block — which uncovered a second, worse bug: ioSender's own G-code streaming pipeline (`GCode.AddBlock`, used by every `RunStreamedJobInPlace` burst) silently drops bare O-word `ELSE/ENDIF`-only lines, leaving the controller's o-word engine waiting for a matching `ENDIF` that never arrives — it kept accepting and acking every subsequent line, including real motion commands, without ever queuing them, until a reboot cleared the stuck state. Reverted that approach entirely: Setup's two program builders (`BuildProgram/BuildViseProgram`) now decide whether to preserve or reset `#<_tlo_ref>` in C#, via the controller's own `$TLR`-backed `IsTloReferenceSet` flag, so the streamed program never contains O-word branching at all. `tc.macro`'s own `o199/o200` guard is unaffected — it's uploaded as a raw file (YModem), never tokenized through the same pipeline.
- **Work Order's live run status now shows where you're actually looking.** Found chasing a Work Order screenshot: Run's live per-line status (`ok/*`) was showing in a separate floating panel, not the real docked Job-tab program list, whose status column just sat dead. Root cause: `MacroProcessor.Run`'s shared streaming pipeline deliberately never writes into the loaded job, by design, for every other macro caller (Setup, calibration, fixture tools) — a run must never silently hijack the Job tab. Work Order is the one legitimate exception (it already made itself the loaded job via `GCode.File.Push()/LoadText()` before streaming), so it now opts in via a new `bool preferJobView` parameter threaded through `Run→Flush→StreamProgram→RunStreamedJobInPlace` (default `false` everywhere else). When set, the sanitized burst is built directly into `GCode.File` instead of a disconnected transient copy, so `ProgramPanel`'s own docked list — permanently bound to `GCode.File.Data` — gets the live writes for free, with no separate view or redirect needed. `GCode.File.Push()/Pop()` is unaffected; the extra in-place rebuild happens inside the already-pushed slot.
- **Removed a dead Edit button.** Work Order's Generate-preview panel carried an Edit button (`ProgramView.EditTargetTab`) that switched to the Work Order tab — but Generate can only ever fire while already on that tab, so it was a permanent no-op. Dropped for Work Order; the generic mechanism stays in `ProgramView` for a future tool that might actually need it.

Confirmed on real hardware: the cold-session crash is fixed (reboot + fresh Setup run completes), and Work Order's docked-list live status is fixed (verified via manual screenshot capture, no floating panel, hover-tooltip explain still works). Debug build clean throughout.

#197 Stepper calibration and squareness folded into Machine Setup → Calibration

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	69	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml	20	2
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	0	628
CNC Controls/CNC Controls/StepperCalibrationWizard.xaml.cs	0	247
CNC Controls/CNC Controls/StepperCalibrationWizard.xaml	0	127
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	0	117
CNC Controls/CNC Controls/ToolsView.xaml.cs	0	2
Locale/*/csv (7 locales)	161	0

Stepper calibration used to live in two separate top-level wizards — one driven from a measured cut, one from a scratch test — reached from the Tools tab, entirely disconnected from the rest of commissioning. Both are deleted. Calibration is now a **step inside Machine Setup**, in the same guided sequence as homing, soft limits, squareness and the TLO reference, so a machine gets set up in one pass instead of by remembering which wizard lives where.

- **One place, one order.** Calibration sits with squareness in Machine Setup → Calibration; both are steps in the wizard rather than destinations you have to go find.
- **1,119 lines of wizard deleted.** The two standalone wizards duplicated a lot of the probe/measure/apply plumbing that Machine Setup already had; folding them in removed the duplicate rather than porting it.
- **Tools tab keeps shrinking.** Another two registrations gone from `ToolsView` — the tab is progressively emptying out as its contents move to where they are actually used.

Machine Setup Calibration step verified on real hardware.

#198 Spoilboard surfacing becomes a Work Order Surface toolpath

File	+	−
CNC Controls/CNC Controls/WorkOrderCompiler.cs	187	6
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	63	11
CNC Controls/CNC Controls/WorkOrderModel.cs	34	4
CNC Core/CNC Core/FeedsSpeedsAdvisor.cs	19	7
CNC Controls/CNC Controls/WorkOrderView.xaml	6	0

Surfacing the spoilboard was its own wizard, with its own stock/origin assumptions, sitting outside the Work Order flow that every other job goes through. It is now a **Surface toolpath** you add to a work order like any other operation — same tool list, same feeds and speeds advisor, same generate/run path, same dry run.

- **Entire Spoilboard** is a checkbox on the toolpath rather than a separate mode. It deliberately does not trust the work order's WCS at all: it touches off its own fresh Z0 and works in machine coordinates, because the point of resurfacing is that the board is *not* where you last thought it was.
- **A scratch WCS slot** is chosen and restored around the operation, so surfacing can establish its own origin without stepping on the work order's own coordinate system.
- **Feeds and speeds** for a surfacing cutter come from the same advisor as everything else, instead of the wizard's separate hardcoded defaults.

Surface toolpath, including Entire Spoilboard, verified on real hardware.

#199 User-addable custom tools, seeded from config instead of a hardcoded enum

File	+	−
CNC Controls/CNC Controls/OddJobsFeedsSpeedsDialog.xaml.cs	206	235
CNC Controls/CNC Controls/OddJobsSettingsControl.xaml.cs	165	78
ioSender XL/ioSender XL/Default-App.config	114	0
CNC Controls/CNC Controls/AppConfig.cs	54	6
CNC Controls/CNC Controls/CustomTool.cs	100	18
CNC Controls/CNC Controls/WorkOrderModel.cs	82	2
CNC Controls/CNC Controls/CustomToolEditDialog.xaml.cs	72	0
CNC Controls/CNC Controls/ConfigStore.cs	51	3
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	42	48
CNC Controls/CNC Controls/CustomToolEditDialog.xaml	35	0
CNC Controls/CNC Controls/OddJobsToolMemory.cs	6	6

The tools a work order could use were a hardcoded `enum`, so adding a cutter meant editing and rebuilding the app. They are now a single **editable list seeded from `Default-App.config`**, with an add/edit dialog — your own tools sit alongside the built-ins and behave identically.

- **Operations self-describe.** Each operation carries its own `ToolName` rather than pointing at an enum ordinal, so a work order stays readable even if the tool list changes underneath it.
- **Stale Ids self-heal.** A work order saved against a tool that has since moved reconciles on load instead of silently binding to the wrong cutter — the built-in Ids 0–13 keep their old ordinals precisely so existing saved work orders still resolve.
- **Config promotion, not a migration.** The seed list arrives through the existing template-fallback mechanism, so an install that has never seen custom tools picks them up without a migration step.

Verified against existing saved work orders on this install.

#200 App.config / Default-App.config processing on first run after an install

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	64	3
CNC Controls/CNC Controls/ConfigStore.cs	51	3

Two related problems in how settings are loaded the first time a build runs against an existing install — both confirmed as real failures, not hypotheticals.

- **A missing section fell back to the bare C# field initializer.** A section the user's own `App.config` has never contained — because it is new in this build — now falls back to the **shipped `Default-App.config` template's curated value** instead, after first trying any real legacy data. The difference matters: the template is a considered starting configuration, the field initializer is whatever `= new()` happened to produce.
- **One bad section took out every section after it.** `ConfigStore.ReadDocument` now isolates each section's `Read()` in its own `try/catch`. Previously a single section holding data a newer build no longer recognised threw, and every section registered *after* it silently never loaded. Confirmed case: a custom Work Order tool persisted with `Kind="Mill"`, from before that value was split into EndMill/OFlute/BallEnd/Surfacing.
- **Failures are visible.** A section that fails to read is left at its owner's default and reported through a startup dialog plus `ConsoleLog` — the old behaviour discarded the data with no indication anything had gone wrong.

Matters more than it used to: ioSender V2 now has real users beyond its author, so saved installs cannot be silently broken.

#201 Fix batch: unhomed Go To, Work Order park height, dry run, alarm aborts, jog pad targets

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	105	6
CNC Controls/CNC Controls/WorkOrderCompiler.cs	92	23
CNC Controls/CNC Controls/MacroProcessor.cs	79	7
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	75	32
CNC Controls/CNC Controls/UIUtils.cs	74	1
CNC Controls/CNC Controls/JogBaseControl.xaml	61	11
CNC Controls/CNC Controls/JobControl.xaml.cs	41	16
CNC Controls/CNC Controls/GotoBaseControl.xaml.cs	35	0
CNC Controls/CNC Controls/StreamPump.cs	23	4
ioSender XL/ioSender XL/MainWindow.xaml.cs	20	0
CNC Core/CNC Core/GrblViewModel.cs	10	0
CNC Controls/CNC Controls/NumericTextBox.cs	6	4
CNC Controls/CNC Controls/LayoutModel.cs	6	2
CNC Controls/CNC Controls/WorkOrderView.xaml	5	1

A batch of fixes found by running work orders end to end on real hardware. Several turn out to be the same mistake in different places: **trusting a value in the wrong reference frame**.

- **Go To refuses to run unhomed.** Every Go To is a machine-coordinate move, so it is only as good as the machine-coordinate reference. After an unexpected controller reboot grblHAL came back *Idle* — not Alarm — with `MPos:0,0,0` at wherever the tool happened to be; a G30 then chased the stored coordinates as a blind displacement from a false zero and rapided into the hard stops, stalling three steppers. Soft limits are no backstop, as they are only enforced when homed. The guard lives in the shared `SafeGoto` routines, because a missing button-level guard on the offset flyout is exactly what let this through.
- **Work Order park and opening retract are machine-referenced.** Both ends of a generated program used `G0 Z{SafeZ}`, treating a *work*-Z height as if it were a machine height. From a G30 park that opening "retract" was a ~90 mm plunge; at the end it traversed the whole table ~93 mm below machine top. Both now go through the shared `EmitGotoG30` idiom.
- **Dry Run actually protects again.** `MacroProcessor.Run` silently ignored Dry Run mode, so a work order run in dry run could still fire the spindle for real. The neutralization also ran twice on the normal path; it is now scoped to the callers that need it.
- **Alarms always abort the stream.** A tab-gated abort left the pump stuck active after an alarm raised on a non-Job tab, silently swallowing every later response including jog acks. Macro alarm-abort now latches an event counter rather than sampling current state, closing a race where a fast manual Reset let a macro continue past an alarm it never saw.
- **Jog pad targets the machine, not the job.** The centre button always targets the machine envelope instead of the loaded program's bounding box, and four corner buttons (20 mm holdback) were added.
- **Work Order tab UX.** Unsaved-changes prompting, right-click opening the correct menu, Tab no longer escaping the panel, Generate saving first, name-field select-all, and fractional input such as `1 1/8"`.

All verified on real hardware — the work order that exposed most of them now drills all 12 dimples clean.

#202 Touch-plate offsets no longer gated on stock conductivity; Work Order picks its own WCS

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	26	13
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	41	13
CNC Controls/CNC Controls/WorkOrderModel.cs	18	6
CNC Controls/CNC Controls/WorkOrderView.xaml	13	0

Two independent fixes that both came out of running a work order end to end on the machine.

- **Touch-plate compensation was gated on stock conductivity.** Plate thickness and lip offsets applied only when `touchPlate && !stockConductive`. That is the wrong test: the circuit for a touch-plate probe closes between the *bit* and the *plate*, both metal, and never through the stock underneath — so what the stock is made of has no bearing on whether a plate was physically used. Probing conductive stock *with* a real plate is an ordinary workflow (thin aluminium sheet, too thin for a reliable direct stylus touch), and it silently produced zero compensation. Confirmed on real hardware: work Z0 landed exactly one plate thickness (12 mm) high, and X/Y landed on the plate's own lip edge instead of the stock's. Now driven by whether a touch plate was actually selected, in all three program generators. Stock conductivity still drives the reliability warning it was always meant to drive.
- **A work order chooses its own coordinate system.** The compiler hardcoded G59. A work order now carries a WCS field: *Follow Setup* resolves live at generate time, or it can be pinned to G54–G59. Setup is shared with the plain Start Job tab, so its live selection is not something a saved work order can treat as a stable reference.
- **Entire Spoilboard no longer writes to the wrong slot.** It emitted `G10 L20 P1 Z0` regardless of which scratch WCS it had actually chosen — which could have overwritten a real G54 origin when the work order's own WCS was G54.

Z error dropped from 12 mm to within 1–2 mm on retest; the remainder was a stale machine-wide TLO reference, recalibrated separately.

#203 Feed rate readout stops clipping; run bar shows its unit in the tooltip

File	+	−
CNC Controls/CNC Controls/FeedControl.xaml	10	2
CNC Controls/CNC Controls/FeedControl.xaml.cs	1	1
ioSender XL/ioSender XL/MainWindow.xaml	13	2
Locale/*/csv (7 locales)	7	0

Reported against V2.32: the *Feed rate* panel's readout only showed about two digits. The box was a hard `Width="50"` with its unit label absolutely positioned beside it, and once the control style's 8px horizontal padding and 1px border came out of that only ~32px of text area was left - `Format="####0"` renders five digits but they had nowhere to go and clipped.

- **Readout and unit share a layout panel** now instead of the label being pinned at a fixed offset, and the box has a minimum width sized for the digits it can actually produce - growing rather than clipping beyond that.
- **Run bar drops the static unit label.** On that bar the room is better spent on digits, so the unit moved into a tooltip covering the whole *Feed:* group. The wording lives in a localizable run rather than a binding format string, so it translates like every other tooltip.

The tooling that generates locale rows learned inline runs at the same time, which backfilled rows for several probing descriptions that had been silently English-only.

#204 Tools tab holds only the hardware-gated tools, and hides when none apply

File	+	−
CNC Controls/CNC Controls/ToolsView.xaml.cs	34	21
CNC Controls/CNC Controls/SurfaceSpoilboardWizard.xaml.cs	0	611
CNC Controls/CNC Controls/SurfaceSpoilboardWizard.xaml	0	97
CNC Controls/CNC Controls/LayoutModel.cs	12	9
CNC Controls/CNC Controls/AppConfig.cs	15	8
CNC Controls/CNC Controls/LibStrings.xaml	0	9
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	0	2
ioSender XL/ioSender XL/MainWindow.xaml.cs	3	1
ioSender XL/ioSender XL/JobView.xaml.cs	3	2
Locale/*/csv (7 locales)	0	336

Three of the six things the Tools tab registered had already moved elsewhere and were pure duplication: **Stepper calibration (probe)** and **Squareness** live in Machine Setup's Calibration step, and **Surface spoilboard** became a Work Order Surface toolpath. All three are deregistered, and the standalone spoilboard wizard is deleted now that the Work Order toolpath - including Entire Spoilboard - fully replaces it.

- **What remains is Tool table, Trinamic tuner and PID tuner**, none of which moved anywhere, and all three of which were already gated on the controller actually having a tool table / TMC drivers / a PID log.
- **The tab now gates itself on its own children** rather than re-testing their prerequisites, and loses its always-visible flag - so the existing connect-time prune drops the whole tab on a controller that supports none of them, and keeps it with just the relevant sub-tabs on one that does.
- **Persisted layouts are migrated**: the retired components join the one-time Tools-slot fixup, so an existing profile has them stripped rather than carrying dead entries.

Also swept 336 locale rows orphaned by the deleted wizard, and renamed the internal tab-identity value that Work Order had quietly inherited from it.

#205 Rapid single-step jogging can no longer wedge the controller

File	+	−
CNC Core/CNC Core/JogGate.cs	78	0
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	16	41
CNC Core/CNC Core/ControllerMapper.cs	8	0
CNC Core/CNC Core/GrblViewModel.cs	11	1
CNC Controls/CNC Controls/JobControl.xaml.cs	9	0

Sending a second jog before the first has been acknowledged can wedge the controller completely - no response to anything, including a bare `$I`, until a power cycle. A guard against that already existed, and had two holes.

- **The Xbox controller was never covered.** The guard was private to the on-screen jog panels; the gamepad's D-pad builds and sends its own jog line, so rapid presses reproduced the original wedge unmitigated. The guard now lives in the core alongside everything else that talks to the machine, and both inputs share it.
- **It could never be acknowledged while jogging.** Responses were suppressed for the whole time the controller reported the Jog state - but the first jog puts it in that state, so that jog's acknowledgement and every later one was swallowed. The guard then fell through to its timeout and eventually emitted an unacknowledged jog anyway: exactly what it exists to prevent, just slower. That also explains why only *rapid* jogging triggered it - jog slowly enough to fall back to idle between moves and the acknowledgement arrived normally.

Responses are delivered unconditionally now, with the Jog filter moved to the one consumer that needed it - the job streamer's acknowledgement accounting, where a stray jog response would otherwise be counted against an outstanding command.

Analog stick jogging deliberately stays outside the guard - overlapping sends are how that loop achieves continuous motion.

#206 Fixture definition dialog: keeps its probe choice, offers where you are standing, stops minimizing the app

File	+	−
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	119	28
CNC Controls/CNC Controls/Fixture.cs	12	1
CNC Controls/CNC Controls/UIUtils.cs	29	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	3	0

- **Test position offers the current machine position.** Test rapids to the *saved* position before probing, so jogging closer and pressing Test silently undid the jog and re-ran the identical failing probe - which is precisely what the failed-search recovery was telling the operator to do. Test now compares live against saved and, past a small tolerance, asks; the default adopts where the machine actually is, exactly as Set position would. Nothing moves until that is answered.
- **The recovery prompt loses an auto-retry that could never fire.** It asked the operator to jog closer and click OK, but it is a modal dialog - the jog pad is dead while it is up, so the position comparison on the way out always saw no change. It now just explains the failure and points at Test, which handles it.
- **The probe selection is remembered per fixture.** The 3D Probe / Touch Plate radio reset on every open, so reopening a touch-plate fixture silently ran the next Set/Test with 3D-probe geometry - plate thickness and lip offset dropped - until the operator noticed. A new fixture with nothing to restore now defaults by which probes are actually defined, rather than always landing on 3D Probe even on a machine that has none.
- **Closing the dialog no longer minimizes the main window**, a side effect of it being non-modal so that jogging stays reachable while it is open.

Existing fixture definitions are unaffected - the stored probe type is absent from an older file and defaults to what those fixtures already do.

#207 Setup runs without the multi-second stalls; ATC macro update no longer needs a restart

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	38	4
CNC Core/CNC Core/YModem.cs	26	11
CNC Controls/CNC Controls/SDCardView.xaml.cs	63	41
CNC Controls/CNC Controls/AtcMacros.cs	13	1
CNC Controls/CNC Controls/TrinamicView.xaml.cs	22	14
macros/pcorner.macro	9	1
ioSender XL/ioSender XL/StartJobView.xaml.cs	3	1

- **Each hold-for-idle gate cost two seconds of nothing.** The wait allowed time to observe motion *starting* before it could trust an idle report - but only a controller-side SD job acknowledges first and moves later; an ordinary burst has already finished moving by the time the gate is reached. The long allowance is now scoped to macros that actually start such a job. A Setup program passes three of these gates.
- **A redundant lift after the tool-length reference** descended to the park position only to climb straight back to machine top.
- **Corner probe retreats drop from 20mm to 5mm.** Twenty is far more than any real bed variance - on a T-slot extrusion bed even five would take some doing - so it just spent Z travel on every retreat between faces.
- **The macro upload could hang the app outright.** The per-packet wait had no guard against its worker faulting, which left the interface spinning with no way out - reliably reproduced against the largest of the macro files. It now fails cleanly and reports a failed transfer instead.
- **A stuck internal flag survived until restart.** The flag that reroutes controller responses during a file read was cleared only on the success path, so one fault left the app misreading everything the controller said for the rest of the session - the cause of the long-standing 'exit and restart, then it works first time' behaviour when updating the ATC macros. It is now restored unconditionally, in all four places that set it.

The retreat clearance is a fixed value rather than a caller-supplied one: reading a parameter no caller had set aborted the run with an expression error.

#208 Settings and Machine Setup: a searchable index replaces the tab strips

File	+	−
CNC Controls/SettingsNavShell.xaml.cs	329	0
CNC Controls/SettingsNavShell.xaml	89	0
CNC Controls/SettingsNavNode.cs	221	0
CNC Controls/SettingsSearchIndex.cs	132	0
CNC Controls/NullToVisibilityConverter.cs	27	0
CNC Controls/ToolTipTextTemplateSelector.cs	28	0
CNC Controls/GrblConfigView.xaml(.cs)	279	243
CNC Controls/MachineSetupWizard.xaml(.cs)	132	16
CNC Controls/MachineSetupView.xaml(.cs)	91	9
CNC Controls/MainPageEditor.xaml(.cs)	75	7
CNC Controls/KeyMapEditor.xaml(.cs)	39	7
CNC Controls Camera/ConfigControl.xaml(.cs)	38	2
CNC Controls/SettingsPanelRegistry.cs	74	1
Config panels declaring their own placement (8 panels)	41	9
CNC Controls/AppConfig.cs	12	0
ioSender XL/App.xaml	25	0
CNC Core/LibStrings.xaml	16	1
Locale/*/csv (7 locales)	140	14

Settings and Machine Setup were top-level tabs whose contents were themselves tab strips - about twenty destinations across two of them, and Machine Setup's Calibration step nested a third tab strip inside the second. Tabs stopped scaling: they compete for horizontal space, offer no grouping, and cannot be searched. Both now use one shared shell - a searchable category tree on the left, the selected page on the right, with the existing Save / Restart / Reset footer beneath.

Every config panel is its own page. The App, Jogging and G Code tabs used to cram unrelated panels into two or three columns because there was nowhere else to put them; that cramming is gone. Camera splits further into *Camera* and *Demo recording (OBS)*. Machine Setup keeps its numbered steps and its green/orange/red grading, now shown as a status dot beside each step.

Search matches the words on the page, not just page names - "backlash" finds Axis information, "OBS" finds Demo recording - with a match count, and a tooltip explaining any hit you cannot see on the page (much of the useful text lives in tooltips, so a match can read as wrong until it says *Matched tooltip:* ...). The index is built by walking the logical tree, so it never has to open a page: opening a Machine Setup step queries the controller's filesystem, and indexing must never talk to the machine.

Page labels are read from each panel's own already-localized group header, so they are correct in all seven languages without a single new string. Placement moved onto the panels themselves, retiring the hardcoded table in the settings host that had to match panels in other assemblies by full type name - a feature can now contribute a settings page without the host knowing it exists.

The Grbl page deliberately keeps its own group tree and $-search: that is live controller data, fetched on connect and absent entirely on classic grbl, so lifting it into the navigation tree would have made the tree change shape whenever the machine connected.

Sub-tab keyboard shortcuts are dropped - the badge and right-click bind menu lived on tab headers that no longer exist - and any already saved are removed on load. **Top-level tab shortcuts are unaffected.**

Prompted by this work, long tooltips now wrap at a sensible width app-wide instead of running off as one line, and sit clear of the mouse pointer: WPF places them assuming a standard cursor, so an extra-large Windows pointer covered the first few characters.

#209 Settings search names the control a matched tooltip belongs to

File	+	−
CNC Controls/CNC Controls/SettingsSearchIndex.cs	72	6
CNC Controls/CNC Controls/SettingsNavShell.xaml.cs	52	9
CNC Controls/CNC Controls/SettingsNavNode.cs	1	1

The navigation tree's search (#208) already told you *why* a page matched — *Matched text* or *Matched tooltip*, quoting the words it hit. A tooltip match still left you hunting, because a tooltip has no visible text of its own: you knew the page held the answer but not which control to hover.

- **Every indexed tooltip now carries its owner's name** — resolved by a naming ladder that tries the control's `Content`, then its `Header`, then a `Label` property, then its automation name, and only gives up if none of those exist.
- **The match explainer quotes the owner** — "Matched tooltip on *Send comments*: ...", so the hover target is named outright.
- **Opening words are preserved** when the hit lands near the start of a tooltip, instead of eliding them into a leading ellipsis.

#210 The main bar carries what a running job needs; everything else opens from the File and Tools menus

File	+	−
CNC Controls/CNC Controls/ViewHostWindow.xaml.cs	195	2
CNC Controls/CNC Controls/ViewHostWindow.xaml	13	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	157	52
CNC Controls/CNC Controls/AppConfig.cs	137	28
CNC Controls/CNC Controls/LayoutModel.cs	35	23
CNC Controls/CNC Controls/TabRegistry.cs	30	5
ioSender XL/ioSender XL/MainWindow.xaml	16	8
Locale/*/csv (7 locales)	7	0

The tab strip had grown to thirteen tabs, most of which are places you visit once to set something up rather than screens you work from. It is now cut back to the four used while a job actually runs — **Setup**, **Job**, **Work Order**, **Offsets** — and each remaining view opens in its own window from **File** or **Tools**.

- **A hosted view keeps tab semantics** — `ViewHostWindow` runs the same Setup / Activate / CloseFile lifecycle a `TabItem` gave it, and caches the instance, so a view reopens in the state you left it. Closing is ESC or the window X; the windows are deliberately non-modal, because the machine keeps running behind them.
- **Placement is a layout slot, not a hardcoded decision** — the File and Tools menus are slots of the same layout tree that drives the tab strip, so what ships is a default rather than a rule.
- **Work Order stays a tab**, and File's Load / New Work Order select it rather than opening a second copy.
- **Existing profiles migrate once**, and only once: a saved arrangement that already has menu slots is left exactly as the user left it.

Host windows inherit the main window's DataContext explicitly — that does not cross a Window boundary on its own, and several views read their model from it rather than from Setup().

#211 Interface preferences gather under User Interface → General

File	+	−
CNC Controls/CNC Controls/UiGeneralConfigControl.xaml.cs	33	0
CNC Controls/CNC Controls/UiGeneralConfigControl.xaml	30	0
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	9	5
CNC Controls/CNC Controls/BasicConfigControl.xaml	8	18
CNC Controls/CNC Controls/BasicConfigControl.xaml.cs	0	1
Locale/*/csv (7 locales)	91	0

The Application page had become a mix of two unrelated things: how ioSender talks to the *controller*, and how the app itself looks and behaves. The second group now has a page of its own at **Settings → User Interface → General**, which sorts first under that heading.

- **Moved:** keep MDI focus, restore last window size, friendly Start/Pause run-bar labels, UI scale, and the auto-save / prompt-before-saving pair.
- **Deliberately left behind:** the grbl auto-save options — those write $ settings to the machine, which is controller interaction, not an interface preference.
- **The Theme dropdown is removed** — it listed five themes and applied none of them. Dead UI that only ever raised the question of why it did nothing.

#212 A shortcut names a view, not a place — and you choose whether that view is a tab or a menu entry

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	198	26
CNC Controls/CNC Controls/MainPageEditor.xaml.cs	163	65
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	77	103
CNC Controls/CNC Controls/ActionKeyBinder.cs	62	17
CNC Controls/CNC Controls/AppConfig.cs	46	22
CNC Controls/CNC Controls/MainPageEditor.xaml	26	13
ioSender XL/ioSender XL/MainWindow.xaml	12	9
docs/manual/index.html	30	11
Locale/*/csv (7 locales)	7	0

Moving ten views into the menus (#210) silently killed every keyboard shortcut bound to them: dispatch resolved a shortcut to a `TabItem` and gave up when there was not one. The fix is the principle rather than the symptom — **a shortcut id names a view, not a place**.

- **A key reaches its target wherever it lives** — tab or menu window — so moving a view never costs it its shortcut. The ids were deliberately not renamed when the views moved, so bindings made before the move still work.
- **One group, top level only.** Seven groups (Views, Settings pages, Machine Setup steps, Probing / Lathe / Odd Jobs tabs, Menu commands) collapse into a single **Top Level Tabs** list holding exactly the tab-strip and menu destinations. Second-level targets are withdrawn: whether something is currently a tab or a menu entry is a layout choice, not a category.
- **Main-menu commands are bindable** — Connect, Load Program, Load/New Work Order, Camera and the Help entries. All unbound by default, and a shortcut is refused while its menu entry is greyed out, so a key can never do what the menu will not.
- **The menu shows the key** — every shortcut-capable entry carries its binding in the gesture column, refreshed off the same event the tab-header badges use, so the two cannot disagree.
- **Top-level tabs becomes a placement editor** — one row per view choosing Tab bar, File menu, Tools menu or Not shown, with ordering within each. The old shuttle could only say "tab bar or nothing", and did not even list a menu-hosted view. Settings, Machine Setup and Job cannot be hidden, because the app puts them back rather than let a layout lock you out.

#213 Job tab: the jog pad and its go-to buttons are optional, and the panels use the room that frees up

File	+	−
ioSender XL/ioSender XL/JobView.xaml.cs	182	33
CNC Controls/CNC Controls/JogBaseControl.xaml	26	44
CNC Controls/CNC Controls/MainPageEditor.xaml	24	14
ioSender XL/ioSender XL/JobView.xaml	18	5
CNC Controls/CNC Controls/AppConfig.cs	17	1
ioSender XL/ioSender XL/MainWindow.xaml.cs	12	1
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	11	21
docs/manual/index.html	5	2
Locale/*/csv (7 locales)	28	0

The jog pad was the widest thing competing with the program view, and nothing could remove it. Both it and its go-to buttons are now yours to switch off, and the panel columns finally use whatever that frees.

- **Show go-to buttons on the jog pad** (Settings → User Interface → General) shows the four corner targets and the centre bullseye. Off by default on a new profile — each one rapids the machine across the table, so you opt in once you know what they do — and an existing profile keeps whatever it already had.
- **The centre button loses its hidden second job.** While a jog was running it silently cancelled the jog instead of going to centre, with nothing on the button saying so; the same branch came out of the corner buttons. Releasing the key or button, Feed Hold and Reset all still stop a jog.
- **Show the jog pad on the Job tab** (Job tab layout) removes the pad entirely for anyone who finds the run bar's compact jog enough. Keyboard and game-controller jogging are unaffected — the run bar hosts the same control and does the key registration either way.
- **Panels fill one column before starting a second.** They used to be balanced by height, so two short panels always landed one per column and the second reserved its width whether or not it held anything. An unused column now collapses to nothing and the workspace gets the width back.
- **The right-most column clears the flyout strip**, so a pinned flyout no longer floats over the top of whatever panel is there.

#214 Fix batch: duplicate Work Order tabs, a setup prompt that could never be satisfied, Esc, and the settings scroll wheel

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	99	8
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	59	4
CNC Controls/CNC Controls/SettingsNavShell.xaml.cs	45	0
CNC Controls/CNC Controls/ViewHostWindow.xaml.cs	36	19
ioSender XL/ioSender XL/MainWindow.xaml.cs	31	16
CNC Controls/CNC Controls/ConsoleWindow.xaml.cs	25	0
ioSender XL/ioSender XL/JobView.xaml.cs	23	1
CNC Controls/CNC Controls/ProbeDefinition.cs	14	2
CNC Controls/CNC Controls/SettingsNavShell.xaml	7	1
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	7	0
CNC Controls/CNC Controls/JogControl.xaml	6	2
docs/manual/index.html	3	2
CNC Controls/CNC Controls/ViewHostWindow.xaml	2	3

Five fixes, four of them fallout from the tabs-to-menus move (#210) and one older.

- **A new Work Order tab on every launch.** A fixup re-added the retired Odd Jobs node whenever the tree lacked one — which, once nothing else created it, was always — and the next fixup promoted that fresh node to a Work Order. An "add it if absent" fixup for a component nothing else creates is not idempotent, it is a generator. A dedupe pass repairs profiles that already grew copies.
- **Height Map could be deleted out of existence.** Its injection tested only the tab strip, but since menus became placement slots "not on the tab strip" no longer means "absent"; and the flat tab list, applied later, dropped nodes it did not name with nothing left to rescue them. Both closed.
- **"Let's finish setting up your machine" every single launch.** The prompt navigated via the Machine Setup tab, which no longer exists, so it left you on the Job screen — and because the wizard was never reached, no step could be completed, so it re-armed next launch. Self-sustaining rather than merely annoying.
- **Esc is free again.** The console toggle defaulted to Esc and is dispatched before any control sees the key, so Esc stopped cancelling edits, closing popups and backing out of things. It defaults to F12 now, existing profiles are moved off the old default, and the console window gained its own Esc-to-close — two-stage, so Esc still clears a part-typed command first.
- **The scroll wheel works over a settings page**, not just over its scrollbar. A list laid out at full height inside the page scroller had nothing of its own left to scroll, but still marked the wheel handled.
- **First-run setup could not be completed on a machine that already worked.** Apply was enabled only when a $ setting would change, and it returned early on zero changes — before the one line that records which machine you picked. On a controller already holding the right settings, choosing your machine changes nothing, so Apply stayed greyed, the machine was never recorded, and the setup gate asked again on every launch with no way out from inside the app. Apply now stays live while the picked machine differs from the stored one, and commits the identity when that is all there is to commit.
- **Menu-hosted views open at a usable size and fill it.** The host window was a fixed 1000×720, so Machine Setup's Apply row sat outside it; and the settings page scroller allowed horizontal scrolling, which makes a page lay out at its own natural width and never stretch — so widening the window just

revealed more of a fixed-width page and right-aligned buttons stayed out of reach. The window now takes a share of the work area and the page is constrained to it.

#215 The views that moved into the menus can actually be opened

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	14	6
CNC Controls Probing/CNC Controls Probing/ProbingView.xaml.cs	82	41
CNC Controls Lathe/CNC Controls Lathe/LatheWizardsView.xaml.cs	15	4
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	12	2
CNC Controls/CNC Controls/ProbeDefinition.cs	4	1
ioSender XL/ioSender XL/JobView.xaml.cs	1	1
ioSender XL/ioSender XL/Default-App.config	820	10

Moving ten views off the tab bar (#210) gave each of them a new first act: instead of being a tab that already existed, a menu entry builds the view and shows a window. Everything below is that one change catching up with itself — and every one of them was reachable from a fresh install, because the affected views live in the menus by default.

- **The Tools menu was decorative.** Menu entries are created disabled and switched on when the controller connects, exactly as the tabs are — but the routine that switches them on was never called from anywhere. SD Card, Probing, Height Map and Lathe Tools were therefore not merely greyed: with no tab to fall back on, they could not be opened at all. The tab and menu paths are now driven from one place, and the routine is named after *views* rather than tabs, which is how the two came apart.
- **Probing crashed on open.** Its view model was built when the control loaded, but a hosted window selects its first inner tab while it is still being measured — before that. The setup also sat behind a flag shared by every instance, so even with the timing fixed, the second Probing window would have opened empty and crashed the same way. Setup now happens on demand, once per window, and the tab selection that arrives too early is replayed rather than lost.
- **Lathe Tools crashed on open**, differently: activating a wizard asks the controller for its parser state, which waits for the reply by pumping messages — illegal during the layout pass the activation was running inside. Activation is now queued to run once layout has finished.
- **A restart that fails to restart says so.** Relaunching after a settings change wrote nothing to the log, so a relaunch that never came back was indistinguishable from quitting the app. The attempt, the new process and any failure are now recorded. The probe library's legacy import gained the same treatment: it silently treated a corrupt file as no file.
- **The shipped default configuration is now a real one** — captured from a genuine first run rather than left for the app to invent at startup, so what a new install begins with is explicit and reviewable instead of implied.

Found while shooting the manual's screenshots against a first-run configuration: the Tools entries looked wrong in a capture, and the machine they were shot on had no saved layout to hide behind.

#216 Large programs load in seconds, not half a minute

File	+	−
`CNC Core/CNC Core/HelperClasses.cs`	23	2
`CNC Controls/CNC Controls/GCode.cs`	35	2
`CNC Core/CNC Core/GCodeJob.cs`	29	1

Loading a big program used to take about half a minute with nothing on screen but a wait cursor, which is indistinguishable from a hang. A 220,197-line file now loads in **2.6 s instead of 32.0 s** — and while it loads, the status line says *Loading <file>…*, then reports *Loaded <file> — 220,197 lines in 2.6 s* when it is done.

- **The G-code parser was never the problem.** It interprets all 220,197 lines in 989 ms. Almost the entire load — 33.4 s of a 34.5 s parse — went into *constructing* the program-list rows.
- **The real cost was cross-thread latency, not work.** Raising a property-change notification from a background thread marshals it to the UI thread and *blocks* until the UI thread runs it. Program loading deliberately parses on a background thread to keep the app responsive, and every row raised about four of these while being built — roughly a million blocking round trips to the UI thread.
- **A row being built has nothing bound to it yet**, so every one of those hops was announcing a change to an audience of nobody. Notifications with no listener now return immediately instead of marshalling. That is a no-op by definition, so nothing about how the UI updates has changed.
- The same fix applies everywhere the app builds view models in bulk off the UI thread, not just to program loading.

Found by measurement rather than inspection: the load timing was split into phases twice before the cause showed up. Four plausible explanations — bounding-box computation, per-line allocation, quadratic scanning, and unoptimised Debug builds — were each measured and ruled out first. The phase timings are left in the debug log deliberately, since this class of regression is easy to reintroduce and invisible without them.

#217 The program list's Data column no longer starts collapsed

File	+	−
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	41	8
CNC Controls/CNC Controls/GCodeListControl.xaml	4	1

Start the app, go to the Job screen with no program loaded, and the program list's *Data* column was a sliver — narrow enough to clip its own heading mid-letter — while the rest of the list sat empty to its right. Nudging a column divider snapped it out to the full width, where it belonged all along.

- **The column is sized as "whatever space is left over"**, and that share is worked out by spreading the available width across the list's *rows*. With no program loaded there are no rows, so the calculation has nothing to spread across and never runs. The widths therefore stayed as they were first worked out — during the very first layout, before the list had been given any width at all, when the only sensible answer was "as narrow as possible".
- **A leftover-space column is never checked against its own heading**, either — it means "a share of what remains", not "at least as wide as what it says". With nothing holding it up it collapsed below the width of the word *Data*. It now has a floor that keeps the heading readable no matter what else happens.
- **While the list is empty the width is now worked out directly** rather than waiting for rows that are not coming, and it is recalculated when the window is resized. The moment a program is loaded the column goes back to taking its natural share.
- The same recalculation now also runs when a program arrives by a route other than opening a file — Work Order handing its generated program to the Job tab, or a wizard supplying its own — which previously skipped it entirely.

No column has been given a fixed size: Data still takes the remaining width and grows with the window.

The cause was measured, not deduced. Two confident explanations — an unbounded measurement from a parent container, and a simple ordering problem — were each written, tried and disproved first; logging the actual widths at each level showed the list knew its size perfectly well and the columns had simply never been asked again.

#218 Portable core: CNC.Core runs without WPF, and builds on .NET 8

File	+	−
CNC Controls/CNC Controls/JobControl.xaml.cs	375	1433
CNC Core/CNC Core/JobRunner.cs	1775	10
CNC Controls/CNC Controls/MacroProcessor.cs	57	850
CNC Core/CNC Core/ControllerValidator.cs	892	0
CNC Core/CNC Core/MacroRunner.cs	871	0
CNC Controls/CNC Controls/ValidateProcessor.cs	54	798
CNC Core/CNC Core/Grbl.cs	54	373
CNC Core/CNC Core/GCodeProgram.cs	410	0
CNC Controls/CNC Controls/KeypressHandler.cs	0	385
138 further files across CNC Core, CNC Controls and ioSender XL	5934	1591
Locale/*/csv (7 locales)	105	0

The whole machine-facing engine — connection, streaming, macros, g-code, machine state — used to be welded to WPF. It referenced `Dispatcher` for thread marshalling, `Media3D` for geometry, WPF dialogs for prompts, and the Windows registry for stored secrets. Nothing about running a machine needs any of that. It is now a plain library with no UI framework in it at all, proven by compiling and running it on .NET 8.

- **Thread marshalling by contract, not by Dispatcher.** A portable marshaller replaces every `Dispatcher.Invoke`. One real bug fell out immediately: the ambient context inherited `ApplicationIdle` priority, so connecting hung behind idle work until something else happened to wake the queue.
- **The engine no longer calls the view.** Streaming, macros and run control moved out of the WPF controls into `StreamPump`, `MacroRunner`, `JobRunner` and `ControllerValidator` — the g-code program, the ATC macros and the controller filesystem client went with them.
- **Machine state has one owner.** `MachineState` holds what the controller reports, instead of it living as scattered fields on a view model.
- **A wire contract exists.** `CNC.Contracts` carries the machine-facing enums and snapshot/delta messages; `CNC.Client` mirrors machine state from them. The Machine Mirror window is the first consumer that reads state purely through that contract.
- **Platform ties cut.** Registry secrets, WMI port enumeration and the hard-link P/Invoke became host-supplied services; the websocket transport moved onto the framework's own `ClientWebSocket`.

Groundwork only — the sender behaves exactly as before. Nothing here changes how the app is used.

#219 Work Order runs in one press: one streaming engine for macros and programs

File	+	−
CNC Core/CNC Core/MacroRunner.cs	163	556
CNC Core/CNC Core/StreamPump.cs	326	22
ioSender XL/ioSender XL/MainWindow.xaml.cs	18	209
CNC Controls/CNC Controls/MacroProcessor.cs	192	12
CNC Core/CNC Core/JobRunner.cs	99	15
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	51	63
ioSender XL/ioSender XL/StartJobView.xaml.cs	28	23
CNC Core/CNC Core/Comms.cs	25	6
CNC Core/CNC Core/GCodeJob.cs	22	2
7 further files across CNC Core, CNC Controls and ioSender XL	62	32

Macros and loaded programs used to run through two separate engines with two different ideas of what "wait" meant, which is why a Work Order needed a press for the setup macro and another for the carve. There is now one engine, and directives in the g-code tell it when to pause.

- **(WAITIDLE)** — a dispatch barrier: hold the next block until the controller is genuinely idle, rather than until the last block was acknowledged.
- **(PREREQ)** — a gate evaluated before a loaded program starts, so a run that cannot succeed is refused instead of started.
- **(MBOX)** and **(PROMPT)** — an operator prompt mid-stream, with the answer available to later blocks. Cancel is wired to a real abort so the stop byte is not suppressed.
- **Work Order is one press.** Setup and carve are one stream, so the race between them is gone by construction rather than by timing.
- **The old engine is deleted** — 736 lines, including the unsleeping message-pump loop that was the reproducible cause of the 32-bit out-of-memory crash.

#220 The run strip: everything a running job needs, and the bottom status bar retired

File	+	−
CNC Controls/CNC Controls/RunStripPanel.xaml.cs	584	76
CNC Controls/CNC Controls/RunStripPanel.xaml	313	52
ioSender XL/ioSender XL/MainWindow.xaml	148	195
ioSender XL/ioSender XL/MainWindow.xaml.cs	126	57
CNC Controls/CNC Controls/StatusControl.xaml.cs	68	0
ioSender XL/ioSender XL/JobView.xaml.cs	21	2
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	12	0
CNC Controls/CNC Controls/ConsoleWindow.xaml.cs	10	1
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
3 further files across CNC Core, CNC Controls and ioSender XL	9	1
Locale/*/csv (7 locales)	418	7

The main page grew by accretion: a run bar, a message strip, and a bottom status bar that repeated things already on screen. It is now one deliberate strip carrying what a running job actually needs, and the bottom status bar has been removed entirely.

- **A five-column right half** — Jogging, Signals and Overrides as real groups, with live feed and spindle values, a shared jog selection, spindle direction, and probe selection.
- **Run controls, DRO and MDI on the left**, with State and job elapsed time raised into view and the MDI console reachable without leaving the page.
- **Overrides read as a fill bar** behind the value rather than as a separate readout, and double-clicking one resets it — which previously never fired at all.
- **The connection summary moved to the menu bar**, and hides itself on a narrow window rather than squeezing the menu.

The bottom status bar is gone. Everything it showed is on the run strip or the menu bar; the Status button opens the full message history.

#221 Click the status line for every message since launch

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	178	6
CNC Core/CNC Core/GrblViewModel.cs	46	9
ioSender XL/ioSender XL/MainWindow.xaml	22	14
CNC Controls/CNC Controls/AppConfig.cs	10	2
Locale/*/csv (7 locales)	7	0

The status line shows one message at a time and overwrites freely, so anything that scrolled past was simply gone. Clicking it now opens the full history since launch.

- **The whole status line is the target**, not a small button on it, and Esc closes the window.
- **It pops itself when a message arrives** — with the message strip retired, a message nothing displays is a message lost.
- **An auto-popped window dismisses itself after 10 seconds**, so an unattended notice clears on its own. One you opened from the Status button never self-closes, and touching an auto-popped one keeps it open.
- **It remembers its size and position**, which matters far more for a window that shows itself repeatedly than for one you open on purpose.
- **Only messages the log records pop it** — not every internal change of the message property, which previously made it appear for things that were never shown.

#222 Text toolpaths: single-stroke engraving

File	+	−
CNC Core/CNC Core/StrokeFont.cs	365	4
CNC Controls/CNC Controls/WorkOrderCompiler.cs	129	10
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	114	16
CNC Controls/CNC Controls/CustomTool.cs	75	0
CNC Controls/CNC Controls/WorkOrderModel.cs	42	4
CNC Controls/CNC Controls/OddJobsFeedsSpeedsDialog.xaml.cs	39	3
CNC Controls/CNC Controls/CustomToolEditDialog.xaml.cs	15	1
CNC Controls/CNC Controls/OddJobsFeedsSpeedsDialog.xaml	10	4
CNC Controls/CNC Controls/WorkOrderView.xaml	11	0
2 further files across CNC Core, CNC Controls and ioSender XL	3	0
Locale/*/csv (7 locales)	91	0

A Work Order can now carry text as a toolpath, engraved with a single-stroke font — the letter is one pass down its centre line, not an outline to be cleared.

- **A stroke font in the core**, so the geometry is generated rather than depending on an installed typeface.
- **V-bit tools carry their included angle**, and an Engrave operation gets one — and can pick one — instead of silently inheriting an unsuitable tool.
- **Strokes are clamped to what the bit can cut** at its angle and depth.
- **Three real defects fixed on the way:** an Engrave operation could be dropped from the generated g-code entirely; the letter S was built from three strokes and carved as a mangled 5; and the operation diameter followed a stale copy of the tool rather than its definition.

#223 V-carving: TrueType outlines carved to shape

File	+	−
CNC Core/CNC Core/VCarve.cs	663	29
CNC Controls/CNC Controls/TrueTypeOutlines.cs	254	0
CNC Controls/CNC Controls/WorkOrderCompiler.cs	215	7
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	104	9
CNC Controls/CNC Controls/WorkOrderModel.cs	30	4
CNC Controls/CNC Controls/WorkOrderView.xaml	9	0
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
CNC Core/CNC Core/CNC Core.csproj	1	0
Locale/*/csv (7 locales)	28	0

Beyond single-stroke engraving, text can be V-carved: the glyph's real outline is carved so the stroke widens and narrows the way the typeface actually draws it.

- **TrueType outlines become toolpath polygons**, so any installed font is available.
- **A distance-field engine** converts the outline into depth-varying passes for the bit's included angle.
- **Carve-quality passes** — merged spine rings, simplified paths and flat clearing — instead of a naive offset ladder.
- **Each letter is cut completely before moving to the next**, rather than the whole job being traversed once per depth.
- **Two costly bugs fixed:** circles carved oversize because the nearest-boundary search stopped a ring early, and a carve spent its opening minutes tracing every letter outline at zero depth.
- **The distance field is memoized**, so regenerating a repeat board takes seconds instead of minutes — keyed so that position, WCS and feeds never invalidate it.

#224 Shape text: text fitted inside Work Order geometry

File	+	−
CNC Controls/CNC Controls/WorkOrderModel.cs	185	7
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	156	13
CNC Controls/CNC Controls/WorkOrderCompiler.cs	50	15
CNC Controls/CNC Controls/WorkOrderView.xaml	38	8
ioSender XL/ioSender XL/StartJobView.xaml.cs	5	0
Locale/*/csv (7 locales)	42	0

Any Line, Circle, Oval, Square or Rectangle toolpath can carry text fitted inside it. The shape still cuts as before; the text is engraved or V-carved within it.

- **The fit is computed, not guessed.** A cap height of zero auto-sizes to the largest that fits; an explicit size that does not fit is refused at Generate, by the same resolver the compiler uses, so a refusal and a cut can never disagree.
- **A line is a baseline** — text runs along it at its angle, sitting on it, hanging below it or straddling it.
- **Curved shapes fit against the curve**, not the bounding box, and are centred only — sliding a block of text inside a curve would void the fit guarantee.
- **The stock preview renders the fitted text live** at its real size and placement as it is typed.
- **Text toolpaths arrive with their Engrave operation** already attached, and the material is set on the panel rather than hunted for elsewhere.

#225 SAFETY: Feed Hold could be starved for the whole length of a dense job

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	17	1

Controller replies were marshalled onto the UI at a higher dispatcher priority than user input. On a dense job the reply stream never emptied, so input never got a turn: a click on Feed Hold or a keypress could sit unprocessed until the job ended.

- **The symptom was indistinguishable from a hung application** — the window redrew, the DRO updated, and the button simply did nothing.
- **The fix is a priority change**, so input is dispatched ahead of routine comms traffic.

This is a safety path: the whole point of Feed Hold is that it works during the densest part of a job, which is exactly when it did not.

#226 Safety batch: envelope, coordinate frames, and a Validate run that crashed a machine

File	+	−
CNC Core/CNC Core/ControllerValidator.cs	90	195
CNC Controls/CNC Controls/ValidateProcessor.cs	69	79
ioSender XL/ioSender XL/StartJobView.xaml.cs	107	3
CNC Core/CNC Core/MacroRunner.cs	89	1
CNC Controls/CNC Controls/MacroProcessor.cs	5	0

Four independent defects, each of which could put the machine somewhere it should not go.

- **A job whose G28/G30 park lies outside the soft-limit envelope is refused** rather than run into a limit.
- **Work origins are stored in the tool-length reference frame**, not the probe's — storing them in the probe's frame meant an offset that was wrong by the tool length.
- **Validate ran in the wrong coordinate frame** for its visualisation job, and alarmed instantly on a machine homed to its minimum. Its "passed moves" program has been removed outright: it crashed a machine.
- **Setup no longer finishes in G49.** Leaving tool-length compensation cancelled at the end of a setup run cut a spoilboard.

#227 The link watchdog stops killing live operations

File	+	−
CNC Core/CNC Core/LinkMonitor.cs	147	7
CNC Core/CNC Core/YModem.cs	68	24
CNC Core/CNC Core/Grbl.cs	39	3
CNC Core/CNC Core/Comms.cs	12	0
CNC Core/CNC Core/EltimaStream.cs	3	0
CNC Core/CNC Core/TelnetStream.cs	2	0
CNC Core/CNC Core/SerialStream.cs	2	0
CNC Core/CNC Core/WebsocketStream.cs	2	0
CNC Core/CNC Core/CNC Core.csproj	1	0

The link watchdog treated any silence from the controller as a dead connection. Two operations legitimately go quiet, and both were being aborted mid-flight.

- **grblHAL is silent for the whole homing cycle** — the watchdog was ending homing runs on a machine that was working perfectly.
- **A YModem firmware upload was killed the same way**, and polling the controller in the middle of a YModem packet corrupted the transfer independently of that.
- **A genuinely dead link is still detected**, and now by a signal that actually means something: a half-open socket reports itself as connected forever because that flag reflects the last I/O, not the live state, so the watchdog counts received bytes against polls sent instead.

#228 Jogging: a cancelled jog no longer wedges the jog path

File	+	−
CNC Core/CNC Core/JobRunner.cs	72	0
CNC Core/CNC Core/JogGate.cs	44	0
CNC Controls/CNC Controls/JogUiConfigControl.xaml.cs	15	0
CNC Controls/CNC Controls/JogUiConfigControl.xaml	13	1
CNC Core/CNC Core/GrblViewModel.cs	5	0
Locale/*/csv (7 locales)	80	0

A jog cancel landing within a few milliseconds of the jog itself makes grblHAL discard the command with no reply at all — no ok, no error. The sender was waiting for that reply before sending anything else, so every later jog queued up behind it and jogging stayed dead until the app was restarted.

- **The wait is released** when an unanswered jog is outstanding, the controller reports idle, and enough time has passed for a reply to have arrived.
- **Stale jogs are purged, never replayed.** This mattered more than the wedge: the queue held a continuous move whose cancel had gone out minutes earlier, so draining it would have produced a large unrequested Z motion with nothing left to stop it.
- **A status report also reopens the separate jog gate**, which had the same underlying firmware behaviour to contend with.
- **Continuous jog is now a setting** under Jogging, rather than implied by how a control was used.

#229 Large programs load in seconds; the console stops starving the UI

File	+	−
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	58	0
CNC Core/CNC Core/Grbl.cs	36	6
CNC Controls/CNC Controls/GCode.cs	21	2
CNC Core/CNC Core/HelperClasses.cs	15	2
CNC Controls/CNC Controls/ConsoleControl.xaml	7	8

Two separate performance defects that both presented as "the application has stopped responding".

- **Property-change notifications were marshalled synchronously** for every row of a program being built off-thread, whether or not anything was listening. Skipping the marshal when there is no listener took a 220,000-line load from 32 seconds to under 3.
- **The console rebuilt its entire scrollback on every response line**, re-joining thousands of lines per line received. The visible symptom was the console still echoing minutes after the wire had gone quiet — a dispatcher backlog, not a slow controller.
- **Loading a large program says what it is doing** rather than appearing hung, and a stack overflow loading one is fixed — a marshal guard was re-raising its own event and recursing.

#230 Generate says what it is doing, and what it will cost

File	+	−
CNC Core/CNC Core/GCodeRunTime.cs	255	11
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	244	6
CNC Core/CNC Core/GCodeProgram.cs	100	0
CNC Core/CNC Core/GCodeRunTimeIndex.cs	58	0
CNC Controls/CNC Controls/Converters.cs	20	0
CNC Controls/CNC Controls/GCodeListControl.xaml	17	0
CNC Controls/CNC Controls/MacroProcessor.cs	13	2
CNC Core/CNC Core/MacroRunner.cs	9	2
CNC Core/CNC Core/GCodeEmulator.cs	11	0
5 further files across CNC Core, CNC Controls and ioSender XL	31	1
Locale/*/csv (7 locales)	7	0

Generate could take minutes on a V-carve with no indication it was working, and once it finished there was nothing to say how long the resulting job would take.

- **Generate reports that it is compiling**, with a wait cursor and the time it took.
- **An estimated run time** is computed for any loaded program, not only a generated one, and shown per toolpath on each outline group header as well as for the job as a whole.
- **Compile statistics survive the "ready — press Cycle Start" prompt** instead of being cleared by it.
- **A Work Order's compiled output is cached** and restored at boot, so returning to a job does not pay the compile again.

#231 Probing and Setup: say what to fit, and stop parking over the spoilboard

File	+	−
macros/pcorner.macro	74	152
ioSender XL/ioSender XL/StartJobView.xaml.cs	90	57
CNC Controls/CNC Controls/ProbeDefinition.cs	37	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	13	7
CNC Controls/CNC Controls/FixtureEditDialog.xaml	8	8
CNC Controls/CNC Controls/SDCardView.xaml.cs	11	2
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	5	7
CNC Core/CNC Core/GrblSDCard.cs	7	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml	3	3
4 further files across CNC Core, CNC Controls and ioSender XL	8	4
Locale/*/csv (7 locales)	63	63

The probing prompts assumed you already knew which probe they meant, and the corner routine had grown a second path that behaved differently from the first.

- **Prompts name the probe and its tip diameter** when asking for it, and the install prompt says what to actually go and fit.
- **The fixture dialog stopped telling you to park over the spoilboard** in its Set/Test position guidance.
- **All four corners build the same program.** The corner macro's discovery path is deleted — every call supplies its own floor, so there is one path through the probe rather than two that could disagree.
- **The probe macros print every input before they move**, so what the macro believes is visible before anything is committed to motion.
- **ATC macros install from Machine Setup** without needing the SD Card window opened first, and a successful install is reported as success rather than as an error.

#232 Fix batch: crashes, startup, and the connect path

File	+	−
CNC Controls/CNC Controls/AppDialogs.cs	65	8
ioSender XL/ioSender XL/App.xaml.cs	58	15
ioSender XL/ioSender XL/CrashReporter.cs	41	5
CNC Controls/CNC Controls/RunStripPanel.xaml.cs	34	10
CNC Core/CNC Core/Grbl.cs	23	5
CNC Core/CNC Core/LogFile.cs	15	2

Four defects around the least forgiving moments in the app's life: starting up, connecting, and crashing.

- **The crash log survives the crash it is recording.** The reporter could itself fail while writing, losing exactly the report that mattered — and the first fix for that introduced two more defects, found by the next crash.
- **Reading settings on connect could crash** — the settings arrived into a list that was being enumerated at the same time, because the marshal that delivered them is asynchronous.
- **A dialog raised during shutdown answers its default** instead of crashing, with the catch filtered on the shutdown flag so genuine dialog bugs still fail loudly.
- **A startup crash from reading configuration before it was loaded** is fixed.

#233 Fix batch: windows, dialogs and small corrections

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	192	12
CNC Controls/CNC Controls/WorkOrderModel.cs	114	5
CNC Controls/CNC Controls/AppDialogs.cs	56	8
ioSender XL/ioSender XL/MainWindow.xaml.cs	58	3
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	41	6
CNC Controls/CNC Controls/Converters.cs	25	19
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	41	1
CNC Controls/CNC Controls/MacroProcessor.cs	13	27
CNC Core/CNC Core/GrblSDCard.cs	36	0
17 further files across CNC Core, CNC Controls and ioSender XL	211	38
Locale/*/csv (7 locales)	77	17

Accumulated small defects, each individually minor.

- **Offsets:** a work origin is set with G10 L20 and the tooltips say so; the DRO tooltip shows the sum and stops naming tool-length offset on axes that do not carry it.
- **Windows:** closing a menu-hosted window returns focus to ioSender rather than to another application; the camera window is resizable and sized to its content, which had been clipping the use-as-probe button.
- **Work Orders:** a title right-click menu for Rename, Open Copy, Close and Delete; Bamboo added to the feeds-and-speeds chart and the material lists.
- **Program view:** lines are marked complete when executed rather than when acknowledged; the source highlight is re-marked when rows appear; a g-code explanation balloon could be left on screen indefinitely.
- **Correctness:** the keyboard spindle coarse override pair was inverted; the controller filesystem is never enumerated mid-stream, which produced a hard error that killed a run; Reset could not clear an error message; loading a file containing #-expressions failed.
- **The Rewind button is removed** — the mechanism is kept, the button was a trap.

#234 SAFETY: a parser gap deleted lines from the program and crashed the machine

File	+	−
CNC Core/CNC Core/GCodeJob.cs	75	16

Loading a program decided what the controller receives by asking whether **ioSender's own parser** understood each line. Anything it could not model, outside a short list of blessed exceptions, was discarded - no error, no log, nothing on the wire. A Setup run lost four lines, one of them a **G59.3** coordinate-system change. The **G0 Z0** that followed then ran in G54, where Z0 was the work origin rather than the toolsetter's top of travel: a 128 mm rapid straight down into the touch plate.

- **The default is inverted.** A block the parser cannot make sense of is now streamed verbatim. The controller is the authority on valid g-code - a line it rejects returns an error the sender already surfaces and the run stops, which is loud and attributable. A line silently omitted is a different program from the one the operator approved, and it fails as motion.
- **One exception, and it is not stylistic.** The realtime characters **!**, **~** and **?** are acted on by grblHAL the instant they appear anywhere in the stream. Passing those through would turn a stray character in a file into a feed hold mid-cut or an unexpected cycle start. They stay out, and are logged when dropped.
- **Nothing is discarded in silence any more.** Every block that fails to parse, and every block lost to an exception, is written to the g-code debug channel.
- **Parsing is for the preview, not for transmission.** The parser exists to draw the 3D view and compute the bounding box. A gap in the drawing code has no business editing the program.

Verified on the simulator: the toolsetter is probed at G59.3 and the corner at the fence, in that order - before the fix both probes went to the fence. Hardware-verified the same day on a full Setup plus work-order run.

#235 A connection whose capabilities were never read

File	+	−
ioSender XL/ioSender XL/JobView.xaml.cs	125	12
CNC Core/CNC Core/Grbl.cs	44	2
ioSender XL/ioSender XL/MainWindow.xaml.cs	16	4
ioSender XL/ioSender XL/StartJobView.xaml.cs	10	1

On some startups ioSender announced **Unexpected response received from controller: ok** and offered to exit - about a controller that had just answered a perfect status report. The same fault, on the runs where no dialog appeared, left the sender believing the firmware had no capabilities at all.

- **The root cause: the handshake read the wrong line.** It returned the comms layer's last-received line rather than the response its own wait had matched. That field is assigned on the read thread for every line the controller sends, so the app's own follow-up acks overwrote the status report with **ok** in the few statements before it was read back. The handshake now keeps what it matched.
- **Which is also why $I was never sent.** A failed handshake skips the rest of initialisation, so the capability query never ran and GrblInfo stayed empty - and empty does not read as unknown anywhere, it reads as every capability ABSENT. That is how a controller reporting NGC expression support was told it lacked it, with expressions, ATC, probing and the filesystem all quietly refusing.
- **Fail closed if it ever happens again.** A connection whose capabilities were not read is now refused outright rather than carried on blind. Choosing to stay holds the machine in check mode - parsed and acknowledged, never executed - until a connect actually reads them, and the lock is re-asserted if a reset clears it.
- **Reconnect now actually reconnects,** and the capability warnings are recomputed when the controller answers instead of being frozen at the moment a tab was first shown.

Root-caused from a captured wire log rather than inference; verified across five restarts over both the network and the simulator with no recurrence.

#236 Setup hands the program to the Job tab at Generate

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	279	39
CNC Controls/CNC Controls/MacroProcessor.cs	73	6
CNC Core/CNC Core/JobRunner.cs	21	0

Setup used to switch to the Job tab at the moment the run started, so the program appeared at the same instant the machine began moving. It now follows the Work Order shape: **Generate** makes the program the loaded job and takes the operator there to look at it, and **Run** streams it from where they already are.

- **Setup keeps the run bar across the switch.** Everything that makes a probing program safe to stream - the expression-support refusal, comment sanitising, the macro arming that skips the dry-run Z shift, the borrow-and-restore of the previous job, and the height-map pass that follows a clean run - lives on the Setup run path, so ownership is retained rather than handed over.
- **The previous job is borrowed exactly once.** Loading at Generate and running afterwards would otherwise push a second snapshot onto the stack and double the watchers; the run now reloads in place instead. A program that is generated and then dropped without ever running hands the previous job back.
- **A run that declines to start says why.** Prerequisites unmet, a cancelled confirmation, a busy machine - each previously left the operator watching a Job tab where nothing happened. Every refusal is now reported and logged.

Verified on hardware: Generate lands on the Job tab with the program drawn, and the run streams unchanged.

#237 3D view: the stock block tells the truth

File	+	−
CNC GCodeViewer/CNC GCodeViewer/CarveView.xaml.cs	300	49
CNC GCodeViewer/CNC GCodeViewer/IToolpathView.cs	36	0
CNC Core/CNC Core/GCodeProgramComments.cs	24	2
CNC Controls/CNC Controls/WorkOrderCompiler.cs	17	0
ioSender XL/ioSender XL/JobWorkspace.xaml.cs	9	3
CNC Core/CNC Core/GCodeEmulator.cs	8	1
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	8	0
CNC GCodeViewer/CNC GCodeViewer/CarveView.xaml	6	0
CNC GCodeViewer/CNC GCodeViewer/RenderControl.xaml.cs	4	1
Locale/*/csv (7 locales)	7	0

A 6.35 mm board was drawn about 80 mm thick. The declared stock size was being read correctly - it simply was not what decided the block, which took the lower of the declared thickness and the program's bounding box. That box grows from every move the program makes.

- **Only a cut may be deeper than the board.** The bounding box counts rapids, the machine-coordinate preamble every work order opens with, and expanded park moves - none of which remove material. The block now takes its depth from cutting moves alone, so a through-cut still pokes out the bottom and nothing else does.
- **A probe is not a cut.** Probing moves are reported as feed moves because that is how they draw, which meant a Setup program - whose deepest motion is a toolsetter probe - was drawing an 83 mm slab. They are now marked as probes and excluded.
- **No invented boards.** Where the size was unknown the view used to draw a 150x150 stand-in, or a 19 mm thickness under a real footprint - a size nobody supplied and indistinguishable on screen from one that was declared. It now says **No stock size information** and draws nothing.
- **The declared board is drawn where it actually is,** from the program's own stock comment or, failing that, from the Setup tab - and the work order compile cache notices when that size changes.
- **The carve clears when the program does,** through an interface that cannot silently miss it, so a finished job never leaves a carved surface for the next one to preview onto.

#238 Job tab: optional split screen

File	+	−
ioSender XL/ioSender XL/JobWorkspace.xaml.cs	105	3
ioSender XL/ioSender XL/JobWorkspace.xaml	25	2
CNC Controls/CNC Controls/AppConfig.cs	16	0
ioSender XL/ioSender XL/JobView.xaml	9	1
CNC Controls/CNC Controls/UiGeneralConfigControl.xaml	1	0
Locale/*/csv (7 locales, 2 files each)	21	0

The Job tab can now show the program listing and the 3D view at the same time rather than one at a time, with a draggable splitter between them. Off by default; turned on in Settings, and the split position is remembered.

#239 V-carve Generate: 144 s down to 19.6 s, byte-identical output

File	+	−
CNC Core/CNC Core/VCarve.cs	162	42
CNC Controls/CNC Controls/WorkOrderCompiler.cs	28	0

Generating a v-carved script font took over two minutes. The distance field it is built on was quadratic per ring and single-threaded.

- **Measured, not guessed.** Both initial theories about where the time went were wrong; the cell loop was split into its two calls and timed, and the answer was neither of them.
- **The output is byte-identical.** This is purely a speed change - the generated program is unchanged, which is the property that was checked before accepting it.

#240 MDI dispatch unified with the streaming pump

File	+	−
CNC Core/CNC Core/WirePacer.cs	361	8
CNC Core/CNC Core/MdiDispatcher.cs	216	0
CNC Core/CNC Core/StreamPump.cs	137	221
CNC Core/CNC Core/JobRunner.cs	23	127
CNC Core/CNC Core/Grbl.cs	3	1
docs/Architecture-MDI-Dispatch-Unification.md	201	2

Typed MDI commands went out through their own send path, with their own flow control, alongside the streaming pump that runs programs. The pacing logic is now a shared primitive and the separate MDI send path is gone.

- **Two behaviour changes are deliberate.** Both are recorded in the architecture note that ships with this change, along with the wire-level checks used to accept it.
- **Probing is the caller that matters,** since it drives the machine from typed commands; its behaviour was verified on the wire rather than by reading.

#241 Jogging: g-code is refused while a jog is outstanding

File	+	−
CNC Core/CNC Core/GrblViewModel.cs	36	1
CNC Core/CNC Core/JogGate.cs	25	0

A g-code command issued while a jog was still in flight was rejected by the controller with **error:9**, once per command, with nothing on screen to explain why. Such commands are now held off until the jog completes rather than sent to be refused.

#242 The simulator comes up as the machine actually is

File	+	−
CNC Controls/CNC Controls/MachineOffsets.cs	379	13
CNC Controls/CNC Controls/SimulatorManager.cs	19	0
CNC Controls/CNC Controls/AppConfig.cs	13	6
CNC Controls/CNC Controls/PortDialog.xaml.cs	13	6
ioSender XL/ioSender XL/JobView.xaml.cs	7	1

Rehearsing a job on the simulator only proves something if the simulated machine resembles the real one. The real machine's reference points are now captured whenever it is connected, and stamped onto the simulator whenever one is - exactly one of those happens per connection.

- **It boots homed when the machine is,** so the work envelope and soft limits behave as they do on iron rather than refusing motion from an unhomed state.
- **The geometry is written before the running check,** and the simulator is restarted for it, so a mirrored change actually takes effect instead of landing in a config the running instance already read.
- **One banner per launch.** The simulator setup file was accumulating a fresh banner on every connect.

#243 Controller filesystem: an unanswered query is unknown, not empty

File	+	−
CNC Core/CNC Core/GrblSDCard.cs	114	17
tools/gen-sim-macros.py	69	9

A filesystem listing that went unanswered - because the controller was busy homing, for instance - came back as an empty result, and empty was then reported as fact: **no ATC macros found** on a controller that had them. Three separate places made that same substitution.

- **Unknown is now distinct from empty,** and reported as such, so a timed-out query no longer becomes a confident negative.
- **A listing already in flight keeps checking whether a job has started,** rather than deciding once on entry - listing the filesystem mid-stream collides with the running job and returns error:9.
- **Generated macros match the file byte for byte,** so what is compared against the controller is what was actually written.

#244 An error is an answer: no more waiting 36 seconds for an ok that never comes

File	+	−
CNC Common/HelperClasses.cs	51	0
CNC Core/CNC Core/GrblSDCard.cs	43	19

The wait for a controller acknowledgement returned only on a literal **ok**. An **error** reply - which is just as final an answer - was ignored, and because the timeout was measured per message rather than per command, ordinary status polling reset it indefinitely. One such wait held for 36 seconds against a controller that had already answered.

- **An error now completes the wait,** and is reported as the outcome.
- **The timeout bounds the whole command,** so unrelated traffic cannot extend it.

#245 The app stops running out of memory during long macro runs

File	+	−
CNC Controls/CNC Controls/UiPump.cs	18	0

Several long-running operations keep the UI responsive by pumping messages in a tight loop while they wait. Unbounded, that loop allocates fast enough to reach the 32-bit address-space ceiling - the app died at around 3.2 GB more than once. The pump now yields between passes, which bounds the allocation rate without changing responsiveness.

Not yet soak-tested over a multi-hour session.

#246 Status pop-up: chosen dwell time, errors only, and focus handed back

File	+	−
CNC Controls/CNC Controls/WindowFocusReturn.cs	115	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	51	8
CNC Controls/CNC Controls/AppConfig.cs	11	0
CNC Controls/CNC Controls/ViewHostWindow.xaml.cs	6	26
CNC Controls/CNC Controls/UiGeneralConfigControl.xaml	6	0
ioSender XL/ioSender XL/App.xaml.cs	5	0
Locale/*/csv (7 locales)	14	0

Three corrections to windows that appear over the main one.

- **The pop-up status window's dwell time is a setting,** rather than a fixed interval that was too short for some operators and too long for others.
- **Only errors take the screen.** Routine messages no longer raise the window over whatever is being worked on.
- **Every pop-up hands focus back to ioSender when it closes,** so the keyboard does not end up pointed at a window that is no longer there.

#247 Configuration overlays — hand someone a config fragment and layer it on

File	+	−
CNC Core/CNC Core/ConfigOverlay.cs	255	0
CNC Controls/CNC Controls/AppConfig.cs	217	10
ioSender XL/ioSender XL/MainWindow.xaml.cs	169	1
ioSender XL/ioSender XL/MainWindow.xaml	6	1
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	4	1
Locale/*/csv (7 locales)	42	0

Settings load only fills a section a profile does *not* already have, so an upgraded install keeps its saved `Layout` for ever and a new screen arrangement can never reach an existing user. An overlay is how you hand someone one specific change.

- **No new format.** An overlay is a trimmed `App.config` — the same `<AppConfig version="2">` root with a subset of its `<section>` entries. You can make one by deleting sections from a working config, and every section type works for free, including keys this build does not own.
- **Two merge modes.** `replace` (default) supplants a section wholesale; `merge` grafts only the child elements the fragment names onto what is already there, so an overlay can flip three fields without carrying all eighty.
- **Help > Support > Apply / Export / Undo.** Apply names the sections it will change *before* doing it, backs the profile up and restarts; Undo restores that backup. Export writes the current screen layout as a `.ioconfig`, scrubbing the port, network host, machine and firmware build so it is safe to hand to someone else.
- **Applying is staged, never written live.** Dozens of call sites save the config during a session, so anything written to `App.config` while the app runs is liable to be overwritten by the in-memory state moments later. Apply drops the fragment next to the config and restarts; the merge happens at load, where the file is authoritative.
- **`-overlay <file>`** is the same merge for one run only, with the session's writes redirected to a side file so the real profile cannot be touched.

Engine verified against real configs: an exported layout scrubs all four machine-identity fields, applying it to the shipped template changes only the Layout section and leaves the other fourteen byte-identical, merge mode replaces one element of 140 in place, and a legacy v1 document is refused as both source and target rather than half-applied. Applying an overlay was confirmed on screen.

#248 The shipped default is the full tab bar again

File	+	−
`CNC Controls/CNC Controls/LayoutModel.cs`	26	16
`ioSender XL/ioSender XL/Default-App.config`	20	21

The 2026-08-03 cutback moved ten views off the tab strip into the File and Tools menus and made that the shipped default. That is the wrong way round now that overlays exist (#247): a saved `Layout` is never overwritten by the template, so an existing install keeps whatever it has — a crowded default that can be slimmed down with one overlay beats a slim default nobody can opt out of.

- **Tab bar:** Settings, Feeds and Speeds, Start Job, Job, Offsets, SD Card, Work Order, Machine Setup, Lathe Wizards — the pre-cutback order.
- **The Tools *container* tab does not come back.** The Tools menu replaced it and is kept: Probing, Height Map, Tool Table, Trinamic and PID live there. Probing and Height Map came off the default bar in July when Start Job's Dynamic mode folded their functionality in, so restoring the old bar does not bring those back either.
- **File menu is empty of views** — Machine Setup and Settings are tabs again.

Written in *both* places that decide it, because either alone is silently wrong: the layout tree and the flat tab list in the template, and the code-level default so “Reset to default” cannot disagree with what a fresh install gets. The menu slots are emptied rather than removed — a missing menu slot reads as “not migrated yet” and skips the placement invariants entirely.

Reviewed on screen against a fresh profile; tree and flat list list the same nine components in the same order.

#249 A first connect opens on the Serial tab, not Network

File	+	−
ioSender XL/ioSender XL/Default-App.config	3	3
CNC Controls/CNC Controls/PortDialog.xaml.cs	6	4
CNC Controls/CNC Controls/AppConfig.cs	5	1

The Connect dialog hardcoded the Network tab for a profile with no target, on the reasoning that most grblHAL controllers are networked and the Scan button lives there. In practice a first connect is far more likely to be a USB cable already plugged in, and Network is one click away.

- **A saved target still selects its own tab** — that logic is untouched, so only a first run (or a profile with nothing configured) is affected.
- **`Prefer network` now defaults off.** It is what probes the IP reported by `$I` after a serial connect and silently migrates the link to the network; leaving it on would have contradicted the tab default. An existing profile keeps whatever it saved.

Set in both places that state it — the shipped template and the code-level default — because a profile predating the element falls back to the field initializer, and the two disagreeing would give old and new profiles different behaviour.

Confirmed on a fresh profile: the dialog opens on Serial.

#250 Jog go-to buttons: a refusal is now visible, and the set-back applies at both ends

File	+	−
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	19	10
CNC Core/CNC Core/GrblViewModel.cs	9	2

Two separate defects in the go-to buttons, both found on the machine.

- **The refusal was invisible.** Each button declines unless the machine is homed, idle, has max travel set and soft limits on — they rapid to a machine coordinate. Each guard wrote its reason with a plain status message, whose setter clears the error flag; since the run strip lost its message line the only thing that displays an arriving message pops the log window *for a flagged message only*. So the reason went into a log nobody had open and the button looked broken. The eight refusals are now flagged, and the method that flags them is public with a widened contract: a command the UI refuses is as actionable as one the controller rejected.
- **The set-back applied at one end of each axis only.** With force-set-origin, the envelope clamp held the far end off by the clearance but pinned the home end to exactly zero — the limit switch itself. Caught from the wire log: a go-to-corner emitted a safe-Z retract straight to `Z0`. It was a no-op that run because Z was already home, which is why only this end had gone unnoticed. The same asymmetry put the centre button half a clearance off true middle.

Wire-log verified on the machine: correct far-end targets, and the Z retract now stands off by the homing pull-off. The invisible-refusal half was reported from the machine as "they don't work".

#251 A setting changed outside the settings pages now reaches the rest of the app

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	114	0
CNC Core/CNC Core/Grbl.cs	73	0
CNC Core/CNC Core/MdiDispatcher.cs	5	0

Reported as “Machine Setup is showing stale info”, and it was — but the page was faithfully rendering a cache that had itself stopped matching the controller. Three gaps, closed in order of how far the staleness reached.

- **An MDI write never updated the cached settings.** `$132=120` is acked by the controller and goes nowhere near a save or a `$$` reload, so nothing refreshed the copy. The dispatcher's ack path — the one place that sees a command and its reply together — now records an accepted `$n=value`, rebaselines it as controller-held and re-derives the cached projections. That last part is the one that mattered: max travel is derived from the same collection, and the jog envelope clamp and the 3D machine frame read *those*, so after an MDI change they were all clamping against the old number.
- **Nothing told a view the settings had changed.** A `SettingsReloaded` notification is raised on both the read path and the write path; Machine Setup subscribes while active and re-reads. It refuses to run over edits in progress — an event from the controller may not discard someone's typing — tested by an explicit edit flag, because a stale page makes the pending-change list non-empty by itself.
- **Opening Machine Setup re-reads $$.** The two above only catch a change this app saw go out; a pendant, another sender or a controller-side macro leaves no trace on the wire. Opening the page whose numbers get written back to the machine is the right moment to ask. It stands down when disconnected, mid-job, or when unsaved edits exist anywhere — the settings pages share that collection.

Not cosmetic: the page's Apply diffs the on-screen values against the live settings, so a stale entry comes back as a pending change and gets written to the machine. A travel envelope corrected from the MDI was one Apply away from being un-corrected.

The MDI parse is verified against the real commands in a session log — setting writes match, and jogs, queries, filesystem and reset commands are all ignored. The live refresh paths build clean but are not yet exercised against hardware.

#252 Moving the Job view into a menu quietly cost you the connect handshake

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	25	0
CNC Controls/CNC Controls/MainPageEditor.xaml.cs	25	10
ioSender XL/ioSender XL/MainWindow.xaml.cs	34	12

Every view on the tab bar can be sent to the File or Tools menu instead — that is the point of #210, and a keyboard shortcut follows its view wherever it lives. The Job view was offered the same choice, and it is the one view that cannot survive it.

Nine places in the code look the Job view up by asking the tab bar for it. A view that lives in a menu is not on the tab bar, so all nine got nothing back. Most did nothing at all in response; one closed the loaded file through it and would have thrown outright.

- **The silent ones were the damaging ones.** Four of those nine are the connect paths, and their whole job is to wake the Job view so it runs the handshake: `$I`, the settings load, parser state, the status poller. Skipping that looks exactly like a controller that never answered — no dialog, no log line, capabilities simply unread.
- **The invariant is now true instead of assumed.** Rather than teach a dozen call sites to open a window mid-connect, the Job view is no longer offered the File/Tools menu at all. It is not an ordinary destination: it boots the controller, owns the poller and carries the run controls.
- **A profile that already moved it is repaired on load**, in both places that decide the tab bar — the layout tree and the older flat tab list. Fixing only the tree undoes itself on the next launch, because the flat list is replayed over it.
- **If it ever goes missing again, the log says so.** The lookup is one helper now, and it records which caller came up empty rather than shrugging.

Verified: the Job row offers only Tab bar; connect, reconnect, simulator switch and network migration all still run the handshake.

#253 Forty waits on the controller that had no way to end

File	+	−
CNC Core/CNC Core/Comms.cs	44	0
CNC Core/CNC Core/Grbl.cs	49	95
CNC Controls Probing/CNC Controls Probing/ProbingViewModel.cs	28	45
CNC Controls Probing/CNC Controls Probing/Program.cs	8	20
CNC Controls/CNC Controls/AppConfig.cs	8	20
CNC Core/CNC Core/GrblSDCard.cs	4	10
CNC Controls/CNC Controls/SDCardView.xaml.cs	4	10
CNC Controls/CNC Controls/GrblConfigControl.xaml.cs	4	10
CNC Controls/CNC Controls/OffsetView.xaml.cs	4	10
CNC Controls/CNC Controls/UiPump.cs	6	0
ToolView / TrinamicView / ControllerValidator / JobRunner / Grbl Config App (5 files)	10	25

One idiom appears about forty times in this codebase: start a background thread to wait for the controller, then pump the UI until that thread reports a result. Nothing in it can end the wait if the result never arrives. An exception inside the thread dies there — unobserved, no crash log, nothing sets the flag — and the app pumps for ever. It stays responsive and ignores you completely, which is a good deal harder to work out than a crash. #245 fixed what that spin *costs*; this fixes that it need never stop.

- **The wait now owns its worker.** A single helper runs the work, pumps while waiting, and re-throws the worker's failure on the calling thread with its original stack, so it surfaces where it can be handled instead of becoming a hang. Thirty-nine call sites converted.
- **Two of them were not waiting on bad luck at all.** The two “wait for a work-offset update” helpers started their worker only when the status poller was running, but waited outside that check. With the poller off, no thread ever ran, so nothing could ever end the wait — a guaranteed hang, no exception required. Both now report “no update” instead.
- **No timeout, deliberately.** These wait on a controller that legitimately goes quiet for minutes — grblHAL says nothing for a whole homing cycle, a file upload runs long — and each operation already carries its own per-message timeout. A backstop short enough to be useful would abort work that was merely slow, which is the worse failure.
- **Two waits were left exactly as they were.** The filesystem listings already catch their own failures and report “did not answer”, which is a real answer there and must not become an error — reporting an unanswered query as “nothing found” is a bug this app has paid for before.

The four stream transports wait on comms state rather than a worker thread; same unboundedness, different cause, not addressed here.

#254 MDI and Status can be put on a key

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	21	0
CNC Controls/CNC Controls/ActionKeyBinder.cs	7	0
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	1	1

The run strip's two smallest buttons are the ones most worth reaching without the mouse. Both are now assignable under *Keyboard & Controller* > *Program*: **MDI** opens the console with the caret already in its input box, and **Status** opens the message history.

- **Each presses the actual button.** The shortcut invokes that button's own click handler rather than a copy of it, so there is one implementation and the key cannot drift from what clicking does — and it refuses while the button is disabled, so a key can never do what a click currently cannot.
- **They stay live during a run**, unlike the bindable menu commands, which the menu bar disables wholesale while a job is going. Reaching the MDI console and the message history is exactly what you want mid-job.
- **Unbound out of the box**, like every other hand-bound command here. Claiming a key uninvited collides with whatever you already use.

Verified: both bind, both fire, and both are refused while their button is disabled.

#255 F12 opens the MDI console, and the MDI input has its own text size

File	+	−
CNC Controls/CNC Controls/ConsoleControl.xaml	23	7
ioSender XL/ioSender XL/MainWindow.xaml.cs	19	31
CNC Controls/CNC Controls/AppConfig.cs	14	32
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	16	1
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	5	15
docs/manual/index.html	4	3
ioSender XL/ioSender XL/Default-App.config	1	2
Locale/*/csv (7 locales)	42	21

Two changes to the console.

- **`F12` now opens the MDI console instead of toggling it**, with the caret already in the input box, ready to type. `Esc` dismisses it. Same key, same window, and the one difference is how you put it away — which is the reason for the change: `Esc` is how every other floating window in ioSender is dismissed, and the console was the odd one out with a key that closed it only if you remembered it was also the key that opened it. The separate "Toggle console window" action is retired; F12 is simply the default for the MDI action (#254) now, so it is rebindable like any other, and its two leftover config keys drop out of your profile on the next save.
- **The MDI input line has its own text size**, defaulting to roughly double what it was. It was never tied to the log's size — it was simply fixed and not adjustable. The console toolbar now carries two steppers, *Log* for the scrollback and *MDI* for the input, and both persist. They are deliberately separate: the scrollback is a log you skim, the input is a line you type g-code into and read back before pressing Enter.

Retiring the toggle came within an inch of taking F1 context help, every tab-switch shortcut and keyboard jogging with it — the routine that parsed its key also installed the window-wide key handler. That hookup stayed.

#256 Searching the console marked one match, and often not even that

File	+	−
CNC Controls/CNC Controls/SearchHighlightAdorner.cs	146	0
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	72	1

The console's Search box found matches and counted them, but showed you almost nothing. Every match except the current one was unmarked by design, so the only way to reach them was to press F3 and watch the counter climb. And the current one was usually invisible too, for two further reasons.

- **Focus never leaves the search box** while you type, so the scrollback is never focused — and WPF will not reliably paint a selection in a text box that has never held focus.
- **The scrollback is rebuilt wholesale four times a second** while output arrives, and rebuilding it drops the selection outright. Any highlight was gone within a quarter of a second.
- **So the highlighting no longer goes through the selection at all.** Every match gets a soft fill, the current one a stronger fill and outline, drawn over the log — which ignores focus and survives the rebuild. Stepping with F3/F4 or the arrows moves the strong one, and the view still scrolls to it.
- **Match positions are recomputed after each rebuild.** Once the scrollback hits its cap it trims from the top, which shifts every position — stale ones would confidently highlight the wrong text, which is worse than highlighting none.
- **Not searching costs nothing.** Only matches on visible lines are drawn, so a query hitting thousands of lines still costs one screenful; with the box empty nothing is drawn and nothing is recomputed.

Verified in both the Console tab and the floating console window, with output arriving and across a scrollback trim.

#257 Settings search points at the match on the page, not just at the page

File	+	−
CNC Controls/CNC Controls/SettingsNavShell.xaml.cs	196	19
CNC Controls/CNC Controls/ElementHighlightAdorner.cs	125	20

Searching the settings list narrowed it to the right pages and stopped there. Opening one of them left you to find the word yourself — and the search also matches tooltip text, so a page could be a genuine hit with the word nowhere on screen at all. The tree explained that in words; explaining is not pointing.

- **Choosing a page now marks every control on it that matched** and scrolls the first into view. Captions, group headers, text boxes, checkbox and button labels, and tooltips all count.
- **A tooltip-only hit is drawn dashed** rather than solid, so you can tell at a glance that the reason is hidden and the control is worth hovering — instead of a highlight that looks simply wrong. Tooltip text itself is left exactly as written.
- **It scrolls to a hit you can read.** If a page has both kinds, the visible one wins the scroll; landing on an invisible match while the word sits in plain sight further down is the same confusion in smaller form.
- **A page that matched but could not be marked says so in the log** rather than showing nothing, which would be indistinguishable from the bug this fixes.

The $-setting search on the controller settings page is untouched; it already highlighted its matches.

Marks are drawn over the page rather than by restyling it, so no settings panel had to be touched, and none can lose the highlight to a background of its own.

#258 Controller ack waits are bounded — and a settings write that never got an answer now says so

File	+	−
CNC Core/CNC Core/Comms.cs	55	4
CNC Core/CNC Core/SerialStream.cs	16	21
CNC Core/CNC Core/TelnetStream.cs	19	17
CNC Core/CNC Core/WebsocketStream.cs	19	18
CNC Core/CNC Core/EltimaStream.cs	16	21
CNC Core/CNC Core/Grbl.cs	15	3
CNC Controls/CNC Controls/PIDLogView.xaml.cs	11	1
tools/delta-probe/Program.cs	2	4

Each of the four connection types (serial, Telnet, websocket, Eltima) waited for the controller to acknowledge a command by spinning on a state flag with no clock on it. If the answer never came the wait never ended, and since it ran on the UI thread the sender froze until it was killed and restarted. Two of those loops span with no message pump and no sleep at all, so on a Telnet or websocket link the same operation froze the window solid and saturated a CPU core, where the serial and Eltima paths merely waited — the same action behaving differently depending on how the machine was connected.

- **Every ack wait is now capped** at five seconds and pumps the UI on all four connection types. The cap sits deliberately above the shorter per-operation timeouts already in use, so it acts only as a last resort for a wait that would otherwise never end — not as a limit on a controller that is merely slow. Nothing that legitimately goes quiet for minutes, such as a homing cycle or a file upload, waits on it.
- **A settings write that gets no answer is now reported as an error** and the setting stays pending. Previously the code read back a shared "last reply" value that still held some earlier exchange's response, and reported *that* as the outcome — so a setting the controller may never have accepted was marked written and taken as the new baseline. This was unreachable while the wait was unbounded, because it hung rather than reaching the check.
- **The PID log plot** gives up cleanly instead of parsing a half-collected report.
- Eight of the eighteen wait loops had no callers at all and were removed rather than repaired.

The unbounded waits had been a known hazard for some time; an earlier fix capped two of them in the file-upload path and another reduced what the spinning cost, but neither addressed the loops that could never end.

#259 Corner reliefs — cut a pocket a square part will actually seat in

File	+	−
CNC Controls/CNC Controls/OddJobsGeometry.cs	38	1
CNC Controls/CNC Controls/WorkOrderCompiler.cs	22	5
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	18	0
CNC Controls/CNC Controls/WorkOrderModel.cs	10	0
CNC Controls/CNC Controls/WorkOrderView.xaml	6	0
Locale/*/csv (7 locales)	14	0

A round cutter cannot leave a square inside corner. Pocket a 50 × 30 recess for a 50 × 30 inlay and the part will not seat: each corner carries a radius equal to the cutter's, and the square corners of the part foul on it. The usual answer is a relief cut — a dogbone — that removes the material the corner needs, and it is now a checkbox.

- **Corner reliefs** appears on Square and Rect toolpaths (nothing else has corners). Ticked, the wall pass pokes out along each corner's diagonal until the cutter's circle passes through the true corner point, then comes straight back and carries on. Each wall gets a small round nick just under a third of the cutter radius deep — that is the trade, and it is why the box is off by default.
- **The relief is sized by the cutter, not the shape**, so the same rectangle roughed with a quarter-inch bit and finished with an eighth gets the right relief in each case.
- **It lands on whichever pass leaves the final wall.** Where a Side finish is present the pocket stands down and the finish pass cuts the reliefs, so a wall still holding stock for finishing is not cut away early.
- **Contour is untouched.** It is the external operation, cutting a part free rather than a recess into one; its corners are convex, a round cutter already reproduces them exactly, and there is nothing there to relieve.

Verified on a cut pocket, not only in the preview.

#260 A jog key jogs and a shortcut fires — wherever you are

File	+	−
CNC Controls/CNC Controls/GlobalKeys.cs	143	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	24	22
ioSenderJogging.md	14	1
ioSender XL/ioSender XL/JobView.xaml.cs	10	0
CNC Controls Probing/CNC Controls Probing/ProbingView.xaml.cs	8	0
CNC Controls/CNC Controls/JogFlyoutControl.xaml.cs	8	0
CNC Controls/CNC Controls/KeypressHandler.cs	6	1
CNC Controls/CNC Controls/CNC Controls.csproj	1	0

Whether a key did anything depended on which view was showing and which control happened to hold focus — a model no operator should have to carry in their head. Keyboard jogging was wired up inside four individual views, plus a forwarder that ran only while the Job tab was the current one, so the arrow keys were silently dead on every other tab. Keyboard shortcuts were dispatched from the main window alone, so they worked across the top-level tabs and stopped at the first dialog. Open Machine Setup or Fixture Definition — the two places you are most likely to want to nudge an axis while lining something up by eye — and neither jogging nor shortcuts responded at all.

There is now one rule: **a jog key jogs and a shortcut fires, unless you are typing.**

- **Every window, including dialogs** — and any dialog added later, with nothing to wire up for it. Jogging while a setup dialog is open is the case this was built for.
- **Suppressed only by an input field** — a text box, combo, list, slider or open menu, where an arrow key already means something. A shortcut carrying Ctrl or Alt still fires with the caret in a field, because it cannot be mistaken for typing; an unmodified one does not.
- **A key release always stops a jog**, whatever has focus by then. A continuous jog runs while the key is held, so if focus moved into a text box mid-jog — a click, a dialog opening — the release had to be honoured regardless. That is a moving machine, not a UI detail.
- **Shortcuts are no longer eaten while the controller is busy.** A separate fault swallowed *every* key whenever anything was queued for the controller, without ever checking whether it was a jog key — so shortcuts failed intermittently even on the tabs where they were supposed to work, seemingly at random. The hold-off now applies to jog keys only, which is all it was ever meant to cover.

Tab-switch and action shortcuts firing from inside a modal dialog is deliberate: consistency was judged worth more than the odd case of a shortcut acting behind an open dialog.

#261 Bore feed — compensated on a wall pass, left alone on a full-immersion helix

File	+	−
CNC Controls/CNC Controls/WorkOrderCompiler.cs	41	2

A feed rate on an arc governs the path the machine is told to follow — the centre of the tool. Where a bore is *following a wall*, the engaged edge sits out on a bigger circle and covers more ground per revolution than the centre does, so an uncompensated feed runs the edge faster than asked. That is real, and those passes are now compensated by the ratio between the two circles.

It is not true of every bore, and assuming it was cost a cutter. When the tool is wide enough to cover the hole centre — a 2.5 mm cutter in a 5 mm hole, say — the helix is not following a wall at all. It is cutting a full slot, centre to wall, 180° of engagement. In that regime the chip thickness is set by how far the tool *axis* advances per tooth, not by how fast the wall contact point sweeps round, and compensating the feed simply halves the chip.

A cutter taking a chip thinner than its own edge radius rubs instead of shearing. Measured on that exact bore at 12,000 rpm: an entered 550 mm/min became a programmed 275, an axis chip load of 0.023 mm against a 0.03–0.05 mm target. Fine powdery chips, chatter, and a 60 × 60 × 4 plate too hot to hold — roughly 1,400 J into the work against the ~110 J the metal removal actually needed. Ten times the energy, all of it friction. The cutter seized and broke on the second hole.

- **A wall-following pass is compensated**, so the feed you enter is the speed of the cutting edge — the number the Feeds and Speeds advisor recommends and the one an operator can reason about.
- **A full-immersion helix is emitted exactly as entered.** That is what a Bore always did before any of this, and it was never the thing overloading cutters.
- **Decided per pass.** A bore too wide for one orbit is cut as several: the centre-clearing helix is full immersion and the passes stepping out after it are wall passes, so within one operation the two rules can both apply.
- **Each helix states which rule it took** in a comment, with the feeds involved — so the next problem of this kind is diagnosable from the program instead of from the workpiece.

Upgrade note: the feed emitted by an existing Bore changes only where the cutter is narrower than the bore's path radius. A bore whose feed was dialled in by hand against the old behaviour is worth re-checking.

#262 SVG artwork as a Work Order toolpath — V-carve a logo the way you engrave text

File	+	−
CNC Controls/SvgOutlines.cs	373	0
CNC Controls/WorkOrderModel.cs	76	6
CNC Controls/WorkOrderView.xaml.cs	115	7
CNC Controls/WorkOrderCompiler.cs	36	1
CNC Controls/WorkOrderView.xaml	16	0
docs/Architecture-SVG-Import.md	197	0
tests/svg/*.svg (2 test files)	121	0

A toolpath can now be an **SVG file**. Choose *SVG artwork* as the geometry, browse to a `.svg`, set how wide the artwork should be, and it V-carves exactly the way lettering does — same depth field, same passes, same stock placement, same preview.

- **Nothing new under the hood.** The carve engine takes polygon contours and has never known whether a glyph or a logo produced them, so this is one new producer feeding the existing seam. Curves are flattened once, at the same tolerance the font path uses.
- **The size readout answers the real question.** It reports the artwork's ink extent at the width you chose — *"Artwork is 3.81:1. At 150 mm wide it cuts 150.0 x 39.3 mm, 42 outlines."* — while you are choosing the file, not at Generate. Width measures the *ink*, not the page, so padding around a logo does not count.
- **Rotation and Engrave come free.** The Angle field turns artwork about its anchor, and the Engrave operation is added automatically since it is the only one that applies.
- **An import it cannot fully read is REFUSED, with the reason named** — at Generate and again in the editor. Silently cutting the half it understood is the dangerous outcome: the preview looks plausible and the missing part is invisible until it is missing from the wood.
- **Holes come from containment nesting, not winding direction.** TrueType guarantees counters wind opposite their outline; SVG guarantees nothing, and a real vendor logo had 2 of its 42 subpaths wound against that assumption.

Verified by running it against two purpose-built test files and a real 23 KB vendor logo (42 contours, 30 outer / 12 holes) with an ASCII fill map to check the topology, not by a clean compile. Not yet done: preview rotation on the stock canvas is unrotated for anchoring, and locale rows for the new controls.

#263 The Status window is written to a log file, like the console

File	+	−
`CNC Core/StatusLog.cs`	92	0
`CNC Core/GrblViewModel.cs`	14	0
`ioSender XL/App.xaml.cs`	1	0

The Status window was memory-only. Its log held 1200 lines and dropped the *oldest* 200 on overflow — so the lines explaining how a session got into trouble were the first ones lost, and none of it survived a restart. The console log has been durable for a long time; this closes the gap for the other one.

- **Same scheme as the console log** — `status_<stamp>.log` in the day-of-week folder, with `latest_status.log` hard-linked in the logs root, and the same retention.
- **Each line carries its kind and source** — error/info, and `app` versus `firmware` for a line that arrived as a controller `[MSG: … ]`. So *"what did the machine actually say"* is one grep, which is the question this log gets opened to answer.
- Fed from the single funnel that already knows both, and written on a background thread — the funnel runs on the UI thread and inline file I/O there could freeze the UI during a burst.

Verified on the machine: the file appears at startup and carries firmware-tagged [MSG:] lines through a real run.

#264 A dry run's Z offset could outlive the job and silently shift every later cut

File	+	−
`CNC Core/JobRunner.cs`	115	1
`macros/tc.macro`	24	0

Stopping a dry run part-way left its Z-clearance offset on the controller **permanently**. Every job afterwards cut at the wrong height, and nothing said so — until hours later a tool change reached past the Z envelope and reported a soft-limit alarm that looked like a toolsetter fault.

- **The cleanup lost a race it did not know it was in.** It was sent as a queued G-code line; the Stop path then sends a *realtime* byte, which bypasses the queue and flushes the controller's receive buffer — discarding it 4 ms later. The give-away in the log is that no `ok` ever came back for it.
- **And it persisted.** With G92 persistence enabled (the default) grblHAL writes the offset to non-volatile storage, so the orphan survived every reset and power cycle.
- **The restore now waits for Idle** and is retried on each status report until it lands. Alarm is deliberately excluded — G-code is locked out there, and Alarm is exactly where a Stop leaves the machine.
- **A leftover offset is now named at Run**, per axis, before the program streams. A warning rather than a refusal: an offset can be deliberate, and the failure worth catching is the silent one.
- **The tool-change macro guards itself too.** Cancelling the old tool's length offset never touched this one, so the move over the toolsetter was not the machine move it read like. It now suspends and *restores* the offset rather than destroying it — a tool change runs mid-job, and a job may legitimately be using one.

Verified on the machine: stop a dry run part-way and the offset now reads zero once the controller settles. The tool-change guard was verified by a clean TLO set at the toolsetter after the alarm that exposed it.

#265 Three faults found running a real job: a crash after a probe fail, a refused fence, a truncated comment

File	+	−
CNC Controls/WorkOrderCompiler.cs	38	10
CNC Controls/AppConfig.cs	24	7
CNC Controls/Fixture.cs	20	3
CNC Core/JobRunner.cs	10	2
ioSender XL/StartJobView.xaml.cs	9	8
CNC Controls/FixtureEditDialog.xaml.cs	6	0
CNC Controls/StepperCalibrationProbeWizard.xaml.cs	4	2

Three unrelated faults, all found the same afternoon while setting up and running an actual job.

- **A probe failure could kill the application.** When a macro alarmed mid-probe its program was torn down while the controller was still reporting the line it stopped on, and the next status report indexed past the end of it. That threw out of a status-report handler, which is fatal. The streaming path already guarded exactly this one line lower down; this sibling had been missed.
- **Generate refused a fence that had just been probed successfully.** The check for "has this fixture's corner been located" inferred the answer from the offset being non-zero — but Test position parks the machine *at* the corner, so setting the reference from there makes an offset legitimately zero. One fence in the operator's own configuration sat at 0.012 mm and passed only by luck. Whether the corner was measured is now recorded explicitly instead of guessed from a value. The same check ran on every launch (despite living among the one-time fixups) and would have silently un-validated the fence on the next restart.
- **A work order comment truncated its own block and failed the run.** A toolpath description containing brackets ended the G-code comment early, and the remainder was parsed as G-code — rejected 12 seconds into a 32-minute job. The operator's text was already being sanitised; the compiler then appended a bracketed note of its own *after* that step. Everything entering a comment now passes through one place, replacing four scattered copies of the same guard — none of which were at the site that broke.

All three verified on the machine: no crash after a probe fail, the fence generates, and the comment now survives as one well-formed block.

#266 A stuck probe is named — and the check waits for the machine

File	+	−
`macros/pcorner.macro`	74	3

grblHAL refuses a `G38.2` whose probe is already triggered and reports a bare `ALARM:4`. That reached the operator as "*Warning: error 9 in named sub pcorner.macro*" plus a triple `error:9` cascade — three unrelated-looking failures naming nothing. Finding the real cause took a wire-log trace: the probe had latched on the X face and stayed asserted through a 10 mm retract, a Z lift and a diagonal traverse, so the next seek was refused before it started.

- **The corner probe now checks before each seek** and aborts with a sentence — "*Probe is still triggered before the Y face seek — it did not release. Check the probe, its wiring and $6/$I inversion.*" It reads `#<_probe_state>`, grblHAL's own `hal.probe.is_triggered`, and **fails open** when the driver cannot report it — a controller that cannot answer must not lose the ability to probe.
- **The check waits for the machine to arrive.** As first written the gate aborted *healthy* probes: grblHAL evaluates an o-word condition when the block is *parsed*, and none of `ngc_flowctrl.c`, `ngc_expr.c` or `ngc_params.c` drains the planner first. The latch probe ends with the tool sitting on the plate; the lift and the whole face approach are merely queued, so the parser reached the gate while the tip was still in contact. A `G4 P0` before each read flushes the planner at no time cost.
- **The face-probe descent backs off further** before dropping to depth. Every caller passed 10 mm and 10 was not enough — the margin is measured from the operator's saved reference, and that reference sat about 7 mm inside the true face.

Reported from a real touch-plate run: the operator was watching, nothing was in contact, and the gate fired anyway. Both firmware behaviours were read out of the grblHAL source rather than assumed, and the embedded macro was verified by reflecting the built assembly. Needs a macro re-upload to the controller, which the app does at Setup when the checksum changes.

#267 The carve cache served the wrong artwork

File	+	−
CNC Controls/WorkOrderCompiler.cs	86	17

V-carve passes are memoized because building them is by far the most expensive thing Generate does. The key was text, font, style, cap height, half angle and max depth — the full input set when text was the only carvable thing. Adding SVG as a second geometry source did not extend it.

- **Changing the artwork width and regenerating hit the cache** and reused the old passes — indistinguishable from the width field doing nothing, which is the fault fixed one entry earlier.
- **Pointing a toolpath at a different .svg carved the previous logo**, and two SVG toolpaths in one work order collided with each other.
- The cache is static and lives as long as the process, so restarting hid it — the worst shape a stale cache can take.

The key is now built from a geometry-source fragment that leads with the kind, so a text carve and an SVG carve cannot collide however their other fields line up, and then carries that kind's real inputs.

- **An SVG is identified by a SHA-256 of the file's bytes**, not its path or timestamp. Iterating on a logo means re-exporting over the same filename, and a restore from backup or any exporter that preserves timestamps changes content while path, length and modified time all hold still. Hashing a few hundred KB is free against a carve measured in minutes.
- **Artwork that cannot be fingerprinted is computed but not cached** — serving a possibly-stale hit is the one outcome worth refusing outright.

Verified by reflecting the built assembly and exercising the real key function: the width changes it, a different file changes it, and the same path with identical length and modification time but new bytes changes it — the case a timestamp check fails.

#268 A depth cap for V-carve, so a narrow bit can cut fine detail

File	+	−
CNC Controls/CustomTool.cs	41	0
CNC Controls/WorkOrderModel.cs	33	0
CNC Controls/WorkOrderView.xaml.cs	28	7
CNC Controls/WorkOrderCompiler.cs	13	12
CNC Controls/WorkOrderView.xaml	2	0
Locale/*/csv (7 locales)	70	0

A V-carve has no depth setting — depth is a consequence of each shape's own local width — and the only ceiling was derived from the tool: half its diameter over the tangent of its half angle. That is the deepest the bit *can* go, never a depth anyone chose, and it is far deeper than any artwork wants once the bit gets narrow.

Measured on a real logo at 100 mm wide, whose widest feature has an inscribed radius of 1.355 mm and whose tagline strokes are 0.5 mm with counters only 0.33 mm across:

- a 60° bit cuts those strokes 0.43 mm deep, and the widest feature 2.35 mm;
- a 15° bit cuts them **1.88 mm** deep — the difference between legible and mush — but drives the widest feature **10.29 mm** into the stock as a pure V, with a slender tip taking the whole depth. The derived clamp does not help: it sits at 12 mm for a 1/8 in bit at that angle.

- **New *Max carve depth* on the operation.** 0 means automatic — the old derived limit — so every saved work order cuts exactly as before.
- **A cap cannot touch anything already shallower than it**, because depth follows width. Capping that carve at 2.5 mm takes the deepest pass from 10.292 mm to 2.5 mm while the 396 passes at or below 1.9 mm — the tagline — come out byte-identical to the uncapped run. Only areas wide enough to want more flatten off and get cleared.
- **The bit's own limit stays a hard ceiling.** A cap may lower it, never raise it: past its cutting diameter the cone has run out and the shank would be doing the work. A request that had to be limited is stated rather than quietly cut shallower.
- **The editor and the compiler no longer disagree about what a V-carve is.** The editor asked whether the toolpath had a font, which is false for artwork, while the compiler carves every SVG unconditionally. So an SVG toolpath was offered a stroke-width field the compiler ignores, quoted a stroke plunge depth for a cut whose depth follows the artwork, and denied the new cap — on the one geometry that needs it most, since a logo is exactly where thick and hairline detail share a bit. Both sides now ask one property.

Verified by reflecting the built assembly rather than from a clean compile: the depth helper across automatic, honoured and clamped requests; the real carve engine capped against uncapped on the actual artwork; and the pass cache recomputing when the cap changes. The editor's own rendering of the field is not covered by that and was checked by hand.

#269 Duplicate quietly dropped every field added to a toolpath since it was written

File	+	−
`CNC Controls/CNC Controls/WorkOrderModel.cs`	20	2
`CNC Controls/CNC Controls/WorkOrderView.xaml.cs`	35	30

Duplicating an SVG toolpath produced one with **no artwork at all**. Both copy routines built their result from a hand-written list of fields, and between them seven were missing: the toolpath's `SvgFile`, `SvgWidth` and `EntireSpoilboard`, and the operation's `CarveMaxDepth`, `EngraveWidth`, `Direction` and `ToolName`.

Only the carve depth cap was noticed, and only because its value happened to differ from the default. Engrave width and cut direction reverted *invisibly*, because the values in use matched the defaults they fell back to. That is the character of this fault: a field nobody copied reads back as a plausible default rather than as an error, so it surfaces as a wrong cut hours later.

It had happened before. A comment sat mid-routine recording that the text block "*was missing here entirely — duplicating a Text toolpath used to come up with the TEXT defaults, silently*" — fixed by hand, then lost again as soon as new fields arrived.

- **The default is inverted** rather than the seven fields added. Every public field is now copied automatically, and the two that must *not* be copied verbatim carry an attribute stating their own reason next to the field — the name (a duplicate needs a unique one) and the operations list (copying it verbatim would share one list between both copies).
- Adding a field to a toolpath or an operation now needs no thought in the copy path at all.

#270 An Indirect toolpath's X/Y are absolute, or relative to the toolpath it copies

File	+	−
CNC Controls/CNC Controls/WorkOrderModel.cs	75	4
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	51	4
CNC Controls/CNC Controls/WorkOrderCompiler.cs	6	1
CNC Controls/CNC Controls/WorkOrderView.xaml	5	0
Locale/*/csv (7 locales)	14	0

An Indirect toolpath borrows its geometry from another one, so it has no shape of its own for an anchor to name a corner of. The *X/Y refers to* dropdown was nevertheless shown for it, fully editable, and did **nothing** — the half-extents it works from are zero for a toolpath with no geometry. A live control that silently changes nothing is worse than no control: it reads as a setting that did not take.

Replaced, for Indirect toolpaths only, with the choice that actually means something there — not *which* point of a shape the offsets name, but what they are measured **from**:

- **Absolute** — a position in the work order's coordinate system. What Indirect has always done, and still the default, so no saved work order moves.
- **Relative** — an offset from the source's own position, so moving the original moves the copy with it and a row of copies stays a row.

Position now has a single authority that both the editor and the compiler read, rather than each doing its own arithmetic. That was survivable while the answer was always "X/Y, unchanged"; it is not once there is a mode, and a preview that disagreed with the g-code about where something is would be the worst of both.

#271 Group toolpaths, and switch a whole set on or off together

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	207	41
CNC Controls/CNC Controls/WorkOrderModel.cs	42	0
CNC Controls/CNC Controls/WorkOrderView.xaml	6	0
Locale/*/csv (7 locales)	14	0

A toolpath can carry a **group** label. Toolpaths sharing one nest under a header row in the tree whose tick reads on when any member is on, and drives all of them when clicked — the same rule a toolpath already applies to its own operations.

Purely an authoring construct: nothing downstream of Generate learns that groups exist, so this cannot change any g-code. That is deliberate. A group that were atomic in the program would defeat the cross-toolpath tool grouping that keeps tool changes to a minimum.

- **Joining a group moves the toolpath** next to its groupmates rather than only relabelling it. The default program order *is* tree order, so a header drawn around toolpaths scattered through the run would draw a grouping the machine does not cut.
- Contiguity also means a run *is* the group, so the tree needs no second membership structure to keep in step with the first.
- A group may not take a toolpath's name: the two share one namespace when an Indirect toolpath picks its source, and the toolpath would always win, leaving the group permanently unreachable.

The tree previously paired its rows with the model *by position*. Group headers joined that level and broke the correspondence at the first row — a header wearing the first toolpath's summary, the toolpaths under it wearing that toolpath's operation summaries. Both walkers now match rows by identity at any depth, so a row's place in the tree need not mirror its place in the model.

#272 Point an Indirect toolpath at a group and it copies the whole set

File	+	−
CNC Controls/CNC Controls/WorkOrderModel.cs	256	42
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	243	144
CNC Controls/CNC Controls/WorkOrderCompiler.cs	123	71

An Indirect toolpath's source may now name a **group** as well as a single toolpath. The point is the part that cannot be got any other way: the copy is of whatever is in the group *at the time*, so adding a toolpath to the original adds it to every copy without going to find them. Five hand-maintained copies is exactly the kind of list that silently stops matching what it describes.

- **Positioned rigidly by the group's first member** — that member lands on the copy's X/Y and the rest keep their offsets from it. Not the bounding-box centre, which would need every member's true extents and would shift the whole copy whenever any single member was resized.
- **A tick per borrowed toolpath**, so one member can be held back in the copy without touching the original. Recorded by name, and absence means it runs — so a toolpath added to the group appears in the copy and cuts, and a renamed one leaves a harmless stale entry. The failure mode is only ever "did not hold something back", never "silently dropped geometry".
- **Groups hold leaf toolpaths only.** That one rule, enforced where membership is set, makes a group unable to contain a reference to itself however the references are arranged — no cycle detection anywhere.
- The copy's own tick is the **only** gate on its output. It cuts what the source *defines*, not what the source currently has ticked, so switching the originals off to cut only the copies does what it looks like it does.

Expansion is one function, and the compiler, the preview, the tree and validation all read it. Each had grown its own copy of "what does this borrow", and one of them reported every valid group reference as a missing toolpath, in red, beside a preview drawing it perfectly correctly.

#273 Burn an SVG on a diode laser, straight from File > Open

File	+	−
CNC Converters/SvgToLaser.cs	259	4
CNC Converters/SvgLaserSettings.cs	165	1
CNC Converters/SvgLaserFill.cs	153	0
CNC Converters/SvgLaserDialog.xaml	65	0
CNC Converters/SvgLaserDialog.xaml.cs	63	0
CNC Converters/CNC Converters.csproj	11	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	4	0
Locale/*/csv — Converters (7 locales)	182	0

Pick an `.svg` in File > Open and it becomes a laser program in the Job tab, streamed by the machinery already there — alongside the existing HPGL and Excellon importers.

Deliberately separate from the Work Order's SVG carving. That path emits grblHAL — expressions, named parameters, tool changes, tool-length offsets, Z per pass — and a diode laser controller is typically plain Grbl, which rejects every one of those. What the two share is the **geometry**: the same loader that carves artwork on the mill already produces closed contours in millimetres.

- **Outlines, or shaded.** Shading scans back and forth across the enclosed areas, leaving holes and counters unburned, then traces the boundary so the edge lands crisp over the fill.
- **The head's position when the job starts becomes the artwork's corner.** That is how a diode laser is actually set up — there is no fixture and often no homing, so an absolute coordinate means nothing. The program says so in its own comments, including that an aborted job leaves the temporary origin set.
- **Laser mode is checked, not assumed**, and the rapids carry a zero-power word anyway — the failure it guards against is a lit rapid dragging a scar across the work.
- Power is shown as a percentage of the controller's own maximum, because the same number is a light mark on one machine and full power on another.
- Artwork the importer cannot fully read is refused unless confirmed: a partial logo looks like a finished job until you notice a piece missing.

Verified end to end on an Elegoo Phecda: streaming holds one block in, one block out over a 256-byte receive buffer with no errors; a 40 mm square marks cleanly; the logo burns as line art and, shaded, with depth.

#274 Depth of cut now drives the V-carve step, instead of cutting each groove six times over

File	+	−
CNC Controls/CNC Controls/WorkOrderCompiler.cs	30	12
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	6	1
CNC Controls/CNC Controls/WorkOrderView.xaml	2	2
Locale/*/csv (7 locales)	7	7

Watching a carve run, the V-bit adds fine detail that the next, deeper pass then removes. That is exactly what was happening, and the geometry says why: consecutive carve passes tile the cone exactly — a deeper pass's flank runs through the *tip* of the one above — so in any region that reaches full depth, the deepest pass alone recreates the entire V profile.

The shallower levels are not geometry. They are depth-of-cut stepping, and the step was fixed at 0.5 mm whatever the bit or the material, so a 2.5 mm emblem always got six levels. Thin detail was never the cost — a hairline stroke only ever has one level, because no deeper contour exists there.

- The operation's own **Depth of cut** — which already exists and already means "axial step per pass" — now drives it, and the field is shown for carve operations.
- Existing carves take their stored value, so a 2.5 mm carve drops from six levels to two on the next Generate. Fewer, heavier passes: worth watching the first one, particularly on a fine-tipped bit.

The carve cache had to learn about the step in the same change. Its own comment already named the condition — the step was excluded only because it was a compile-time constant, and exposing it as a setting required it to join the key. Without that, changing the field and regenerating would have reused passes built at the old step, and the field would have looked inert.

#275 A disabled Generate now says why, and a 4.2 mm drill is a drill

File	+	−
CNC Controls/CNC Controls/WorkOrderModel.cs	62	5
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	20	2
CNC Controls/CNC Controls/JobControl.xaml.cs	14	1
CNC Controls/CNC Controls/MacroProcessor.cs	12	0

Two faults from one work order that would not build: a program that could not be generated, and a Generate button that would not say why.

- **The button.** Its enabled state came from one condition and its tooltip from another, so whenever a tab blocked generation the button greyed out wearing an *enabled* tooltip — hovered, it cheerfully described the run it was refusing to start. It now names the actual reason, published by whichever tab set the gate.
- **The drill size.** The list of standard sizes was a metric index — 1 to 13 mm in half-millimetre steps — plus imperial fractions. That is what a boxed set contains, and it omits every metric *tapping* drill, which are bought individually and are most of what a machine build uses. 4.2 mm is the M5 tapping size: five of the seven holes in the reported file were 4.2, each one refusing to generate, while the operator held the bit.

The tapping and close-clearance series are now recognised, each named for the thread it serves. More importantly, **the check no longer blocks**: it advises. That list is a list of sizes that usually exist and can never be complete — fractional, number and letter drills, and whatever a particular shop owns — and a rule whose premise is about the world outside the program has no business being the one that says no. Everything else still blocks.

#276 A fix could appear to do nothing: the program cache keyed on version, not build

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	17	0

After a change to the compiler, rebuilding and regenerating produced **the previous build's program**. Generate never ran: the cache fingerprint covered the work order, the stock and the controller settings — and the assembly *version*, which is identical across every development build. The stamp on the cached file still matched, so it was handed straight back.

A fix appears to do nothing. Worse, a line that had been *reverted* came back, because the file predating the revert still matched.

- The fingerprint now includes the module version identifier of the two assemblies that shape the output — a value the compiler stamps afresh on every build.
- Within one run of a released build these never change, so the cache still avoids the multi-minute carve rebuild it exists for. It invalidates across a rebuild, which is the one time it must.

The same method's comment already recorded this happening once before, when a cached program missing a stock declaration kept being restored after the compiler had started emitting one — "which read as *the fix did nothing*". That was repaired by adding the missing input to the key. The input still missing was the code itself.

#277 The connect handshake could read the wrong reply, and the controller's settings came back empty

File	+	−
CNC Core/CNC Core/Grbl.cs	88	0
CNC Core/CNC Core/MacroRunner.cs	46	3

The connect sequence asks the controller several questions in a row — the alarm codes, the error codes, the setting descriptions, then the settings themselves. Each wait ends on the first literal `ok` it sees, and each one subscribes *before* sending its own command. So when the previous reply is still streaming, a wait takes **that** command's `ok` and returns having collected nothing of its own, and the whole handshake runs one command behind itself.

```
09:52:34.053  > $EE            sent while $EA's replies are still arriving
09:52:34.056  < ok             $EA's ok - the error-code read takes it and returns
09:52:34.061  < [ERRORCODE:0||]        its real answer, now unwatched
09:52:34.064  > $ES            the settings read fires into it
09:52:34.099  < ok             $EE's ok - ends the settings-detail wait
09:52:34.099  < [SETTING:0|...]        its real answer, ONE LINE too late
```

The settings collection came out **empty** while the load reported success. Nothing downstream could tell, because none of the accessors can express "not loaded": a lookup miss reads as −1, a missing number parses as 0, and the cached travel limits derive from those.

- The stored-position check skipped every code — it reads `$20`, which answered −1.
- The oversized-program check skipped every axis, because the travel read as 0.
- The jog soft-limit clamp and Go To bounds read soft limits as switched off.
- Setup generated `#<_bottom> = -9999` as a probe floor from that zero travel, the controller-side corner macro turned it into `G38.2 Z-9998`, and the job stopped on `ALARM:2`.

Each connect query now waits for the link to fall quiet before it speaks, so no wait can be listening while someone else's reply is still arriving.

Pure timing, which is why it came and went: 90 ms between two of those queries collected 104 settings, 11 ms collected none — same build, same machine, minutes apart. Possible since the chained query sequence landed upstream in December 2021. This does not make the underlying wait correct — it still cannot tell whose `ok` it is reading — it removes the condition that lets it read the wrong one. Soft limits caught the `-9999` probe; with them off it would have been a full-depth plunge.

#278 A stored WCS origin is checked against the travel envelope before the job starts

File	+	−
CNC Core/CNC Core/MacroRunner.cs	17	5

A generated program declares what it needs before it runs — homed, a tool-length reference, `G30`, `G59.3`. Each of those was checked for being *set*. For the stored machine positions `G28` and `G30` it was also checked for being *reachable*, because a position recorded when the pull-off was smaller stays in the controller at a coordinate the parser will now refuse.

The work coordinate systems were deliberately left out of that second check, on the reasoning that `G54`…`G59.3` are offsets rather than targets. Selecting one and moving to its origin makes it a target, which is exactly what the tool-change macro does:

```
G59.3
G0 X0 Y0        → ALARM:2
```

With the Y envelope running to −834 mm and `G59.3` stored at −837.996, the move was refused at planning — after the program had parked at `G30` and prompted for the touch plate. The check now covers the work coordinate systems too, so it is caught before anything moves:

G59.3 is stored outside the machine's soft-limit travel: Y −837.996 is 3.996 mm beyond the limit of −834. Move the machine inside its travel and set G59.3 again.

No false alarms: this only runs for a coordinate system a program explicitly names as a prerequisite, and naming one is declaring it will go there. `G92` stays out — it is a transient offset applied on top of the active system, not somewhere anything rapids to.

#279 A G53 move kept its axis words — the 3D view and the travel check were mixing frames

File	+	−
CNC Core/CNC Core/GCodeEmulator.cs	25	1
CNC Core/CNC Core/GCodeParser.cs	22	7
CNC Core/CNC Core/JobRunner.cs	11	0

`G53 G0 X… Y…` — the park line every generated program ends with — was parsed into **two** tokens: a `G53` marker carrying no axis words at all, and an ordinary `G0` carrying the machine coordinates with nothing left to mark them as machine coordinates.

Only the `G53` case converts those out of the machine frame. Handed a token with no axes it converted nothing, and the real motion arrived as a plain rapid that was taken for work coordinates. Machine points then landed in the program's bounding box beside work-frame ones:

```
box  X 0..742.589   Y -820.001..38   (span 858.001)
"Y: the program spans 858 mm, the machine travels 845 mm"
```

858.001 is 38 — a work-frame Y — minus −820.001, which is `G30`'s *machine* Y. The program held exactly two distinct Y values, 6.000 and 38.000.

- The `G53` token now keeps its axis words whether or not the block also restates `G0`/`G1`, and the modal-motion flag is cleared so the fall-through cannot emit a second, axis-less move for the same block.
- The same mis-framing drew park moves in the wrong place in the 3D view, for every program that parks.

Parsing only — the controller is streamed the raw lines and never saw this. Introduced upstream in May 2021; before then the case was a single line that always passed the axis words along.

#280 The status log now records what actually happened: connects, jobs, generates and setting changes

File	+	−
CNC Controls/CNC Controls/LibStrings.xaml	1	1
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	21	0
CNC Core/CNC Core/Grbl.cs	52	0
CNC Core/CNC Core/GrblViewModel.cs	27	1
CNC Core/CNC Core/JobRunner.cs	32	0
ioSender XL/ioSender XL/JobView.xaml	1	1
ioSender XL/ioSender XL/JobView.xaml.cs	38	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	50	15
Locale/*/csv (3 locales)	3	3

A connect was one line that read the same whether the handshake had read everything or nothing. A run left no mark at all. A settings change left none either. Everything that would have explained a later symptom lived in a developer-only log the operator has no access to.

```
Connecting to controller (192.168.1.247:23)...
Connected: 192.168.1.247:23
- 104 controller settings read
- grblHAL 1.1f.20260712 build 20260712 - options: ENUMS,RT+,HOME,PROBES=3,EXPR,ATC=1,SD
Generate for LeftZRailSpacer.workorder
Work order compiled - 77 lines in 0.5 s, est. run 00:02:14.
Program start - LeftZRailSpacer.workorder, 77 lines
Program end - LeftZRailSpacer.workorder, runtime 00:01:23
Disconnected from controller (192.168.1.247:23) - reason: reconnecting
```

- **Connect** reports how many settings were read — *0* is the failure above, and it now says so in plain words — the firmware's options, and whether the controller is already sitting in an alarm. That last one blocks g-code *and* filesystem access, which otherwise reads as "the ATC macros are missing".
- **Disconnect** states its reason: reconnecting, switching to the simulator, restoring afterwards, migrating from serial to the network, or the application closing.
- **Runs** log start with the file and line count, end with the runtime, and mark themselves `[DRY RUN]` or `[CHECK MODE]` — a dry run and a real one were indistinguishable afterwards. A run stopped by the operator or aborted by a reset is recorded as an error, with the elapsed time.
- **Generate** names the work order it was for, on the cached path too.
- **Refusals** are recorded — prerequisites unmet, or a program larger than the travel. These were dialogs that left no trace, so "why did it not start" was unanswerable afterwards.
- **Controller settings** log one line per change with old and new value, including a `$n=` typed at the MDI, which bypasses the settings editor entirely and otherwise leaves no record anywhere.

"Waiting for controller" is now "Connecting to controller" — it describes what is happening rather than what the sender is doing about it, and no longer reads as a stall. The controller's own terse `Pgm End` is filtered now that the end of a run is reported properly.

#281 A connect announced the previous connect's result, and skipped its own report

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	12	1
ioSender XL/ioSender XL/JobView.xaml.cs	6	1

```
Connecting to controller (COM6:115200,N,8,1)...
Connected: 192.168.1.247:23
```

The connect routine opened by saving whatever status line happened to be on screen, and the caller treats that saved text as "a message the connect set for itself" — giving it precedence over the connect's own outcome. So "nothing to report" was represented by the *previous* message, and every connect after the first re-announced it.

Worse than the wrong target: taking that branch skipped the outcome branch, and with it the settings count, the firmware options and the already-in-alarm warning. The one situation where "how many settings did this connect read" matters most — a reconnect after trouble — is the one that silently omitted it. The first connect of a session looked right only because the status line happens to start empty.

It now starts empty each time, so only a message the connect deliberately sets — the homing-required notice — takes precedence. An outcome that is neither success nor no-response states itself rather than assigning nothing, since silence there is the "connected or not?" ambiguity the block exists to close.

#282 Menu commands get their own keyboard group, apart from the tab destinations

File	+	−
CNC Controls/CNC Controls/ActionKeyBinder.cs	29	16
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
CNC Controls/CNC Controls/Features.cs	34	0
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	31	10
ioSender XL/ioSender XL/MainWindow.xaml.cs	21	15

The built-in menu commands shared the *Top Level Tabs* group with the views, on the reasoning that both are just destinations. In practice it read as clutter: a list of tabs you can rearrange, with fifteen menu commands you cannot, interleaved among them.

- A new **Top Level Menu Items** group holds `Connect`, the `File >` and `Help >` commands, ordered immediately above the tabs — adjacent because they are related, separate because a tab can be moved on and off the strip and a File command cannot.
- Both group descriptions were corrected; the old one claimed the tabs group contained the menus.
- Existing bindings are unaffected — the action ids did not change, only the group they display under.

Also adds a small feature-flag holder so a command held back from a build disappears from the menu, its shortcut registration *and* its row in Settings > Keyboard together. Hiding the menu item alone had left the command listed and bindable to a key that would then do nothing.

#283 Height Map: probe the board, read the program before it moves, and see every reading

File	+	−
ioSender XL/HeightMapView.xaml.cs	427	27
ioSender XL/HeightMapView.xaml	73	35
Locale/*/csv (7 locales)	56	0

A probing run that drives a tool the length of the Z axis is worth reading before it starts. The generated program now goes through `MacroProcessor.PublishGenerated` — the same seam Start Job publishes its Setup program through — so it appears where a generated program is already looked for, and is saved as a generated copy alongside the others.

- **The listing is a rendering, not the literal program.** Three of the engine's lines are markers rather than g-code: a status message, a hold, and the latch retract whose captured position is discarded. Handing those to a g-code viewer gets them dropped or misparsed — the worst failure available in a preview, because the operator reads it as *this is what will run*. They are rendered as comments that say what they do; everything else passes through untouched.
- **The right-hand pane follows the run** — Steps, Program, Surface Map, in the order the run goes through them. Start switches to Program at the moment the program is built, because that is when what is about to be sent stops being hypothetical; a run that *completes* switches to Surface Map. A run that fell short leaves Program up, which is where the failure is legible.
- **Published into the page, not the main window's overlay.** Height Map is menu-hostable, and the overlay renders on the main window — behind the window the operator is actually looking at. The generated copy on disk was proof it had worked; the screen showed nothing.
- **Every probed height reaches the status log**, not just the range. The range answers how much has to come off; it cannot answer *where* it is high, which is the question you have the moment a surfacing pass leaves something behind — and a 3D view cannot be read to a hundredth. Each point logs its grid cell, its table position, the board height there, and how far it sits from the first point, with the touch plate already taken off so the numbers agree with the Z0 the run sets.
- **The selected probe's own parameters are used.** The tab carried its own probe depth, feed and settle dwell and overrode the probe with them, so a probe set up in Machine Setup was ignored here. Forcing the latch distance to zero is why a touch produced no retract and no second pass — the tab had disabled the probe's fast-search-then-slow-retouch, not the probe. Only the XY offset is still forced to zero, and that stays: it is an edge finder's tip offset, and a height map probes straight down on the centreline.
- **Full work surface sets the origin itself** rather than requiring one. Mapping the envelope in whatever work frame was already set needed an origin, which meant Setup, which for a spoilboard means probing a corner that sits too close to the limits to probe. There is nothing to find — the board *is* the envelope — so X0 Y0 goes on the table corner before probing and Z0 is set afterwards to the highest point probed. Grid size is stated as divisions here, not millimetres: over a whole table the useful thing to say is "sample it four by four". It asks before writing the origin, and waits for the offset report before reading the area back.
- **Full work surface holds at each point**, because MDF will not close a circuit and the touch plate has to travel with the probe. Without the hold the run rapids to point 2 while the plate is still at point 1 and drives the full probe depth into bare board that can never make contact. A Continue button came with it — a hold with no way out is a run that stops and cannot be resumed from the app at all.

Both the highest and the lowest point are reported: a cutting plane only flattens the board where it sits below the existing surface, so the range is the difference between skimming the high spots and cleaning up the whole table. Run on real hardware across sixteen-point and full-table surveys.

#284 Every probe on a height map aimed somewhere it could not reach

File	+	−
ioSender XL/HeightMapView.xaml.cs	205	46
ioSender XL/HeightMapView.xaml	7	3
CNC Controls Probing/Program.cs	7	0
Locale/*/csv (7 locales)	14	0

Four separate faults in how far each probe travelled, found in the order they could be reached — each one had to be fixed before the next became visible.

- **The first probe aimed at the soft limit.** Its search distance was the axis *travel* (135 mm here) while the probe starts at machine Z0, so `G38.3 Z-135` targeted the far limit itself, which grblHAL rejects as readily as a target past it. The full travel is the one distance guaranteed unusable from the top of that travel. It is now the drop from the start height to the safe floor.
- **A 4×4 grid produced 20 probes.** The view model couples the two grid sizes while its Lock is on — setting Y writes X too — so assigning X spacing then Y spacing left both holding the Y value, and the wider X axis silently gained a point. The lock is a convenience for typing one number into two millimetre fields; for a spacing derived separately per axis it is simply wrong. Cleared on the engine's copy only, so the operator's own setting is untouched.
- **Later probes searched the whole way down.** After point 1 triggered near Z−96.8 and lifted 15 mm, point 2 asked for the probe definition's full 50 mm — targeting Z−131.8, below the usable floor once grblHAL's homing pull-off is reserved. The first probe's target had cleared that floor by a millimetre, which is why point 1 always worked. A later probe never needed 50 mm: the surface height is known, so it only has to cover the lift plus however far the board falls away between two points.
- **The retract ate the search.** The lift between points was the probe drop less 3 mm — sensible against a 5 mm "probe depth", nonsense against a real probe definition: with a 50 mm search it lifted 47 mm and then searched 50, leaving three millimetres of margin for the whole board's variation plus wherever the plate is set down. Point 15 of 16 ran its entire search and touched nothing; fourteen points had passed on luck. The lift is a *clearance* — enough to slide a plate under the tool — so it is now a small constant capped against the search, not derived from it.

The drop allowance between the lift and the search is now an **input, defaulting to 5 mm**: it is the one number here that depends on the board rather than the machine, and a chewed-up spoilboard wants more of it than a flat table. Floored at 0.5 mm rather than accepted as typed — the search is the lift plus this, so an allowance of zero is a probe that stops exactly where the last point triggered and finds nothing the moment the surface falls away. The field states its own consequence underneath: what the tool lifts, what it then searches, and how much lower a point may be and still be found.

A failed run also left the controller in **G91 relative mode** with nothing on screen saying so — the distance-mode restore was going through an ungated dispatch and being dropped on precisely the runs that need it. The next hand-typed move would have gone somewhere nobody intended.

Instrumentation came with the first fix, because both of the early faults were invisible from outside: every point now logs its target in the work frame, the distance it will search and the retract after it, and the run logs its whole geometry up front. A serpentine of relative increments is unreadable after the fact, and "which point, going where, searching how far" is the first question any probing alarm asks. All four confirmed against wire logs on real hardware.

#285 The Surface Map had been empty since it was written, and its colours meant a different height on every board

File	+	−
ioSender XL/HeightMapView.xaml.cs	186	15
ioSender XL/HeightMapView.xaml	42	4
CNC Controls/HeightMapConfig.cs	51	0
CNC Controls Probing/HeightMap.cs	12	4
CNC Controls/AppConfig.cs	4	0
CNC Controls/CNC Controls.csproj	1	0
Locale/*/csv (7 locales)	7	0

The bindings could never have worked. Every `Visual3D` in the viewport bound with `RelativeSource FindAncestor`, which walks the FrameworkElement tree — and a `Visual3D` is not in it. The bindings failed silently, the mesh never reached the visual, the scene had no bounds, and zoom-to-extents correctly did nothing with them. The tab has therefore been empty since this view was written; moving the viewport into a tab only made it obvious, because a permanently visible empty 3D view reads as "nothing probed yet". The Probing tab's copy of the same viewport binds through its `DataContext` and renders, which is the whole difference between them.

- **Nothing was ever pointed at the surface.** The zoom-when-loaded hook fires when the *viewport* loads — once, at startup, long before any surface exists. It now frames when the surface is built *and* when its tab is opened, because a tab's content is not measured until it is shown, and zooming to the extents of something with no size does nothing at all.
- **And again once the scene has actually landed.** The mesh's binding is only evaluated when the tab's content enters the visual tree, which on the first showing happens in the same layout pass as the framing call. So it zooms a second time at idle priority, unconditionally rather than trying to detect which case applies — it costs one camera move.
- **The colour scale was wrong.** The mesh's texture coordinate *multiplied* the height by the map's range where it should have divided, so the ramp depended on the *square* of how flat the board is. A 0.837 mm board reached only 0.70 of the ramp — blue through orange, never red or purple; a 2 mm board would saturate over most of its area and a 0.1 mm board come out uniformly blue. The same colour meant a different height on every board, which is the one thing a colour scale must not do. Normalised now: blue is always the lowest point probed, purple the highest.
- **A legend says what the ramp is worth in millimetres.** On a board flat to under a millimetre the full spread of colour is genuinely alarming until the numbers show it covers 0.8 mm. Labelled as depth *below the high point* — the high point is where the cutter touches first and the rest is how much further it has to go. The bar reuses the same gradient stops in the same order as the mesh material, so the two cannot drift into disagreeing about what a colour means.
- **The page's settings persist, including the Area radio.** Divisions and the drop allowance were fields on the view; Area came from the XAML on every build and so reverted on every visit. That combination is worse than forgetting everything: it teaches you the page remembers, then quietly resets the one setting that changes what the run does — whether the grid covers the table and sets the origin from it, or trusts an origin already there.

Two rounds of instrumentation to get here, and each was needed: the first established that geometry existed and the viewport was measured, which killed the obvious explanations; the second logged the camera either side of the zoom, and two identical readings of its XAML position said the scene had no bounds at all — which is neither a camera problem nor a timing problem, the two things already tried.

#286 A height map run you can stop, resume, and not get stuck inside

File	+	−
ioSender XL/HeightMapView.xaml.cs	382	129
CNC Controls/AppDialogs.cs	23	0
ioSender XL/HeightMapView.xaml	5	4
Locale/*/csv (7 locales)	14	0

A run that dies at point 15 of 16 has fifteen good readings in it. Throwing them away and asking for the plate to be placed fifteen more times is the difference between a recoverable interruption and an afternoon — and completing sixteen consecutive probes without a stumble is exactly what has been hard.

- **Retry clears the alarm, lifts Z, positions *absolutely* over the point that stopped it, and continues.** Absolute positioning is what makes the resume safe: the relative chain that carries the run from point to point is precisely what an interruption breaks, so it is not relied on to get back. The resumed first probe searches long for the same reason — the height it would have started from is no longer known.
- **Offered only when the resume can be trusted.** Readings must exist, the grid must still be the one they belong to, and the alarm must be one grblHAL retains position through — soft limit, the probe-fail pair, safety door — never a hard limit, a reset while moving, or a homing fault, where the readings already taken refer to an origin the machine can no longer find. Merging across that would build one map out of two coordinate systems, and it would look entirely plausible. A fresh Start discards kept readings: they belong to the run that was interrupted, not to a new survey.
- **The serpentine point order is now defined once** and used by the builder, the resume and the read-back alike. It was written out as a nested loop in two places, which is a standing invitation for them to disagree about which reading belongs in which cell — and a transposed map looks fine right up until it cuts.
- **Start locks during a run, and Stop works throughout it.** Both were bound to the probing engine's own "executing" flag, so through the setup before that — writing the work origin, parking at machine Z0, traversing to the first point — Start could be pressed again to stack a second run on the first, while Stop could not be pressed at all. That window is exactly when the machine is making its longest unattended moves, which is when Stop is most wanted. The primary (blue) treatment now moves with the run: Start while idle, Continue while holding.
- **Writing an origin that was already right was read as a failure.** Full work surface aborted after a four-second pause with "the controller did not report the new work offset", having moved nothing: the origin was already where the run wanted it, so writing it again changed nothing, so no notification fired. The guard was watching for an *event* when the question was about a *value*. It now checks whether the origin already matches, and gates on the offset's actual value either way — which answers correctly whether the write changed something, changed it before the wait was armed, or had nothing to change. Same family as the connect handshake reading a shared "last received" field (#277): a notification is not an answer to a question you asked.
- **A dialog nobody dismissed kept the app busy indefinitely.** A cancelled run left ioSender refusing to shut down long after the machine had gone idle. It was not a job: the map builder raised its own short-capture warning, which is modal, and the call that clears the running flag sat behind it — so the run stayed "running" for exactly as long as a message box went unanswered. The warning is handed back now, and the caller ends the run first and talks second.

The ordering that did matter is unchanged: the map is still built before the run is ended, because ending it unsubscribes the probe handlers and could drop the final captured point. Retry exercised on real hardware against a soft-limit abort mid-grid.

#287 The step after a probe was silently dropped, and the run waited for ever

File	+	−
CNC Controls Probing/Program.cs	30	0

Every full-surface height map stopped dead on the first touch. The probe succeeded, the controller acked, and the next command vanished:

```
13:40:14.286  < [PRB:6.995,-780.000,-96.844:1]
13:40:14.286  < ok
13:40:14.290  PM:ok
13:40:14.290  [jobrunner] SendCommand DROPPED "G0Z2" -
                        streamingState=Send is not in the allowed set
```

The job runner drops g-code, silently and without queueing, while the streaming state is `Send`. This engine dispatches its next step the instant the previous `ok` arrives — four milliseconds here — which for a long move is before the streaming state has finished transitioning back. So the command was never sent: no traffic, no error, and a run left waiting for a reply to something that had never gone out.

- **It then resumed on the next unrelated `ok`.** On the previous attempt that was the operator's own jog — so the run carried on from the wrong step, which is a far worse outcome than the stall that preceded it.
- **The engine now waits for the dispatcher to accept commands before handing one over.** Deliberately not by widening the allowed set: that gate keeps typed commands out of an active stream, which is correct. This engine is simply faster off the ack than the state machine is, so the race is ours and the wait belongs here.
- **Bounded and pumping**, so a state that never clears cannot freeze the UI.

Diagnosed from the wire log and the job-runner trace together — neither alone showed it, because a command that is dropped before transmission is invisible from the wire and a command that is never sent produces no error.

#288 The assert-the-probe dialog is gone — it asserted water is wet

File	+	−
CNC Controls Probing/ProbeVerify.xaml.cs	0	93
CNC Controls Probing/ProbeVerify.xaml	0	14
CNC Controls Probing/ProbingViewModel.cs	9	17
CNC Controls Probing/*Control.xaml.cs (7 probing controls)	1	22
CNC Controls Probing/LibStrings.xaml	0	2
CNC Controls Probing/ConfigControl.xaml	0	1
CNC Controls/AppConfig.cs	0	2
ioSender XL/HeightMapView.xaml.cs	0	5
Locale/*/csv (7 locales)	0	35

Every probing operation stopped to demand the operator trigger the probe by hand before it would run. That confirms nothing anyone did not already know: you are about to probe, so the probe is fitted.

- **It was not even free to satisfy.** The check captured its result *before* showing the dialog, so having asserted the probe you got “Press [Start] again to start probing”. That was deliberate rather than a slip, but a second button press to get past a question that should not have been asked is not a better bargain.
- **On a touch plate the ritual is genuinely awkward** — asserting it means holding the plate against the bit, and the height map now parks at the top of travel before its first probe, so it was asking the operator to reach up there.
- **A probe that does not work still reports itself**, and reports something real: the probing action fails to make contact. That is a signal. A dialog the operator satisfies by hand before every run is a ritual, and the two are not substitutes.
- **Removed rather than defaulted off.** There was already a “Manually validate probe is connected” setting, which made it worse — a toggle nobody should have to find, for a question nobody should be asked. Gone with it, along with the window, the view-model plumbing, and the 35 locale rows they left behind across the seven CSVs.

If connection validation is ever wanted back, it belongs where the probe is defined rather than in front of every run.

#289 The spoilboard is not the travel envelope, and a cutter went there

File	+	−
CNC Controls/WorkSurface.cs	134	0
CNC Controls/MachineSetupWizard.xaml.cs	73	0
CNC Controls/MachineSetupWizard.xaml	30	0
ioSender XL/HeightMapView.xaml.cs	14	40
CNC Controls/WorkOrderCompiler.cs	9	3
CNC Controls/AppConfig.cs	4	0
CNC Controls/CNC Controls.csproj	1	0
Locale/*/csv (7 locales)	70	0

Two features asked the *machine* how far it could travel and called the answer the spoilboard: the Height Map tab's Full work surface and the Work Order's Entire Spoilboard toolpath both derived their extent from `$130`/`$131`.

That is only true when the board fills the table. This machine mounts the toolsetter off the front of the board — `G59.3` sits at Y−838 while the board stops near Y−780 — so the last stretch of Y travel is air with the toolsetter standing in it. The height map merely wasted probes out there. **Entire Spoilboard ran its raster across the same strip with the cutter turning.**

- **The board's extent is now its own machine fact** — a Work surface section of the configuration, entered in Machine Setup — and both callers read it. Undefined means "the board fills the table", which is exactly what they assumed before, so a machine where that holds needs no setup and behaves identically.
- **Deliberately not expressed by shrinking the travel limits**, which was the obvious shortcut: the gantry genuinely can reach Y−838 and has to keep being allowed to, or the tool-change macro can never drive to the toolsetter at all. Travel describes the gantry; this describes the board.
- **The extent is clamped to the safe travel envelope rather than trusted.** A number typed too large, or carried over from another machine, must not become permission to drive into a limit switch. Machine Setup states the clamped result back rather than echoing the input, so a value that will not be used cannot look like one that will.
- **One copy of the envelope arithmetic.** Both callers had grown their own, which is part of how they came to disagree; there is now one, next to the thing that overrides it.

Found on real hardware, after the fact, by noticing where the raster had been.

#290 Spoilboard surfacing proves the depth on one plunge before committing the table

File	+	−
CNC Controls/WorkOrderCompiler.cs	108	6
ioSender XL/HeightMapView.xaml.cs	44	0
CNC Controls/HeightMapConfig.cs	18	1
CNC Controls/WorkOrderModel.cs	13	1
CNC Controls/WorkOrderView.xaml.cs	10	1
CNC Controls/WorkOrderView.xaml	3	1
Locale/*/csv (7 locales, 2 files each)	28	0

Surfacing commits the whole board to a number arrived at by measurement and arithmetic, and the first physical evidence that the number is right normally arrives when the cutter is already several hundred millimetres into the work.

- **One plunge makes that measurable beforehand.** The tool cuts to depth once, dwells, lifts to Z0 and holds. Put a rule in the pocket, then OK to surface the board or Cancel to stop with nothing cut but the pocket. The hold is non-modal on purpose — the machine stands still with the prompt up and jogging stays live, so the gantry can be moved away to look at the pocket and brought back before answering.
- **At the board's high point, not wherever the raster begins.** That was the least informative place it could be: at a low point a shallow pass can miss the board entirely, so the pocket proves nothing either way — and "nothing was cut" reads identically to "the depth is wrong". The high point is where the cutter takes the most material and where Z0 was defined, so the pocket measures exactly what the pass will remove. The height map records that point when a surface is built *or* loaded, since a map read back from a file describes the board as well as one probed a minute ago — and it is bounds-checked rather than trusted, so a stale value is harmless instead of a plunge somewhere nobody chose.
- **This is not the height map being applied to the toolpath**, which for flattening would be exactly wrong — it would follow the dips and reproduce them. It decides where to *look*, not where to cut. The cut stays a flat plane.
- **The test plunge is capped at 300 mm/min.** It first plunged at the operation's own plunge feed, described as "a slow cut rather than a bang" — on a real surfacing setup that feed is 7920 mm/min, and even clamped to the machine's Z maximum it is about 76 mm/s. A wrong depth at that rate is an impact, not the observable cut the test is supposed to be. The justification had come before the arithmetic.
- **Entire Spoilboard can cut on the origin already set.** It is self-contained by design — park machine-referenced, jog to touch, anchor a scratch WCS — which is right when nothing is set up. After Full work surface has run it is actively harmful: it replaces a Z0 measured as the highest of sixteen probed points with one eyeballed touch at one spot, and a single touch cannot tell you whether you touched a high spot or a low one. Off by default, and enabled only while Entire spoilboard itself is ticked.

Both surfacing paths get the test plunge — the one that trusts an origin already set and the one that touches off its own. The depth is the thing being checked, and neither way of arriving at Z0 makes it self-evident.

#291 A program bigger than the machine can travel is refused before it runs

File	+	−
`CNC Core/JobRunner.cs`	75	1

A 600 mm ruler was run on a 400 mm machine. It drove 200 mm past the end of X and sat grinding against the stop with the stepper rattling. The Limits panel had been showing the program's extents the whole time; nothing ever compared them to `$130`–`$132`.

With soft limits off — `$20=0`, which is normal on a machine that cannot home — **nothing else is going to stop it.** The controller does not know it is out of travel and the sender never asked.

- **Compared as a span, not against absolute coordinates**, because the span is the part that is knowable. Without homing there is no trustworthy machine origin, so "where will it go" has no answer; "is it wider than the axis can travel" does, and a program wider than the axis cannot complete from any starting point. That also makes it free of false alarms.
- **It asks rather than refuses.** `$130`–`$132` are left at a nominal default on plenty of controllers, and this must not become the thing that stops a machine working because a setting was never filled in. It only has to make sure nobody drives into a stop without having been told — and it says explicitly, when soft limits are off, that the controller will not save them.
- **Silent when travel is not configured at all.** A limit invented from an unset value is worse than none — the same lesson the Machine Setup zeros taught the same week (#292).

This is the check whose frame handling #279 then corrected: a 32 mm job was refused for spanning 858 mm because machine-coordinate park moves were being measured as if they were work coordinates.

#292 Machine Setup showed 0 for settings that had not arrived, and Apply would have written them back

File	+	−
CNC Controls/MachineSetupWizard.xaml.cs	102	13

The page reads the controller's settings synchronously when it activates, and the `$$` reply can land afterwards. Measured on real hardware: the wizard read `$130`–`$132` at 00:55:46.328 and the values arrived at 00:55:46.769 — 441 ms later. The getter returned NaN, and the code turned NaN into 0.

That is the whole of the "my machine setup is trashed" report, three times now. **0 is a number an operator cannot tell apart from a measurement.**

- **A value that has not arrived now leaves its field alone.** Whatever is there — a real value from an earlier load, or the untouched default — beats a fabricated zero. The load flags itself incomplete and comes back for the rest, bounded at six attempts so a controller that never answers cannot leave a timer running for the session, and never running over the operator's own typing.
- **Apply could have written the zeros back.** It wrote whatever the field held, so an unloaded or unanswered setting was indistinguishable from a real value and would have gone to the controller as `$130=0 $131=0 $132=0` — soft limits enabled over a zero envelope, where every move is out of bounds. The guard already existed three lines below for steps/mm, with the comment "never clobber with 0", and had simply never been extended upward to travel or max rate. It is the same argument for all three: 0 is not a value, it is the absence of one.
- **Both halves are needed.** One stops the bad number being shown; the other stops it being written. The Apply guard is what kept this a display fault rather than a second round of overwritten settings — verified across every wire log of that day: travel was written exactly three times, all with the correct values, and a zero has never been sent to a travel setting.
- **Why the table read 0 at all is instrumented rather than theorised.** Three explanations were tried and each contradicted by the logs. Rather than a fourth theory, the load now records the axis, its index and the setting id it actually asked for whenever travel comes back non-positive — a wrong index and a missing value produce the identical empty answer from the table, and the log can tell them apart where staring at the code cannot.

There is a settings-reloaded event meant for exactly this, and it did not repair the case above. A page that invents the machine's travel is not something to leave resting on one mechanism, so this covers for that hook rather than replacing it.

#293 Two connects that could never succeed: a controller left in check mode, and a board slower than 2.5 seconds

File	+	−
CNC Controls/AppConfig.cs	40	0
ioSender XL/JobView.xaml.cs	29	1
CNC Core/Grbl.cs	12	2

Check mode looped for ever. Connecting to a board left in check mode by an earlier session produced an endless cycle: capabilities unread, reconnect, capabilities unread, reconnect. Grbl answers *every* `$` query with `error:8` while in check mode, so `$I` cannot succeed there — and neither can the reconnect the dialog offers, because check mode survives it. The wire log shows it plainly: the first status report already said `<Check|…>` before ioSender had sent anything. Being in check mode is not a mystery to report, it is the whole explanation, so it now leaves check mode and reconnects instead of asking a question whose answers both lead back to the same place.

- **The second half made it self-sustaining.** `$C` *toggles* check mode; it does not set it. The "lock in check mode" path sent `$C` unconditionally, which on a controller already in check mode *left* it — and the enforcement that re-asserts the lock then saw it was not in check mode and sent `$C` again, putting it back. Two `$C` a couple of seconds apart, achieving nothing, once per cycle. The other five sites were swept; this pair were the unguarded ones.

And a handshake that gave up after two and a half seconds. The startup handshake retried ten times at 250 ms, and a grblHAL board answers on the first. Cheap clone controllers reset when the serial port is opened and can take five or ten seconds to come back, so connecting to one failed every time with nothing wrong.

- **It asks whether to keep waiting** and retries for about ten seconds if so, rather than lengthening the default and making every connect pay for the slowest board that exists.
- **Asked rather than silently waited out**, because the two cases are indistinguishable from inside the handshake and cost opposite things: a slow board wants patience, while a wrong port, a dead board or an unplugged cable wants to be told promptly rather than after a minute of hopeful silence. The operator can see which they are looking at and the code cannot.
- **Defaults to Yes** — someone who has just plugged a board in and pressed connect almost always means to wait — and is bounded at three offers, so a machine that will never answer cannot become an endless sequence of identical dialogs.

The check-mode diagnosis came from the wire log rather than from the symptom: `$G` answered normally and `$#` came back `error:8`, which is the signature of check mode and of nothing else.

#294 A coordinate system that was never reported is zero, not null — loading a file could take the app down

File	+	−
CNC Core/Machine.cs	41	6
CNC Core/GCodeEmulator.cs	21	3

Two crashes, one shape. A `NullReferenceException` in the g-code emulator, taking the whole app down through the 3D viewer's toolpath build, on Load File of a perfectly valid program.

- **Any file containing `G92`, loaded while disconnected.** `Machine.g92` comes from a lookup over the controller's coordinate systems, so it is null until those have been read — and stays null when there is no controller at all. `Machine.cs` guards every one of its own uses for exactly that reason; the validator guards it; the job runner guards it and returns early. These two did not, and they were the only two in the codebase that did not. Reading a program before connecting to the machine that will run it is a normal thing to do, and it is the *safe* order to work in — and the laser files this project generates all begin with `G92 X0 Y0`.
- **And the laser job itself, connected.** The laser is plain Grbl 1.1h on an MKS DLC32, and the reset path only asks for work parameters when the controller is grblHAL. `$#` is never sent, so the coordinate systems list is empty, every lookup yields null, and the first consumer to dereference one goes down. The CNC is grblHAL, which is why this had never appeared before.

Null guards had been accreting one crash site at a time — `Machine.cs` guarded all six of its own uses, the emulator two of thirteen. That fixes each report and leaves the next one waiting; this was the third such guard in a single session. So the coordinate system, `G28`, `G30` and `G92` are now **non-null by construction**: initialised to a zeroed stand-in and only ever *replaced* by a real one, never set back to null. Every unguarded site becomes correct at once and the existing guards stay harmlessly true. Three assignment sites had to change, not one — the reset, the coordinate-system setter, and the emulator's own `G54`…`G59.3` handler, which would otherwise have put a null straight back.

Zero is honest here rather than invented. It is what the offsets and origin already hold before any g-code runs, and WCS rotation is a grblHAL feature plain Grbl does not have. It is *not* a claim that the controller has no offsets — only that none were asked for, and the loaded flag still distinguishes the two. The emulator also still applies the offset when `G92` is absent rather than skipping the case: a viewer that silently ignored a coordinate offset would draw the toolpath in the wrong place, which is a worse failure than the crash it replaced.

Verified with a headless harness driving the exact path that threw, against both builds: before the fix all four references are null and stepping a `G0` throws; after it, the same program walks through.

#295 A macro global documented “0 (default)” had no default, and aborted the probe

File	+	−
macros/pcorner.macro	31	1

Fixture *Test position* died with `error:2` — “error 2 in named sub pcorner.macro” — on the read of `#<_ls_inside>`.

grblHAL does not hand back 0 for a named parameter nobody set. The failed lookup becomes `Status_BadNumberFormat`, which is `error:2`. The macro’s header documents `#<_ls_inside>` and `#<_ls_faces>` as “0 (default)”, but **a documented default is not a default**: only one of the four callers in the app sets them. Start Job does, and so it worked and hid this; Fixture Test position and the stepper calibration wizard do not, and both aborted part-way into a probe.

- **The file’s own notes had warned about exactly this since 2026-08-02**, having been bitten once already, and prescribed the fix — guard the read with `EXISTS[]`. The two reads added after that note did not follow it.
- **Guarded in the macro, not at the three call sites.** The default belongs with the macro that defines what the parameter means, and a fifth caller cannot forget what it never has to remember.
- **`#<_ls_edgemargin>` gets the same guard.** Every caller happens to set it today, so it is not currently broken — but its note claimed an omitting caller “gets 0 from grblHAL”, which is false in exactly the way proved above. The note now says where the 0 really comes from, because the next person to read it would otherwise write a fourth unguarded read on the strength of it.

Verified against the wire log: every `_ls_` global the fixture macro sends is there with its `ok`, and `_ls_inside`/`_ls_faces` are absent from that list.

#296 Restore picks a moment, not a file — and says what that moment holds

File	+	−
CNC Controls/RestorePoint.cs	163	0
CNC Controls/RestorePointDialog.xaml.cs	67	10
CNC Controls/GrblConfigControl.xaml.cs	32	7
CNC Controls/RestorePointDialog.xaml	31	7
CNC Controls/AppConfig.cs	29	0
CNC Core/Grbl.cs	4	38
CNC Controls/CNC Controls.csproj	1	0
Locale/*/csv (7 locales)	56	0

Two different things are snapshotted into the daily backup folder: the controller's $ settings and ioSender's own configuration. Same folder, same timestamp convention, written at the same moments — everything about them says "one backup in two parts".

They were not treated as one. Restore listed only the controller settings, and the configuration snapshots had no user-facing restore at all — they existed purely as a crash fallback inside the config loader. So the operator had to already know *which kind* held the thing that broke before Restore was any use, and knowing what broke is precisely what they are trying to find out. Someone who knows only "it was fine an hour ago" was given a list that could not tell them.

- **Restore now lists moments.** Each row says when, how long ago, and what that moment contains, and the operator chooses: both, machine settings only, or app configuration only. Options the chosen point cannot satisfy are disabled rather than hidden, so a point holding one kind says so instead of the dialog quietly changing shape.
- **Pairing is by time proximity, not equality.** The two snapshots come from different code on different triggers and land seconds apart. A generous window, because pairing two that were merely close offers a choice the operator can decline, while splitting a real pair recreates the original problem: a restore point silently missing half of itself.
- **App configuration restores through the staged-restore slot and a restart.** It cannot be applied in place: the live configuration is read once at startup and held in memory, so overwriting the file under a running session would be undone by the next Save writing the in-memory copy back over it.
- **Controller settings are restored first**, before the restart is offered, so declining the restart still leaves the machine itself put back. The dialog says which half has already happened before asking.

The old settings-only scan is removed rather than left beside the new one — two independent scans of the same folder is how the two kinds came to look unrelated to begin with.

#297 Feeds and speeds are held to what the machine can actually deliver

File	+	−
`CNC Core/FeedsSpeedsAdvisor.cs`	74	1

The chart knows the cutter and the material and nothing about the machine. On a gantry whose `$110`/`$111` is 16510 mm/min it recommends 24000, and grblHAL clamps rather than complains — so the number is quietly not the number.

That alone would be tolerable. What is not is that **the chip load was computed from the feed that was asked for**: 24000 with 20000 rpm and 3 flutes reads 0.400 mm/tooth, while the machine actually cuts 16510 and delivers 0.275. The operator is told a figure the tool never sees, and in MDF that is the difference between cutting and rubbing.

- **The feed is capped at the axis maximum and the spindle brought down to match**, restoring the chip load the chart intended — for the case above, 13758 rpm at 16510 mm/min, still 0.400 mm/tooth. Slowing the spindle is the right lever: chip load is feed per tooth per revolution, and the feed is the half that has hit a wall.
- **Plunge feed is capped against `$112` the same way**, with no compensation available — there is no second term to trade.
- **Both changes say why** in the field's own notes, rather than silently substituting numbers.
- **Silent when the limits are not known.** The settings lookup returns NaN before the controller has answered, and a limit invented from that would be worse than none — the same mistake that had Machine Setup showing a travel of zero the same morning (#292).

In the advisor rather than the dialog that prompted it, so every consumer of a recommendation gets the same answer.

#298 Startup could die focusing a control that was never chosen

File	+	−
`ioSender XL/JobView.xaml.cs`	14	2

A `NullReferenceException` in the Job view's activation, from the startup sequence, before the window was usable.

The focused-control field is only assigned on the *deactivate* path, where it records what had focus so it can be handed back later. On the activate path — including the very first activation of the session, which is the one startup performs — nothing assigns it and the field is still its null initialiser. The block that dereferences it runs either way.

- **It normally survives** because the run control's own activation returns false on that first call and the block is skipped. It only bites when that returns true instead.
- **The same method appears in crash logs from 2026-07-14 at a different line**, so it has form.
- **It falls back to the view itself**, which is exactly what the deactivate path already does when there is no MDI box worth restoring — not a null check that skips the call. A startup that quietly leaves focus nowhere is a keyboard that does nothing until something is clicked.

Not caused by the connect change made minutes earlier; the crash signature predates it. Checked before assuming, because a crash arriving right after a change is the easiest thing in the world to blame on it.

#299 Dialogs record what the operator was told, and what they answered

File	+	−
CNC Controls/AppDialogs.cs	39	12

Console lines, wire traffic and alarms all reach a log. Message boxes did not — so the most explicit statement the app ever makes ("nothing has been probed", "this will overwrite your work origin", "the controller did not report…") was the single thing that could not be recovered afterwards. Diagnosing from logs meant inferring which dialogs must have appeared, which is guesswork about the one part of the session that was unambiguous at the time. Reported directly: a height map failure popup that appears nowhere in the status or console logs.

- **Caption, message and result** are recorded on every path. The result matters as much as the question — "asked, and the operator said No" and "asked, and they said Yes" are different histories and only one explains what happened next. That includes the harness-intercepted and shutting-down paths, which answer without showing anything at all and were previously invisible twice over.
- **Logged when they go up, not only when they are answered.** Logging only the answer leaves the one state worth diagnosing invisible: a dialog still open. That is when the app looks hung and the log's last word is whatever happened before the box appeared. With both sides recorded, an unanswered prompt shows as a "shown" with no matching answer — which names what the app is waiting on instead of leaving it to be inferred (#286 is that fault caught in the act).
- **One dialog is one line**, newlines flattened, and the logging is wrapped in its own catch: recording a dialog must never become the reason one fails to appear.

Feeds the operator-facing status log added in #280.

#300 SVG import reads the rendered tree, and applies transforms instead of just noting them

File	+	−
CNC Controls/SvgOutlines.cs	197	22

Two defects, both exposed by the first PDF-exported artwork to go through the importer.

- **Non-rendered subtrees were imported as artwork.** The load walked every descendant, so a `<path>` inside `<defs>` or `<clipPath>` became geometry. Every PDF and Illustrator export wraps the page in a clip rectangle, and because that rectangle is the largest thing in the file it also captured the bounding box — the logo would have come out undersized inside a frame it never had.
- **A `transform=` was counted as unsupported while the path was imported anyway**, at its untransformed position. The comment said it was reported "instead of importing it at the wrong spot"; the code below it did exactly that. PDF exports hang a `matrix(1,0,0,-1,tx,ty)` off every path to undo PDF's upward Y, so every shape landed mirrored and displaced.

A recursive walk now skips non-rendered subtrees and accumulates the transform down the tree, supporting the full `matrix/translate/scale/rotate/skew` list. Anything not understood is still *reported* rather than applied as identity — half a transform misplaces artwork more convincingly than none, and a refusal that names its reason is the whole point of the import gate (#262).

- **The matrix goes on the geometry, not on the points afterwards**, so flattening happens in transformed space and the flatten tolerance stays meaningful under scale.

Verified against two real vendor logos: the ink bounding box the importer reports now matches an independent measurement of the source PDF exactly (143.488 × 143.777 and 361.188 × 234.466), which it could not if the transforms were being dropped.

#301 Place the laser artwork, repeat it, and anchor it to the corner the machine actually homes to

File	+	−
CNC Controls/SvgLaserSettings.cs	127	3
CNC Converters/SvgToLaser.cs	85	11
CNC Converters/SvgLaserDialog.xaml	20	2
Locale/*/csv (7 locales)	112	14

The importer normalises artwork to its own *bounding box* — the lower-left of the drawn geometry becomes 0,0 and empty canvas around it is discarded. So the logo could only ever start exactly at the origin, and no amount of editing the SVG would move it: adding whitespace changes nothing, because nothing was drawn in it. Fine for a bench job where you jog to the corner first; useless for a fixture, where the work sits in a known place and the file has to reach it.

- **Origin X/Y says where the artwork belongs**, and **Copies and pitch repeat it** — so a fixture holding three identical parts is one drawing engraved three times at the pocket spacing, rather than three copies pasted into the SVG. One drawing stays the single source of the shape, and a change to it cannot be made twice and differently.
- **An anchor for a back-left-homed machine.** A diode laser homed to its back-left corner is the mirror of the front-left convention: the whole table lies at *negative* Y, so artwork placed the ordinary way runs off the back edge into the stop — not a wrong-looking job, a stalled axis, the same failure as the 600 mm ruler on a 400 mm machine (#291). Ticked (the default) puts the artwork's *top*-left corner on the origin so it occupies Y 0 down to −height; cleared restores the front-left convention. A setting rather than a constant because both conventions are real and this dialog serves both machines.
- **Which corner is named in the `.nc` as well as the dialog.** The file outlives the dialog, and the anchor is not recoverable from the coordinates. Both localised strings that said "lower-left corner" are updated across all seven locales — a stale CSV row wins over the XAML, and this is the one instruction that ruins the material if followed wrongly.
- **The offset is applied where every coordinate is formatted**, not at each of the five places coordinates are emitted, so there is one place a copy's offset can be forgotten instead of five. It is cleared before the closing moves: the program ends with `G0 X0 Y0` to return where it started, and that has to mean the origin rather than the origin plus wherever the last copy sat.
- **The summary states where the far corner of the last copy lands**, because that is the number worth checking against the travel and it is not apparent from four separate boxes. A modest-looking pitch adds up to a reach that does not fit.

The anchor shift lives in its own field rather than folded into the placement offset, which is rebuilt per copy — a value that must be re-added whenever something else is recalculated is one that eventually gets forgotten once.

#302 The laser dialog fits the screen, keeps its settings, and shows the exposure the S values hide

File	+	−
CNC Converters/SvgLaserDialog.xaml	147	72
CNC Controls/SvgLaserSettings.cs (moved from CNC Converters)	185	12
CNC Converters/SvgToLaser.cs	31	7
CNC Controls/GCode.cs	29	1
CNC Converters/SvgLaserDialog.xaml.cs	25	4
ioSender XL/MainWindow.xaml.cs	9	0
CNC Controls/AppConfig.cs	6	1
ioSender XL/MainWindow.xaml	2	0
CNC Controls/ActionKeyBinder.cs	2	1
Locale/*/csv (7 locales, 2 files each)	77	28

Power alone says nothing. A fill at S400/F3000 looks like a large power increase over an outline at S150/F1200 and is in fact the same burn — 0.133 against 0.125. Power divided by feed is what browns or chars the wood, and nothing in the dialog showed it, so climbing 200 → 300 → 400 on the fill while its feed ran 2.5× faster bought nothing. Each S field now carries an exposure line, and the shading's is stated as a ratio against the outline's, which is the comparison actually being made.

- **For a fill that measure is areal, because a fill is not a line.** The readout computed power over feed and did not look at the line interval at all. Halving the interval puts twice the energy into the same square millimetre while every number on the Burn tab stays exactly where it was. Found the hard way: a fill at S600/F800 with the default 0.1 mm interval removed nearly 7 mm of cedar, and the readout said "8× the outline" — which reads like a strong engrave rather than a cut, and was not wrong about *lines*. The summary now leads with power over feed × interval, naming the interval in the same breath so it is obvious which field moves the number.
- **Three tabs and a height that fits the screen.** The dialog was one column of five groups, grown taller than a screen, with size-to-content, no resizing and no scrolling: OK and Cancel sat below the bottom edge and Escape was the only exit. Now Artwork / Copies / Burn, with "Before you run this" pinned below the tabs rather than inside one — which corner to jog to is the instruction that ruins the material if it is missed, and a tab you are not on hides it.
- **Enable beam can be left off for a dry run** — identical motion at identical feeds, every S word zero.
- **The settings persist**, which the file header had claimed since it was written and which was never true: the converter is constructed afresh on *every* load, so each import started from the field initialisers, not just each session. Registered as an owned configuration section the way the other converters are.
- **Its own File entry.** Burning a logo started with Load Program and picking "SVG files (laser)" out of the filter combo, which is a poor way to find a whole workflow. Deliberately a separate entry rather than a mode on an existing one: this path emits plain Grbl for a diode controller, while the Work Order's SVG carving emits grblHAL. They are different jobs that happen to start from the same file type.

The layout took three measured fixes, each of which had to be found before the next was visible — size-to-content measures at infinite width so nothing wrapped and the window sized to its longest line; the client area comes back 11 DIP short of what the content asks for, and a minimum height on the tab control turned that shortfall into buttons pushed off the bottom. **The File > Load SVG Laser Job entry is present but not offered yet** — it is gated off pending time on a real machine (#282), since a laser job that reaches the table with the wrong exposure is not a defect to let someone else find.

#303 “Save Drawing” — a work order as a dimensioned shop drawing, in PDF

File	+	−
CNC Core/CNC Core/PdfDocument.cs	407	0
CNC Controls/CNC Controls/WorkOrderDrawing.cs	657	0
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	109	13
CNC Controls/CNC Controls/WorkOrderView.xaml	11	1
Locale/*/csv (7 locales)	14	0

A work order is what you take to the machine, and until now the only way to carry it there was the screen. **Right-click the stock diagram → Save Drawing…** writes it as a PDF sheet: the stock dimensioned overall with its origin corner and keep-out margin marked, every toolpath tagged with a lettered balloon, and a feature table underneath listing each one’s geometry, X/Y, quantity, and every operation with its depth, bit, feed and speed.

The drawing plus a table is the split a real shop drawing uses: a hole chart carries forty features where forty leader lines would just fight each other. So the drawing itself dimensions only what has nowhere else to live — the blank, the keep-out, the origin — and everything per-feature is read off the table.

- **The sheet is drawn by the code that draws the screen.** The diagram builder was parameterised to take a target canvas, and the export points it at an off-screen one sized to the paper. There is one body of code that knows what a work order looks like, so the sheet taken to the machine cannot describe a different arrangement from the preview. The table is built from the same `DescribeGeometry`/`Summarize`/`Expand` helpers the tree uses, for the same reason.
- **PDF is written directly, with no new dependency.** A drawing is lines, filled outlines and text; that is a small enough slice of PDF to write outright, and the base-14 Helvetica every viewer already has needs no font embedded. `CNC.Core` keeps its zero-NuGet position and the new writer passes the net8 portability probe.
- **Long feature lists spill rather than squeeze.** A table too tall to share the page yields page 1 to the drawing entirely and starts clean overleaf, and a page break never lands between a feature and its own operations.

Verified by rendering the output and reading it, not by compiling — which is what caught the drawing being squeezed into a stamp while half the sheet sat empty, and operations orphaning from their feature across a page break.

#304 The stock diagram names features with colour-coded balloons instead of writing their names across it

File	+	−
CNC Controls/CNC Controls/WorkOrderPalette.cs	109	0
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	155	39
CNC Controls/CNC Controls/WorkOrderDrawing.cs	39	31

The diagram used to write each toolpath's *name* across the stock. A name is many times wider than the feature it names, so on a busy work order the labels collided with each other and buried the geometry — the drawing became unreadable exactly when there was most to read.

Identity moved onto a **lettered balloon**: a fixed-size disc whatever the toolpath is called, placed clear of its shape and of every balloon already placed, with a leader back to the shape's edge. Colour ties it together — each toolpath's outline, its envelope wash and its balloon share one hue, and the saved drawing's feature table repeats that exact balloon in its ID column.

- **Colour is never the only channel.** The letter always carries the identity, so the sheet still works printed in mono or photocopied. The palette holds no grey or near-grey, because grey is spoken for as "held back" — a slate blue sat in the list at first and the eighth feature came out looking switched off.
- **The balloon keeps its colour unconditionally.** It is the key that ties the drawing to the table, and a key that changes colour depending on whether a toolpath happens to be held back is not a key. "Won't cut" is carried by the greyed geometry and the dimmed table row instead.

The preview and the print are now the same picture.

#305 The material-removed envelope described a footprint the cut never had

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	143	12

The envelope exists to answer one question — is this toolpath about to run into its neighbour — because that, not the nominal outline, is what actually collides. It was answering wrongly in two different ways.

- **A contour removes a band, not an area.** The envelope was drawn filled whatever the operations were, so a through-contour around a part claimed to have cleared the whole part; on a work order with an outline toolpath that washed the entire stock in one colour and buried everything else. A toolpath whose operations are all perimeter passes (Contour, Side finish, Chamfer) now draws the band along its outline, with the geometry taken from the compiler rather than guessed — the cut runs from the nominal line inward by a full bit diameter, plus whatever a side finish was told to leave, and a chamfer spreads back outward by its depth. Anything that clears area keeps the filled envelope. A band deeper than the shape's narrowest half-span falls back to filled, because then the interior really is all removed.
- **Text and artwork had no rectangle to grow.** Both fell through to the rect default and drew a box from `Width`/`Depth` — fields neither geometry kind ever sets, so what appeared was the model's defaults, 40×25 mm, for every engraved line and every logo. This is the same fault the tree's own summary carried and had fixed; the envelope was missed in that sweep. It matters more here because it *understates*: a 150 mm logo claiming a 40 mm footprint says "clear" when the truth is "collision". Both now draw through the same helpers the outline pass uses, at the width the cut actually is.

Geometry that cannot be resolved — a text fit that fails, an SVG that will not import — draws nothing, which is the honest answer: a missing envelope says "unknown", a rectangle says "this much", and only one of those is true.

#306 A work order records the blank it was authored for

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	160	5
ioSender XL/ioSender XL/StartJobView.xaml.cs	37	0
CNC Controls/CNC Controls/OddJobsStockCanvas.cs	30	7
CNC Controls/CNC Controls/WorkOrderView.xaml	20	2
CNC Controls/CNC Controls/WorkOrderModel.cs	19	0
CNC Controls/CNC Controls/WorkOrderDrawing.cs	12	7
Locale/*/csv (7 locales)	28	0

A work order is a recipe for a known piece of stock — you feed it a blank of a given size and run it — so the size belongs in the file. It wasn't there, so the diagram drew every toolpath against whatever Setup happened to be showing, and opening a work order composed for a different blank produced a layout that looked wrong with nothing on screen to explain why.

The `.workorder` file now carries its stock width, depth and thickness, and the diagram — along with the saved drawing's dimension lines and title block — is drawn against those. Saving a new work order captures the size automatically, so authoring one never has to think about it.

- **Setup is offered the size, never given it.** Setup's Width/Height/Thickness are the operator's own numbers for the material clamped to the table right now, feeding probing and the keep-out area. Silently applying a loaded job's stock to those fields was tried once before for g-code programs and rolled back as confirmed unwanted. So a banner appears *only* on a discrepancy, with **Apply to Setup** one way and **Use Setup's size** the other — the second is also how you change a work order's stock.
- **An existing work order shows its layout again in one click.** A file saved before this change records no size, and rather than quietly falling back to Setup — the right toolpaths on the wrong blank is a drawing that lies, and lies convincingly — the diagram says so and the banner offers Setup's current size. One click per file, then save.
- **Not a material.** Material is one shared value already edited from both tabs; a per-file copy would make a second authority over it. Size is different — it is a property of the blank, not of the session.

#307 Help > Support > Restart ioSender

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	27	0
ioSender XL/ioSender XL/MainWindow.xaml	6	0
CNC Controls/CNC Controls/ActionKeyBinder.cs	1	0
Locale/*/csv (7 locales)	14	0

Some state that wedges a session lives only in memory, and nothing in the UI clears it. The case that prompted this: when the capability query `$I` goes unanswered during connect, ioSender can take a check-mode lock and then re-assert `$C` every time the controller leaves check mode - but `$` queries answer `error:8` in check mode, so the lock blocks the one query that would release it. A fresh instance is the way out, and "close it and start it again" is a poor thing to have to explain.

- **At the top of the submenu**, alone above a separator - it is a recovery action, not a configuration one. You reach for it when the app is misbehaving, not while setting things up.
- **Asks first**, and says plainly that settings are saved and the connection is remade.
- **No new relaunch mechanics.** It calls the same path the settings footer button and both config-overlay items already use, so the `--self-relaunch` handshake with the single-instance probe stays in one place.
- **Bindable** like its neighbours - it appears in Settings > Keyboard under the Menu group.

No job-running guard was needed: the menu bar is already disabled while a job streams, so this cannot be reached mid-burn.

#308 A failed connect could hide its own prompt behind the main window

File	+	−
`ioSender XL/ioSender XL/JobView.xaml.cs`	7	0

Reported during repeated reconnect attempts: ioSender reached a state where every click just beeped, the close button did nothing, and ending the task was the only way out - twice in one sitting.

That is not a frozen UI, it is an **unowned modal**. The "Could not read this controller's capabilities (`$I`)" prompt used the message-box overload that passes no owner. An unowned box still disables every window in the application, but nothing keeps it in front - so it can end up *behind* the main window, leaving the app modal-blocked by a dialog the operator cannot see or reach. Clicks on the disabled window beep, and closing is refused because a modal is up.

- This is the prompt a failing connect shows **over and over**, so of all the dialogs in the app it was the likeliest to land behind something.
- 37 call sites already passed an owner; 13 did not. This fixes the one in the connect path.

The remaining ownerless call sites are worth a sweep but are not all wrong - some run where there is no window to own them.

#309 SVG laser: exposure as four named constants, resolved on load

File	+	−
CNC Core/CNC Core/NgcConstants.cs	295	0
CNC Core/CNC Core/GCodeJob.cs	71	0
CNC Converters/SvgToLaser.cs	33	6
CNC Controls/CNC Controls/GCode.cs	6	1
tools/ngc-roundtrip (round-trip harness)	146	0

A generated laser program is far easier to retune when its exposure values sit at the top

```
#<s_line> = 200
#<f_line> = 1200
```

than when the same two numbers are repeated on hundreds of cut moves. Retuning becomes editing four numbers in the `.nc` - the difference between a file you can adjust on the machine and one you have to regenerate.

Named parameters are an NGC/grblHAL feature the target controller does not have; the laser this was built against is an MKS DLC32 running stock Grbl 1.1h. That is not a contradiction: **the file is canonical and each reader resolves it.** ioSender substitutes on load when the controller does not report `EXPR`, and passes them through verbatim when it does. What reaches the wire is always literal `S` and `F` words.

- **The resolver refuses rather than guesses.** Constant assignments and plain references only - no arithmetic, no numbered parameters, no system parameters, no O-words. A resolver that passed through what it did not understand would be the same failure as the parser gap that once dropped a `G59.3` and put a rapid 128 mm into a touch plate.
- **`#<_abs_x>` and friends are refused on principle**, not as a gap to close later: their value is whatever the machine reads at *parse* time, which during a streamed program is not where it will be when the line executes.
- **Save writes the source form.** Each block keeps the line as it was before substitution, so saving a program with variables does not quietly flatten them to literals - which would undo the whole point on the first save.
- **Line count is preserved** - an assignment becomes a comment rather than vanishing - so resolved line N is always raw line N.

Verified through the real loader, not only the resolver's own tests: a harness drives GCodeJob.ParseFileLines and asserts what actually comes out, including that neither a system parameter nor an undefined reference ever reaches the program.

#310 SVG laser: step the power per copy, to burn a test strip in one job

File	+	−
`CNC Controls/CNC Controls/SvgLaserSettings.cs`	75	4
`CNC Converters/SvgToLaser.cs`	47	4
`CNC Converters/SvgLaserDialog.xaml`	12	0
`Locale/*/csv (7 locales)`	49	0

Copies already repeat the artwork at a pitch. Two more pitches - **Power step** and **Fill step** - make copy *n* burn at base + n × step, so five versions of the same logo at rising exposure come off one job, on one piece of material, at one focus setting. That is a far better comparison than five separate runs, and it is how you find the right power without guessing.

- **It works because the exposure values are declared, not inlined.** Each copy opens with a fresh `# <s_line> = 250` and every reference below it takes the new value. Redeclaration was already a property of the resolver, so the whole feature is two settings, a helper and two call sites.
- **The clamp is announced, never silent** - in the dialog before the job is built, and as a comment in the file on any copy that clamped. Power cannot exceed `$30`, so a ramp that overshoots burns the last copies at the *same* power while the file claims they differ. A test strip whose last two squares are identical reads as a result, which makes a silent clamp worse than no ramp at all.
- **Outline and shading step independently**, and a beam-disabled dry run still declares 0 for every copy.

Verified through the real loader: a three-copy ramp resolves to S200 / S250 / S300 in order, the feed stays put because it was never re-declared, and all three declarations survive into what Save writes.

#311 SVG laser: a placement that leaves the table is refused, and the fix offered

File	+	−
CNC Controls/CNC Controls/SvgLaserSettings.cs	128	11
CNC Converters/SvgToLaser.cs	42	2

Reported: an Origin Y of `−9.125` with a Pitch Y of `+100` happily emitted **positive Y** on a machine whose entire work area is `[−400..0]`. That controller runs with `$20=0` and `$21=0` - soft limits and hard limits both off - so nothing downstream catches it. The gantry simply drives into the back stop.

The placement summary already worked out where the copies reach, and then told the operator to "check that is within travel". **A number computed and acted on by nobody is not a guard.**

It was also **wrong**, in exactly the case that mattered. The far corner was taken as `origin + (−artHeight) + (copies−1) × pitch`, which is correct only when the pitch marches the same way the artwork extends. With the pitch signed the wrong way it named the *bottom* of the last copy rather than the top, understating the reach by a whole artwork height: at two copies it reported `Y−9.3` - comfortably inside the table - while the head actually reaches `Y+90.9`. Minimum and maximum are now tracked separately, which is the only form that cannot hide a reach behind a sign, and the summary reports the whole occupied rectangle.

- **Enforced between the dialog and the emit.** Sign comes from the anchor; magnitude from the reported travel, and only when it is actually known - inventing an envelope would be worse than checking none.
- **The anchor is used rather than the machine-envelope helpers on purpose** - those read $22/$23, which decide nothing on a controller that never homes.
- **A wrong-signed Pitch Y is offered back** as "use -100 instead?", not silently flipped. Flipping a sign the operator typed is inventing intent, and if they did mean +Y the job would run and quietly not be the one they pictured.
- **Answering yes re-validates** rather than assuming: the sign was only the first fault found, and a corrected pitch can still overrun the travel. Rejection returns to the dialog with the numbers intact.

The understated reach was measured against the real artwork dimensions, not reasoned about - the old formula and the true extents were tabulated side by side across one to five copies before anything was changed.

#312 A generated laser job could be streamed but never re-loaded

File	+	−
`CNC Core/CNC Core/NgcConstants.cs`	7	2
`tools/ngc-roundtrip/Program.cs`	34	0

The block of comments every generated laser job opens with wrapped one sentence across two lines and put the closing parenthesis only on the second, leaving line 10 of the file as `(A controller reporting EXPR evaluates them itself; ioSender and the` - an unterminated comment.

That reads like a typo and is not one. `NgcConstants.IsComment` recognises a block only when it both opens `(` and closes `)`, so the resolver did not treat the line as a comment - and `ParseFileLines` then refused the file after the **first block**.

- **Why nobody hit it.** A job built by the converter is handed to the Job tab block by block and never re-parsed, so every job burned so far was fine. It bites only when a **saved .nc is loaded back** - which is exactly what happens when the file is taken to a standalone controller and opened again later.
- **Each line now closes its own parenthesis**, rather than one sentence spanning two blocks.

The round-trip harness (`tools/ngc-roundtrip`) grows a section over the header the emitter actually produces: every line is a balanced comment or an assignment, the header loads, nothing survives as an unresolved reference in an executable block, and the exposure constants resolve to what was declared.

Proven by making it fail: with the missing ')' put back, three of those checks go red - including "the emitted header loads".

#313 A power ramp restated the value it had just declared

File	+	−
CNC Converters/SvgToLaser.cs	30	2

With a ramp in effect the emitter re-declared the exposure constant before every copy - including the first, whose value is by definition the one declared at the top of the file:

```
#<s_fill> = 850          <- header
(shading copy 1 of 4)
#<s_fill> = 850          <- says "this changed here" about nothing
```

The same restatement appeared wherever a ramp had hit the `$30` ceiling, since consecutive clamped copies share a value.

- **The emitter now tracks what each constant currently holds**, seeded from the header's own declarations, and writes an assignment only when the number actually moves.
- **The CLAMPED comment still prints per copy.** That one is worth keeping - it is the difference between two squares that read as a result and two squares that are the same burn.
- The map is cleared as it is seeded: it is per-program state on a converter the placement-retry loop can reach more than once, and a value carried over would suppress a declaration the next program needs.

#314 SVG laser: the placement as two constants, and the geometry below it 0-based

File	+	−
CNC Converters/SvgToLaser.cs	81	18
CNC Core/CNC Core/NgcConstants.cs	24	0
tools/ngc-roundtrip/Program.cs	46	0

The placement chosen in the dialog used to be added to every coordinate in the file - tens of thousands of them on a shaded job - so moving the job meant regenerating it. It is now declared once, holding exactly the numbers the dialog showed, and everything below the header is the artwork's own geometry from 0,0:

```
#<x_org> = 16.5
#<y_org> = -9.525
G21 G90 G17
G92 X0 Y0                     the PARKED corner is 0,0
M4 S0
G0 X#<x_org> Y#<y_org> S0     move in by the placement
G92 X0 Y0                     and that is the artwork origin
...artwork, 0-based...
M5
G0 X0 Y0                      back to the artwork origin
G92 X#<x_org> Y#<y_org>       re-label: the parked frame again
G0 X0 Y0                      back to the parked corner
G92.1
```

- **It is a rapid, not `G92 X#<x_org> Y#<y_org>` at the top**, which is the shape it first looks like it should be. G92 *labels the point the head is standing on*: naming the placement there declares the parked corner to be 16.5 and sends the artwork the same distance the other way. A single G92 could carry these only negated - and a file whose constants are the negatives of what the operator typed gets hand-edited in the wrong direction exactly once, on a machine running with soft and hard limits off.
- **The same two constants bring the head home.** After the artwork the head is in the artwork's frame, where the parked corner sits at minus the placement; re-declaring the current point as the placement restores the frame the job opened in, so the return is a plain `G0 X0 Y0` and no literal in the file can drift from `x_org`. Return first, *then* `G92.1` - clearing while still at the artwork would send the next move to whatever the machine calls 0,0.
- Both exits - shading-only and full - now share one closing routine instead of carrying a copy each. The copy **pitch** stays on the coordinates: that is not placement, it is where copy n sits relative to copy 1.
- The off-table guard added in #311 is unaffected - it works from the settings, not from the emitted coordinates.

The round-trip harness covers the new shape through the real loader, including the case that decided the design: a **negative** `y_org` substituting into `Y#<y_org>` gives `Y-9.525`, where the negated form `Y-#<y_org>` would have produced the malformed `Y--9.525`.

Verified on the laser: job generated, sent to the engraver, correct result - and the 3D preview is unchanged, so the viewer honours both G92s.

#315 A held-back feature turned on by a launch flag instead of a commit

File	+	−
CNC Controls/CNC Controls/Features.cs	16	9
CNC Controls/CNC Controls/ActionKeyBinder.cs	10	2
ioSender XL/ioSender XL/App.xaml.cs	10	0

`Features.SvgLaserJob` had been committed **true** with a comment saying it must be reverted before the branch was pushed - a note that only works if whoever pushes reads it. The shipped default is now the held-back one, and daily use is a launch flag:

```
.\build.ps1 -Launch -enableSVGLaserJob
```

- **Matched case-insensitively**, not as a switch label. It is typed by a human, its natural spelling mixes cases, and a flag that silently does nothing when you type `-enablesvglaserjob` is worse than no flag.
- **It is no longer a `const`.** A const is inlined into every reading assembly at compile time, so no command line could move it - the readers would keep the value they were built against.
- **`ActionKeyBinder.Catalog` stopped being a snapshot.** It was a `static readonly` array that filtered on the flag, which was safe only while the flag was compile-time. Against a value set during startup it becomes a snapshot taken whenever that type is first touched - a race decided by unrelated code. It is computed per read now; two callers, neither hot.

One switch still governs all three places the feature shows itself: the File menu entry, its keyboard action registration, and its row in Settings > Keyboard.

#316 Park lift is a bare `G53 G0 Z0`, not the live machine position

File	+	−
CNC Core/CNC Core/MacroRunner.cs	24	7
CNC Controls/CNC Controls/WorkOrderCompiler.cs	10	8
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	5	7
ioSender XL/ioSender XL/StartJobView.xaml.cs	5	4

Every generator that parks at G30 opened with `G53 G0 X[#<_abs_x>] Y[#<_abs_y>] Z0` - lift Z, hold X/Y at the live position. Naming the live position failed twice over.

- **Parse-time read.** grblHAL evaluates `#<_abs_x>` when the block is parsed, not when the machine gets there. In a streamed program with a full planner buffer it froze G30's coordinates into a later rapid.
- **Soft-limit trap.** grblHAL's jog clamp (`$40=1`) parks an axis exactly on the envelope boundary, and re-commanding that coordinate is refused: Alarm:2 on the park line, machine motionless. Observed with Y clamped to -839.004 against a -839.000 floor.
- **The justification was a disproven firmware bug.** The comments claimed a bare Z-only G53 sign-flips a $23-inverted axis. That was tested and disproven on 2026-08-11; bare `G53 G0 Z0` was already in production at six other sites.
- The hand-rolled copy in the stepper calibration wizard now uses the shared `EmitGotoG30`.

Hardware-verified on the mill: Setup runs through the park it used to alarm on.

#317 Soft-limit reachability guard never evaluated - it read a homed mask that `Get()` had just cleared

File	+	−
CNC Core/CNC Core/Grbl.cs	24	2
CNC Core/CNC Core/MacroRunner.cs	28	1

`StoredPositionUnreachable` exists so a G59.3 stored outside the soft-limit envelope is refused with a dialog before the program starts, instead of an Alarm:2 mid-run with the probe already fitted. Across the entire log history it had logged *SKIP - HomedMask -1* four times out of four: it had only ever failed open.

- **Cause.** `GrblWorkParameters.Get()` cleared `HomedMask` to -1 on entry and let the `[HOME:]` parse set it. Every reader treats -1 as "can't know" and fails open, so anything reading it between the clear and the report being consumed had its check silently disabled. The $# reply carried `[HOME: … :7]` and the guard still read -1 three milliseconds later.
- **Fix.** The mask is staged and published once the report is consumed, so a concurrent reader gets the previous (real) value rather than "unknown"; -1 now means only that the read failed. Every `Get()` logs the published value and calls out a completed report with no `[HOME:]`.
- **Belt and braces.** The guard falls back to the live `HomedState` when the mask is unknown, mirroring what the `homed` PREREQ already did.

Verified on the mill: the G59.3 stored 4 µm outside the floor is now refused at the PREREQ with the reachability text instead of alarming on the park line.

#318 CNC.Svg: the SVG importer and laser emitter as a portable assembly - no WPF, no native dependencies

File	+	−
CNC Svg/SvgPath.cs	570	0
CNC Svg/SvgLaserProgram.cs	395	0
CNC Svg/SvgLaserOptions.cs	115	0
CNC Svg/SvgMatrix.cs	111	0
CNC Svg/SvgOutlines.cs (moved from CNC Controls)	50	85
CNC Svg/SvgLaserFill.cs (moved from CNC Converters)	2	3
CNC Svg/CNC.Svg.csproj, CNC.Svg.net8.csproj, Directory.Build.props	126	0
CNC Common/NgcConstants.cs (moved from CNC Core)	8	1
CNC Common/Geometry.cs	25	0
CNC Converters/SvgToLaser.cs	20	326
CNC Controls/CNC Controls/SvgLaserSettings.cs	48	0
CNC Controls/CNC Controls/TrueTypeOutlines.cs	4	15
CNC Controls/WorkOrder{Compiler,Model,View} (3 files)	3	0
CNC Common / CNC Controls / CNC Converters / CNC Core .csproj + .sln (5 files)	20	3
tools/svg-compare (3 files)	851	0

The SVG-to-laser converter could only run on Windows: `SvgOutlines` leaned on WPF's `Geometry.Parse` + `GetFlattenedPathGeometry`, and PresentationCore has no Linux build. That put it out of reach of the EngravingBox appliance that actually burns the files.

- **One thing replaced.** `SvgPath` parses the SVG `d` grammar and flattens Béziers and elliptical arcs itself (adaptive subdivision, transform applied before flattening so the tolerance keeps its meaning under scale). `SvgMatrix` keeps WPF's row-vector semantics exactly, because the transform walk's Append/Prepend calls are only correct for that convention. Written rather than taken from SkiaSharp so nothing ships a per-architecture native `.so`.
- **Everything else moved as it was:** `SvgOutlines`, `SvgLaserFill`, and the emitter lifted out of `SvgToLaser` as `SvgLaserProgram` returning lines behind a plain `SvgLaserOptions`. ioSender keeps its dialogs and delegates via `SvgLaserSettings.ToOptions()`.
- **Kept off the comms assembly.** `OutlineContour` and `NgcConstants` moved to `CNC.Common` (same namespace, no call-site churn) so the appliance takes one small DLL, not CNC Core.
- **The net8 probe is the acceptance test:** `CNC.Svg.net8.csproj` fails the moment a Windows-only reference creeps in.
- **Proven, not compiled.** `tools/svg-compare` holds the original WPF implementation verbatim and diffs it geometrically against the port: identical contour counts, outer/hole classification and bounds on both test files and the real 42-contour logo, areas within 0.003%. Point counts differ by design - a different flattener subdivides differently.

Builds for net462 and net8.0. Not yet run on ARM64/Linux - pure managed, so expected to be a straight xcopy, but unverified there.

#319 Console: an MDI command no longer resets the scrollback to line 0

File	+	−
`CNC Controls/CNC Controls/ConsoleControl.xaml.cs`	25	0

`FlushLog` rebuilds the whole TextBox text on every log change. Assigning Text snaps the internal ScrollViewer to offset 0, and the follow-the-bottom behaviour infers "the user scrolled" from a ScrollChanged event whose extent did not change - which is exactly what that snap looks like once the 2000-line scrollback is full (one line trimmed off the top per line added). It recorded the snap as a manual scroll to the top and stopped following.

- The console now reads "were we at the bottom?" from its own scroll state *before* the text is replaced and restores it afterwards.
- An MDI Enter always jumps to the end, scrolled-up or not - typing a command means "show me what happens".

Reported and fixed while squaring the laser; verified on the next restart.

#320 Laser skew compensation from a burned test square

File	+	−
CNC Controls/CNC Controls/SvgLaserSettings.cs	53	1
CNC Svg/SvgLaserOptions.cs	35	0
CNC Svg/SvgLaserProgram.cs	20	8
CNC Converters/SvgLaserDialog.xaml	16	5
Locale/*/csv (7 locales)	70	0

The 400×400 diode machine's gantry is racked about 0.45° and would not hold a mechanical correction - belt phase and frame push both failed to move it over two days of trying. A commanded square burned as a parallelogram.

- **The shear.** `SvgLaserOptions.SkewXY`, Y added per mm of X, applied at the emitter's one coordinate formatter as `Y' = Y + s·X`. Y is sheared by X because that is what a racked gantry physically does; the square test cannot distinguish the two shears and the other differs only by a rotation of the whole job, which a fence built to the machine's own etch absorbs.
- **Measured, not typed.** The Burn tab of the SVG-to-laser dialog takes the three numbers the operator actually reads off a burned square - D1 (home corner to far corner), D2, Side - and derives `s = (D1² − D2²) / (4·side²)`. Only the difference of the diagonals matters, so a ruler that reads both a few millimetres short against charred corners cancels out. Persisted; zero any field to switch it off.
- The three fields were side by side at first and the third clipped clean off the 400 px dialog; now one per row.

Hardware-verified 2026-09-06: an uncompensated 395 square read 560/555; a compensated 385 square read 543/543. Skew is a per-machine property living in a global slot for now - it moves to the machine profile when the profile store is built.

#321 Parametric programs: (PROMPT) fields answered at load, bracket arithmetic folded for controllers without EXPR

File	+	−
CNC Common/NgcConstants.cs	231	21
CNC Core/CNC Core/GCodeProgram.cs	58	1
CNC Core/CNC Core/GCodeJob.cs	57	3
CNC Core/CNC Core/JobRunner.cs	14	1
CNC Controls/CNC Controls/MacroProcessor.cs	6	1
tools/ngc-roundtrip/Program.cs	100	1

One `square.nc` for every test size, with the sizes asked for when the file is opened - runnable on a controller with no expression support.

- **PROMPT already existed** - collected by JobRunner before Cycle Start, values sent as an EXPR preamble or substituted at send time. On a non-EXPR controller that path never worked: the loader resolves constants at load, so `#<_name>` references were refused as system parameters, or the file's defaults were baked in and the operator's answers never reached the wire.
- **Now the dialog fires at load, on every controller.** Both file loads and in-memory programs collect the fields after `Action.New`, ask once on the calling thread before the background parse, and install the answers before line 1 is resolved. Non-EXPR: references resolve to the answers and the 3D preview shows the real geometry. EXPR: the file's declarations are rewritten to the answers so the preamble is not undone by the program's own default. JobRunner sees `PromptedAtLoad` and does not ask twice; unattended macro runs keep the defaults; Cancel loads the defaults.
- **Bracket folding** in `NgcConstants` (CNC.Common, so EngravingBox gets it too): `[#<inset> + #<side>]` with + − * /, unary sign, nesting and precedence, both as an assignment and inline. A function, `**`, `MOD`, or any name without a value still refuses the whole line - the class's refuse-don't-guess rule is untouched.
- `Raw` keeps the parametric form, so Save writes the variables back out.

Proven by tools/ngc-roundtrip (folding incl. nested precedence, a refused SIN, an answer beating a declaration, a seeded name with no declaration, LoadText collecting fields) and on the laser: square.nc prompted on open and burned the answered size.

#322 SVG carve: Negative — carve the background and let the artwork stand proud

File	+	−
CNC Svg/SvgOutlines.cs	52	0
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	33	3
CNC Controls/CNC Controls/WorkOrderModel.cs	17	3
CNC Controls/CNC Controls/WorkOrderCompiler.cs	13	4
CNC Controls/CNC Controls/WorkOrderView.xaml	8	0
Locale/*/csv (7 locales)	35	0

A V-carved logo cuts the logo *into* the wood. Sometimes the piece wants the opposite: the logo left standing at the surface and everything around it cut away, the way a raised-letter sign is made. That is now a **Negative** tick on the SVG toolpath, with a **Border** field for how far the recessed panel extends past the artwork's ink on every side (5 mm by default).

- **One extra contour is the whole feature.** The carve engine decides what is inside the shape by even-odd containment over every contour at once, and the preview fills by the same rule. Adding a single rectangular frame around the artwork flips that answer for every point inside it: the background becomes the carved region, the logo's outer shapes become islands, and their counters — the hole in an O — are carved again. Nothing downstream had to learn anything.
- **The panel floor is the carve depth cap.** Where the background is wider than the bit can span, the engine's flat-clearing generations already floor it at the cap; with no cap set it goes as deep as the bit allows, and the field's tooltip says so. The panel edge gets the same sloped wall as any carved edge.
- **Everything that describes the cut knows.** Negative and Border are in the carve cache key (the lesson from #267), in the toolpath's extents so the anchor and the stock check see the panel rather than the ink, in the toolpath summary and the operation comment, and in Validate. The editor's info line restates the panel size, since that is what has to fit the stock.

Engine-verified in a harness before it reached the UI: on the nested-islands test file and the real logo, a point in the border strip carves to the cap in the negative and not at all in the positive, and islands inside counters stand proud again. The UI flow is hand-off-verified only.

#323 V-carve: the hairline detail passes ran first — a negative logo spent 45% of its program barely scratching before the first real ring

File	+	−
CNC Core/CNC Core/VCarve.cs	12	6
CNC Controls/CNC Controls/WorkOrderCompiler.cs	3	2

Found on the first negative carve (#322): three runs on the machine were feed-held and stopped at about 8% because the bit was *barely touching the surface*, whatever depth cap was set. The wire log agreed — the deepest plunge ever sent was 0.4 mm — while the program's own minimum Z was −8. The depth was all there. It was at the end.

- **The cause was the cut order, not the depth.** The engine cuts stepped rings shallow to deep, and adds one extra *spine* pass per constant-width feature at that feature's own full depth — the pass that puts the crisp bottom line in a V-carve. Those spine passes were merged into the ring sequence by depth. For lettering that was invisible, because each glyph is regrouped and cut completely before the next. A negative logo is one region with no regrouping, and every hairline sliver between the letters and the frame has a spine at 0.1–0.4 mm — so on the operator's file those were the first 45% of the program, the 2 mm rings started at 45%, and the 8 mm panel floor was the last 5%.
- **Spines now come last:** rings shallow to deep, then the flat clearing, then the spine passes. Physically nothing changes — a spine sits at its region's maximum distance, so every ring in that region is shallower and still precedes it, and a spine with no rings at all is a hairline plunge into solid stock whichever way round. The same file now reaches its first 2 mm ring at 5% of the program, and the only thing ahead of it is the depth-0 boundary rings the emitter already skips.

Measured in a harness on the operator's own artwork and settings before and after; pass counts and geometry are identical in both the positive and negative harnesses, only the order moved.

#324 SVG carve: a negative bounded by the artwork's own outline — carve only what is inside a badge's circle

File	+	−
CNC Svg/SvgOutlines.cs	52	0
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	42	8
CNC Controls/CNC Controls/WorkOrderModel.cs	38	4
CNC Controls/CNC Controls/WorkOrderCompiler.cs	18	6
CNC Controls/CNC Controls/WorkOrderView.xaml	6	0
Locale/*/csv (7 locales)	14	0

The first negative carve (#322) inverts everything inside a rectangle around the artwork. On a circular badge logo — a 60 mm border circle with a rim and the art inside it — that carved the four corners outside the circle as well, and the operator asked how to carve only what the circle contains.

- **Panel: Rectangle or Artwork outline.** Rectangle is the existing frame with its Border field. Artwork outline *drops* the artwork's single enclosing contour instead of adding a frame. Removing a contour lowers the containment depth of everything inside it by one, so the even-odd parity flips exactly there and nowhere else: outside the circle nothing is cut, the rim stands at the surface, the inside floors at the depth cap, and the art stands proud. No border is needed, so the field hides.
- **Only honest where it applies.** The choice needs one contour that encloses all the others, which badge-style artwork has and bare lettering does not. The editor's info line says whether the chosen file can do it, Validate names the problem at Generate, and the compiler refuses by name rather than quietly substituting a rectangle — the same stance as an incomplete import.
- **One place applies the choice.** The toolpath answers “which contours does an Engrave carve” for the compiler and the preview alike, so the preview shows the corners and the rim untouched before anything is generated. The panel kind joins the carve cache key, the extents, and both descriptions.

Harness on the operator's badge logo: the enclosing contour is the 60 mm circle; a corner point and a rim point carve to 0.00 mm, the inside floors, the art stands. The nested-islands test file, three top-level rings, reports no enclosing outline.

#325 Work Order: Clear floor — an end mill flattens the corrugated floor a V-carve leaves

File	+	−
CNC Controls/CNC Controls/WorkOrderCompiler.cs	190	36
CNC Core/CNC Core/VCarve.cs	78	0
CNC Controls/CNC Controls/WorkOrderModel.cs	56	1
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	24	3

A V-bit has no flat bottom. Where a carve is wider than the cone can span it floors the region with rings of its tip at the cap depth, and the V-walls of adjacent rings meet in a ridge half the depth step tall — a 1.5 mm step leaves 0.75 mm corrugations across the whole panel of a negative badge (#322, #324). The walls and the standing art are crisp; the floor is not. The only lever was a smaller step, which multiplies the rings everywhere they buy nothing.

- **Clear floor is a second operation on the same toolpath**, offered only where the toolpath V-carves, with its own end mill, feeds, depth of cut and stepover. Its floor depth and the cone angle are not settings: they are read from the sibling Engrave operation's V-bit and cap every time, so the mill's floor and the V-bit's floor are one number by construction. Remove the Engrave, or give the Engrave a pointed tool that is not a V-bit, and Validate, the editor's note and the compiler all say so by name rather than guessing a depth.
- **The mill's path comes from the same distance field the carve uses.** Its centre runs on the iso-contour at the V-bit's tip distance plus the mill's radius, so its edge reaches exactly the toe of the V-wall, then every stepover inward until nothing remains, cut inside-out so each pass is a side cut at the stepover and the toe ring goes last. The field has to reach the middle of the region rather than one cone-width in, so it is built to the region's half-diagonal on a coarser 0.25 mm cell — under half a second on the badge.
- **The V-bit keeps its own floor passes.** Anything narrower than the mill's diameter plus two V-wall toes stays the V-bit's; on the badge that is two 3.5 mm strips between the art and the rim, 13 cells of 650. The mill skims 0.75 mm ridges everywhere else, which is a light cut, so the order of the two operations is the operator's.
- **Shared placement.** The frame and outline resolution BuildVCarve used inline are now two helpers both builders call, so the mill lands where the V-bit did — same anchor, same rotation, same negative and panel choice.

Engine harness on the badge: 5 rings in 418 ms, coverage map shows the mill reaching every part of the V-floor it can fit into. Compiler harness with a stand-in tool table: floor resolved at 2.5 mm / 60°, plunges at exactly -2.0 and -2.5 over two passes, every XY inside the placed badge, a toolpath without an Engrave refused in both the g-code and Validate. UI flow hand-off verified only.

#326 Clear floor burned the channel bottoms in maple — the mill now cuts a tenth below the V-bit's floor, and starts as one pass

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	42	6
CNC Controls/CNC Controls/WorkOrderCompiler.cs	12	4
CNC Controls/CNC Controls/WorkOrderModel.cs	9	0
CNC Controls/CNC Controls/WorkOrderView.xaml	2	0
Locale/*/csv (7 locales)	21	0

First maple badge with Clear floor (#325): the V-walls came out clean and the channel bottoms came out scorched, and only where the 1/8" mill had been. That is the mill's end face rubbing. At exactly the carve's depth the face rides a surface the V-bit already finished, removing nothing but the ridge tops, and at a 40% stepover it re-rubs most of the previous pass on top of that. A face that rubs instead of cutting burns hardwood.

- **Below carve floor**, a new field on the operation, default 0.1 mm. The rings stay placed for the carve's floor, so the toe ring still meets the V-wall, but the cut goes that much deeper and the face takes a continuous thin chip. The step it leaves at the wall is a tenth of a millimetre. Passes step over the carve depth and the last one absorbs the extra, so a step that divides the carve depth exactly never gains a tiny final pass.
- **One pass by default.** Run after the V-bit, the mill only skims ridges half a depth step tall, so a new Clear floor now seeds its depth of cut from the carve's floor. The first attempt at that was silently overwritten a few lines later by the tool memory's recall of the mill's last confirmed step, which a harness with an empty memory never saw; the seed now runs after the recall, and the harness plants a remembered value to prove it.
- **The note says the pass count** — “Cuts to 2.6 mm, 0.1 mm below the 60° carve's floor, in 1 pass” — because a 2 mm depth of cut beside a 2.5 mm floor reads as a depth and is a step.

Compiler harness: depth of cut 2.5 -> one pass at -2.600; 1.25 -> -1.250 then -2.600; below 0 -> -2.500 only. Factory harness with a remembered 2 mm step planted under the picked tool: feed recalled, depth of cut still the floor's. Also worth doing: put Clear floor before the Engrave in the operation list, so the mill pockets solid wood and cannot rub at all.

#327 Work Order: Mark — a V-bit traces any shape's outline as a shallow groove

File	+	−
CNC Controls/CNC Controls/WorkOrderCompiler.cs	47	0
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	22	6
CNC Controls/CNC Controls/WorkOrderModel.cs	20	1
CNC Controls/CNC Controls/OddJobsFeedsSpeedsDialog.xaml.cs	1	1

The badge (#324, #325) wanted a 1 mm groove on a 63.5 mm circle around it, marking where the stamp ends. Contour is the wrong operation for that: it offsets the path by an end mill's radius, cuts to a total depth, and does not offer a V-bit at all.

- **Mark is a marking line along the outline.** It is offered on every shape geometry — circle, square, rectangle, oval, line — and the V-bit's tip rides the nominal outline with no offset. The one thing to set is the groove's *width*; the depth follows from it and the bit's angle, by the same answer a stroke engrave uses, including the clamp at the bit's own diameter. The editor's note says what that depth will be on the chosen bit.
- **Same seams as the rest.** Cut direction applies, the depth of cut steps it if asked, the feeds dialog offers the V-bits, Validate wants a V-bit and a positive width, and the preview's envelope shows the groove's width rather than the V-bit's shank — a 12.5 mm bit drawing a 12.5 mm band around a 1 mm line would have hidden the geometry it marks.

Harness: a 63.5 mm circle with the 60° bit and a 1 mm width - every emitted point on the 31.75 mm radius, one plunge at -0.866 mm, Validate clean. UI flow hand-off verified only.

#328 Mark: Dashed — a dash-and-gap line along the outline, with whole periods on a closed shape

File	+	−
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	78	12
CNC Controls/CNC Controls/OddJobsGeometry.cs	73	0
CNC Controls/CNC Controls/WorkOrderCompiler.cs	24	8
CNC Controls/CNC Controls/WorkOrderView.xaml	9	0
CNC Controls/CNC Controls/WorkOrderModel.cs	5	0
Locale/*/csv (7 locales)	56	0

A marking line (#327) is often better dashed: a cut-here line, a fold line, a stitching guide, or a border that reads as decoration rather than an edge. Since the mark is already a polyline along the outline, dashing it is a matter of where to lift.

- **Dash and Gap are lengths along the line**, defaults 4 and 2 mm, shown only when Dashed is ticked. A dash can run round a rectangle's corner, which is what a marking line looks like and avoids any rule about where dashes may start.
- **The pattern closes on itself.** A circle or rectangle is divided into a whole number of dash-plus-gap periods, the pitch stretched or shrunk by under half a period so the last gap meets the first dash without a stub. A line gets a dash at each end with the gaps fitted between.
- **Each dash is its own plunge**, with the V-carve's 2 mm hop over the gap. A 63.5 mm circle at the defaults is 33 short plunges, well under a minute at a normal plunge feed.
- **The preview shows the dashes** — a dashed stroke at the groove's width on the nominal outline — when a toolpath's only operations are Marks and one is dashed. Mixed with anything else the envelope band stays, since it is the more important thing to see.

Harness: the 63.5 mm circle at 4/2 gives 33 dashes of exactly 4 mm at a 6.045 mm pitch and the pattern meets itself; a 50 mm line gives 9 dashes from one end to the other; the emitted g-code plunges 33 times at -0.866 mm with 32 hops. Preview hand-off verified only.

#329 FIX: Mark cut a 1 mm groove in five laps — a depth of cut the editor never showed

File	+	−
CNC Controls/CNC Controls/WorkOrderCompiler.cs	12	10

Mark stepped by the operation's *depth of cut* — a field the editor does not show for a Mark — and tool memory recalled the V-bit's 0.2 mm into it when the operation was created. So a 1 mm groove ran as five 0.2 mm laps, with nothing on screen to say why.

- **A Mark is always one pass.** Its depth follows from the groove width and the bit angle (#327), so it is a couple of millimetres at most — there is nothing for a depth step to divide.

Harness: depth of cut 0.2 on the operation, one plunge emitted at -0.866.

#330 Stepper calibration (scratch) is back — a machine with no 3D probe had no way to calibrate at all

File	+	−
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	636	0
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	117	0
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	123	18
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	31	8
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml	15	1
CNC Controls/CNC Controls/CalibrationView.xaml.cs	11	9
CNC Controls/CNC Controls/AppConfig.cs	8	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml	4	0
CNC Core/CNC Core/LibStrings.xaml	2	1
CNC Controls/CNC Controls/IGrblConfigTab.cs	1	0
Locale/*/csv (7 locales)	35	0

The scratch wizard was deleted in 2026-08 on the grounds that the probe method was the only one worth supporting. That traded away the only method with no hardware dependency: the probe wizard needs a 3D probe *and* a located Corner Fence fixture *and* a reference block of known size, and it *hides* itself when no 3D probe is defined — so a profile with a tool setter and a touch plate saw a Calibration step containing nothing but Squareness, with no hint that a stepper page existed.

- **Restored and rehomed.** It is a sub-tab beside the probe wizard, with no availability gate of its own: a V-bit and a pair of calipers is the whole dependency list, so it is always offered. The nav labels now disambiguate — Stepper (probe) and Stepper (scratch).
- **The method.** N candidate steps/mm values are cut as N pairs of scratch lines. Steps/mm cannot change mid-program, so each candidate is realised by commanding span × candidate / current — physically identical motion. Measure each pair with calipers, type the numbers in, and the row closest to the commanded span is highlighted. No homing, no G53, no probe.
- **A touch plate works here too.** The earlier reasoning that it could not — that the plate is the sensor and so cannot feel an unwired block — is wrong, and Setup's own Measure option disproves it on hardware: the plate goes *on* the corner and the tool touches it. A probe-type picker now offers 3D Probe or Touch Plate, feeding the macro the plate's thickness and lip width exactly as Start Job does, and pausing between corners so the plate can be carried.
- **Three availability faults fixed while tracing why the probe page was invisible.** Adding a first 3D probe refreshed the step colours before recomputing availability, so the nav tree sampled the old answer; editing a probe's *type* never told the tree at all; and the wizard's view model is now re-resolved on activation, since a null model turns Generate and Run into silent no-ops.
- **Crash fix.** Opening Calibration took the process down: the new dropdown carries *SelectedIndex=0*, and WPF raises SelectionChanged from EndInit while the BAML is still loading, so the handler ran with every other named field still null. Guarded on IsInitialized — the picker's real state is established on activation anyway.

The touch-plate program's pause placement and its Z-offset cancellation argument both want a hardware pass; the scratch method itself has since been run end to end (#334-#342).

#331 FIX: a disabled Generate button now says what it wants — the reason was never once displayed

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	22	3
ioSender XL/ioSender XL/StartJobView.xaml.cs	19	0
CNC Controls/CNC Controls/JobControl.xaml.cs	10	1
CNC Controls/CNC Controls/AutoSquareWizard.xaml.cs	7	1
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	7	1
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	3	0

A blocked-reason channel was added to the run strip so a greyed-out Generate could carry the tab's own explanation rather than a generic "nothing to run yet". It had never worked: the reason was read 55 lines below the branch every Generate-first tab actually takes. Work Order — the only tab that ever published a reason, added after the complaint "it doesn't tell me what it doesn't like" — took that early return every single time. The fix and the complaint that motivated it never met.

- **The Generate branch now prefers the published reason**, falling back to the generic text when no tab supplied one, or when the bar is disabled for an unrelated reason (disconnected, mid-job).
- **The reason clears itself.** These are statics shared by every Generate-first tab, so a reason left behind by the tab you just left would otherwise describe the wrong thing on the next one. Becoming ready clears it, and so does tab teardown.
- **All five gates publish one** — Auto Square (travel legs), Stepper calibration probe (3D probe for Z, Corner Fence for XY), Stepper calibration scratch (axis resolution unavailable), Start Job probe and Start Job fixture (no fixtures, none selected, kind unsupported, alarmed, no probe — most specific first).

#332 A view that needs the run strip opens as a TAB, not a window — and Calibration leaves Machine Setup

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	127	33
CNC Controls/CNC Controls/CalibrationView.xaml.cs	117	0
CNC Controls/CNC Controls/CalibrationView.xaml	33	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	7	115
CNC Controls/CNC Controls/MachineSetupWizard.xaml	2	34
CNC Controls/CNC Controls/TabRegistry.cs	13	2
CNC Controls/CNC Controls/LayoutModel.cs	5	2
CNC Controls/CNC Controls/AppConfig.cs	5	0
CNC Controls/CNC Controls/ICNCView.cs	2	1
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	1	0
CNC Controls/CNC Controls/LibStrings.xaml	1	0
Locale/*/csv (7 locales)	49	0

The run strip is docked in the main window and exists nowhere else. It carries the Generate/Run button — the only one any Generate-first wizard has — plus Feed Hold, Stop and the jog pad. A view opened from a menu opens in its own top-level window, so its controls were in a *different* window: behind it, or on another monitor, and clicking them deactivates the popup being worked in. On a profile that had not dragged Machine Setup onto the tab bar, its Probe and Fixture steps had no jog pad and the calibration wizards under it had no Generate button.

- **A view can now declare that it needs the run strip**, and the main window creates a tab for it on demand. The menu entry stays exactly where the operator put it — only the presentation is forced — and the tab is not written to the layout tree, so it lasts the session. Set on Machine Setup, Start Job, Work Order and the new Calibration view.
- **Calibration is its own view** (Tools menu by default) holding the same three wizards as sub-tabs, instead of being buried at Machine Setup step 8. Probe availability is re-checked on every entry rather than pushed from the probe editor — the push version had its call order wrong twice, and a question asked when the answer is needed cannot go stale.
- **Existing profiles get it automatically**: the menu-placement pass adds any default entry the saved tree does not mention, so Calibration appears in the Tools menu on next load rather than being unreachable.

#333 Closable tabs — an X on any tab the menu can reopen, and an opt-in ESC to do the same

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	98	8
CNC Controls/CNC Controls/TabKeyBinder.cs	40	2
CNC Controls/CNC Controls/AppConfig.cs	5	0
CNC Controls/CNC Controls/UiGeneralConfigControl.xaml	2	1
Locale/*/csv (7 locales)	14	0

A tab opened from a menu (#332) had no way back out — the tab strip grew every time one was opened and stayed grown for the session.

- **Closability is "can I get it back", not "was this made on demand".** A tab gets an X when its component sits in a File or Tools menu slot, which is the same question the menu builder answers. So a view the operator deliberately placed on the tab bar has no X and ESC will not touch it — closing that one would remove it for the session with no way back short of the layout editor.
- **The X is on component tabs too** (tool table, Trinamic, PID), with a hit area well beyond the glyph, and the click is marked handled so closing a background tab does not first select it.
- **Closing selects a neighbour first.** That is not cosmetic: the selection change is what releases the shared Generate-mode state, so closing a Generate-first tab without it would leave the run strip's Generate button pointing at a view that is no longer on screen.
- **New setting "ESC closes current tab if closable"** (Settings > UI, default off). It only fires for the active window, never while focus is in a text box, and never with a modifier; a non-closable tab lets the key fall through, so turning it on cannot make ESC dead for the console or a flyout.

#334 CRASH FIX: corner 1 rapided into the touch plate — it had no trusted height, so the probe aimed at the machine floor

File	+	−
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	54	7
CNC Core/CNC Core/Grbl.cs	38	0

Running stepper calibration drove the tool into the touch plate at rapid on the first corner. From the wire log the feed was 4570 mm/min — rapid, not the 500 search feed — because the corner macro's slow seek only starts *after* a rapid to an approach height, and that height is computed from a parameter meaning "the surface the stock sits on". This wizard was passing the machine's Z *travel limit*: correct for the only other thing that parameter does (cap the seek depth), meaningless as a material datum. The approach was aimed most of the Z envelope below where the block actually was.

- **Corner 1 now passes a trusted safe height.** It reached that branch because it passed zero, which the macro reads as "the caller has no trusted height". Machine top *is* trustworthy here — the operator has just confirmed the probe at G30 and nothing sits above Z0 — so it passes a hair below it. With a real height the macro travels at the top and leaves the entire descent to its probe-guarded seek at search feed, which is what a tool that by definition knows nothing about the stock height should always have done. It costs time and nothing else.
- **Then the seek floor itself was out of range.** With the rapid fixed, corner 1 reached its top seek for the first time and the controller refused it at planning — Alarm:2 with the position unchanged, the tell that the *target* was out of range. The wizard derived its floor as raw travel with no pull-off, four millimetres outside the reachable envelope.
- **One reachable-limit helper now, not a fourth private copy.** It mirrors the firmware's own work-envelope calculation — force-set-origin, the axis' homing direction, and the pull-off — and both the seek floor and the face-probe floor come from it. The face-probe floor was wrong by the same amount and exists precisely so that descent cannot trip a soft limit; it would have fired on the next thin reference block.
- **"Travel unknown" no longer substitutes the deepest possible target.** The helper returns nothing, and Generate refuses with a sentence.

Not fixed and not assumed fixed: the jog clamp still holds its own copy of the same formula, and Start Job's floor clears this machine's pull-off only by luck.

#335 CRASH FIX: no pause between corner 1 and corner 2 — the plate was never moved, so the tool probed into the stock

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	36	12

Setup with Measure and a touch plate crashed at corner 2. From the wire log, corner 1 finished and corner 2's setup ran 0.2 s later — no prompt, no pause. The plate was still sitting on corner 1, so nothing at corner 2 could trigger the probe and the tool drove 36 mm down through the stock to the seek floor before alarming.

- **Three transitions, two prompts.** The comment above them explained why none is needed *before* corner 1 (the plate is placed before Run is pressed) and said nothing about after it — and after corner 1 the machine goes to corner 2, where the plate is not.
- **Fixed structurally, not by adding a third hand-placed prompt.** The emitter now produces one for every corner after the first whenever a touch plate is selected, so a corner cannot be added without its pause. The two hand-written ones are gone with it.
- **Swept the other callers**, since the same omission looks identical anywhere: the vise program probes two corners and had no prompt between them either.

Not fixed, and worth deciding separately: corners 2-4 inherit corner 1's seek floor - the machine floor - so a missing or misplaced plate still seeks the full depth instead of giving up a few mm below the top corner 1 already measured.

#336 Scratch calibration runs from the origin that is already set — no jogging, no Zero All mid-run

File	+	−
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	173	11
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	11	1
Locale/*/csv (7 locales)	14	0

The prompt in the middle of the run — "jog to a stock corner, then Zero All" — was the slowest part of this tool, and it asked for a promise rather than reading a signal. After Setup the work origin *is* the stock corner with Z0 on its top, so there is nothing to ask.

- **It also carried a trap.** The prompt fired before the program's own G54 line, and Zero All zeroes whichever WCS is *active* — so an operator working in G55 zeroed G55, and the program then ran against whatever G54 held. Both are gone: the program inherits the active WCS and never changes it.
- **G49 added.** A tool length offset left live by an earlier probe shifts every Z in a program whose entire cut is a 0.3 mm scratch, so any offset at all either buries the V-bit or marks air.
- **The editor shows the live work origin and the pattern's reach**, and Generate refuses when the pattern leaves the machine envelope — through the shared reachable-limit helper (#334), so the homing pull-off is accounted for. It follows touch-offs made while the tab is open rather than showing a snapshot.
- **What it deliberately does not test is "is G54 defined".** There is no such state: an unset WCS holds zero, which is also a legitimate origin. So it tests consequences instead — does the pattern fit, and could work Z0 be a stock top. A work Z0 at the machine's Z home *is* refused: nothing can sit at the top of Z travel.

#337 FIX: the Measured cell fought the caret, and the scratch depth default was optimistic

File	+	−
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	13	4
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	5	1
Locale/*/csv (7 locales)	21	0

Typing 500 then .2 into the Measured column left "500.2.000". The column bound with a three-decimal format *and* updated the source on every keystroke, so each character pushed a half-typed number to the source, which raised a change notification, which wrote the formatted text back into the box being typed in.

- **The text is left alone while it is being typed** and formatted once, on commit — Enter, Tab, or leaving the cell — which is also when the closest-row highlight and the new steps/mm refresh. Swept for the same pairing elsewhere; this was the only one in the repo.
- **Scratch depth default 0.3 → 0.5.** 0.3 assumes a reference surface flatter than an unsurfaced spoilboard is, and a pair with one mark missing cannot be measured at all. Depth does not affect the result, so erring deep costs nothing. New profiles only.
- **Real tooltips on the three cutting fields**, which had none: the depth only has to be deep enough to *see* (a V-bit's line centre sits under the spindle axis whatever the depth, so deeper widens the groove rather than moving it); the scratch feed is the rate the line is drawn at, and says outright that nothing is probed in this program, because sitting beside a tab with a real search feed made 500 mm/min read as one; and the plunge feed goes straight down, so keep it below the scratch feed.

#338 FIX: changing an input left the OLD program loaded and the run strip still saying Run

File	+	−
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	78	18
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	21	9
CNC Controls/CNC Controls/AutoSquareWizard.xaml.cs	9	0
CNC Controls/CNC Controls/MacroProcessor.cs	6	1

Reported on the Tool combo — switched to "Loaded" and the bar stayed on Start instead of reverting to Generate. It was not the Tool combo. The program is built once and held, and the bar reads Run while one is held, so editing the span, depth, feeds, spindle or tool left the bar offering to run the *previous* program while the panel described the new one. Pressing it cut the old pattern.

- **Axis was worse.** It was not even a watched property, and the axis letter is baked into every line of the program — so generating for X, switching to Y and pressing Run would have cut the X pattern with the panel saying Y.
- **Swept the siblings**, since a stale program looks exactly like a fresh one. Auto Square had the same hole and no discard hook at all, so all eleven of its inputs — blade and tongue lengths, hole depths, peck, feeds — were silently ignored by an already-generated program. It drills holes.
- **The prolog parks at G30 before asking for the bit**, instead of prompting for a tool change wherever the previous operation left the spindle, possibly down in the work. That makes the program require a homed machine, and its prerequisite line now says so.
- **The safe-Z move moved to where it does something.** It sat immediately before the tool change, which retracts and parks on its own, so it was discarded a line later; without it the pattern's opening absolute plunge began at machine top and crawled all the way down at plunge feed. It now sits after the tool change and spindle start.
- **The program ends itself** rather than waiting on a race. The run watcher terminates on the sender's own streaming state — reached when the last line is acked, which for a buffered rapid is long before the motion finishes — while the discard needs the controller's program-end message, which arrives later still. Run now takes a completion callback and the wizard discards its own program, unconditionally: the pattern regenerates instantly and re-running a stale one is the exact hazard above.

The depth row is a shared-size grid now rather than a hand-set width, so it lines up with every other field on the panel.

#339 tlo.macro — one puck reference on the controller, called by the tool change and by generated programs

File	+	−
macros/tlo.macro	143	0
CNC Controls/CNC Controls/MacroProcessor.cs	132	42
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	59	15
macros/tc.macro	13	97
ioSender XL/ioSender XL/StartJobView.xaml.cs	10	57
CNC Core/CNC Core/AtcMacros.cs	2	2
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	3	0
Locale/*/csv (7 locales)	14	0

The sender carried a second copy of the tool change's puck-reference sequence, emitted inline into generated g-code, and it had already drifted three ways: it sought to Z-80 where the macro had been raised to Z-90 after a short V-bit alarmed five millimetres short of the puck; it hardcoded G54 on return where the macro restores the *caller's* WCS and says in as many words that it must not be hardcoded; and it picked one probe input unconditionally where the macro chooses between three.

- **The last two could never have been right in a streamed program.** Both need o-word branching, and o-word flow control cannot be streamed to grblHAL — a bare ELSE or ENDIF is dropped by the line pipeline and leaves the controller's o-word engine waiting for an ENDIF that never comes. The copy was not merely at risk of drifting; it was structurally incapable of matching.
- **So it is a controller macro now, called like the corner probe.** Extracted from the tool change and verified line by line: every motion line is identical — the seek and its latch, the probe-input pair, the puck approach, the offset apply and the reference store. The only changes are the two lines it absorbed to become self-contained and the parameter renames that give it an interface.
- **The emitter passes a tool *id*, not a "touch plate" flag.** Which probe input to use is the macro's decision; the caller's job is to pass the fact, not pre-empt it.
- **It ends at the puck, deliberately**, because the tool change goes home its own way and would only undo a return baked in.
- **Scratch calibration reuses it instead of cancelling the offset with G49.** The G49 added a few commits earlier was backwards: on this machine the tool length offset is what makes one Z0 mean the same thing for every tool, and cancelling it leaves Z0 referenced to whatever tool last had one. With Tool = Loaded nothing re-applies it, so the V-bit cuts at the previous tool's depth — precisely the fault that rapided a job 15 mm into the material on 2026-08-06. A new "Reference the loaded bit at the puck" option, default on, gated on a tool setter being defined.

REQUIRES A MACRO REINSTALL - tlo.macro is part of the required set, so Machine Setup > Controller macros will offer the upload. Until it is uploaded, any program calling it gets error:81.

#340 SAFETY: a tool-length reference must hand back a parked machine in the caller's own WCS

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	36	12
macros/tlo.macro	22	2
ioSender XL/ioSender XL/StartJobView.xaml.cs	19	10
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	6	1

Two failures on real hardware, from the same root.

- **A 60° V-bit was driven into the toolsetter at 15000 rpm and destroyed it.** The macro ends standing *on* the puck, lifted 10 mm — a documented post-condition — and the caller-side G30 park was then removed from the scratch program as "a pointless round trip". It was the only thing standing the spindle off the puck. What followed was spindle start and a G0 to a *work* Z, far below it.
- **Setup then alarmed right after its Z-top probe.** The macro also leaves G59.3 active. The corner probe's face seek is deliberately not a G53 move, so it addressed the puck's frame and asked for a target 816 mm outside travel.

The emitter now guarantees both: it re-selects the caller's WCS by name and parks at G30 after the call, with each caller passing its own WCS. That does not replace the macro's own restore — it makes that restore not load-bearing, which is the only arrangement where getting it wrong is survivable. The macro's header, which told the next reader that leaving the machine on the puck was the caller's problem, is corrected: that sentence is what the V-bit followed.

Why the macro's own WCS restore did not take is NOT yet answered - one theory was already disproven by the log. It now prints what it captured and what it is restoring, so the next run says which. REQUIRES A MACRO REINSTALL.

#341 FIX: the first tool-length reference after a controller reset died on an unset parameter

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	15	0
macros/tlo.macro	8	2

Setup's Generate and Run, ninety seconds after a reset: the macro's IF line came back as *error 2* — a missing or badly formatted value. It branches on a touch-plate flag, and its header said that flag is "set by the tool change's own header; read here so both callers see one rule". That described the machine truthfully and the *contract* wrongly: it is an input, and only one of the two callers was supplying it.

- **Named parameters do not survive a controller reset**, so the branch read an undefined parameter. A tool change earlier in the same session leaves it set, so only the *first* reference after a reboot fails — and a reboot is followed by homing and setup, not by a second attempt at the thing that just failed.
- **Both inputs are emitted on every call now**, with the value taken from the controller's own answer to the question the flag asks rather than a copy of the tool change's literal constant — a second hardcoded constant beside that one is exactly what the extraction (#339) existed to prevent.

Sender-side only: the generated g-code changes, the macro does not, so nothing has to be reinstalled on the controller.

#342 Scratch calibration: a depth per mark, so an uneven spoilboard still gives two crisp lines

File	+	−
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	32	5
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml	14	3
Locale/*/csv (7 locales)	14	0

One depth across a 500 mm span assumes the reference surface is flat. It is not: a spoilboard that dips more than the cut depth marks one end of a pair and misses the other, and a pair with one mark missing cannot be measured at all — which is what happened on the first real run.

- **The row is now two depths**, first mark then second. They may differ freely: the measurement is the spacing between the two line *centres*, and a V-bit's centre sits under the spindle axis whatever the depth. Deeper only widens the groove.
- **An existing profile keeps the depth it already had** for both marks rather than silently reverting one to the shipped default. Zero is a safe "unset" sentinel here in a way it usually is not, because a zero-depth mark is not a mark and nobody sets one on purpose.

#343 One handoff for every Generate-first tab — push to the Job tab, pop and come back

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	379	23
CNC Controls/CNC Controls/AutoSquareWizard.xaml.cs	60	23
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	57	24
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	52	23
ioSender XL/ioSender XL/StartJobView.xaml.cs	32	52
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	27	104
ioSender XL/ioSender XL/MainWindow.xaml.cs	15	0

Work Order and Setup each had their own copy of "Generate makes this the loaded job, and the run's end gives the previous one back", and the three Calibration wizards had none at all — their program appeared only in a floating preview of their own while the Job tab went on showing someone else's. Two displays, two mental models, and the program about to run was not the one on screen.

- **One record now owns the bookkeeping.** Both copies had to learn the same two guards the hard way — never push a second snapshot over a program that is already ours, and never pop when the loaded job is no longer ours — and "exactly one push is outstanding" is something that can be checked in one place instead of a property two copies each maintain and a third never had.
- **The wizards' own program views are gone.** The display is the Job tab's docked list, the same one every loaded file uses. Setup's preview goes too: it was connected by one line and disconnected by the next, so it existed for a single frame.
- **Esc discards a handed-off program** and takes you back to the tab that built it. Until now the only complete discard was to nudge an input on the originating tab or reset the controller. The key is free exactly where it is needed — the Job tab has no menu entry, so the close-tab binding (#333) already fell through there — and this branch refuses while a job is running, and returns the key to everyone else when there is nothing to cancel. The status line the handoff writes now says so.
- **The switch back happens only on a clean finish.** A run that alarmed or was stopped leaves the operator on the Job tab where the failed program is still loaded and the stopped line still marked; the old unconditional switch-back moved them off the evidence.
- **Two things the conversion forced out into the open.** Auto Square named its program twice — one name at Generate, another at Run — and the handoff record *is* a name match, so that read as a foreign program. And an aborted run leaves the operator on the Job tab while the run strip would go on pointing at the off-screen tab that generated the program, so their next Cycle Start over a file of their own would have run the wizard's program instead.
- **A latent bug this uncovered, affecting every macro run:** restoring the previous program reconnects the Job tab's view synchronously, so the watcher's following disconnect was detaching that view from the program it had just restored.

#344 One prologue and one epilogue for all nine program builders

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	79	0
CNC Controls/CNC Controls/AutoSquareWizard.xaml.cs	9	3
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	8	12
ioSender XL/ioSender XL/StartJobView.xaml.cs	8	11
CNC Controls/CNC Controls/WorkOrderCompiler.cs	8	3
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	3	4

Nine builders across five tabs each opened and closed a generated program by hand, and the duplication had already drifted — the modal line was spelled two different ways for no reason anyone recorded.

- **Three methods, not one.** The *order* is not shared: Setup and the probe wizard put the modal line straight after the gate and park much later; Work Order puts its tool declarations in between; the scratch wizard parks before establishing units at all. One call would have silently reordered machine-moving g-code, and a refactor does not get to do that.
- **The prerequisite list stays a string the caller passes whole** rather than something assembled from flags. "Connected, homed" is the only condition all nine share; the rest genuinely differ, and a program must state what it needs rather than have it added behind its back.

Two things this surfaced and deliberately did NOT fix, both now stated in the code: Auto Square does not park - it ends at safe Z over the last hole, where every other generated program hands the machine back at G30, and adding the park is a safety change that wants a decision; and the scratch wizard rapids to G30 before its modal line establishes millimetres.

#345 FIX: a nested paren in a message prompt ate every space after it

File	+	−
CNC Core/CNC Core/MacroRunner.cs	61	3
CNC Controls/CNC Controls/MacroProcessor.cs	11	18

"Fit the 6.354 mm (1/4") bit or dowel it is set up for, clip the lead…" reached the operator with the sentence up to the inner bracket intact and everything past it run together into one word.

- **The parser tracks comment state with the controller's own rule** — a comment ends at the *first* close paren — and keeps spaces only while inside one. A nested paren therefore ends the comment early as far as it is concerned, and it strips every space in the rest of the line. That is precisely the corruption the paren sanitiser exists to prevent.
- **It never ran on these lines.** Directive rows were skipped outright when sanitising, on the reasoning that the pump consumes them and they never reach the wire. The reasoning is correct and the conclusion was too broad: a directive is exempt from the length *limit*, not from the *syntax*, because it is still parsed at load time like every other line.
- **The paren rewrite is now applied per line** — full sanitising for ordinary comments, parens only for directives — and it runs before the handoff (#343), so the diagnostic copy, the Job tab's list and the run all show the same program. A prompt that reads as gibberish in the list is one the operator distrusts before it is ever shown.

Verified by porting the trim and sanitise functions exactly and running them over the real generated program: 30 spaces restored to the install prompt, 3 to each corner prompt. It reaches any macro with parens in a directive message, hand-written ones included.

#346 FIX: the controller-macro upload ran at the status-poll rate — 56 seconds instead of 6

File	+	−
CNC Core/CNC Core/Grbl.cs	44	1
CNC Core/CNC Core/YModem.cs	7	2

Measured from the wire log over ethernet: one macro install took 56.4 seconds, with 261 status polls injected into the transfer and a median gap between packets of 210 ms — exactly the poll interval. 98% of the window was stalls. The transfer was running at one packet per poll.

- **This is the second time.** The earlier fix had the upload suspend polling, and that call is still there and still correct. It came back because the suspension is a single shared flag, and the thirteen short query helpers nest *inside* the transfer — the upload pumps the UI once per packet while waiting for each acknowledgement — so an inner resume clears the outer suspend. A tab activation does the same thing.
- **A 56-second transfer cannot be protected by a flag any 20 ms helper can switch off underneath it**, so the transfer takes its own claim on the link, checked alongside the flag and released in a finally. It cannot be orphaned by the helpers and it cannot be cleared by them.
- **A nesting counter is the textbook answer and is the wrong trade here:** those thirteen pairs have no try/finally, so an orphaned suspend would kill polling for the session. The flag's own sloppiness is currently what un-sticks them.

Narrower blast radius, not fixed, now written down: setting the poll interval still clears the flag, so a tab activation during a settings load resumes polling in the middle of it - those are ~20 ms, not 56 s.

#347 FIX: Clear floor defaulted to a ball nose, the one bit that cannot flatten a floor

File	+	−
CNC Controls/CNC Controls/OddJobsFeedsSpeedsDialog.xaml.cs	9	1
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	6	2
CNC Controls/CNC Controls/WorkOrderModel.cs	6	0

Adding a Clear floor operation (#325) picked a 1/4" ball end mill where the operator expected their 1/8" two-flute. The suggestion came from the "finishing" bucket, which is defined as a ball nose.

- **The justification was about the cut, not the requirement.** "A small end mill flattening a carve's floor is a finishing cut in every sense" is true and is the wrong question: the tool here is defined by the shape it must *leave*, a flat bottom, and a ball nose is the single profile that cannot produce one. It leaves a field of scallops.
- **A "flattening" bucket**, distinct from finishing, picking a two-flute end mill — placed after the aluminium branch on purpose, so an aluminium job still gets its single-flute bit, which flattens perfectly well.
- **And the guard that should have caught it.** Validation already rejected V-bits, countersinks and drills with "a pointed bit cannot flatten a floor". A ball nose is not pointed, so it read as harmless — the check was written against the failure someone imagined rather than the requirement, which is FLAT. It now rejects a ball nose with its own message.

#348 Work Order: Mark only — dimple every hole centre and cut nothing else

File	+	−
CNC Controls/CNC Controls/WorkOrderCompiler.cs	172	41
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	136	40
CNC Controls/CNC Controls/WorkOrderModel.cs	101	21
CNC Controls/CNC Controls/WorkOrderView.xaml	12	5
Locale/*/csv (7 locales)	70	0

The mill is too flexible to drill aluminium, so the job splits: the router marks where the holes go, a punch deepens the marks, and a mag drill makes the holes. Stripping the work order down to 1.4 mm dimples by hand destroys the very thing the second half of the job needs — the record of what size hole goes at each dimple.

- **Mark only is a property of the RUN, not of the content.** The work order keeps every real drill size and depth; the substitution happens on the way to the program, at the seam where the authored work order was already being resolved. Nothing downstream needs to know it exists — scheduling, tool declarations, the single tool change, the 3D view and the time estimate all come out right on their own.
- **One dimple per toolpath, not per operation.** A circle has one centre, so a toolpath carrying both a Drill and a Bore describes one hole and must not be punched twice in the same spot. Patterns come across intact, so a 3×2 grid still yields six dimples. A Bore counts as a hole: milled rather than drilled, but its centre is just as much a place a mag drill is about to go.
- **Two guards, because a program that looks like the job and is not the job is the whole hazard.** The program says so in its own header, beside the existing partial-run line and for the same reason — the file outlives the UI state that produced it — and Mark only with no enabled Drill or Bore anywhere is refused up front rather than emitting a file that parks and does nothing, which reads exactly like a machine that failed to start.
- **The summary counts what will actually be emitted** — "9 hole centres dimpled with 1/8in twist drill. Every other operation is left out." — so the number can be checked against the part in front of you.
- **FIX: it dimpled with a 1/4" END MILL.** The tool setting defaulted to 0 as a "not chosen" sentinel, and tool ids start at 0 — so the default *was* the first tool in the list, the not-chosen check passed, and the dimples were cut with whatever that happened to be. Underneath that, the fallback ran through the drilling suggestion whose no-drill-found path returns an end mill, and the dimple's diameter was read from the tool, where a drill entry's nominal diameter is typically a placeholder. The dimple now carries its own diameter, the resolve takes the first pointed tool directly, and validation refuses the run when nothing in the list comes to a point.
- **FIX: Mark only did nothing at all on a work order with indirect toolpaths.** The resolve step rebuilt the work order from a hand-written field list that never copied the flag, so the mark step received a work order whose flag had already been reset and returned immediately — generating the real job, ten 13 mm through-drills and a 35 mm bore, with no marker anywhere in it. Both copies go through the shared field copy now, the same lesson this method already documented one level down.
- **The dimple is a real operation, edited by the ordinary Feeds and Speeds dialog**, restricted to drills with its depth field relabelled. That settles three complaints at once: it supplies the tool picker Mark only never had, it brings the advisor's recommendations and highlighting along, and there is no longer a second, poorer editor of the same six numbers. The defaults come from the advisor's own aluminium drill reference rather than a general-purpose milling constant that was never about drilling.

#349 Save Drawing: list every instance with its X and Y, not just a count

File	+	−
CNC Controls/CNC Controls/WorkOrderDrawing.cs	36	0

The sheet's job is to be carried to the second machine. "Qty 6" says a pattern exists and says nothing about where to put the drill — the operator then reconstructs six positions from a grid pitch and a corner, by hand, next to a running spindle.

- **The numbers were already there.** The quantity is literally the count of the same position sequence the drawing walks to place its balloons. Printing them costs one row each and removes the arithmetic.
- **An indented row per instance**, labelled with the feature's own key letter and an index, carrying its resolved X and Y in the same right-aligned columns the feature rows use — so a pattern's positions read straight down beneath the centre they were laid out around. Only when there is more than one, since a single feature's position is already on the row above it.

#350 FIX: two warnings the reader could do nothing about

File	+	−
CNC Controls/CNC Controls/WorkOrderModel.cs	50	10
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	7	11
CNC Controls/CNC Controls/WorkOrderCompiler.cs	3	10

Both said something true and neither could be acted on, which is worse than saying nothing: it teaches the operator to scroll past the panel that also carries the real warnings.

- **"1/4" 2-flute endmill has no point"** named a tool Mark only never offered to change. The validator read the raw setting while the compiler resolved past it to a drill, so the message described a bit the program was never going to use — and a work order saved with the old default hit it every time. The "pick a pointed bit" rule existed in three places that disagreed; there is one now, and the warning is reduced to the single case the operator can fix: nothing in the tool list comes to a point.
- **"Ø5.2 mm is not a size this build knows as a standard drill"** was a statement about the list, not about drills. Metric jobber drills are sold in tenths and the series stepped in half-millimetres, so 5.2 was refused while 5.0 passed. This is the second report of that exact shape — the M5 tap drill was refused the same way — so the step is fixed rather than the symptom: 0.1 mm, 121 entries, 1.0 to 13.0. The named entries stay, but only for the name they give a match, never for whether it is accepted.

#351 FIX: an assignment's right-hand side cannot be a bare binary expression — error:1 on rotation

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	8	1
macros/pcorner.macro	7	2

Setup with Measure and Set rotation probed all four corners cleanly, printed its results, and then died on its own rotation line with *error:1 — G-code words consist of a letter and a value*.

- **Read from the firmware rather than guessed.** A named-parameter assignment parses as name, equals, then a *real value* — and that accepts exactly four things: a bracketed expression, a parameter, a unary sign on a non-digit, or a plain number. There is no binary operator anywhere in it. So the line read the leading zero, assigned it, returned, and left the rest of the expression sitting in the block for the g-code word scanner.
- **Which is why it survived review:** the expression is valid NGC, just not valid as an unbracketed right-hand side.
- **Swept rather than spot-fixed** — every named-parameter assignment in the macros and in emitted g-code, checked for a top-level operator outside brackets. The second real site is the corner macro's inside-corner sign flip, which has the identical fault and would abort the probe the same way. It is latent only because that block runs for an inside corner, which is not what this machine has been doing.

pcorner.macro changed, so this triggers a controller macro update on the next connect.

#352 Controller macros: send only the ones that changed, not all five

File	+	−
CNC Core/CNC Core/AtcMacros.cs	128	14
macros/pcorner.macro	2	0

Measured: a full set is about 500 transfer packets, the controller takes ~50 ms to commit each 128-byte block to its filesystem, and the whole thing takes 23 seconds. After a one-line edit to one macro, 13 of those seconds were for the file that changed and 10 were for the four that had not.

- **The sidecar is a per-file manifest now** — one hash-and-name row per macro — and provisioning decides per FILE: send it if it is absent, the wrong size, or its hash differs. Editing the tool change or the tool-length macro, the common case when working on tool changes, drops from 23 seconds to about three.
- **Hashes rather than sizes**, because size already had this job and cannot do it: it catches truncation and misses any edit that preserves the byte count, which a one-line change very often does.
- **The migration does not cost what it appears to.** An old single-hash sidecar parses to zero rows, which reads as "everything stale" — so that exact shape is recognised, and when it still matches, every macro is treated as current and the sidecar is rewritten without sending a single file. Otherwise the upgrade whose entire purpose is avoiding needless uploads would have opened with one, on every machine.
- **The Machine Setup grid marks only the macro that actually changed** as outdated, instead of all five.
- **FIX: a freshly written sidecar re-prompted on every connect.** The migration worked — first connect recognised the legacy hash, sent nothing, rewrote the manifest — and the connect after that reported it stale with nothing to upload, forever. The file read back from the controller is assembled with the platform's line ending and the manifest is written with newlines, so the raw string comparison could not match whatever the controller held. It compares the PARSED manifests now: does the controller hold exactly these files at exactly these hashes, which is what the question meant in the first place.

Worth naming the process failure as well as the bug: the first version was verified by porting the functions and feeding them text generated to match - which tested the model of the read path rather than the read path. The second test is built from the bytes the controller actually returned.

#353 FIX: WCS rotation was read once at connect and never updated again

File	+	−
CNC Core/CNC Core/Grbl.cs	25	2
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	15	0

Setup with Measure and Set rotation, then Generate and run a work order: Alarm:2 on the program's first machine move, with the machine parked at G30 and nothing moved. The rule that refuses to generate a work order against a rotated WCS — whose own comment calls it the only thing standing between a rotation set by ordinary Setup use and a program that cuts skewed — had run, and passed.

- **Its input was stale.** The coordinate-system cache is refreshed only when something asks for it, and nothing does when a program changes a WCS: Setup writes the rotation itself and the cache keeps its old answer. The log shows the read at 17:29 returning zero rotation, Generate passing on it at 17:35, and the next read at 17:35:51 returning −0.10°. A safety gate cannot read a cache whose age is unbounded, so Generate re-reads first — one round trip on an explicit action.
- **That was necessary and did nothing**, because the refresh could not update the field being asked about. The rotation in the payload was parsed only in the coordinate system's *constructor*; every later read went through a position parse that keeps only the coordinates, by design. So the value stayed frozen at its connect-time reading, and the log shows a refresh running with the work order generating six seconds later against a rotation still reported as zero.
- **It is parsed in exactly one place now**, called from both the constructor and the update path. The split between those two was the entire bug.

Belongs to the stale-cache family, where this same shape has now appeared seven times.

#354 WCS rotation: fix the one operation that breaks, and stop refusing all the others

File	+	−
CNC Controls/CNC Controls/WorkOrderModel.cs	17	2
CNC Controls/CNC Controls/WorkOrderCompiler.cs	16	1

The gate refused to generate *any* work order against a rotated WCS, on the grounds that its toolpaths do not account for rotation and would cut skewed. That reason does not survive reading the code: every cutting move the compiler emits is a WORK-coordinate move and the firmware rotates those itself — which is the entire point of measuring a skewed stock — while the retract and park are machine moves, which the firmware exempts from rotation outright.

- **Exactly one operation genuinely could not tolerate it.** Entire spoilboard is the only place in the compiler that mixes frames: it takes the travel-envelope corner as machine coordinates, uses it both as a machine target and as a scratch WCS origin, then cuts the whole raster in work coordinates. A rotation on that slot turns the raster while the machine-referenced arithmetic assumes it is square — and it is the one toolpath sized to the full envelope, so there is no margin to absorb the error.
- **Fixed at its source rather than by forbidding the feature:** the write that claims the scratch slot now clears the rotation with it, so the frame is unrotated by construction. Claiming a slot with coordinates does not clear a rotation on its own, which is why it could arrive carrying one. Gated on the controller reporting rotation support, since the word is an error without it — and a controller without it cannot have a rotation to clear.
- **What is left is an advisory, not a block:** the stock canvas and 3D preview draw unrotated, so with a rotation set the picture will not match the finished part.

This also unblocked the diagnosis of the unexplained Alarm:2 (#353, #358) - the gate was refusing to generate the very program needed to reproduce it.

#355 Run strip: hide Feed Hold and Stop when they mean nothing

File	+	−
CNC Controls/CNC Controls/JobControl.xaml.cs	51	0
CNC Controls/CNC Controls/JobControl.xaml	4	2

A greyed-out button still asks to be read.

- **Feed Hold appears whenever the machine CAN move** — Run, Hold, Jog, Tool, Door. Jog is the one that matters and is easy to miss: grblHAL reports it as its own state rather than as Run, and a feed hold during a jog decelerates and cancels it. Hiding it there would remove the brake in precisely the case it is armed at idle for.
- **Stop is about a JOB.** With nothing streaming there is nothing to stop, so it is visible exactly when pressing it would do something.
- **Collapsed rather than hidden**, because both are the last two controls in the strip, so reclaiming their space shifts nothing. If anything ever lands to their right this has to become hidden — a control bar whose buttons move under the cursor is how a mis-click happens.

The enabled-ness logic is untouched; this only decides what is worth showing.

#356 FIX: prefer network proved a port was open, not that the controller answers — and retried forever

File	+	−
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	78	6

Two faults in the same path, both seen during a network outage, and together they put the machine in CHECK MODE.

- **The probe asked the wrong question.** It completed a TCP handshake and called that "reachable" — but the caller is about to throw away a working serial link on the strength of that answer, so the bar has to be "the controller responds". Something accepted the socket and never spoke g-code; the migration dropped serial onto a mute link, the connect handshake went unanswered five times, and the resulting "could not read capabilities" prompt has a "stay connected" option that sends the check-mode command. It now sends a status request and requires a report back — that is answered from the controller's own input handler rather than the g-code queue, so a live controller replies even while running, held or alarmed. There is no state in which it stays silent. Anything between the angle brackets is proof; an open port is not.
- **It retried forever.** The migration is attempted every time controller info loads, which includes every reconnect, and its only guard was cleared in a finally: connect, load, migrate, fail, back to serial, load, migrate. It is latched per app run now, at the point the attempt is *committed* rather than where it succeeds — an unreachable controller has to spend the shot too, or a network down for the whole session re-probes on every reconnect.
- **Both outcomes now report**, and say they will not be retried until restart. The declined path used to say nothing at all, which reads as the option being ignored.

#357 FIX: WCS rotation pivots about MACHINE zero — the origin was 0.9 mm out and the sign was backwards

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	55	17

grblHAL's rotation block computes machine = R(rot) × (work + offset) + G92 + TLO. It rotates the SUM, so the pivot is machine zero, not the WCS origin — and the offset is stored raw, with nothing counter-rotating it. Two things were wrong because of that.

- **SIGN.** Work +X maps to machine direction (cos rot, sin rot), so aligning work X to a front edge at +θ needs R = +θ. We emitted −θ, pointing the work frame 2θ the wrong way — 1.21 mm across a 458 mm stock at θ = 0.075°. The comment that justified the negation blamed the un-negated form for exactly that symptom; the stock behind that reading was itself 1.7 mm out of square, which is the likelier culprit.
- **ORIGIN.** Work zero landed at R(rot) × C instead of the probed corner C. The error is |C| × rot with |C| measured from MACHINE zero, so it grows with how far out the stock sits — 0.89 mm at 674 mm on the panel measured that day — and it cancels nowhere. The counter-rotated origin is stored instead.
- **The two compounded:** on that panel the work frame was out by 0.89 mm at the origin and 1.63 mm at the front-right corner, displaced and mis-rotated at once.
- **Measured on the machine, not inferred:** a known origin, a 5° rotation, and a commanded work X0 Y0. The formula predicts 186.602,−648.068; the controller reported 186.605,−648.070.
- **The plain origin write stays as it was.** It has to survive firmware without rotation support, where any R word errors and halts, and a raw origin is exactly right when there is no rotation to counter. Only the rotation block, which runs where rotation is known supported, restates the origin in counter-rotated form.

Verify skew now writes the origin with its own R word for the same reason: the button is enabled on four probed corners alone and never required the measure to have applied a rotation, so inheriting the origin could test a frame nobody built.

#358 FIX: a G53 Z-only lift traversed the whole table — clearing a rotation corrupts the parser's position

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	46	7
CNC Controls/CNC Controls/WorkOrderCompiler.cs	12	0

Root cause is in grblHAL, found by reading the g-code parser and confirmed by reproducing both observations arithmetically. When a coordinate-system write touches the rotation of the system that is CURRENTLY ACTIVE, it converts the parser's position machine → work, applies the new data, then converts work → machine. The two halves are guarded independently when they have to be paired: the first runs if the OLD rotation is non-zero and differs from the new; the second runs if the NEW rotation is non-zero.

- **So clearing a rotation converts to work and never converts back** — the parser is left holding work coordinates it believes are machine coordinates. Setting one from zero has the mirror fault, and rewriting the same non-zero rotation runs the second half alone. Any later move that leaves an axis unnamed then holds that axis at the corrupted value, and a G53 Z-only lift is exactly that.
- **Two reproductions, both matching to the millimetre.** Clearing 5° on an active G55 leaves the parser at 0.003,−0.003, and the next "G53 G0 Z0" — a Z-only line, with Z already at zero — traversed the table to machine 0,0 at rapid. Clearing −0.10° the night before dragged the same lift's X by 1.140 and Y by 0.224; the formula predicts 1.156 and 0.226.
- **The real fix is to pair the guards in firmware**, which is a different repository and is tracked separately. Here the damage is repaired where it is caused: every rotation write is followed by a dwell and a machine move naming X and Y from the parameters that report the STEPPER position, not the parser's — so the no-op move overwrites the parser's position with the truth however corrupt it had become.
- **The dwell is load-bearing.** Those parameters are read at PARSE time, which runs ahead of motion, so mid-stream they would answer with a position the machine has not reached and the "no-op" move would drive backwards to it. Z is deliberately not named — only the plane axes are corrupted, and naming Z would turn the repair into a plunge.

Seven sites in Setup - the measure prolog, its rotation block, the vise prolog and Verify skew's four - plus the compiler's scratch-WCS slot, which is exposed because nothing deactivates that slot at the end of a run. A G53 Z-only lift is NOT a safe lift on its own.

#359 FIX: a bare origin write turned the next Z-only lift into a 662 mm rapid

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	46	0
ioSender XL/ioSender XL/StartJobView.xaml.cs	9	4
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	8	2
CNC Controls/CNC Controls/AutoSquareProbeWizard.xaml.cs	4	1
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	4	1

Observed on hardware, on the first run of the new Squareness (probe) tool, aborted by the operator. The machine stood at 20.001,−20.003 with G54 at 128.392,−662.315 and a 0.10° rotation. The program's fifth line zeroed G54's offset; its next line was the park's "G53 G0 Z0" — a Z-only lift, naming no X or Y — and the machine rapided to 148.4,−682.3 at 16824 mm/min with a probe in the spindle. That is the position plus the old offset, matched to a thousandth of a millimetre.

- **Same firmware fault as #358, through a door that commit did not cover.** This write carries NO rotation word at all, so old and new are both 0.10: the first guard fails on "old differs from new", the second passes, and the machine-frame half runs alone. It is enough for the active WCS to merely HAVE a rotation — which, since rotation shipped, is an ordinary state rather than an exotic one.
- **The repair already existed and simply was not reachable** from the three generators outside Start Job. It is moved verbatim to the shared emitter, so there is one implementation.
- **Swept every caller**, which is the point — the new tool was not the only one standing in this door, only the one that happened to run with a rotation live: the squareness probe wizard, the stepper probe wizard's two programs, the fixture dialog's test position, and Setup's vise program.

RESIDUAL, recorded rather than patched: pcorner, pcenter and pvisecorner each carry the same bare line. In practice every caller zeroes the WCS before calling them, so the offset delta is zero and nothing moves - but that is an argument about callers, not a guarantee from the macro, and changing it costs a controller reinstall. The repair itself is HARDWARE-VERIFIED: the re-run's lift moved 0.5 and 0.8 mm - the blend into the next line - not 662.

#360 FIX: the console threw you to line 0 on every flush while you were scrolled up

File	+	−
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	34	12
CNC Controls/CNC Controls/ConsoleControl.xaml	11	2

Assigning the text box's content snaps its scroll to the top, so the flush has to put the view back afterwards. It only ever did that for the at-bottom case — the scrolled-up case kept the snap. With a connected machine the log changes several times a second, so reading anything above the fold was impossible.

- **The same TEXT is held still, not the same pixel offset.** Once the scrollback is at its 2000-line cap each line added drops one off the top, so an uncorrected offset creeps down the log at the stream's rate. The lines removed are counted — the cap's own trim, and "clear up to here" — and subtracted at the line height the current extent implies.
- **The follow-the-end helper is dropped from this box.** It decides whether to follow from inside a scroll handler, where a wholesale text replacement and a user dragging the thumb are indistinguishable — both are an offset change with no extent change once the scrollback is capped. It read the replacement's snap as "scrolled to the top" and stopped following, which is the other half of this same bug reported on 2026-09-06. The flush knows a replacement is coming, so it is now the only thing deciding where the view sits.

#361 FIX: degree, delta and diameter signs were UTF-8 encoded twice

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	3	3
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	2	2

The stock readout showed "skew −0.08Â°" with a mangled delta beside it. The bytes in the source were a degree sign encoded to UTF-8, read back as Latin-1, and encoded again.

- **Four sites** — the skew and vise-skew readouts, the angle label on the result drawing, and the two bore-strategy strings. A repo-wide sweep for the double-encoding signature finds nothing else.
- **Both files keep their byte-order mark**, which is what makes the compiler read them as UTF-8 rather than the system codepage; without it this fix would re-break at compile time.

#362 Verify skew: probe the whole descent, and a fly-over mode that needs no probe at all

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	149	38
ioSender XL/ioSender XL/StartJobView.xaml	14	3
Locale/*/csv (7 locales)	28	14

The per-corner approach was a RAPID to a work Z ten above where work Z0 was set, and then a 13 mm probe. Work Z0 only means the stock top while the tool-length offset active during the measure is still active — and this program's own prompt asks you to fit a 3D probe, which is exactly a tool change. Measure with the touch plate, verify with the probe, and that rapid aims at a height wrong by the difference in tool length: into the stock if the probe is the longer one.

- **The descent IS the probe now.** Cross to the point at machine top, then a single probing move all the way down. A probing move stops on contact, so a stale offset can only make it touch high or miss — it cannot drive in. Each corner targets its own measured top, so the stock's flatness is followed rather than assumed away; the board it was written against spans 0.3 mm.
- **New "Touch corners" checkbox.** Unticked makes it a FLY-OVER: the six frame points are visited, nothing descends, nothing is probed. That needs no probe defined or fitted and has no tool-length dependence at all — you sight the tip against each corner. Worth having because the touching form's resolution is set by the tip radius anyway, so a frame rotated or displaced wrong shows up to the eye about as well as it does to the probe.
- **FIX: the no-probe mode was unreachable, behind a message that named no probe.** Shipped default-ticked, so on a library of tool setter plus touch plate — a normal setup, and the one it was first run on — the button's only reachable behaviour was a dialog saying "select a probe definition first", which reads as nonsense against a library that already has two. The checkbox follows the probe library now: with no 3D probe defined it is cleared and disabled, so Verify skew does the fly-over. Only forced when unavailable — once a 3D probe exists the operator's own choice is left alone — and refreshed when the library is edited, not only after a measure. Its tooltip shows on the disabled control, since the disabled state is the one that needs explaining, and if the guard is ever reached it says which kind of probe is missing and why a touch plate cannot stand in.
- **The fly-over descends to Safe Z delta above each corner**, not machine top: on this machine that is ~104 mm over the work, and you cannot judge a corner down a sight line that long, which was the whole point of the mode. The descent happens only once the tool is over the point and it retracts before the next crossing, so the low height is never held while travelling. Plate thickness is added on top, exactly as the measure does — the corner Z is already plate-corrected, so clearing the stock top by a margin under the plate's own thickness would fly straight into it.

Also drops the old note claiming no machine move may run while a rotation is active. The firmware exempts them; what actually needs care is a machine move right after a rotation write, which #358 repairs.

#363 Persist the measure result, so a rebuild no longer costs a four-corner probe

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	152	10
CNC Controls/CNC Controls/StartJobConfig.cs	12	0

Re-running a measure purely to get back to a state where Verify skew is available costs minutes of machine time, and every code change was costing one.

- **One opaque string rather than twenty fields.** It serialises through the existing settings section with no schema work, and a format that fails to parse restores NOTHING instead of half a frame — which matters because these corners are what Verify skew writes an origin and rotation from. Values are millimetres and machine coordinates, the frame the corners are probed in, never the display unit the operator can toggle.
- **The readout says when a restored measurement was taken** — "re-measure if the stock has moved". Restoring silently would make a measurement from days ago look like one taken just now, against stock that may have been re-clamped or replaced. A fresh measure clears the marker.
- **FIX: it never restored — two off-by-ones in the parser.** The length check tested for one field too many and the corner loop read from one index too far. Either alone would have been caught; together they hid each other, because the length check rejected every saved string before the loop could throw. Both indices derive from the format's own constants now.
- **FIX: a failed restore DELETED the measurement it could not read.** The save path serialised "nothing complete" as an empty string and wrote it over whatever was stored, so a restore that failed for any reason left the next save to destroy the data. A reader must never be able to destroy what it could not read: the string this session loaded is handed back verbatim when there is nothing complete of its own, including one that failed to parse. That also covers the ordinary case with no bug at all — a session that simply has not measured yet no longer wipes the last one on exit.
- **FIX: the restore threw on its own timestamp** and wiped the corners it had just parsed. Two date-parsing flags that cannot be combined do not return false, they throw — after the four corners had already been assigned, so the catch-all cleared a measurement that had read perfectly well. The timestamp is parsed in its own try now: it is a caption, and the corners an origin and rotation are written from must not ride on it.
- **The silent return is gone.** The restore logs why it rejected a string and what it restored when it worked. It returned silently through two builds while the readout showed a dash, which is indistinguishable from "never measured" — so a broken parse looked like a missing feature, twice.

Found by reading the log rather than reasoning about it: the debug line added in the previous fix named the exception on the first run, after two builds spent inferring, both wrong.

#364 FIX: a debug build logged ONLY the status poll — every other instrument was filtered out

File	+	−
CNC Core/CNC Core/DebugLog.cs	22	7

The debug-build branch set the category filter to the poll channel when no flag was given, and the comment two lines above calls that additive. It is the opposite: a null filter already means "log everything", so replacing it with one category subtracts the other twenty.

- **It looked exactly like working instrumentation** — a growing file, a poll line every second — while every trace anyone actually reaches for was dropped at the category check.
- **It cost a diagnosis.** A false "program does not fit" warning has two log lines written specifically to tell a bogus bounding box from a real one, added after that warning misfired once before. Neither was in the file, and an empty log answers "did we instrument this?" with a confident no.
- **Debug builds log every category now.** Volume is the price and a debug build is not where that is worth optimising: a development session is precisely the session where something unexpected happens, and nobody gets to re-run it with the right flag afterwards. An explicit category flag still narrows to what it asked for.

#365 Verify skew fly-over: sight from 2 mm, inset by the tool radius, and cross each pair at height

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	155	34
ioSender XL/ioSender XL/StartJobView.xaml	12	0
CNC Controls/CNC Controls/StartJobConfig.cs	4	1
Locale/*/csv (7 locales)	14	0

Three reports from running it, each narrowing the check until it can actually be read.

- **2 mm, not 37.** Safe Z delta plus the plate came to 37 mm on this setup, and from there you cannot tell whether the tip is on the corner or 3 mm off it — which is the entire question the fly-over exists to answer. 2 mm is honest here in a way it would not be anywhere else, because the height is measured from the stock top *as the measure probed it*, in the tool-length frame active at the time: same tool fitted, and the tip is exactly 2 mm over the surface. What breaks that is a tool change between the measure and the check — and restoring corners (#363) makes that likelier, not less — so the prompt states the gap, states the assumption, and names the measure's date when the corners came out of storage. The plate term goes with it, since 2 mm above the stock top is *inside* a plate still sitting on it; the prompt says to take it off.
- **Inset by the tool radius.** The fly-over put the tool CENTRE on each corner, which sounds like the honest choice and is the wrong one: a 1/4" tool centred on a corner covers it, so the operator is sighting at the one thing the tool is hiding. Inset is the convention the probing mode already uses, where it is forced — the tool then sits fully on the stock with its circumference passing through the corner point, both edges visible, and the judgement becomes "does it just kiss the corner". The diameter is whatever is in the collet, not a probe tip, since there may be no 3D probe defined at all.
- **Each back-corner PAIR is crossed at the sight height.** Those corners are visited twice — the ideal rectangle point, then the corner actually probed — and the whole value of the pair is seeing the gap between them, often well under a millimetre. Retracting 100 mm and coming back down somewhere new destroys exactly that: you compare two positions from memory with a full-height move in between. The second of each pair now crosses at the same height, so the difference is something you watch happen. Fly-over only — in probing mode the tool is standing on the surface and moving at that height would drag it across the stock.
- **Change to a V-bit first, and put its point on the corner.** A V-bit's tip IS a point, so it sights a corner exactly, with no inset — the right answer for a round tool is a compromise either way. And the tool change applies an offset against the machine-wide baseline, which puts the V-bit's length in the SAME frame the corners were measured in, so the 2 mm gap becomes correct by construction instead of an assumption about what is in the spindle. A picker rather than a hardcoded tool number, restricted to V-bits and chamfer tools, with "(none)" leaving the spindle alone and keeping the radius inset. The change goes in before the rotation is applied and before any corner is visited.

#366 FIX: the V-bit dropdown was full of blank rows, and now defaults to the sharpest

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	41	6
CNC Controls/CNC Controls/StartJobConfig.cs	1	1
ioSender XL/ioSender XL/StartJobView.xaml	1	1

The tool type exposes its id, name and kind as public FIELDS, and WPF's display and value paths resolve properties only — a field resolves to nothing without complaint. So the list rendered one empty row per tool and the selection never matched anything. It looked like an empty dropdown; it was a full one with no text in it.

- **Projected onto a small item type with real properties.** The label carries the angle, because the names in the tool table do not reliably agree with the angle recorded against them — one called "45° V-bit" carries 90 — and the number shown is the one the sort and the default actually use.
- **Sorted sharpest first**, and an unchosen setting selects the sharpest rather than "(none)": the finer the point, the less there is between it and the corner being sighted.
- **"(none)" is a real answer**, so "never chosen" needed a sentinel of its own — reusing the obvious one would have made the sharpest-V-bit default overwrite a deliberate choice of none on every load.

#367 Scribe square — the first verification in this panel you can measure instead of eyeball

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	158	0
ioSender XL/ioSender XL/StartJobView.xaml	15	3
Locale/*/csv (7 locales)	14	0

Every other check in this panel judges a POINT, and a point can only be judged by eye — which is why a 1.2 mm sign error (#357) survived for weeks behind a check whose resolution was the probe tip radius. A scribed line turns an angular error into a TAPER read with calipers over the full length of the stock: get the rotation right and the gap from the line to the front edge is a constant 10 mm end to end; get the sign wrong and it tapers by twice the length times the sine of the angle — 2.6 mm on this stock, unmissable with a rule.

- **A rectangle inset 10 mm from every edge of the measured frame**, cut with the selected V-bit 0.25 mm below the surface. The other three gaps taper by however far out of square the stock is, and the panel states the predicted numbers — predictions to check, not just output.
- **Built on the PROBED frame, not the measured means.** The means average two opposite spans, so a rectangle from them sits 10 mm from nothing in particular, while one built from the frame being verified makes "exactly 10 mm from the front edge" an exact claim.
- **Depth comes off a bilinear interpolation of the four probed tops**, so the 0.3 mm of flatness in this stock cannot make it dig at one corner and skip at another. Along each side one parameter is constant, so the interpolation collapses to linear and a single move carrying X, Y and Z is exact over that side — no segmentation.
- **Its own button, not a third mode on Verify skew.** This one runs the spindle and leaves a permanent mark, and that must not sit one checkbox away from a check that touches nothing. The prompt says it cuts, and names the depth, the size, the speed, and that the mark lands on the part.
- **FIX: the button shipped invisible.** That row is a horizontal stack and had grown to two buttons, a checkbox, a label, a combo and now a third button — wider than the column at its minimum width. A stack panel does not wrap: it lays its children out past the edge, and with horizontal scrolling disabled the control is simply not there, with nothing on screen to suggest it exists. It wraps onto a second line now.

#368 Squareness (probe) — measure the gantry against a reference square instead of sighting pins

File	+	−
CNC Controls/CNC Controls/AutoSquareProbeWizard.xaml.cs	1103	45
CNC Controls/CNC Controls/AutoSquareProbeWizard.xaml	117	0
macros/pcorner.macro	36	4
CNC Controls/CNC Controls/CalibrationView.xaml.cs	15	3
CNC Controls/CNC Controls/CalibrationView.xaml	10	2
CNC Controls/CNC Controls/AppConfig.cs	7	0
CNC Controls/CNC Controls/IGrblConfigTab.cs	1	0
Locale/*/csv (7 locales)	273	14

A second Squareness tab, paired with the existing one the way stepper calibration pairs (probe) with (scratch). Neither replaces the other: the pin method needs no probe, so it stays as the answer for a machine that has none.

- **The inversion is the point.** In the pin method the machine is the WRITER — the drilled holes are the machine's own idea of a right angle — and a framing square is the READER, at eyeball resolution. Here the square is the artifact and the probe is the reader: a sighted gap at a pin is ±0.05–0.1 mm against probe repeatability of 0.01–0.02, which over a 600 mm blade is 0.002° rather than 0.01°.
- **And it iterates freely.** After Apply and a re-home the gantry angle has CHANGED, so the L just drilled sits at the old angle and cannot verify the new one — every pin iteration costs a fresh L, a bit, a Z touch-off and more spoilboard. A re-probe costs minutes.
- **It probes the heel, the blade end and the tongue end** through the same corner macro Start Job and the stepper probe wizard already drive — all three are real outside corners with both faces, so the macro handles them unchanged. There is no fourth corner and nothing here wants one: the skew is the angle at the heel, which takes three points.
- **The face-probe depth is a caller parameter now, not a fixed 5 mm.** That fixed depth silently assumed the stock is thicker than 5 mm, and this tool probes a steel framing square — 4.8 mm on the one measured for it. Its face probes would land 0.2 mm *below* the square's underside and seek through air: a probe-fail alarm, or a confident reading of whatever is underneath. Note the trap: raising the square on a spacer does not help, because the depth is relative to the top just probed and rises with it. The new parameter is optional and unset means the old behaviour exactly, so the four existing callers are untouched.
- **The SIGN of the correction is deliberately not derived.** Which way a positive squaring offset racks the gantry depends on which rail the firmware's ganged motor drives, and that is not readable from the controller. So there is an invert checkbox, and the how-to and the Apply dialog both say to re-measure after the first Apply. The alternative was to reason it out and ship a sign — which is exactly what was done for the rotation word (#357), where a plausible derivation defended by a confounded hardware reading stood for weeks while every job cut double the error it was correcting.
- **The offset arithmetic is a DELTA on the current offset**, where the pin tab maps its measured gap to an absolute one. The probe measures the RESIDUAL error with whatever offset is in force, so a delta is what that measurement means; the two agree only when the offset started at zero, which is why this tab's how-to says to clear first.
- **FIX: it stopped handing the machine back cancelled.** The program's fifth line threw away a live tool-length offset. For the measurement that is harmless — it is XY only and the macro works in machine coordinates — but the run then *ends* cancelled, so work Z0 sits a whole tool length too deep, silently,

waiting for the next job that emits no tool change. What it does instead is reference the probe now fitted, at the puck: fitting a probe IS a tool change, so restoring the offset the run inherited would hand back a confident number measured against a tool that is no longer in the spindle. A new "Set TLO reference" option, default on, gated on a tool setter being defined, with a red line when it is ticked and no puck exists.

The settings lookup, rail span and Apply/Clear/Re-home are duplicated from the pin tab rather than factored out, deliberately: that tab is hardware-proven and this one had never run, so a shared refactor is the one change that could break the working method to serve the unproven one. The two tabs will not agree on a machine already carrying an offset, which is worth knowing.

#369 Squareness (probe): the reversal test — separate the square's error from the gantry's

File	+	−
CNC Controls/CNC Controls/AutoSquareProbeWizard.xaml.cs	433	75
CNC Controls/CNC Controls/AutoSquareProbeWizard.xaml	13	0
Locale/*/csv (7 locales)	56	0

A single reading is (square + machine) and cannot be split, so correcting it to zero squares the gantry TO the square. On this bench the residual reached 0.0034° while the framing square's own tolerance is several times that — the measurement had run out of meaning, not out of precision.

- **Flipping the square mirrors it**, which reverses the sign of ITS error and not the machine's. The square's error is half the sum of the two readings; the machine's is half the difference.
- **The flip forces the heel to MOVE, and that drove the design.** A mirrored L cannot lie with both arms in the same quadrant, and no rotation undoes a mirror — so the reversed orientation registers the far end of the BLADE in the existing corner fence and puts the heel out at blade length, tongue running back from there. The heel becomes the skew's vertex instead of the fence corner.
- **Points are stored by ROLE — heel, blade, tongue — not by corner number**, and that is what keeps the orientation out of the arithmetic: the skew, the span readouts and the adrift-span guard are all unchanged and orientation-blind. Only the generator branches, and even there the first two probes are identical in both orientations, because reversing swaps which END of the blade sits in the fence.
- **The square's error is PERSISTED once measured**, which is the real payoff: it describes a physical object, so every later single reading yields the machine's error by subtraction and the reversal never has to be repeated unless the square changes. Apply and Clear drop the reversal halves — they were measured against a gantry that no longer exists — but KEEP the square calibration, which is exactly the asymmetry the physics has.
- **Two ways to get a confidently wrong answer, both refused rather than averaged.** Halves taken at different squaring offsets are not comparable, so the offset is stored with each half and a mismatched pair reports that instead of splitting. And measuring the same orientation twice averages to "the machine is perfect" — nothing can detect that after the fact, so the generated program's setup prompt states which way round the square must be for THIS run, where the operator is looking when they clamp it.
- **The panel says at every stage what the number rests on** — raw, square-calibrated, or a completed reversal — because an operator who forgets a raw reading includes the square will read squaring-the-machine-to-the-square as success.
- **Both halves persist, each carrying the offset it was taken at.** They lived only in memory at first, so an app restart between them threw one away — which is what happened: a good normal half was measured, the app was rebuilt, and the pair had to be split by hand outside the tool. A half is twenty minutes of clamping and probing, and the second is normally measured after the operator has been away from the machine, precisely when a restart is likely.
- **The square's error is an editable field**, not a value only a completed pair can produce — the reversal that measured it may have happened in a session this install has no memory of, and typing it in is a legitimate way to know it. A pair must also match the offset in force NOW, not just each other: once halves survive restarts, a pair can be read back on a machine whose settings have since been changed elsewhere, and it would report a machine error for a gantry that no longer exists.
- **FIX: the reversed run told the touch plate to go to the wrong corner.** The install prompt was made orientation-aware; the two prompts BETWEEN corners were not, and still said "the far end of the blade"

for the second probe — true normally, wrong reversed, where the second probe is the heel a blade-length away. That failure is the quiet kind: a plate on the wrong corner does not alarm, it returns a perfectly confident coordinate for a corner nobody asked about, and in a reversal pair that would be averaged with a good half and reported as a square calibration. Found by the operator asking, before running, what order the corners are probed in.

#370 FIX: a controller setting the machine ACCEPTED was reported as a failed write

File	+	−
CNC Core/CNC Core/Grbl.cs	33	0
CNC Controls/CNC Controls/AutoSquareProbeWizard.xaml.cs	24	6

"Could not write $171" — while the wire log showed it sent and acknowledged nine milliseconds later. The save returned false 5.035 seconds after that, and the console names the branch: no acknowledgement within the timeout.

- **The wait raced the poller.** It waits while the stream state is "awaiting" or "data received", and every reply reclassifies that state — ok, error, or anything else. A status report is anything else. So with the poller running there is exactly ONE window in which the wait can observe its acknowledgement, between this command's ok and the next status report, and there is no second chance because every later reply is another status report. Over the network link it is worse: the receive handler classifies a BATCH and the last reply wins, so an ok arriving in the same socket read as a status report never exists as an observable state at all.
- **Fixed in the save, not at a call site.** It is the function that owns the synchronous write-and-acknowledge exchange, so it owns the requirement not to race the poller — and fixing it there fixes every caller at once, including the pins tab and the settings editor, equally exposed whenever they are reached from a view that leaves polling on. Polling is suspended around the loop in a finally, and only when we were the ones who stopped it.
- **Second fix, in the tool that reported it.** The dialog did not merely report a failure, it asserted a CAUSE — "only the actually-ganged axis accepts a write" — an invented diagnosis of the operator's machine configuration, delivered confidently, about a write the controller had already accepted. It now names the possibilities instead of picking one, and points at the only thing that settles it: the value the controller actually holds.

The fault was never in the controller or the value: the setting held the written value on the machine at the moment the call reported failure.

#371 FIX: a typed value was displayed while a DIFFERENT one was written to the controller

File	+	−
CNC Controls/CNC Controls/NumericTextBox.cs	42	0
CNC Controls/CNC Controls/NumericField.xaml.cs	36	0
CNC Controls/CNC Controls/AutoSquareProbeWizard.xaml.cs	32	1
CNC Controls/CNC Controls/AutoSquareWizard.xaml.cs	26	2
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	4	0
CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs	4	0

Observed on hardware. 0.37 typed into Squareness (probe)'s new offset, Apply pressed, and the wire log shows 0.450 sent — the value the panel had computed BEFORE the operator typed over it. No error, no warning: the number on screen and the number sent to the machine simply disagreed, on a setting that racks a gantry.

- **Two correct decisions combining into a wrong one.** A length-unit field parses its text only on focus-lost or Enter, never per keystroke — deliberate and right, since a per-keystroke parse cannot handle a typed unit suffix and would commit "1" on the way to "10". And the action buttons are non-focusable so they cannot steal the jog keys — also right. But a non-focusable button takes no focus, so clicking one raises no focus-lost on the field being typed into, and the handler reads the stale value underneath it.
- **A pending edit is now flushed before the handler reads anything.** The walk is over the LOGICAL tree, not the visual one, so it works before first render and reaches collapsed branches. Swept rather than spot-fixed: five views pair these fields with non-focusable buttons, and all four whose handlers read values are done.
- **FIX: Apply stayed grey with a hand-typed offset — a genuine deadlock.** Both squareness tabs gated Apply on the new offset differing from the current one, and a typed value does not reach the underlying value until focus leaves the field — which these buttons never cause. So the comparison read the OLD value, saw no difference, and disabled the button, which then never received the click that would have committed the edit. The gate depended on the very state it was preventing from being updated: type over the current value and the button is dead, permanently, with no way out but tabbing away first. Apply is now enabled whenever there is a setting to write and the handler decides after the commit, where the real value is known. A press with nothing to write says so, rather than returning silently — a button that does nothing when clicked is indistinguishable from the bug just fixed.
- **FIX: committing a READ-ONLY field destroyed its binding.** The flush walks every field under a panel, which is what makes it safe to call without knowing which has focus — and it wrote to read-only displays, which lacked the guard the keystroke path had always had. That is not a harmless no-op: these displays chain a one-way binding into a two-way one, so writing the value sends it back up and a local write to the target of a one-way binding CLEARS that binding. The field stops tracking its source permanently, frozen on whatever it was showing. The Apply press broke its own readout, and only the redundant heading beside it made that visible rather than merely confusing. Guarded at the source, so the scratch wizard's steps/mm display and all six machine-position readouts are covered too.

#372 Peek - step away from a running job, and pick it up where you left it

File	+	−
CNC Core/CNC Core/JobRunner.cs	337	14
CNC Controls/CNC Controls/JobControl.xaml.cs	63	1
CNC Controls/CNC Controls/JobControl.xaml	12	0
CNC Controls/CNC Controls/ActionKeyBinder.cs	5	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	5	0
docs/Architecture-Peek.md	298	0
Locale/*/csv (7 locales)	28	0

Mid-job you want to *look* - at the cut, at the chips, at whether the tab is still holding. Until now that meant Stop and lose the run, or Feed Hold and peer past the gantry. Peek is a button on the run strip: it pauses at the end of the block in flight, parks at G30 with the spindle off, and waits. Press Resume - or Cycle Start - and the spindle restarts, the tool goes back to where it was, and the program carries on from the next line.

- **The job is never interrupted, it is STARVED.** That distinction is the whole design. The sender stops dispatching and lets the controller drain its own planner to Idle. Nothing is cut short, so nothing needs recovering - and because no reset is involved, the controller's parser state is never lost. The work offset, tool-length offset, feed rate, units, plane and distance mode are all still exactly as the program left them.
- **The first design was abandoned after reading the firmware.** Pausing mid-block with a feed hold cannot work: grblHAL refuses a jog in Hold, and g-code sent during a hold does not execute - it queues BEHIND the interrupted block, so Cycle Start would finish the cut and only then drive to the park. Recovering from that needs a soft reset, which is safe only from a completed hold and destroys the modal state on the way. Pausing at a block boundary instead removes every one of those problems, at the cost of a short wait.
- **Only two things are restored, because only two are disturbed.** The spindle, which Peek turned off, restarted with a dwell so it reaches speed before the tool re-enters the cut. And the motion mode: the park ends in G0, G0 is modal, and a following bare X/Y line that relied on a live G1 would have rapided into the work.
- **A stall watchdog would have restarted the job while the operator leaned over it.** A parked machine is streaming AND idle, which is exactly the condition that nudges a pump believed to be stuck - and the pump's suspend flag does not stop it, because that flag gates replies while the nudge arrives by another route entirely. The tool-change pause escapes this only by luck, reporting a different state. Guarded at the one place that decides, deliberately not by silencing the shared signal path, which also carries aborts.
- **Refused, with the reason said out loud**, when there is no program running, on an SD-card job (the controller streams those itself, so the sender cannot starve it), or on a controller without expression support, which the G30 park needs. If the query for the parser state goes unanswered the peek is abandoned with the job still streaming rather than parking on a guessed frame.

Hardware-verified: peeked mid-cut while marking a 120 mm square and the groove carries on with no witness mark - a Mark is one shallow pass, so any error in the captured position, the motion mode or the spindle restore would have shown in the cut. Left parked for over five minutes with a live job and the watchdog never fired. The design, the firmware reasoning and the traps are written up in docs/Architecture-Peek.md.

#373 FIX: Machine Setup's Overview listed six steps for an eight-step wizard

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml	11	4
Locale/*/csv (7 locales)	48	34

The Overview page exists to tell you what the setup steps are. Its list stopped at six, and had since before Fixture definitions and Build simulator were added - so it omitted both, and numbered Controller macros as 6 when the step header beside it says 7. The page describing the wizard disagreed with the wizard.

- **One entry per page the wizard actually has**, in the same order and with the same numbers as the step headers: 6 Fixture definitions, 7 Controller macros, 8 Build simulator. Step 8 says that it is not shown while the active connection IS the simulator, which is the one case it hides itself.
- **Three sibling statements fixed with it**, all the same stale mental model: the step strip was templated away when the steps became nodes in the navigation tree, so nothing is "below" or "left to right" any more. The intro said "work through the step tabs below", the body said "work through the tabs left to right - each setup step is its own tab", and the footer said "confirm each tab and Apply".

Long-standing: flagged as an app bug in the manual's own debt list since 2026-08-03, and left there because it was never the thing being worked on. It surfaced again while correcting the manual to say eight steps - at which point the app was the only thing still claiming six.

#374 Entering a new stock size discards a measurement carried over from a previous session

File	+	−
ioSender XL/ioSender XL/StartJobView.xaml.cs	41	2

Reported from the machine: a new stock size was entered on Setup and the four-corner measurement from a previous session was still sitting there, still arming Verify skew and Scribe square, with no way in the app to get rid of it. There was none - the only routes were running a fresh Measure, or closing ioSender and editing the profile by hand.

- **Editing Width, Height or Thickness by hand now drops a carried-over measurement**, greys out the checks that depended on it, and says on the status line why - rather than letting two buttons silently stop working.
- **Restored measurements only, deliberately.** One taken in this session was taken with the operator standing at the machine, and a nominal figure corrected afterwards - a typo, a unit toggle, a second look at the calipers - must not throw away minutes of probing. A carried-over one has no such witness, and it is the case that bites: a frame probed days ago, against stock since unclamped, still able to write an origin.
- **The stored copy is cleared with it**, which is the part that makes it stick. The save path deliberately hands the loaded string back verbatim whenever a session has no complete measurement of its own - that is what stops a failed READ from destroying data - so clearing only the on-screen values would have let the discarded frame return on the next launch. This is the one path where the erasure is the operator's own instruction rather than a side effect.

The date stamp added to the readout when measurements first became persistent was a half-measure against exactly this: it tells you the frame is old, and then lets you use it anyway.

#375 FIX: an offset test that answered backwards, and a start-from-line that skipped its prolog

File	+	−
CNC Core/CNC Core/Grbl.cs	12	1
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	15	0

Two defects found while specifying another feature, recorded at the time rather than fixed, and cleared here.

- **The offset test was inverted.** "Does this position carry a real offset" was written as NOT(is-not-a-number OR is-not-zero), which reduces to "is exactly zero" - it answered true when there was no offset and false when there was one. Both callers then did the opposite of what they meant: a machine holding a real tool-length offset had G49, offset cancelled, synthesised into its parser state; one with no offset got G43.1; and a G92 of all zeros was reported as an active offset. It is reachable only through the workaround for controllers that do not report those words in their parser-state reply, so grblHAL has never taken this path - which is why it survived. The machine this is developed against is the thing that hid it.
- **Starting from a selected LINE restored no modal state.** Starting from a toolpath queues a short prolog re-establishing distance and feed mode, plane and units, because a mid-program start skips whatever set them earlier. Starting from a line skips exactly the same thing and never sent it - two routes into one operation, one of them arbitrarily unprotected, resuming on whatever modal state happened to be live from the last thing the machine did.

The prolog was deliberately NOT widened while making the two paths agree. It restores units, plane and distance/feed mode and does not restore the work offset, the tool-length offset, the feed rate or the spindle - a real gap, and the reason Peek captures its own frame rather than reusing this one. Closing it is a design change, not a bug fix: a blind M3 in a prolog would start a spindle on a resume the operator may not be standing at.

#376 Toolpath outline on any program with tool changes, and run one toolpath from it

File	+	−
CNC Core/CNC Core/GCodeJob.cs	223	0
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	61	2
CNC Core/CNC Core/StreamPump.cs	37	1
CNC Core/CNC Core/GCodeProgram.cs	18	3
CNC Core/CNC Core/GrblViewModel.cs	14	0
CNC Core/CNC Core/JobRunner.cs	10	1
macros/sample_toolchange_outline.nc	40	0

The program list could already collapse into a toolpath outline, but only for files carrying the ioSenderV2 Fusion add-in's own section-marker comments. Every other post - stock Fusion, Vectric, Carbide Create, hand-written, and the app's own Work Order output - got a flat list, with nothing on screen hinting an outline was possible however many tool changes the file had. That also meant "Start from this toolpath" and "Run just this toolpath" were unreachable on ordinary files.

- **Sections are derived from the program's TOOL CHANGES** when a file carries no markers. Applied after the parse rather than during it, so a marker-bearing file is untouched - sectioning on both interleaved would have split every operation in two, once at the marker and again at the M6 a line later.
- **Naming is a convention anyone can opt into**: a comment immediately above the tool change becomes the toolpath's name, so "(Finish contour)" above "T2 M6" reads as "T2 - Finish contour", and a tool change with nothing above it reads as "T2". The search stops at the first line that DOES anything - a comment separated from the tool change by g-code belongs to that g-code - which is what keeps it from wandering up into the file header. The T-number is always the identity and the comment only a suffix, so a bad guess is noise rather than a wrong answer.
- **Program start and Program end** bracket the toolpaths: everything before the first tool change, and the wind-down from the last spindle stop after the last tool change. The epilogue is anchored after that last tool change deliberately - programs routinely stop the spindle between toolpaths, so an unanchored search would have swallowed a real cutting section.
- **Run just this toolpath** now runs Program start, the chosen toolpath, and Program end - in that order. Previously it ran the section alone, which set no units, no plane and no work offset, and finished with the spindle still turning and the tool wherever the cut stopped. **Start from this toolpath** likewise runs the real preamble first instead of a three-word synthetic prolog that never covered the work offset at all.
- **Skipped toolpaths are not sent.** The first attempt neutralised them to empty comments so the per-line status column would still mark them; on a 31,000-line V-carve that meant streaming and acknowledging tens of thousands of no-ops before the machine cut anything, and it was reported as unusable within minutes of shipping. The controller never knew those lines existed - it sees only what is written to it - so there is nothing to keep in step and nothing to skip past. The status column simply does not mark rows that did not run.

Both commands hardware-verified end to end on a three-toolpath work order. macros/sample_toolchange_outline.nc exercises the naming in three comment styles; it is air only, with nothing below Z0.

#377 The manual's twenty-one missing topics — section 2 of the audit, paid in full

File	+	−
docs/manual/index.html	727	18
docs/manual/MANUAL-AUDIT-2026-09-17.md	86	32
docs/manual/MANUAL-DEBT.md	75	22

The audit of 2026-09-17 listed twenty-one features that shipped without a word in the manual. All twenty-one are now written, with the links that make them reachable from the topics an operator is already reading.

- **Four new topics.** The diode-laser path end to end; the Height Map — what a map is, how to probe one and what applying it does; Calibration, including the reversal test that separates the square's own error from the gantry's; and carving.
- **Sub-sections filled in** across Work Order (corner reliefs, Indirect and groups, Save Drawing, Mark only), Job, Settings, Machine Setup and Setup.
- **Work surface** is described as the board it is, rather than as the machine's travel — the confusion that made the whole section read wrongly.

Published to gh-pages 2026-09-19. Section 4 of the audit (thirteen topics that NEED UPDATING) and the screenshot reshoots remain.

#378 Help > User manual on Shift+F1 — and context help that lands on the topic

File	+	−
CNC Controls/CNC Controls/ManualHelp.cs	118	15
ioSender XL/ioSender XL/MainWindow.xaml.cs	55	2
ioSender XL/ioSender XL/MainWindow.xaml	10	0
CNC Controls/CNC Controls/ActionKeyBinder.cs	17	2
CNC Controls/CNC Controls/WorkOrderModel.cs	11	3
CNC Controls/CNC Controls/ViewHostWindow.xaml.cs	4	0
docs/manual/README.md	29	22
Locale/*/csv (7 locales)	14	0

The manual had exactly one way in: F1. On this machine F1 is the first macro — F1 to F9 were reserved for macros in the original ioSender and still are — so the manual was unreachable from the keyboard. It now has a **Help > User manual** menu item, bindable like any other menu action and bound to **Shift+F1** by default.

- **It opens at the topic for the screen you are on**, not the front page. “Current view” was answering with the view the app STARTED on, so it was right once per session and wrong after that.
- **ShellExecute strips the #anchor off a file: URL.** Proven directly after two fixes on the wrong suspect: the browser was being handed the anchor and never saw it. The default browser is now resolved from the shell’s own UrlAssociations and launched with the full URL, so the fragment survives.
- **Four anchors did not resolve at all** — F1 from those screens landed on the front page whatever else was fixed.

Hardware-verified: Shift+F1 from Setup and from Work Order both land on their own topic.

#379 FIX: a message box could open BEHIND the window that raised it, which reads as a hang

File	+	−
CNC Controls/CNC Controls/AppDialogs.cs	18	0
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	8	0

A prompt raised while another dialog is up was owned by the MAIN window, so it opened behind that dialog. The app then sat there blocked on an answer to a box nobody could see — indistinguishable from a hang, and diagnosed as one: the fixture dialog's OK button "stopped working" when in fact a hidden probe-failed box was waiting behind it.

- **The owner is now the ACTIVE window**, then a visible Topmost one, then the main window — so a prompt is always above whatever raised it.

Found on real hardware by the operator, who worked out the hidden dialog before I did.

#380 FIX: Set position captured the CACHED machine position, and could capture a stale one

File	+	−
CNC Controls/CNC Controls/Fixture.cs	84	0
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	29	14

Defining a corner-fence fixture captured 20, −20, −6 with the spindle nowhere near it: the Set button read the cached position from the last status report, which can be arbitrarily old when the machine has been sitting idle or the poll has been suspended.

- **The choice belongs to the accessor**, as the operator put it: rather than a second capture path, the position accessor now takes a request for a FRESH read — an explicit status request with a 500 ms fallback to the cache.
- **The log says which it used**, cached or fresh, and says loudly when the cached one turned out to be stale.

Hardware-verified: the capture matches the DRO, and Test drives back to the captured point.

#381 Setup's height map is KEPT, and the Work Order applies it at Generate

File	+	−
CNC Controls Probing/CNC Controls Probing/SetupHeightMap.cs	340	2
CNC Controls/CNC Controls/HeightMapCompensation.cs	70	1
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	143	1
ioSender XL/ioSender XL/HeightMapView.xaml.cs	49	18
ioSender XL/ioSender XL/App.xaml.cs	42	2
CNC Controls/CNC Controls/WorkOrderModel.cs	12	0
CNC Controls/CNC Controls/WorkOrderView.xaml	3	0
ioSender XL/ioSender XL/StartJobView.xaml.cs	3	3
Locale/*/csv (7 locales)	14	0

Setup used to probe a height map and IMMEDIATELY apply it to whatever program happened to be loaded. That forced the workflow backwards — generate first so there is something to apply to, then go back to Setup — when the order has always been Setup first, then Work Order. So Setup no longer applies anything: it probes, and the map is kept. **Generate is now the one and only place a map is applied**, which also means there is no second, hidden path that could double-compensate a program.

- **Apply height map** is a tick box on the work order, with a summary line saying which map and how old it is. An age is the one thing that makes "is this still my setup?" answerable at a glance.
- **A staleness REFUSAL, not a warning** (the operator's explicit choice). The map is stored stamped with the setup it belongs to — where work zero sat in machine coordinates and what stock extent was mapped — and Generate refuses, with the reason, when the live machine no longer agrees. A map applied to the wrong setup shifts every Z by a surface that is not under the cutter, and the program still looks like the program.
- **A .map saved beside a work order is picked up with it**, and a saved .map now carries its own stamp, so a job reopened next week either finds its map or is told plainly why it cannot have it.
- **The work order could never see a map at all** before this, for three separate silent reasons — including a lazy load that ran before the config path was resolved and looked for the file beside the executable.

The seam between the two assemblies is a set of null-by-default hooks wired at startup; unwired, it refuses rather than silently skipping compensation the operator asked for.

#382 FIX: two false refusals from the height map's staleness check — both the app's own arithmetic

File	+	−
CNC Controls Probing/CNC Controls Probing/SetupHeightMap.cs	80	22
CNC Controls/CNC Controls/HeightMapCompensation.cs	11	4
ioSender XL/ioSender XL/App.xaml.cs	4	3
ioSender XL/ioSender XL/HeightMapView.xaml.cs	2	3
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	2	1

The refusal that guards a stale map fired twice on setups nothing had touched. Both were measured on hardware and both were arithmetic, not movement.

- **A tool change is not a moved origin.** The check compared the work position offset, and grblHAL's WCO is the WCS offset plus G92 *plus the tool length offset* — so fitting a different tool moved it. G54 read the same before and after; the TLO went from −25.766 to −16.540 and the map was refused for a 9.226 mm move that never happened. The comparison now takes the tool out: work Z0 means the same physical plane whatever is in the spindle, which is what a tool length offset IS. G92 stays in, because that genuinely does move the origin.
- **Then the stamp was written with a ten-minute-old tool offset.** WCO rides every status report; the TLO arrives only in a $# report, which nothing had asked for since 20:30. The subtraction mixed a live value with a stale one and put the stamp 2.325 mm out. The offsets are now re-read from the controller at the two points that DECIDE something — storing a map and the check before applying one — and never on a UI refresh or mid-stream, where a blocking round trip does not belong.

The refusal text now also says outright that a tool change does not move the work origin.

#383 CHG: probing the height map is an explicit step in the Height Map tab, not a hidden pass in Setup

File	+	−
ioSender XL/ioSender XL/HeightMapView.xaml.cs	156	35
ioSender XL/ioSender XL/StartJobView.xaml.cs	72	18
ioSender XL/ioSender XL/HeightMapView.xaml	38	21
CNC Core/CNC Core/GrblViewModel.cs	22	1
CNC Controls/CNC Controls/HeightMapConfig.cs	14	0
CNC Controls/CNC Controls/MacroProcessor.cs	12	2
ioSender XL/ioSender XL/StartJobView.xaml	1	1
Locale/*/csv (7 locales)	21	21

Ticking “Probe height map” in Setup was supposed to run a probing pass at the end of the run. It silently did not — and a hidden pass is the wrong shape anyway. Setup now finishes, and the height map is **an extra step you take in Tools > Height map**, where you can see the grid, the area and the probe before anything moves. Measure does not have to be re-run to get there.

- **“From setup”, not “From program”** — the area comes from the authoritative source, the stock Setup measured, rather than from whatever program happens to be loaded.
- **The tab remembers its settings** — grid size, probe selection and the hold tick box — instead of resetting every visit.
- **“Cancelled” now says what happened**: a probe that never made contact says so, rather than reporting the same word as a deliberate stop. (The real cause on the day was a disconnected magnetic probe lead.)
- **The run buttons stay on screen** at the default window width, where Save and Load used to sit off the edge.
- **A modal popup mid-run cost the run its JobFinished edge**, so Setup’s continuation never fired. The finish is now a counter rather than an edge, and Setup’s stock-mismatch warning waits until the end instead of interrupting the sequence.
- **The mismatch threshold is 1 mm**, not 0.5 mm — 0.5 was calling for a calibration that was not needed.

Hardware-verified: a 4x5 map at 100 mm grid completed and was kept.

#384 FIX: the height map's own motion — a G91 traverse on a rotated WCS, and a first point probed with no plate under it

File	+	−
ioSender XL/ioSender XL/HeightMapView.xaml.cs	63	10
CNC Controls/CNC Controls/MacroProcessor.cs	21	4
CNC Controls/CNC Controls/WorkOrderCompiler.cs	4	1
ioSender XL/ioSender XL/HeightMapView.xaml	2	2
Locale/*/csv (7 locales)	7	7

Four faults in how the probing run moves, three of them found as soft-limit alarms on the real machine.

- **It traversed in G91.** grblHAL's incremental branch lacks the rotation awareness its absolute branch has, so on a rotated WCS every relative move computed from the wrong frame — ALARM:2 on the second point. The traverse is absolute now. (The firmware side is filed separately; the app must not depend on it.)
- **It assumed it started at machine top.** It did not lift first, so it crossed the work at whatever height it happened to be at. It now lifts to a machine-referenced safe height before the first traverse.
- **The rotation-corruption repair left Z out.** Clearing a rotation corrupts grblHAL's parser position; the repair re-syncs from the steppers, but naming only X and Y left Z holding a work coordinate believed to be a machine one — the next Z move computed from a number 100 mm wrong. All three axes are named.
- **The first point was probed without waiting.** Hold-at-each-point held before points two onward, so the operator had no chance to place the plate for point one — the head is wherever it was parked, not where you left it. It holds before every point now, the first included.
- **"Hold at each point" was invisible** in the mode most people use, while still governing the run — which is exactly how a map came to be probed with no holds and no plate.

A setting that governs a mode must be visible in that mode.

#385 A Bluetooth shutter remote drives the machine from arm's length

File	+	−
CNC Controls/CNC Controls/ShutterRemote.cs	181	16
CNC Controls/CNC Controls/RemoteActions.cs	178	0
CNC Controls/CNC Controls/AppMessageBox.xaml.cs	24	1
ioSender XL/ioSender XL/HeightMapView.xaml.cs	25	12
CNC Controls/CNC Controls/AppConfig.cs	18	7
CNC Controls/CNC Controls/MacroProcessor.cs	10	1
CNC Controls/CNC Controls/JobControl.xaml.cs	9	0
ioSender XL/ioSender XL/HeightMapView.xaml	6	4
Locale/*/csv (7 locales)	28	14

Probing a height map with a touch plate means: place the plate, press Continue, walk back, place the plate, press Continue — sixteen times for a 4x4 grid with the laptop across the shop. A Bluetooth camera shutter remote solves that for a few pounds, but only if the app can hear it. These remotes are HID keyboards that send a MEDIA key, and Windows routes media keys to the shell and the audio endpoint — they never reach a WPF window at all, so no key binding can see one. A low-level keyboard hook is the only thing that does.

- **What a press means depends on what is happening**: a prompt on screen takes volume up for OK and volume down for Cancel; a RUNNING job takes either key as Feed Hold; a HELD one takes volume up to resume and volume down to Stop; a height-map hold takes either key as Continue.
- **Either key, where it matters.** The two modes of the same remote send different keys, and an operator reaching for it mid-cut wants the machine to stop cutting — a press that does nothing because the remote is in the other mode is the worst outcome of that reach. The same reasoning keeps both keys meaning Continue at a probe point, where one of them meaning Stop would throw away twenty minutes of probing.
- **It presses the real buttons.** Start, Feed Hold and Stop are the Job control's own buttons and only fire when they are enabled, so the remote inherits every gate the app already applies and cannot start a job the app would refuse to start.
- **The key is swallowed only when the press MEANT something**, so with nothing waiting the volume keys are still Windows'. It debounces, because a held button auto-repeats every ~30 ms and one press must not release several holds in a row while a hand is on the plate.
- **Opt-in and off by default** — nobody who has not asked for it gets a global keyboard hook.

Measured on the operator's own PICO remote: iOS mode sends VOLUME UP, Android mode VOLUME DOWN, and it has no mode that sends Enter.

#386 FIX: applying a height map took thirty seconds and showed an empty program list while it worked

File	+	−
CNC Core/CNC Core/BulkObservableCollection.cs	43	1
CNC Controls/CNC Controls/WorkOrderView.xaml.cs	23	3
CNC Core/CNC Core/GCodeJob.cs	18	0
CNC Controls Probing/CNC Controls Probing/GCodeTransform.cs	17	3
CNC Core/CNC Core/GCodeProgram.cs	7	1

The transform hands every rebuilt line through AddBlock, which appends to the live DataGrid-bound collection — so a 43,000-block program spent thirty seconds notifying the grid once per block, showing an empty list throughout. A height map that had applied perfectly was indistinguishable from one that had silently failed.

- **Notifications are held back for the rebuild** and raised once at the end. The same shape froze the app outright on a 220k-line file in August, which is why the bulk collection exists at all; this path simply never used it.
- **The operator is told what is happening** — a message before, a wait cursor during, and the elapsed time and the map's identity after. A step that takes seconds should say it finished.

The log also states the block count after the transform, which tells three otherwise identical-looking failures apart.

#387 FIX: a transformed program had its `#<var≥value` lines replaced by the word “Parameter”

File	+	−
`CNC Core/CNC Core/GCodeParser.cs`	18	0

Every case in the token-to-g-code writer casts the token to its own class before calling ToString, and that is not stylistic: those overloads are declared `new`, not `override`, so they HIDE the base. A token reached through the base-typed loop variable therefore calls the base ToString, which returns the COMMAND NAME — so any type missing a case is silently rewritten to a single word.

- **Two were missing.** A work order put through the height-map transform came back with the literal text “Parameter” where its `#<name≥value` setup lines had been — a program that had quietly lost the values every later line reads. The other was the token carrying a rotation write.
- **Swept:** every token class with a hidden ToString was checked, and those two were the only ones lacking a case.

Seen on hardware at blocks 120 and 130 of a compensated work order.

#388 The toolpath outline names each section after the TOOL, and survives a transform

File	+	−
CNC Core/CNC Core/GCodeJob.cs	26	2
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	20	0
CNC Core/CNC Core/GCodeProgramComments.cs	16	2
CNC Core/CNC Core/GrblViewModel.cs	16	0
CNC Core/CNC Core/GCodeProgram.cs	7	1

A section of the outline is one tool's worth of the program, and with Group by tool it holds every operation that shares the bit — so naming it after the first of them described a fraction of what was in it and hid the one thing an operator wants at a tool change: which bit to fit.

- **The program already said so.** The Work Order writes one `(TOOL T=n D=.. TYPE=.. - name)` declaration per distinct tool into its header, and the shared comment parser was discarding the description. The outline reads them from the header — everything above the first tool change — rather than from the global map, which is refreshed too late to describe the right program.
- **A program that declares no tool** falls back to the nearest-comment name, unchanged.
- **The outline stopped vanishing.** A rebuilt program answers the same "yes, it has sections" as the one it replaced, so no notification was raised and the Job tab's docked list — whose only trigger that is — was left grouped by the null section every new block carried while it was refilled.

Hardware-verified on a five-tool work order.

#389 FIX: a chamfer on a Line ran down the middle of the groove instead of along its edges

File	+	−
CNC Controls/CNC Controls/WorkOrderCompiler.cs	62	6

A Line has no interior, so the Contour that cuts it straddles it: a 100 mm line cut 10 mm deep with a 1/4″ end mill leaves a 6.35 mm wide groove with the line down its MIDDLE and its two long top edges 3.175 mm to either side. The chamfer traced that centreline — the one place in the groove where a chamfer pass meets nothing.

- **Both edges now get a pass**, offset perpendicular by the ROUGHING op's bit radius. The chamfer bit's own diameter stays out of it, exactly as on a closed shape, because the cone does the work.
- **A toolpath with no roughing operation** has no groove and no pair of edges, and still traces the path itself.
- **The pass lifts to safe Z BEFORE its XY rapid**, not after — with two passes the second rapid would otherwise cross the work at chamfer depth.

The offset is taken from the line's own angle rather than mitred, which is exact: the only open geometry that reaches here is a straight Line.

#390 FIX: a rotated WCS drew the whole toolpath outside the machine, in front of the stock and tilted

File	+	−
CNC Core/CNC Core/GCodeEmulator.cs	43	31
CNC Core/CNC Core/GCodeJob.cs	32	0
CNC Core/CNC Core/Grbl.cs	8	0
CNC Core/CNC Core/Machine.cs	5	0

With a rotation set on the active WCS, the 3D view drew the stock correctly inside the machine envelope and the entire job floating 600 mm in front of it, below the table and visibly turned. Two faults in one branch, both measurable.

- **Frame.** For a rotated WCS the emulator added the WCS origin to every point, rotated it about (0,0) — MACHINE zero — and left it there. So a rotated program's every point came out in the machine frame while the stock, the envelope and every unrotated point stayed in the work frame. That is the 600 mm: the G54 origin.
- **Unit.** The same code read the rotation as radians. It is degrees — that is how $# reports it, how the R word writes it, and how the Work Order's own advisory prints it. A 0.09 degree rotation was drawn as 5.16 degrees, 57x too much.
- **The rule, now written into the class:** the emulator yields work coordinates, always. In work coordinates a rotated WCS is square — the rotation says how the work frame sits on the table, which matters only converting work to machine, which the controller does and this does not. The stock is drawn square for the same reason, so the two agree by construction.
- **The bounding box gets it too**, and needed it: the same rotation inflated a 224 mm Y span to 417 mm, and the does-it-fit check reads that box — a big enough job would have been refused for not fitting a machine it fits. The box now also logs which program line set each edge.

Hardware-verified by the operator: "3D View looks perfect now." The dead Rotation field on the emulator's action struct is removed rather than left as a slot for someone to refill.

#391 NEW: bind a Bluetooth shutter remote and choose what its buttons do

File	+	−
CNC Controls/CNC Controls/RemoteDevices.cs	762	0
CNC Controls/CNC Controls/UiRemoteConfigControl.xaml.cs	157	0
CNC Controls/CNC Controls/UiRemoteConfigControl.xaml	142	0
CNC Controls/CNC Controls/RemoteFunctions.cs	91	0
CNC Controls/CNC Controls/ShutterRemote.cs	146	7
CNC Controls/CNC Controls/RemoteActions.cs	128	17
CNC Controls/CNC Controls/AppConfig.cs	80	0
CNC Controls/CNC Controls/ActionKeyBinder.cs	24	0
JobControl.xaml.cs + GrblConfigView.xaml.cs (registration)	5	0
Locale/*/csv (7 locales)	105	0

A cheap Bluetooth camera shutter remote already answered height-map holds. It is now a proper pendant: bound to one device, with each button's job settable per machine state.

- **Binding is one press.** Tick the remote on under Settings → App → User Interface → Remote and the next button press claims the device. Untick and retick to rebind after replacing one, or use the Rebind button. The status line names the device the way Windows does.
- **That same press decides which button is PRIMARY.** The two buttons carry no marking, and on a common remote the larger top button is the one that sends volume *down* — so rather than document that, the button you bind with becomes the primary one.
- **Only the bound device acts.** Every other keyboard is ignored, so the machine keyboard's own volume keys are untouched — previously any volume key meant Feed Hold while a job ran.
- **Six rows, each switchable on its own:** a prompt on screen, the height map waiting, running, holding, idle, and in alarm. The last four are assignable from ten functions — Nothing, Cycle Start, Feed Hold, Stop, Peek/Resume, MDI, Status, Reset, Unlock, Reset and unlock. Defaults reproduce exactly what the remote did before.
- **Both buttons mean Feed Hold while a job runs**, deliberately: a hand reaching for the remote mid-cut grabs whichever button it finds. A prompt with no Cancel button takes either button as OK, which is what the “move the plate to the next corner” prompt needs.

The buttons are read from the remote's own HID report rather than from the key Windows makes of it. That translation turned out to be intermittent — present in one session and absent the next — and the keyboard hook never saw this remote's buttons at all.

The system volume still changes when the remote is pressed and cannot be prevented: raw input cannot swallow a key, and the flag that would suppress it is rejected alongside the one that lets a remote work while the app is not focused.

#392 NEW: a probe result that disagrees with itself is refused

File	+	−
macros/tlo.macro	26	0
macros/pcorner.macro	20	0

Every probe cycle already takes two independent measurements — a fast seek, a back-off, a slow latch. On a real surface they land a fixed distance apart, because the only difference between them is the overshoot at search feed versus latch feed. Across six days of logs on one machine that gap was **0.100 to 0.143 mm**, nine cycles, and 0.047 mm once a wiring fault was repaired.

A false trigger stops the seek at a random point in mid-air. The latch then fires at another random point, so the gap stops being overshoot and becomes noise. On the day this was written, with a bare cable shield intermittently shorting the probe input: **0.411 mm and 1.375 mm**. Both cycles reported success and applied a tool offset 65 mm wrong, and the second fed a corner probe that drove into the touch plate at rapid.

- **Enforced on both Z probes** — the toolsetter reference and the stock-top probe. Outside 0.3 mm the result is rejected with a message naming both numbers, rather than applied.
- **Placed before anything is derived from it**, so a bad stock top cannot become a travel height that is then rapided to.
- **The X and Y face probes log their gap but are not gated**, deliberately: their back-off is 4 mm rather than 2 and the contact is a deflecting ball, so the healthy band is not yet known. Inventing a threshold is how a guard ends up aborting good probes.

Nothing inside a single trigger can tell a true touch from a false one. Two triggers disagreeing can, using numbers the cycle already produces and discarded.

The underlying fault on that machine was electrical — a trimmed-back foil shield left bare at a connector — not a software one. This is the check that would have caught its effects.

#393 FIX: a corner probe's travel height could sit above the top of travel, and the move was refused

File	+	−
CNC Core/CNC Core/Grbl.cs	72	3
ioSender XL/ioSender XL/StartJobView.xaml.cs	10	3
AutoSquare / StepperCalibration / FixtureEdit (same shape)	5	3

Six places computed the height a corner probe travels at the same way — a measured machine Z plus a clearance margin — and none of them clamped the result. pcorner then rapids to it with a bare G53 G0.

So any margin larger than the gap between that datum and the top of travel aims the move **above the envelope**, and grblHAL refuses it at planning: Alarm 2, nothing moves, which reads like a dead probe rather than an out-of-range target. Hit with a toolsetter puck sitting 12.5 mm below machine top and a 25 mm travel margin.

- **One clamp, used by all six.** Clamping down to the top of travel is never less safe than the value it replaces — the top is the highest the tool can go — and it fails open when the envelope is unknown.
- **Clamped to just below the top, not onto it.** The macro reads exactly zero as "no trusted height", and on a machine that homes Z to the top that *is* the top — so landing on it would quietly turn a known height into the sentinel and drop into an unguarded rapid.
- **The shared envelope formula was wrong at the home end** when force-set-origin is on: it applied the homing pull-off where the firmware writes a literal zero. The reachable top of Z read as −6 while G53 G0 Z0 demonstrably executes. Only the home end was affected, which is why every existing caller — all of which ask for the floor — was unaffected and it stayed latent.

Checked against the firmware's own limits_set_work_envelope() rather than re-derived; this is the third time that arithmetic has been worked out by hand in this codebase.

#394 NEW: press a shortcut in the key bindings list to find it, instead of firing it

File	+	−
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	119	1
ioSender XL/ioSender XL/MainWindow.xaml.cs	36	2
CNC Controls/CNC Controls/GlobalKeys.cs	30	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	5	0
Locale/*/csv (7 locales)	21	0

A bindings list is the one place you want to ask “what does this key already do?”, and pressing the key is the obvious way to ask. Scrolling a grouped list of a hundred rows hunting for a combination is not.

- **A bound shortcut expands its group, scrolls to its row and selects it** — and does not run. Groups default to collapsed, so expanding first is most of the work: a row inside a collapsed group is not realised, and scrolling to it does nothing at all.
- **An unbound combination with a modifier, or a function key, beeps.** Those are never navigation, so saying “nothing is bound to that” costs nothing.
- **Unbound bare keys are left alone.** Tab, the arrows, Enter, Space and the page keys are how the list is driven without a mouse — and on the jog tab the arrows and numpad *are* the bindings — so swallowing everything would have made the panel mouse-only.
- **Mid-capture it stands aside**, so setting a binding still works exactly as before.

It also closes a gap that was already understood on the other half of the same editor: gamepad dispatch has been paused while that tab is visible since it was written, so testing a button cannot move the machine. The keyboard never got the same care — a bound key pressed while editing bindings ran the real thing, starting a job or firing a macro from the settings page.

The panel says so on screen. An invisible mode where keys quietly stop doing their job is a trap for whoever meets it cold.

#395 CHG: one place to set the remote up, and an honest line about the volume

File	+	−
CNC Controls/CNC Controls/UiRemoteConfigControl.xaml	4	3
CNC Controls/CNC Controls/UiRemoteConfigControl.xaml.cs	4	3
ioSender XL/ioSender XL/HeightMapView.xaml	1	4
docs/manual/index.html	10	7
Locale/*/csv CNC.Controls.WPF (7 locales)	7	7
Locale/*/csv ioSender (7 locales)	0	14

Two corrections to what #391 shipped, both found while writing the manual topic for it.

- **The Height map tab's shutter-remote tick is gone.** It was the same setting as the one on Settings → User Interface → Remote — one property, two ticks. That made sense while the height map was the only place a press meant anything and there was nowhere else to put it. With a page of its own, a binding status line and a six-row function table, the second tick was just another place to look for a switch that is not there. The Hold-at-each-point tooltip points at the Remote page instead.
- **The Remote page no longer says a discarded press "never reaches the volume control".** That is true only where Windows also delivers the press as a keystroke the low-level hook can swallow — and on the remotes measured so far it does not: the buttons arrive as raw HID input, which can be read but not intercepted. The page now says the volume will usually still move, why, and what to do about it, which is to turn the machine's volume down and ignore the on-screen bar.
- **The Height map tab's own tooltip was stale in a second way** — it still described the fixed mapping from before #391 made the buttons configurable, naming volume up for OK and volume down for Cancel. It now points at the Remote page for what the buttons do.

Both strings were updated in the English baseline row of all seven locale CSVs, which carried the old text verbatim. A string that is wrong in one place is wrong in eight.

#396 CHG: the height map's hold at each point follows the probe you picked

File	+	−
ioSender XL/ioSender XL/HeightMapView.xaml.cs	96	12
docs/manual/index.html	5	1

A touch plate has to be carried to every point by hand — a spoilboard will not close a circuit. A 3D probe triggers mechanically and reaches the surface on its own. So whether the run should hold at each point is not really a preference; it follows from the probe. It was a tick the operator had to remember to change on every probe swap, and forgetting it in either direction wastes a run: sixteen pointless walks to the machine, or a map probed with nothing under the bit.

- **The probe decides, every time the tab is opened.** A touch plate ticks it; a 3D probe or an edge finder does not. Unticking it by hand holds for that visit rather than for ever.
- **That asymmetry is deliberate.** A hold you did not want costs one walk to the machine. A missing hold wastes the whole map, or drives a bit at a board that was never going to trigger.
- **Full work surface no longer forces the hold on regardless of probe.** It asks which probe is selected first: that mode is the movable-plate case, but a 3D probe reads a bare spoilboard perfectly well, and forcing a hold for one was asking the operator to stand at the machine for nothing.

The first attempt applied the rule only when the probe's CLASS changed, so that an explicit untick would survive. It did nothing at all on the machine it was written for: that operator owns two touch plates and no 3D probe, so the selection STARTS in the plate class and never transitions into it. A rule that fires only on a transition cannot fire on a machine with nowhere to transition from.

A programmatic selection never moves the hold. Activate refreshes the probe list BEFORE loading the saved config, and the handoff from Setup's “Probe height map” does not load it at all — and that list prefers a 3D probe when it has to choose. Unguarded, arriving from Setup would have quietly turned the hold off, which is the same fault recorded on this view in September: a map probed with no holds and no plate.

#397 CHG: picking your machine in step 1 no longer overwrites the controller's own settings

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml	9	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	58	1
docs/manual/index.html	22	1
Locale/*/csv (7 locales)	14	0

Choosing a manufacturer/product/model in Machine Setup step 1 seeded steps/mm, max rate, travel, homing enable, the home corner and force-set-origin from the catalogue — on top of the values ioSender had just read off the controller on connect.

That is the wrong way round. The catalogue holds what a model *typically* ships with. The controller holds what this machine is *actually* set to, and it may have been tuned, re-geared, or had its travel trimmed to clear a fixture. The pick is documentation, and documentation should not overwrite a measurement.

- **Connected, with settings: the pick changes nothing.** It is recorded, its note is shown, and steps 2 to 4 keep the controller's values.
- **No settings: it seeds as before.** Not connected, or a download that came back empty — which is exactly what the `$ES` handshake race produced — and the catalogue is the only thing there is.
- **New button: Apply catalogue values**, beside the model dropdown, for the case the seeding existed to serve — a reflashed controller, cleared settings, a board swap. It asks first, and it sends nothing: the values land as pending changes and **Apply** is still what writes them, with its own count and a tooltip listing what will go.

The test for "do we have settings" asks whether the collection was populated at all, rather than reading a particular setting. An absent setting comes back as a value rather than as an absence — that is how a live machine with no `$20` silently lost its soft-limit guard.

Found while answering a question about what a -default-config session would do to a configured machine. The answer turned out to be "nothing, because every $ value is read from the controller on connect" - and then this one path that could.

#398 FIX: Machine Setup answered step 1 itself, then offered to save the answer

File	+	−
`CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs`	27	2
`docs/manual/index.html`	5	0

On a configuration with no saved machine, Machine Setup opened on step 1 with **Apply already enabled** and a review containing nothing — its own tooltip read "*No changes - the target settings already match the controller*". Clicking it closed the wizard and jumped to the Job tab.

The dropdowns were answering their own question. `RestoreOrDefaultMachine` fell through to a default that sets `SelectedIndex` 0 on all three lists, so the page came up showing a machine nobody had chosen. That auto-selection was then compared against the empty stored identity, reported as an uncommitted pick, and lit Apply — whose identity-only branch **recorded the generic machine** and marked step 1 complete.

- **It had already reached a real install.** *Generic / custom | 3-axis CNC | With limit switches (homing)* — index 0 of each list — was sitting in an operator's configuration as though they had selected it.
- **The dropdowns now start empty** when the settings came from the controller. Its values are already in the fields, so the pick is documentation and it is yours to make; Apply stays greyed until you make it. Step 1's dot is gated on the recorded machine rather than on the dropdowns, so the red dot and the screen finally agree.
- **The default still applies where it earns its place:** with no settings from the controller, the catalogue is the only source for the fields below, so it is selected and seeds them.
- **The tooltip says which case it is in.** With nothing to write but an unsaved pick, it names the machine Apply will record and says it completes the step.

Not fixed the way it was reported. “Always disable Apply when the tooltip says no changes” would also disable the case that behaviour exists for: a controller that already holds exactly the right settings, where recording the machine is the only thing left to do and refusing it strands the operator.

The tell was a button and its own tooltip disagreeing - enabled, over a review it rebuilt live and found empty. Worth hovering a control that looks wrong before theorising about it; it named the half of the condition that was not the settings.

#399 FIX: Test position travelled diagonally to the fixture, from wherever the spindle was

File	+	−
`CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs`	27	1
`docs/manual/index.html`	9	1

A fixture's **Test position** emitted one `G53 G0` carrying X, Y and Z together, so the machine cut the corner from wherever the spindle happened to be standing straight to the fixture position.

From anywhere low — beside a clamp, inside a vise, below the stock top after a previous probe — that diagonal goes through whatever is in the way. And the operator has no reason to expect it, because *every other travel move in the app lifts first*.

- **Up, across, then down.** Retract to the top of travel, travel in X and Y, descend last. The same rule the jog pad's **Go to** menu already states in as many words: the only clearance anyone has actually checked is "at the top".
- **The retract height is refused rather than guessed.** If Z travel or pull-off cannot be read, it says so and points at Machine Setup steps 3 and 4 instead of inventing a safe Z. `GoToMachineXY` already refuses on the same grounds, and a retract to a made-up height is the one move in this program whose whole job is to make the rest safe.

Two siblings found in the same sweep and deliberately NOT changed here, because both alter how the machine moves during or around a live cut and neither should ride along with a fixture fix: the Peek resume move (JobRunner), which returns to the interrupted point as one diagonal, and Setup's G53 rapid to origin before its G92.

#400 CHG: The probed corner IS the fixture position — the separate corner offsets are gone

File	+	−
CNC Controls/CNC Controls/Fixture.cs	78	25
CNC Controls/CNC Controls/FixtureEditDialog.xaml.cs	57	15
CNC Controls/CNC Controls/AppConfig.cs	1	27
CNC Controls/CNC Controls/AutoSquareProbeWizard.xaml.cs	4	3
CNC Controls/CNC Controls/StepperCalibrationProbeWizard.xaml.cs	5	4
ioSender XL/ioSender XL/StartJobView.xaml.cs	4	2
docs/manual/index.html	18	4

Test position parks at the true stock corner, and that corner is now the fixture's position. It used to be kept as a *correction* held alongside the reference the operator had jogged to by eye, and every consumer then had to remember to add the two together — two numbers for one place. Nobody reads "the corner is 0.4 mm from where I guessed" and does anything with it.

- **The measurement replaces the guess.** The corner-offset fields are deleted outright rather than kept at zero: a field nothing can ever set to anything is a question left lying around for someone to answer wrongly later. Each consumer loses its addition and keeps the inset it already had — Setup's corner 1, the squareness probe, and the stepper calibration probe all aim from the measurement now.
- **Z is deliberately not rewritten.** The stored Z is the height the operator jogged to and is the *start* of the capped downward search — a travel reference, not a datum. Overwriting it with the stock top would have the next Test position begin its search already touching the surface.
- **The delta is reported, not stored.** "Reference was out by X 0.369 Y 0.000" on the completion message and in the fixture log. Of no use to the machine, which is why it is not a field — but it is the one thing an operator wants after a Test, and *on a fixture tested before, a delta that has grown is the fence having moved*.
- **Existing fixtures need nothing done to them.** A fixture saved by an earlier build has its correction folded into its position as the library loads, keeping the validated tick it already had, and the retired elements leave the configuration on the first save. Idempotent by construction — once folded the correction is zero and adding zero does nothing — so it needs no "already migrated" marker and cannot run twice to any effect. That matters: the last fixture migration guarded by a standing check silently un-validated freshly probed fences on the next restart.
- **Test position is what validates a fixture.** Writing the position normally clears the validated tick, which is right for a re-jogged reference and wrong for a probed corner — that write *is* the validation. **Set position** records where you think the fixture is; **Test position** proves it.

Two bugs in this change were found and fixed the same night, before any release carried it: the fold wrote a type name instead of coordinates, because the position type has no ToString() override and only typed overloads (both call sites now use one named CSV helper); and the write-back re-asserted the validated flag before the write rather than after it, so Test position finished by un-validating the fixture it had just proved. Worth recording why the first got through - the debug line printed the right numbers one statement EARLIER than the corruption, so it was an accurate log of the arithmetic and said nothing about what was then stored. It confirmed the sum, not the write.

#401 FIX: The tool-length probe asked for more travel than the machine had left

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	36	3
docs/manual/index.html	7	3

`ALARM:2` on the first real run of step 5's **Probe it**. Standing at machine Z −61.254 it asked for the probe's full 100 mm **Max search distance**, so the target was −161.254 against a Z envelope of −146 to 0 — over the bottom by 15.254 mm, refused by the controller before it moved. The confirm dialog had just promised "up to 100 mm down".

The descent was bounded by how far the *probe* was willing to look, and nothing ever checked how far the *machine* could still go. Two different questions.

- **The descent is the smaller of the two** — the probe's Max search distance, or the travel left below the current Z.
- **The dialog names the bound that applies**, rather than quoting a number that silently became something else: "all the travel left below Z −61.254, which is less than Touch plate's 100 mm search distance".
- **With no room at all it refuses outright**, showing both figures, instead of emitting a move that can only alarm.

A silent clamp is precisely how a stored position 94 mm out of range walked past the reachability guard earlier the same day, which is why this one announces itself. One inconsistency deliberately left standing: the live line on the step-5 row still names only the probe's Max search distance, because it is a plan line and does not follow the machine position. The dialog is the authority.

#402 NEW: `ALARM:2` now names the axis and shows the arithmetic

File	+	−
CNC Core/CNC Core/SoftLimitExplainer.cs	178	0
CNC Core/CNC Core/StreamCommsBase.cs	16	1
CNC Core/CNC Core/GrblViewModel.cs	14	1
CNC Core/CNC Core/CNC Core.csproj	1	0
docs/manual/index.html	1	1

“Soft limit alarm. G-code motion target exceeds machine travel.” is the controller's whole answer. It names neither the axis nor the number, and four times in one day of setup work that cost a round trip through the wire log to establish something ioSender already knew every part of: where the machine was, what it had just asked for, and where the travel ends.

The alarm now carries the sum. The worked example it was written from:

- `N70 G38.2 Z-100.0 F500.0` went out with the machine at Z −61.254, and `ALARM:2` came back.
- What is now displayed with it: *“Z: −61.254 − 100.000 = −161.254, which is 15.254 mm below the limit of −146.000”*, naming the line that asked for it.

It says nothing rather than guessing. A `G90` move is expressed in *work* coordinates, so resolving it to a machine target needs the work offset — and on a *rotated* coordinate system an X/Y target does not resolve by adding an offset at all. Where the frame or the envelope cannot be settled honestly the axis is skipped, and if that leaves nothing to say the explanation is dropped altogether. A confident wrong number here would be worse than the controller's silence, because it would be believed.

The last motion line and the distance mode in force when it went are remembered from the one place every outgoing write funnels through, NOT from the wire log - that returns early when wire logging is switched off, and an explanation that only works under a debug flag is no use to the operator who hit the alarm. The g-code regexes are lookaheads rather than word boundaries, and that is not a style choice: a word boundary looks right and is wrong on every compound line ioSender actually sends, since G90 in N30G21G90G94G17 is followed by a word character. Checked against real wire-log lines rather than reasoned about.

#403 FIX: The tool-length probe seek distance was a hardcoded 90 mm

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	57	0
macros/tlo.macro	25	5

`tlo.macro` probed downward with a literal `G38.2 Z-90`, measured incrementally from the park height. That distance was sized for one particular toolsetter. Replace the toolsetter with something shorter and the seek simply stops in mid-air and raises `ALARM:5` — the probe never contacts, and nothing in the message suggests a distance is at fault.

- **The seek depth is now derived.** `MacroProcessor.EmitTloSeekDepth` computes it from the stored tool-length reference baseline, less a 10 mm allowance for a tool shorter than the one that set the baseline, measured against the live G59.3 Z offset rather than a remembered one.
- **Hard-clamped so it can never reach the table.** The depth is capped at the Z max-travel setting ($132) less a 5 mm envelope margin, whatever the arithmetic asks for.
- **Unknown stays unknown.** If there is no reference baseline, no fresh `$#` report or no travel setting, nothing is emitted at all and the macro keeps its own built-in default — no invented number is passed to a probe.
- **A diagnostic print that never printed.** One `(PRINT, … )` line in the macro carried nested parentheses, which silently suppress the whole comment on grblHAL — and it was the print for the exact branch that was failing. Now punctuated with dashes.

Verified on hardware: the new 15 mm touch plate asserts at machine Z −122.498 against a baseline of −122.473, and the derived seek of 132.473 mm reaches it where the old fixed 90 mm fell 32.5 mm short.

#404 NEW: Setup asserts a clean machine state before it runs, or refuses with the reason

File	+	−
CNC Core/CNC Core/MacroRunner.cs	34	1
ioSender XL/ioSender XL/StartJobView.xaml.cs	20	5
tools/waitidle-probe/Program.cs	23	1

A Setup program used to start in whatever modal state the previous job, macro or aborted probe cycle happened to leave behind. An aborted macro does not run its own cleanup, so a work offset selected inside it stayed selected — and the next Setup run inherited it silently, then failed somewhere further downstream where the cause was no longer visible.

- **The state is now asserted, not assumed.** All five Setup program builders emit `G21 G90 G94 G17 G49` and select their own work offset at the very top of the generated program, before the first motion, rather than part-way through it.
- **Three of the five were missing the no-alarm prerequisite** and would begin streaming into an alarm state. All five now carry it.
- **A new `nog92` prerequisite refuses on an active G92** — and deliberately refuses rather than clearing it. A G92 belongs to whoever set it; cancelling it would shift every remaining cut of that job. The message names the offset that is in the way and how to clear it.
- **It reads live state, not a cached report.** The new condition is on the coordinate-report refresh list, so it cannot pass on a stale snapshot taken minutes earlier.

The prerequisite is verified against a live simulator in three states — clear passes, an active G92 refuses with the reason, and clearing it passes again, so the gate cannot latch. Hoisting the offset select also cures a spurious "program does not fit the machine" refusal; the underlying bounding-box bug it works around is separate and still open.

#405 NEW: A run that aborts with the probe still touching now retracts Z instead of deadlocking

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	98	0

The corner-probe macro checks that the probe is released before it seeks, and refuses if it is not. That check is right, but it leaves the tool exactly where it was — resting on the plate, holding the probe closed. Because the check tests the probe's *initial* state, every retry then trips instantly on the same condition. There is no way out from the sender: the machine sits there until the operator jogs clear by hand or presses Reset.

- **Z now retracts to machine zero** when a run ends in an error or a halt with the probe reading asserted, for handed-off runs and dialog-driven ones alike — the fixture position test and the probe wizards can strand the tool the same way.
- **It fires on a live reading only.** The probe signal is cleared on any status report that omits the pin field, so this acts on the current state of the input, never on a remembered one.
- **It refuses rather than guessing.** The retract carries its own prerequisites, so it will not move on a machine state it cannot confirm, and it is armed once per abort so it cannot loop.
- **It reports the pin, not the command.** If the probe is still asserted after the lift, the message says so plainly — something other than a resting tool is holding it closed, and that must not be reported as recovered.

Changed, not yet exercised on hardware: the trigger needs a real aborted probe cycle with the probe closed, which the headless test harness cannot reach. To check it, rest the tool gently on the plate until the probe closes and start a corner probe — expect the macro's refusal, then an automatic lift to machine zero.

#406 Sound: hear a limit, a probe touch or an alarm without watching the screen

File	+	−
CNC Controls/CNC Controls/UiSounds.cs	263	0
CNC Controls/CNC Controls/UiSoundConfigControl.xaml.cs	189	0
CNC Controls/CNC Controls/UiSoundWatcher.cs	115	0
CNC Controls/CNC Controls/UiSoundConfigControl.xaml	62	0
CNC Controls/CNC Controls/AppConfig.cs	11	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	5	0
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	1	0
Locale/*/csv (7 locales)	77	0

A new settings page, **Settings > User Interface > Sound**, that attaches a sound to the three things worth hearing about while you are looking at the work rather than at the screen:

- **A limit switch is hit** — any limit input, the X/Y/Z lamps in the run strip and the limits of any other axis.
- **The probe or tool setter touches** — the probe input. Both the P and T lamps report the same input; which one lights depends only on where the probe is currently routed, so it is one event and one sound.
- **The machine goes into alarm** — the controller entering the alarm state.

All three default to None. A sender that makes noises nobody asked for is one people mute, and then the sound they did want is inaudible too.

The list offers the sounds Windows ships with, *filtered by existence* so a trimmed install is never offered a sound that cannot play, and labelled by what they sound like rather than by Microsoft's file names. Browse accepts any file: rather than filtering by extension, the chosen file is **loaded through the player there and then**, and if it cannot be played you are told the reason and the previous choice is put back. Choosing an entry plays it, so you can hear what you are picking. A custom file that is later moved or deleted is marked "(missing)" in the list and goes quiet rather than failing mid-job.

Sounds fire on the **transition** into asserted, never on a state that was already true when ioSender connected — a machine parked on a limit switch is not the switch being hit, and chiming on every connect teaches you to ignore the sound, which is the one thing a warning sound must not become. Each event has its own floor on how often it can repeat, because a marginally closed limit switch chatters and status arrives several times a second.

It watches the controller state rather than the run strip that shows the same lamps, so a sound still happens while you are on another tab. That is the entire point of wanting one.

Note that grblHAL sends one status report as it accepts $H and then stays silent for the whole homing cycle, so limits are not reported during homing unless $10 bit 12 ("Enable when homing") is set. With it off, the limit sound means what you want it to mean: you hit something you did not intend to.

#407 Startup could die in the connect path and leave a half-running instance holding the lock

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	20	1
ioSender XL/ioSender XL/JobView.xaml.cs	19	3
CNC Controls/CNC Controls/TrinamicView.xaml.cs	6	2

A `NullReferenceException` during startup left ioSender in a state that looked, from outside, exactly like an application that would not start at all. Three things were wrong, and the visible symptom was none of them.

- **The connect result was not acted on.** Two outcomes of the controller handshake end the application - close, and exit - and both continued into the rest of the method instead of returning, so everything after them ran against a window that had already been told to go away. One of those steps reached the dispatcher through `Application.Current`, which is null once shutdown has begun.
- **The dispatcher was reached the long way round.** That call now uses the view's own dispatcher, as every other one in the same method already did - the same dispatcher, but one that cannot be null there. The same pattern was swept up in the Trinamic view, where it is reached from comms data arrival, which keeps delivering while the app shuts down.
- **The throw aborted the rest of startup** - and this is what actually cost the session. What follows the connect is not optional: the window key hooks, the global shortcut dispatcher, the tab shortcuts and the console restore. None of them ran. The result was not a dead process but a worse one: a window that stayed up and kept polling the controller for nine hours with no keyboard handling, still holding the single-instance lock, so every later launch quietly forwarded to it and exited. A failed connect is now logged to the debug log and the console, said plainly on the status line, and startup carries on with a usable window.

Diagnosed from the crash log alone: the debug log that would have confirmed which path was taken had already rotated the startup away, because comms polling writes several lines a second and the run lasted nine hours. The fix is deliberately written not to depend on that - the early return stops the code being reached on the closing path, and the dispatcher change removes the dereference even if it is.

#408 FIX: the tool-length baseline was stored in the wrong frame, silently switching the derived seek off

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	115	32
macros/tlo.macro	21	2

Machine Setup stored the tool-length baseline from the probe program's own `TLOREF_Z` print, which is `#5063` — the probe result expressed in whatever work coordinate system happened to be active. `MacroProcessor.EmitTloSeekDepth` subtracts that baseline from G59.3's **machine** Z, so a work Z there is not a small error, it is a different quantity.

Measured: a probe at machine Z −50.002, under G54 with a WCO Z of −115.473, stored **65.471**. A positive baseline drives the computed depth negative, into that method's own "depth ≤ 0" refusal — so it emitted nothing and `tlo.macro` fell back to its 90 mm literal, the very number that threw `ALARM:5` against this touch plate and prompted the derived seek in the first place. It failed safe, which is the clamp working, but the feature was off and the screen read "Baseline: 65.471 mm" like a working measurement.

- **Both writers now store the machine Z** from the controller's own `PRB` report. The adjacent line setting the target surface already did exactly that, under a comment saying to use the machine Z and not arithmetic on the work position — two lines apart, two frames.
- **The whole chain is one frame now, and ignores G59.3.** `tlo.macro` publishes `[#5063 + #5383]`, adding G59.3's own Z offset back so it cancels identically. Verified from the firmware source: `#5061`–`#5069` return the probe position with only the active-at-probe-time G5x offset subtracted, so machine Z is the sum. Previously the macro's `G43.1` subtraction and the sender's seek derivation agreed *only* while that offset was 0 — and it has been both 0.000 and −42.000 inside one week here, which at −42 would have put the applied tool length offset 42 mm out.
- **A positive baseline now reads as NOT USABLE** rather than being printed as a measurement. The refusal was correct and completely invisible.
- The probe program's `TLOREF_Z` print carried a work Z and was only ever a "got past the probe" signal, so it carries no value at all now — it showed one number while the configuration held another.

Hardware-verified: with G59.3's Z offset at −41.999 the macro reported a seek depth of 91.206 (exact for that park, not the 133.205 a park of zero gives), `probe_z` came back as the machine −123.214 where a bare `#5063` would have read −81.215, and the applied offset was −0.009 rather than the ≈42 a broken conversion produces.

#409 FIX: Reference TLO's seek stopped 2 mm short of the trigger, so the probe refused in mid-air

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	143	57
CNC Controls/CNC Controls/MacroProcessor.cs	27	5

`ComputeTloSearchDistance` built the Probing tab's descent as `origin − surface − 2 mm standoff`, and its remarks explained why: the stored Z was a **floor** — the spoilboard the plate stands on — so travelling to just above it is correct and tool-agnostic.

That meaning had been deliberately migrated. The field now means "the top of whatever measures tool length", i.e. the machine Z at which the *reference tool* triggers — the same quantity the baseline holds. **The arithmetic was never updated.** Subtracting a standoff from a trigger Z lands the seek 2 mm *short* of the trigger, so the probe refuses in mid-air for the very tool that recorded the number. It stayed latent only while the stored value was still a floor-era number; the first button to write a trigger Z made it live.

- **It mirrors `EmitTloSeekDepth` now** — the same probe against the same plate, differing only in who emits the g-code. The allowance replaces the standoff *with its sign flipped*: the seek goes *past* the recorded trigger, far enough to find a tool up to the allowance shorter. Bounded descent instead of guaranteed refusal.
- **The short-tool allowance and envelope margin are shared constants** rather than two copies. This seek has been wrong three times; a second copy of the number is how that keeps happening.
- **It refuses when Z travel is unknown**, as the macro path does, instead of clamping to nothing. The clamp is the safety property, so an unclamped descent must not be the fallback.
- **Reference TLO now stores the target surface from the same touch**, so the two numbers for one surface cannot drift apart, and a disagreement over 1 mm is flagged on the row as stale with the gap named.
- The displayed text no longer promises "stopping 2 mm above the surface". It states the allowance, and says a shorter tool runs out of travel in air and alarms — a refusal rather than a crash. A sentence like that is read as the reassurance that makes the descent safe to start, so a wrong one is worse than none.

Hardware-verified: the wire log carries the corrected seek as a literal, `G38.2 Z−118.628`, exact for the stored baseline. The old arithmetic would have emitted `Z−106.628` against a trigger at −112.98 — 6.35 mm short, alarming in air. The new seek reaches with 5.65 mm to spare.

#410 FIX: the applied tool-length offset was the one number never in the log, and the console ate a trailing bracket

File	+	−
CNC Core/CNC Core/GrblViewModel.cs	15	1
CNC Controls/CNC Controls/MacroProcessor.cs	8	2
ioSender XL/ioSender XL/StartJobView.xaml.cs	10	2
macros/tlo.macro	14	2

Two independent faults, both in the instrument rather than the machine, and both found by reading a hardware Setup run.

- **The console ate a bracket.** The `[MSG:]` case used `TrimEnd(']')`, which strips *every* trailing bracket — so any message ending in one lost its own. The wire carried `[MSG:TLO06 TLO APPLIED tlo=[-123.192 - -123.205]]` and the console showed `tlo=[-123.192 - -123.205`. Silent, and it corrupts the console copy of precisely the line someone reads when a probe has gone wrong, while the wire log looks perfect.
- **The expression was never evaluated.** A g-code `(PRINT, ..)` substitutes on `#` but does *not* evaluate brackets, so `tlo=[#<_probe_z> - #<_tlo_ref>]` logged the expression with its operands filled in and never the answer. That number is the applied tool length offset — the value that decides whether a job cuts at the right depth — and it was the single number missing from every log.

Fixed in all three places that printed it: `tlo.macro`'s `TLO06`, and the sender's `LS_TLO_RESTORED` and `LS_ORIGIN_Z`. Each computes the offset into a named parameter first, applies **that** to `G43.1`, and prints it alongside its operands — so the logged number is by construction the one `G43.1` received, not a second derivation that ought to agree with it.

Hardware-verified: `TLO06 TLO APPLIED tlo=-0.009 probe_z=-123.214 ref=-123.205` — a number, with its operands, and the arithmetic exact. `LS_ORIGIN_Z raw=-116.308 tlo=-0.009 origin=-116.299` is internally consistent too.

#411 NEW: the driver and its version are reported at connect, so you can tell which build you are talking to

File	+	−
`CNC Core/CNC Core/Grbl.cs`	27	0
`ioSender XL/ioSender XL/JobView.xaml.cs`	15	0

Nothing at connect could answer “which build is this?”. The connect line reports the version and build from `[VER:]`, and those are the grbl **core** version — identical for every binary built from a given core. So a freshly built simulator logged `1.1f.20260602 build 20260602` exactly as the previous one had. `$I`'s `[DRIVER:..]` and `[DRIVER VERSION:..]` fell through the parser into the raw system-info list, where nothing read them.

- Both are captured now and logged on their own line: `– driver: <name> version <n>`. On a real board that is a driver revision; the matching simulator change reports its own build stamp there, so the line is what tells two simulator binaries apart.
- **Cleared per `$I`** rather than left to be overwritten. A controller that does not report those lines would otherwise leave the *previous* controller's values standing — and switching between a real machine and the simulator inside one session is routine, which is precisely the case where a stale driver version reads as the truth about what is connected.

Note for anyone extending this: the `$I` parse splits the whole line on `:`, so a value containing colons arrives truncated at the first one. That is why the simulator's build stamp is colon-free, and why grbl's own driver versions are too.

#412 The firmware's build date is gone - ten feature gates were reading 0

File	+	−
CNC Core/CNC Core/Grbl.cs	66	6
CNC Core/CNC Core/NetworkScanner.cs	2	1
CNC Controls/CNC Controls/SDCardView.xaml.cs	3	3
CNC Controls/CNC Controls/SpindleControl.xaml.cs	1	1
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	4	0
CNC Controls Probing/CNC Controls Probing/ProbingViewModel.cs	1	1
Grbl Config App/Grbl Config App/MainWindow.xaml.cs	6	6
tools/version-gate-probe/Program.cs	147	0
tools/version-gate-probe/version-gate-probe.csproj	54	0

grblHAL used to carry two hand-typed constants, **GRBL_VERSION** and **GRBL_BUILD**, and reported them joined together: **[VER:1.1f.20250901:<build info>]**. The sender read the version's trailing field as a **YYYYMMDD** build date into **GrblInfo.Build**, and a decade of "does this controller have feature X yet?" tests are written as numeric comparisons against it.

Firmware 2.0.0 deleted both constants and derives the version from the nearest **v*.*.*** git tag, because a hand-typed build number did not move when the code moved - two option-mismatched builds once shared one, which is how a controller presented as identical firmware while laying out its settings storage differently. The wire now reads **[VER:2.0.0:<build info>]** with *no build-date field*, and the boot banner says **GrblHAL 2.0.0**. There is nothing for the old parse to find, so **GrblInfo.Build** came out **0** and every one of those gates flipped.

Eight of them merely switched something off against the newest firmware in the fleet:

- **$ES/$EE extended settings** - the descriptions and enumerations behind the whole settings UI
- **Reference tool offset** on the Probing tab
- **Probe protection**
- **SD card delete** and **show all files**
- **$SPINDLES enumeration**
- the **FTP PWD re-query** before an upload
- two tabs in the standalone **Config app**

The other two went the other way, and those are the ones that matter: the spindle id/num pair is *swapped* when the build predates 2024-08-12, to work around a controller bug. A build of 0 predates everything, so the workaround came **on** against firmware that does not have the bug - a sender that silently talks to the wrong spindle.

GrblInfo now parses the semantic version into **FirmwareVersion** and absorbs the whole problem in one predicate, **BuildAtLeast(date)**: true outright on a git-tag controller, since that scheme postdates every date any of these gates test; a numeric comparison on a date-stamped build, exactly as before; and false on classic grbl, which never reported a date at all. All ten sites ask it instead of comparing by hand - **six of which were found by sweeping for siblings** rather than from the list of measured ones, among them the tool-offset reference and all four in the Config app.

The version parse is deliberately stricter than the framework's: **System.Version.TryParse** accepts a bare **"1.1"**, and accepting that would read an *ancient* classic-grbl controller as the new scheme and switch every gate on for it. It wants three or four all-numeric parts, which both **"1.1f"** and **"1.1f.20250901"** fail on the letter. Both fields are also now assigned on *every* $I rather than only when the pattern matches, so moving

between the real machine and the simulator inside one session cannot leave the previous controller's scheme standing - the same stale-across-controllers trap the **[DRIVER:]** clear already exists for, and here it would quietly re-arm the spindle workaround.

$I's new **[NVSVER:<major>.<minor>-<fingerprint>]** line is captured too and shown in Machine Setup's copyable firmware block. It is the *settings-storage layout* version, deliberately independent of the firmware version - a git-derived version changes on every commit, so keying storage migration on it would rewrite all of NVS on every flash - and it is what says whether a settings image on the board matches the layout the running firmware expects.

Verified by tools\version-gate-probe, a net462 harness that feeds the real $I lines of all three version schemes through GrblInfo's own parser in the app's own CNC.Core.dll and asserts all ten gates for each - 86 checks. A compile proves nothing about which way ten booleans fell, least of all the two that enable a workaround rather than disable a feature.

#413 Version 3.0 — one assembly number, and a release cutter that can express a major bump

File	+	−
Properties/AssemblyInfo.cs (13 projects)	26	26
ioSender XL/ioSender XL/MainWindow.xaml.cs	1	1
ioSender XL/ioSender XL/BuildInfo.cs	1	1
tools/cut-release.ps1	43	1
tools/test-release-version.ps1	62	0
tools/add-changelog-entry.ps1	1	1
.github/workflows/beta-release.yml	6	5

What leaves V2 is **3.0**, not 2.42. Two things had to change for that to be true rather than just declared.

- **The assembly numbers were never one number.** Nine `AssemblyInfo.cs` sat at `2.0.4x.0`, the app's own at `2.0.1.0`, and `CNC Client`, `CNC Contracts` and `CNC Common` were still at `1.0.0.0`. All thirteen are now `3.0.0.0`. This is not cosmetic: five places read `AssemblyVersion` at runtime — `CrashReporter`, `HangWatchdogReporter`, `DebugLog` and the Work Order signature — so a crash filed from a 3.0 build would have identified itself as `2.0.1.0`.
- **The release cutter could not have cut this release.** `cut-release.ps1` computes the next version as *minor + 1* off the last published one, which is the rolling model and works right up until a major bump: with `v2.41` out it can only ever produce `2.42`, so a push carrying a 3.0 binary would have been tagged as 2.41 + 1. `MainWindow.legacyVersion` — the same constant `beta-release.yml` already reads to stamp a beta — now gets a vote, and it is deliberately **one-directional**: it wins only when strictly higher, so it can raise the version but never hold one back. That makes it self-healing — once 3.0 is published the rolling number becomes 3.1, which outranks a `legacyVersion` still reading 3.0, and the normal behaviour resumes with nothing to clean up.
- **A guard, because a wrong version number ships silently.** `tools/test-release-version.ps1` pins the decision across seven cases, including both string-compare traps that would quietly discard the bump (`"3.0"` sorts *below* `"2.42"` on the second character, and `"2.9"` above `"2.10"`). It extracts the comparison from `cut-release.ps1`'s real text rather than restating it, so the test cannot drift from what ships, and fails loudly if the function is reshaped.

Three things were checked and deliberately left alone. **The update check is already safe across a major bump** — `CompareVersions` parses each component as an integer, so 3.0 against 2.41 is `[3,0]` against `[2,41]` and compares correctly in both directions, and the beta's `3.0.beta-pr20` reads as `[3,0,0]` and stays correctly newer. **Saved settings survive**: real config lives in `%AppData%\ioSender\App.config`, which is not version-keyed, and the only thing keyed to assembly version is `UiState.ImportLegacy`, a one-time jog step/feed import that runs only when the section is absent. **No binding redirect references our own assemblies**, so changing `AssemblyVersion` cannot break a reference.

On pr/portable-core, so the PR beta publishes as 3.0.beta-pr20. master stays on v2.41 until the beta has been run in. The version decision is verified by its own guard (7/7); the displayed version is a build-and-look check.

#414 The status log now says which build wrote it

File	+	−
ioSender XL/ioSender XL/MainWindow.xaml.cs	49	0

The status log carried no version information at all, so the first question asked of any log handed over — *which build is this?* — was the one question it could not answer. It now opens with a single line, written before anything else in startup can post a message, so it sits immediately under the run-started banner:

```
17:36:24.034  info  app  ioSender 3.0 – assembly 3.0.0.0, commit dev, Debug build, x86
```

- **Four parts, because they can disagree.** The *version* is what the title bar, splash and About box show — CI-stamped for a release or a PR beta (`3.0.beta-pr20`), the built-in fallback for a local build. The *assembly* version is how a crash report and the hang-watchdog report identify themselves, so this line is what ties one of those back to a release. The *commit* is the exact source, and reads `dev` on anything not built by CI — which is itself the most useful thing that field can say. Configuration and bitness close it out; the app runs 32-bit under WOW64, which is the context an out-of-memory failure has to be read in.
- **One line, both places.** It goes through the ordinary status funnel rather than straight to the file, so it reaches the Status window and the durable log in `%AppData%\ioSender\logs` without being written twice.
- **It cannot take the screen.** Logged as information rather than an error, and only an error auto-pops the Status window — so a build identity that is worth recording on every launch never costs the operator a window on any of them. It is also kept off the one-line status readout, which belongs to whatever the machine is doing.

Verified by reading latest_status.log after a launch: present, first, under the banner. Extends the operator-facing status log from #280.